CYBERSECURITY COMPLETE NOTES

SYLLABUS:-

Phase 1 — Computer Fundamentals

Objective: Understand how computers work before learning how to attack or protect them.

Module 1: Computer architecture

- CPU and processor architecture
- ALU and control unit
- Registers
- RAM and ROM
- Cache memory
- HDD and SSD
- BIOS and UEFI
- Boot process
- Motherboard
- Input/output devices

Module 2: Operating system fundamentals

- What is an operating system?
- Kernel and shell
- User mode and kernel mode
- Processes and threads
- Multitasking
- Memory management
- Virtual memory
- System calls
- File systems
- File permissions

Module 3: Virtualization

- Virtual machines

- Hypervisors
- Type 1 vs Type 2 hypervisors
- VirtualBox
- VMware
- KVM
- Containers
- Docker fundamentals

Module 4: Internet fundamentals

- What is the internet?
- Client-server architecture
- How websites work
- Domain names
- URLs
- Web browsers
- Web servers
- Data packets

Practical tasks:

- Install a Linux virtual machine.
- Install a Windows virtual machine.
- Understand the boot process.
- Create users and manage permissions.
- Monitor processes and system resources.

Phase 2 — Networking Fundamentals

Extremely important

Objective: Understand how computers communicate with one another.

Module 1: Networking basics

- What is a computer network?
- LAN, WAN, MAN and PAN
- Network topology
- Network devices
- Router

- Switch
- Hub
- Modem
- Access point
- Gateway
- Firewall

Module 2: OSI model

Learn all seven layers:

1. Physical
2. Data Link
3. Network
4. Transport
5. Session
6. Presentation
7. Application

Module 3: TCP/IP model

- Network access
- Internet
- Transport
- Application

Module 4: IP addressing

- IPv4
- IPv6
- Public IP
- Private IP
- Static IP
- Dynamic IP
- Subnetting
- CIDR
- Subnet masks
- Default gateway

- NAT
- ARP

Module 5: Network protocols

Protocol	What to understand
TCP	Reliable communication
UDP	Connectionless communication
HTTP	Web communication
HTTPS	Encrypted web communication
DNS	Domain name resolution
DHCP	Automatic IP configuration
FTP/SFTP	File transfer
SSH	Secure remote access
SMTP	Email sending
IMAP/POP3	Email retrieval
ICMP	Network diagnostics
TLS	Communication encryption
SNMP	Network management
LDAP	Directory services
SMB	File and printer sharing

Module 6: Network security

- Packet analysis
- Port numbers
- Open and closed ports
- TCP handshake
- TCP flags
- DNS security
- VPN
- Proxy
- Firewall rules
- IDS and IPS

- Network segmentation
- VLAN
- Wireless security
- WPA2 and WPA3

Module 7: Networking tools

- Wireshark
- Nmap
- tcpdump
- Netcat
- ping
- traceroute
- ipconfig / ip
- nslookup
- dig
- curl

Practical projects:

1. Capture network packets with Wireshark.
2. Analyze TCP handshakes.
3. Identify DNS requests.
4. Scan your own lab network with Nmap.
5. Create a small network using virtual machines.

Phase 3 — Linux and Windows

Objective: Become comfortable administering both major operating system environments.

Part A: Linux

1. Linux architecture
2. Linux distributions
3. Kali Linux
4. Ubuntu
5. Debian
6. File system hierarchy
7. Terminal and shell

8. File and directory management
9. File permissions
10. Users and groups
11. Root privileges
12. Package management
13. Process management
14. Service management
15. System logs
16. Networking commands
17. SSH configuration
18. Cron jobs
19. Bash fundamentals
20. Linux security and hardening

Important commands:

`ls`, `cd`, `pwd`, `cat`, `less`, `grep`, `find`, `chmod`, `chown`, `sudo`, `ps`, `top`, `kill`, `systemctl`, `journalctl`, `ip`, `ss`, `curl`, `ssh`.

Part B: Windows

1. Windows architecture
2. Windows filesystem
3. NTFS permissions
4. Users and groups
5. Windows Registry
6. Task Manager
7. Services
8. Event Viewer
9. Windows Defender
10. Windows Firewall
11. PowerShell
12. Windows networking
13. Group Policy
14. Windows authentication
15. Windows security policies

16. Windows administration

Practical projects:

- Build a Linux server.
- Create multiple users with different permissions.
- Configure SSH.
- Investigate Windows Event Viewer logs.
- Configure Windows Firewall.
- Harden both operating systems.

Phase 4 — Programming and Scripting

Objective: Write programs to automate security tasks and understand vulnerable software.

Learn in this order:

Language	Purpose	Priority
Python	Automation, security scripts	Essential
Bash	Linux automation	Essential
PowerShell	Windows automation	Essential
SQL	Database security	Essential
JavaScript	Web security	Important
HTML/CSS	Web fundamentals	Important
C	Memory and low-level security	Advanced
Assembly	Exploit development	Advanced
Go	Security tooling	Optional

Python syllabus

- Variables and data types
- Conditions
- Loops
- Functions
- Lists and dictionaries
- File handling
- Exception handling
- Modules and packages

- Object-oriented programming
- Regular expressions
- JSON
- HTTP requests
- Socket programming
- Multithreading
- API interaction
- Automation
- Logging

Advanced programming concepts

- Memory management
- Stack and heap
- Pointers
- Buffer overflows
- Data structures
- Algorithms
- Operating system interactions
- Networking libraries
- Secure coding

Projects:

1. Log analyzer.
2. Password strength checker.
3. File integrity checker.
4. Network information collector.
5. Automated security report generator.
6. HTTP security header analyzer.

Phase 5 — Cybersecurity Fundamentals

Learn the concepts that appear throughout the industry.

Module 1: Security principles

- CIA triad
- Confidentiality

- Integrity
- Availability
- Authentication
- Authorization
- Accounting
- Non-repudiation
- Least privilege
- Defense in depth
- Zero Trust

Module 2: Cryptography

- Encryption vs hashing
- Symmetric encryption
- Asymmetric encryption
- AES
- RSA
- ECC
- SHA-256
- Salting
- Digital signatures
- Certificates
- Public key infrastructure
- TLS
- Key exchange

Module 3: Authentication

- Password authentication
- MFA
- 2FA
- Biometrics
- SSO
- OAuth 2.0
- OpenID Connect
- SAML

- Kerberos

Module 4: Vulnerabilities and threats

- Malware
- Ransomware
- Spyware
- Trojan
- Worm
- Rootkit
- Phishing
- Social engineering
- Brute-force attacks
- Password spraying
- DoS and DDoS
- Man-in-the-middle
- Spoofing
- Privilege escalation
- Zero-day vulnerabilities

Module 5: Security frameworks

- NIST Cybersecurity Framework
- ISO 27001
- CIS Controls
- MITRE ATT&CK
- Cyber Kill Chain
- OWASP

Practical task: Create a security assessment of a small fictional company.

Phase 6 — Security Tools and Practical Laboratory

Objective: Learn to use professional security tools rather than simply memorizing their names.

Kali Linux 2023.2 Release Adds a New Hyper-V Image and PipeWire to XFCE Variant

1. Kali Linux

Security testing operating system, tools and laboratory environment.

20 Awesome Nmap Command Examples in Linux

2. Nmap

Network discovery, port scanning and service identification.

3.3. The Main window - Wireshark Documentation

3. Wireshark

Network packet capture and protocol analysis.

Un guide étape par étape pour l'utilisation de BurpSuite pour les tests de sécurité des applications Web

4. Burp Suite

Web application security testing.

File:Metasploit Framework.png - Wikimedia Commons

5. Metasploit Framework

Vulnerability validation and controlled exploitation.

Zed Attack Proxy (Formerly OWASP ZAP): 7 Key Features & Tutorial | Radware

6. OWASP ZAP

Automated web application security testing.

Additional tools:

- Gobuster
- ffuf
- Nikto
- John the Ripper
- Hashcat
- Hydra
- Netcat
- OpenSSL
- Ghidra
- Autopsy
- Volatility
- Sysmon
- Wazuh
- Splunk
- Elastic Security

Learn what each tool does, how it works, how to interpret its output, and how to verify its findings.

Phase 7 — Ethical Hacking and Penetration Testing

Objective: Understand how authorized security professionals discover and validate weaknesses.

Module 1: Penetration testing fundamentals

- Penetration testing methodology
- Scope definition
- Rules of engagement
- Authorization
- Legal and ethical boundaries
- Vulnerability assessment vs penetration testing
- Reporting

Module 2: Reconnaissance

- Passive reconnaissance
- Active reconnaissance
- OSINT
- DNS enumeration
- WHOIS
- Subdomain discovery
- Technology fingerprinting
- Attack surface mapping

Module 3: Scanning and enumeration

- Host discovery
- Port scanning
- Service detection
- Operating system detection
- Vulnerability scanning
- Service enumeration
- Network mapping

Module 4: Vulnerability assessment

- CVE
- CVSS
- Misconfiguration

- Outdated software
- Default credentials
- Authentication weaknesses
- Access control weaknesses
- Vulnerability prioritization

Module 5: Exploitation fundamentals

- Exploit concepts
- Proof of concept
- Payload concepts
- Remote code execution
- Local privilege escalation
- Authentication bypass
- Session attacks
- Post-exploitation concepts

Module 6: Privilege escalation

- Linux privilege escalation
- Windows privilege escalation
- Misconfigured permissions
- Vulnerable services
- Credential security
- Sudo misconfiguration
- Scheduled tasks

Module 7: Penetration testing reports

- Executive summary
- Technical findings
- Evidence
- Severity assessment
- Reproduction steps
- Remediation
- Retesting

Practical projects:

1. Perform a penetration test on an intentionally vulnerable VM.
2. Discover and document vulnerabilities.
3. Demonstrate controlled exploitation.
4. Recommend fixes.
5. Produce a professional penetration testing report.

Phase 8 — Web Application and API Security

Objective: Become capable of understanding and testing modern web applications.

Module 1: Web fundamentals

- Client-server communication
- HTTP requests and responses
- HTTP methods
- Headers
- Cookies
- Sessions
- Authentication
- REST APIs
- JSON
- WebSockets
- Same-origin policy
- CORS

Module 2: OWASP web vulnerabilities

- SQL Injection
- Cross-Site Scripting
- Cross-Site Request Forgery
- Broken access control
- Authentication failures
- Security misconfiguration
- Server-side request forgery
- File upload vulnerabilities
- Path traversal
- Insecure deserialization

- Cryptographic failures
- Business logic vulnerabilities

Module 3: Advanced web security

- DOM-based XSS
- Blind SQL injection
- Authentication bypass
- JWT vulnerabilities
- OAuth security
- API authorization
- GraphQL security
- Race conditions
- Request smuggling
- Cache poisoning
- Web cache deception

Module 4: API security

- REST API security
- API authentication
- API authorization
- Rate limiting
- Input validation
- Mass assignment
- Broken object-level authorization
- API key management

Practical projects:

- Test deliberately vulnerable web applications.
- Identify and document OWASP vulnerabilities.
- Build a secure REST API.
- Perform security testing on your own API.

Phase 9 — Network and System Security

Topics:

1. Network architecture security

2. Firewall architecture
3. IDS and IPS
4. Network access control
5. VPN security
6. Wireless network security
7. DNS security
8. Email security
9. Endpoint security
10. Server hardening
11. Linux hardening
12. Windows hardening
13. Network segmentation
14. Secure remote access
15. Vulnerability management
16. Patch management
17. Backup and disaster recovery
18. Infrastructure monitoring

Advanced topics:

- Network intrusion detection
- Network forensics
- Lateral movement detection
- Network access architecture
- Zero Trust network architecture

Phase 10 — Defensive Security and SOC

Objective: Learn to detect and respond to cyberattacks.

Module 1: SOC fundamentals

- SOC architecture
- SOC analyst responsibilities
- Security monitoring
- Alert triage
- Incident classification

- Escalation procedures

Module 2: Security monitoring

- System logs
- Network logs
- Firewall logs
- Authentication logs
- Windows Event Logs
- Linux system logs
- DNS logs
- Endpoint telemetry

Module 3: SIEM

- SIEM architecture
- Log collection
- Log normalization
- Correlation rules
- Alert generation
- Dashboards
- Threat hunting

Tools:

- Splunk
- Elastic Security
- Wazuh
- Microsoft Sentinel

Module 4: Endpoint detection

- Antivirus
- EDR
- XDR
- Sysmon
- Endpoint monitoring
- Behavioral detection

Module 5: Threat intelligence

- IOC
- IOA
- Threat feeds
- MITRE ATT&CK mapping
- Threat hunting
- Detection engineering

Module 6: Incident response

1. Preparation
2. Identification
3. Containment
4. Eradication
5. Recovery
6. Lessons learned

Practical projects:

- Build a home SOC lab.
- Collect Windows and Linux logs.
- Configure Wazuh.
- Generate test security events.
- Create detection rules.
- Investigate a simulated attack.

Phase 11 — Digital Forensics and Malware Analysis

Digital forensics

- Digital evidence
- Chain of custody
- Disk forensics
- Memory forensics
- File system forensics
- Browser forensics
- Network forensics
- Windows artifacts

- Linux artifacts
- Timeline analysis
- Deleted file recovery

Tools:

- Autopsy
- FTK Imager
- Volatility
- Wireshark
- Plaso

Malware analysis

- Malware classification
- Static analysis
- Dynamic analysis
- Malware behavior
- Reverse engineering fundamentals
- PE file format
- ELF file format
- Strings analysis
- Process monitoring
- Sandbox analysis
- Indicators of compromise

Tools:

- Ghidra
- Detect It Easy
- PE-bear
- YARA
- REMnux
- FLARE-VM

Practice malware analysis only in properly isolated environments.

Phase 12 — Active Directory and Enterprise Security

Topics:

- Windows Server fundamentals
- Active Directory architecture
- Domain controllers
- Domain users and groups
- Group Policy
- LDAP
- Kerberos
- NTLM
- DNS in Active Directory
- Organizational Units
- Domain trusts
- Windows authentication
- Active Directory security
- Identity and access management
- Privileged access management
- Enterprise security monitoring

Advanced concepts:

- Kerberoasting
- AS-REP roasting
- Pass-the-Hash
- Pass-the-Ticket
- Misconfigured delegation
- Active Directory privilege escalation
- BloodHound analysis
- Lateral movement detection

Phase 0 — Introduction to Cybersecurity

Objective: Understand what cybersecurity actually means.

Topics:

1. What is cybersecurity?
2. What is information security?
3. Cybersecurity vs ethical hacking
4. Why cybersecurity is important
5. Types of hackers:
    - White hat
    - Black hat
    - Grey hat
6. Types of cyberattacks
7. Threat, vulnerability and risk
8. Attack surface
9. Cybersecurity attack lifecycle
10. Red team, blue team and purple team
11. Security analyst vs penetration tester
12. Cybersecurity career opportunities

Understand these specializations:

Specialization	Main responsibility
Penetration testing	Find security weaknesses
Red teaming	Simulate realistic attackers
SOC analyst	Monitor security threats
Incident response	Handle security incidents
Cloud security	Protect cloud infrastructure
Application security	Secure software
Digital forensics	Investigate digital evidence

Malware analysis	Understand malicious software
Security engineering	Build security controls
GRC	Governance, risk and compliance

Practical task: Research five real cyberattacks and explain how they happened.

1. What is Cybersecurity?

Cybersecurity is the practice of protecting networks, devices, applications, and data from unauthorized access, digital attacks, or damage. Its goal is to keep systems functioning correctly and ensure data remains secure.

2. What is Information Security?

Information Security (InfoSec) is a broader concept. While cybersecurity focuses specifically on protecting *digital* assets from cyber threats, InfoSec protects *all* forms of information (physical files, conversations, digital data) from unauthorized access or alteration. It is built on the CIA Triad: Confidentiality, Integrity, and Availability.

3. Cybersecurity vs. Ethical Hacking

Cybersecurity is the overarching field that includes both defensive operations (protecting systems) and offensive operations (testing them). Ethical hacking is a specific offensive subset within cybersecurity where professionals are authorized to legally attack a system to find and patch flaws before malicious hackers can exploit them.

4. Why Cybersecurity is Important

As everything from personal banking to critical national infrastructure (power grids, hospitals) moves online, a breach can result in massive financial loss, identity theft, business closure, or even physical harm. It ensures privacy, maintains business continuity, and protects trust.

5. Types of Hackers

- **White Hat:** Ethical hackers who have explicit permission to hack into systems to find and fix vulnerabilities.
- **Black Hat:** Malicious hackers who break into systems without permission for financial gain, espionage, or destruction.
- **Grey Hat:** Hackers who break into systems without permission but usually without malicious intent. They often find a flaw and then offer to fix it for a fee or bounty.

6. Types of Cyberattacks

Cyberattacks are generally categorized by their . Major categories and types of cyberattacks include:

1. Malware (Malicious Software)

- **Ransomware**: Encrypts a victim's files or systems and demands a ransom payment (often in cryptocurrency) for the decryption key.
- **Spyware**: Secretly monitors user activity, captures keystrokes (keyloggers), or harvests personal and financial data.
- **Trojans**: Disguises itself as legitimate software to trick users into installing a hidden backdoor or payload.
- **Worms**: Self-replicating malware that spreads across networks without needing a host program or human interaction.
- **Rootkits**: Gives attackers privileged, root-level access to a computer while hiding deep within the operating system.
- **Botnets**: Networks of infected "zombie" computers controlled remotely by an attacker to launch coordinated assaults. [1, 2, 3]

2. Social Engineering

- **Phishing**: Uses deceptive emails, texts, or messages to trick users into handing over sensitive credentials or financial details.
- *Spear Phishing*: Targeted phishing aimed at a specific individual or organization.
- *Whaling*: Phishing directed specifically at high-profile executives or "big fish" targets.
- *Smishing / Vishing*: Phishing conducted via SMS (smishing) or voice calls/voicemail (vishing). [1, 2, 3]
- **Pretexting**: An attacker invents a fabricated scenario (a "pretext") to manipulate a victim into releasing information. [1]
- **Business Email Compromise (BEC)**: Attackers impersonate executives or trusted vendors to authorize fraudulent wire transfers or invoice payments. [1, 2]

3. Network & Infrastructure Attacks

- **Denial-of-Service (DoS) and DDoS**: Overwhelms a target server, network, or website with a flood of traffic or fake requests to knock it offline (DDoS uses multiple distributed machines or a botnet). [1]
- **Man-in-the-Middle (MitM)** / **On-Path Attacks**: An attacker intercepts, eavesdrops, or alters communications between two parties without their knowledge. [1, 2]
- **DNS Spoofing** / **Poisoning**: Corrupts Domain Name System records to redirect users to malicious or fake replica websites. [1, 2]
- **Zero-Day Exploits**: Attacks that target a previously unknown software vulnerability before the vendor can patch it. [1, 2]

4. Application & Code Injection Attacks

- **SQL Injection (SQLi)**: Inserts malicious SQL code into input fields to manipulate backend databases and steal or delete data.

- **Cross-Site Scripting (XSS)**: Injects malicious client-side scripts into trusted web pages viewed by other users.
- **Drive-by Downloads**: Unintentionally downloading malware simply by visiting a compromised or malicious website.

5. Identity & Access Attacks

- **Credential Stuffing** / **Password Attacks**: Using automated tools to test stolen username and password pairs across multiple unrelated services (exploiting password reuse).
- **Brute-Force Attacks**: Systematically guessing passwords or encryption keys through exhaustive trial and error.
- **Session Hijacking**: Stealing or intercepting a valid session token to impersonate a user on a web application.

6. Specialized & Modern Threats

- **Supply Chain Attacks**: Compromising third-party software vendors, hardware manufacturers, or service providers to infect downstream customers. [1, 2]
- **Insider Threats**: Intentional or accidental data breaches caused by employees, contractors, or partners with authorized access. [1, 2]
- **Cryptojacking**: Secretly hijacking a victim's CPU or GPU resources to mine cryptocurrency. [1, 2]
- **AI-Powered & Deepfake Attacks**: Using artificial intelligence, voice cloning, or deepfake video to execute hyper-realistic social engineering scams. [1]

7. Threat, Vulnerability, and Risk

- **Vulnerability:** A weakness or flaw in a system (e.g., an unpatched software bug or a weak password).
- **Threat:** An entity or event that could exploit a vulnerability (e.g., a hacker or a malware strain).
- **Risk:** The potential for loss or damage when a threat exploits a vulnerability. (Risk = Threat × Vulnerability).

8. Attack Surface

The attack surface is the total sum of all possible entry points (vulnerabilities) an unauthorized user can use to enter a system. This includes open network ports, employee email accounts, unpatched software, and exposed APIs. Reducing the attack surface makes a system harder to hack.

9. Cybersecurity Attack Lifecycle (The Cyber Kill Chain)

Attacks usually follow a structured progression:

1. **Reconnaissance:** Gathering information on the target.

2. **Weaponization:** Creating a malicious payload tailored to the target.
3. **Delivery:** Sending the payload (e.g., via phishing email).
4. **Exploitation:** Triggering the vulnerability.
5. **Installation:** Establishing a persistent backdoor.
6. **Command and Control (C2):** Opening a communication channel to the attacker's server.
7. **Actions on Objectives:** Stealing data, deploying ransomware, or destroying systems.

10. Red Team, Blue Team, and Purple Team

- **Red Team:** Offensive security professionals hired to simulate real-world attacks to test an organization's defenses.
- **Blue Team:** Defensive security professionals who monitor environments, detect attacks, and respond to incidents.
- **Purple Team:** Not necessarily a separate group, but a collaborative methodology where the Red and Blue teams share insights constantly to improve overall security.

11. Security Analyst vs. Penetration Tester

- **Security Analyst:** A defensive (Blue Team) role focused on continuously monitoring network traffic, analyzing alerts, and responding to active threats in a Security Operations Center (SOC).
- **Penetration Tester:** An offensive (Red Team) role focused on actively trying to break into specific applications or networks during a set timeframe to discover security gaps.

12. Cybersecurity Specializations

Specialization	Main Responsibility
Penetration testing	Find security weaknesses by exploiting systems
Red teaming	Simulate realistic, full-scope attacker campaigns
SOC analyst	Monitor network traffic and security alerts for threats
Incident response	Handle, contain, and remediate active security breaches
Cloud security	Protect cloud infrastructure (AWS, Azure, GCP) and data
Application security	Secure software code and development pipelines
Digital forensics	Investigate digital evidence post-breach or for law enforcement

Specialization	Main Responsibility
Malware analysis	Reverse-engineer malicious software to understand its behavior
Security engineering	Build and implement security controls and firewalls
GRC (Gov, Risk, Comp)	Ensure adherence to security frameworks, laws, and policies

Practical Task: 5 Real Cyberattacks and How They Happened

1. WannaCry Ransomware (2017)

- **How it happened:** The attack exploited a vulnerability in Microsoft's Server Message Block (SMB) protocol. The exploit, known as "EternalBlue" (originally developed by the NSA and leaked by a group called the Shadow Brokers), allowed the malware to automatically spread from computer to computer across networks without user interaction, locking files and demanding Bitcoin payments.

2. Equifax Data Breach (2017)

- **How it happened:** Attackers exploited a known vulnerability in Apache Struts, an open-source web application framework used by the Equifax credit reporting agency. A patch for the vulnerability had been available for months, but Equifax failed to apply it. Attackers accessed the personal data (SSNs, birth dates) of over 147 million people.

3. SolarWinds Supply Chain Attack (2020)

- **How it happened:** Russian state-sponsored hackers compromised the software build system of SolarWinds, an IT management company. The hackers injected a malicious backdoor ("Sunburst") into legitimate updates of the SolarWinds Orion software. When thousands of global organizations and US government agencies downloaded the trusted update, the attackers gained access to their internal networks.

4. Target Retail Breach (2013)

- **How it happened:** Attackers did not hack Target directly at first. Instead, they sent a phishing email to an HVAC vendor that worked with Target. Using the vendor's stolen credentials, the hackers accessed Target's vendor billing portal, moved laterally into Target's corporate network, and installed malware on point-of-sale (POS) systems in stores, stealing 40 million credit card numbers.

5. NotPetya (2017)

- **How it happened:** Disguised as ransomware, NotPetya was actually a destructive "wiper" malware designed to permanently destroy data. It was distributed via a compromised update for a Ukrainian tax accounting software called M.E.Doc. Once inside a network, it

also used the EternalBlue exploit (like WannaCry) and password-stealing techniques to rapidly infect global corporate networks, causing billions of dollars in damage.

Phase 1 — Computer Fundamentals

Objective: Understand how a computer works internally, how operating systems manage hardware, how virtual machines work, and how computers communicate over the internet.

This knowledge is important in cybersecurity because you cannot properly secure or attack a system if you don't understand how it works.

Module 1: Computer Architecture

Computer architecture means understanding the internal components of a computer and how they work together.

From Parts to Powerhouse: Building Your Affordable Gaming Rig | SoftBit
sybergaming.com

1. CPU and Processor Architecture

- CPU (Central Processing Unit): The brain of a computer. It executes instructions and performs calculations.
- Processor architecture: Describes how a CPU is designed and processes instructions. Examples: x86, x86-64 and ARM.

2. ALU and Control Unit

The CPU has two important components:

- ALU (Arithmetic Logic Unit): Performs mathematical calculations and logical operations.
- Control Unit (CU): Controls and coordinates the execution of instructions.

3. Registers

Registers are very small and extremely fast memory locations inside the CPU.

They temporarily store data and instructions that the CPU is currently using.

4. RAM and ROM

- RAM: Temporary memory that stores currently running programs and data. Its contents are generally lost when power is turned off.
- ROM: Non-volatile memory used to store firmware and other essential instructions.

5. Cache Memory

Cache is extremely fast memory located inside or close to the CPU.

It stores frequently used data so the CPU doesn't need to access slower RAM repeatedly.

6. HDD and SSD

These are storage devices used to save files, applications and operating systems.

- HDD: Uses spinning magnetic disks to store data.
- SSD: Uses flash memory and is generally much faster than HDD.

7. BIOS and UEFI

BIOS and UEFI are firmware interfaces that initialize hardware and start the boot process.

- BIOS: Older firmware standard.
- UEFI: Modern replacement with more advanced features.

8. Boot Process

Booting means starting a computer and loading its operating system.

Basic process:

1. Press the power button.
2. Firmware (BIOS/UEFI) initializes hardware.
3. Firmware finds a bootable device.
4. Bootloader starts.
5. Operating system kernel loads.
6. System services start.
7. Login screen appears.

9. Motherboard

The motherboard is the main circuit board that connects all computer components, such as CPU, RAM, storage and other devices.

10. Input/Output Devices

- Input devices: Send information to the computer. Examples: keyboard, mouse and microphone.
- Output devices: Display or produce information. Examples: monitor, speakers and printer.

Module 2: Operating System Fundamentals

An operating system (OS) is software that manages computer hardware and allows applications to run.

Kernel Development Series Phase 0: Orientation & Big Picture - Wasil Zafar
wasilzafar.com

1. What is an Operating System?

An operating system manages hardware, memory, files, processes and applications.

Examples: Windows, Linux and macOS.

2. Kernel and Shell

- Kernel: The core part of an operating system. It manages CPU, memory, hardware and processes.
- Shell: An interface through which users communicate with the operating system.

Example: Bash is a Linux command-line shell.

3. User Mode and Kernel Mode

These are two execution modes of a computer.

- User mode: Applications run with restricted permissions.
- Kernel mode: The operating system kernel runs with privileged access to hardware and system resources.

Cybersecurity importance: Malware often attempts to gain higher privileges to control more parts of a system.

4. Processes and Threads

- Process: A running program.
- Thread: A unit of execution inside a process.

Example: A web browser is a process. It may contain multiple threads for different tasks.

5. Multitasking

Multitasking means an operating system can manage multiple running programs at the same time.

Example: Running Chrome, VS Code and a music player simultaneously.

6. Memory Management

Memory management is the process of managing RAM and allocating memory to running programs.

7. Virtual Memory

Virtual memory allows an operating system to use storage space to support memory management when physical RAM is insufficient.

It also provides each process with its own virtual address space.

8. System Calls

System calls allow applications to request services from the operating system kernel.

Examples:

- Opening files.
- Reading data.
- Creating processes.
- Accessing system resources.

9. File Systems

A file system organizes and manages how data is stored and accessed on storage devices.

Examples:

- NTFS (Windows).
- ext4 (Linux).
- FAT32.
- exFAT.

10. File Permissions

File permissions determine who can read, write or execute files.

Linux has three basic permission types:

Permission	Meaning
Read (r)	View file contents
Write (w)	Modify file contents
Execute (x)	Run a file or enter a directory

Cybersecurity importance: Incorrect permissions can allow unauthorized users to access or modify sensitive files.

Module 3: Virtualization

Virtualization means creating virtual versions of computers or computing resources using physical hardware.

Virtual Machines : Fundamentals for Beginners | by Rahul Rathore | Medium

medium.com

1. Virtual Machines (VMs)

A virtual machine is a software-based computer that runs its own operating system.

Example: Running Kali Linux inside Windows without replacing Windows.

2. Hypervisors

A hypervisor is software that creates and manages virtual machines.

3. Type 1 vs Type 2 Hypervisors

Type	Explanation	Example
Type 1	Runs directly on hardware	VMware ESXi, Hyper-V
Type 2	Runs on an existing OS	VirtualBox, VMware Workstation

4. VirtualBox

VirtualBox is free virtualization software that allows you to run different operating systems inside virtual machines.

Example: Running Kali Linux on your Windows computer.

5. VMware

VMware provides virtualization products used for running and managing virtual machines.

6. KVM

KVM (Kernel-based Virtual Machine) is a Linux technology that turns the Linux kernel into a hypervisor.

7. Containers

Containers package applications and their dependencies together.

Unlike traditional VMs, containers generally share the host operating system's kernel.

8. Docker Fundamentals

Docker is a platform for building, running and managing containers.

Important terms:

- Image: A template used to create containers.
- Container: A running instance of an image.
- Dockerfile: Instructions for building an image.
- Docker Hub: A registry for sharing container images.

Cybersecurity importance: Virtual machines and containers help create isolated environments for security testing and malware analysis. Isolation is not absolute, so use proper security precautions.

Module 4: Internet Fundamentals

The internet is a global network of interconnected computers and devices that communicate using standard protocols.

How the Internet Works: Explained Simply for Beginners
kidexplains.com

1. What is the Internet?

The internet is a worldwide network that connects computers, servers, phones and other devices.

2. Client-Server Architecture

- Client: A device or application that requests a service.
- Server: A computer or software that provides the requested service.

Example: Your browser is the client, and Google's web server responds to your search request.

3. How Websites Work

When you open a website:

1. You enter a website address.
2. Your browser finds the server's IP address using DNS.
3. Your browser connects to the server.
4. It sends an HTTP or HTTPS request.
5. The server processes the request.
6. The server sends a response.
7. Your browser displays the website.

4. Domain Names

A domain name is a human-readable address used to access websites.

Examples:

- google.com
- youtube.com
- wikipedia.org

5. URLs

URL (Uniform Resource Locator) is the complete address of a resource on the internet.

Example:

```
https://www.example.com/login
```

It contains:

- Protocol: HTTPS
- Domain: www.example.com
- Path: /login

6. Web Browsers

A web browser is software used to access and display websites.

Examples: Chrome, Firefox, Brave and Edge.

7. Web Servers

A web server is software or a computer that receives requests and delivers website content.

Examples: Apache, Nginx and Microsoft IIS.

8. Data Packets

When information travels over a network, it is divided into smaller units called packets.

Each packet contains information needed to deliver the data to its destination.

Example: When you send a message, its data travels through networks in packets.

Phase 2 — Networking Fundamentals

Extremely important for cybersecurity

Objective: Understand how computers communicate, how information travels across networks, how devices identify each other, and how network communication can be attacked or protected.

Networking is one of the most important foundations of cybersecurity.

For example, when you perform ethical hacking:

- You use Nmap to discover devices and open ports.
- You use Wireshark to capture and analyze network traffic.
- You use DNS knowledge to understand domain resolution.
- You use TCP/IP knowledge to understand network communication.
- You use firewalls to control network access.

Let's understand every module in simple language.

Module 1: Networking Basics

1. What is a computer network?

A computer network is a group of computers and devices connected together to communicate and share information.

Example: Your laptop, mobile phone and printer connected to the same Wi-Fi network.

Understanding Networking Basics (A Beginner Friendly Guide) | by Zahid (Cybersecurity Student) | Medium
medium.com

2. LAN, WAN, MAN and PAN

These are different types of networks classified by their coverage area.

Network	Full form	Meaning
LAN	Local Area Network	Network in a home, office or building
WAN	Wide Area Network	Network covering large geographical areas
MAN	Metropolitan Area Network	Network covering a city
PAN	Personal Area Network	Network around an individual

Examples:

- LAN: College computer laboratory.
- WAN: The internet.
- MAN: A city-wide network.
- PAN: Your phone connected to Bluetooth earbuds.

3. Network Topology

Network topology describes how devices are physically or logically connected in a network.

Network Topologies (Bus, Star, Ring) Notes — OCR GCSE
Bus Topology in Computer Networks: Concepts, Uses, and Examples
Network Topologies (Bus, Star, Ring) Notes — OCR GCSE
4

Important types:

- Star: All devices connect to a central switch.
- Bus: All devices share one main communication cable.
- Ring: Devices connect in a circular arrangement.
- Mesh: Devices have multiple connections to other devices.
- Tree: Devices are arranged in a hierarchical structure.

4. Network Devices

Network devices help computers communicate with each other.

Network Devices: Router, Switch, Hub, Firewall, Access Point, Modem | Meet Parmar posted on the topic | LinkedIn
linkedin.com

Device	Function
Router	Connects different networks and forwards packets
Switch	Connects devices within a LAN

Hub	Broadcasts received data to all connected ports
Modem	Converts signals for communication over an ISP connection
Access Point	Connects wireless devices to a network
Gateway	Connects networks that may use different protocols or architectures
Firewall	Controls network traffic according to security rules

Important difference:

A switch connects devices inside a network, while a router connects different networks.

5. Router

A router forwards data packets between different networks.

Example: Your home router connects your private home network to the internet.

6. Switch

A switch connects multiple devices in a LAN and forwards Ethernet frames using MAC addresses.

7. Hub

A hub sends incoming data to every connected port instead of intelligently forwarding it to the intended device.

It is an older networking device.

8. Modem

A modem provides the connection between your network and your Internet Service Provider (ISP) by converting signals appropriate to the access technology.

9. Access Point

An access point allows wireless devices to connect to a wired network using Wi-Fi.

10. Gateway

A gateway is a device or system that connects different networks or systems.

Your default gateway is usually your router's local IP address.

11. Firewall

A firewall monitors and controls incoming and outgoing network traffic according to predefined security rules.

Example: Blocking unauthorized connections to your computer.

Module 2: OSI Model

The OSI model is a seven-layer reference model that explains how data communication works between networked systems.

[Please Do Not Throw Sausage Pizza Away! — OSI Model Mnemonic | by Anupriya Verma | Jan, 2026 | Medium
medium.com](medium.com)

Think of sending a message from your computer to another computer. The OSI model divides the communication process into seven layers.

Layer	Name	Main function
7	Application	Provides network services to applications
6	Presentation	Handles data format, encryption and compression
5	Session	Manages communication sessions
4	Transport	Handles end-to-end data delivery
3	Network	Handles logical addressing and routing
2	Data Link	Handles frames and MAC addressing
1	Physical	Transmits raw bits through physical media

Let's understand them.

Layer 1: Physical

Transfers raw bits (0s and 1s) through physical media.

Examples: Cables, radio signals, connectors and electrical signals.

Layer 2: Data Link

Transfers data between devices on the same local network.

Important concepts:

- MAC addresses.
- Ethernet.
- Switches.
- Frames.

Layer 3: Network

Handles IP addressing and routing packets between different networks.

Important concepts:

- IPv4.

- IPv6.
- Routers.
- IP packets.

Layer 4: Transport

Provides communication between applications running on different devices.

Important protocols:

- TCP.
- UDP.

Layer 5: Session

Establishes, manages and terminates communication sessions between applications.

Layer 6: Presentation

Handles how data is represented, encoded, compressed and encrypted.

Layer 7: Application

Provides network services used by applications.

Examples: HTTP, DNS, SMTP and FTP.

Easy way to remember:

Physical → Data Link → Network → Transport → Session → Presentation → Application.

When data travels from your computer, it generally moves from Layer 7 down to Layer 1. At the receiving end, it moves upward.

Module 3: TCP/IP Model

TCP/IP is the practical networking model used for communication over the internet.

It has four layers.

Decoding Network Architecture: A Brief Exploration of TCP/IP and OSI Models | by Abassam | Medium
medium.com

TCP/IP layer	Related OSI layers
Application	7, 6, 5
Transport	4
Internet	3
Network Access	2, 1

1. Network Access Layer

Handles communication over the local network.

Examples: Ethernet and Wi-Fi.

2. Internet Layer

Handles IP addressing and packet routing.

Example: IP.

3. Transport Layer

Handles communication between applications.

Examples: TCP and UDP.

4. Application Layer

Provides network services to applications.

Examples: HTTP, HTTPS, DNS and SSH.

OSI vs TCP/IP: OSI is primarily a reference model for understanding networking. TCP/IP describes the protocol architecture used in real-world internet communication.

Module 4: IP Addressing

An IP address is a logical address used to identify an interface on an IP network.

Converting IP addresses into Binary | NetworkAcademy.IO
networkacademy.io

1. IPv4

IPv4 is a 32-bit addressing system.

Example:

`192.168.1.10`

It contains four numbers called octets, each ranging from 0 to 255.

2. IPv6

IPv6 is a 128-bit addressing system designed to provide a much larger address space.

Example:

`2001:db8::1`

3. Public IP

A public IP address is globally routable on the internet.

Your ISP typically assigns your home network a public IP address.

4. Private IP

Private IP addresses are used inside local networks and are not directly routed across the public internet.

Three private IPv4 ranges:

Range	Address range
Class A private	10.0.0.0 – 10.255.255.255
Class B private	172.16.0.0 – 172.31.255.255
Class C private	192.168.0.0 – 192.168.255.255

5. Static IP

An IP address that is manually configured or reserved to remain consistent.

6. Dynamic IP

An IP address assigned automatically, commonly through DHCP.

7. Subnetting

Subnetting means dividing a large IP network into smaller networks.

Example:

You can divide a large office network into separate networks for HR, finance and IT.

8. CIDR

CIDR stands for Classless Inter-Domain Routing.

It represents an IP network using a prefix length.

Example:

`192.168.1.0/24`

Here, `/24` means the first 24 bits identify the network portion.

9. Subnet Mask

A subnet mask identifies which portion of an IPv4 address belongs to the network and which portion belongs to the host.

Example:

`255.255.255.0`

This corresponds to `/24`.

10. Default Gateway

A default gateway is the device to which your computer sends traffic when the destination is outside its local network.

Usually, this is your router.

11. NAT

NAT stands for Network Address Translation.

It translates IP addresses between networks.

Example: Your home router allows multiple devices with private IP addresses to share one public IPv4 address using NAT, commonly PAT.

12. ARP

ARP stands for Address Resolution Protocol.

It helps an IPv4 device discover the MAC address associated with a local IPv4 address.

Example:

Your computer knows the router's IP address but needs its MAC address to send an Ethernet frame to it.

Cybersecurity importance: ARP spoofing is an attack in which an attacker sends forged ARP messages to manipulate IP-to-MAC address mappings.

Module 5: Network Protocols

A network protocol is a set of rules that determines how devices communicate and exchange information.

Different protocols perform different jobs.

Protocol	Full form / purpose	What it does
TCP	Transmission Control Protocol	Reliable data delivery
UDP	User Datagram Protocol	Fast, connectionless communication
HTTP	Hypertext Transfer Protocol	Transfers web data
HTTPS	HTTP Secure	Encrypted web communication
DNS	Domain Name System	Converts domain names into IP addresses
DHCP	Dynamic Host Configuration Protocol	Automatically assigns IP configurations
FTP	File Transfer Protocol	Transfers files
SFTP	SSH File Transfer Protocol	Secure file transfer over SSH

SSH	Secure Shell	Secure remote system access
SMTP	Simple Mail Transfer Protocol	Sends emails
IMAP	Internet Message Access Protocol	Retrieves and synchronizes emails
POP3	Post Office Protocol v3	Downloads emails from a mail server
ICMP	Internet Control Message Protocol	Network diagnostics and error reporting
TLS	Transport Layer Security	Encrypts communication
SNMP	Simple Network Management Protocol	Monitors and manages network devices
LDAP	Lightweight Directory Access Protocol	Accesses directory services
SMB	Server Message Block	Shares files and printers over networks

Important protocols explained

1. TCP

TCP provides reliable communication between devices.

It ensures:

- Data arrives in the correct order.
- Lost data can be retransmitted.
- Duplicate data is handled.

Example: Loading a website over HTTPS usually uses TCP, although HTTP/3 uses QUIC over UDP.

2. UDP

UDP sends data without establishing a connection or guaranteeing delivery.

Example: Online gaming, voice calls and live streaming often use UDP.

3. HTTP and HTTPS

HTTP allows browsers and web servers to exchange information.

HTTPS uses HTTP over a secure connection, generally protected by TLS.

4. DNS

DNS converts domain names into IP addresses.

Example:

```
google.com → IP address
```

Without DNS, users would generally need to remember IP addresses instead of domain names.

5. DHCP

DHCP automatically provides network configuration to devices.

It commonly assigns:

- IP address.
- Subnet mask.
- Default gateway.
- DNS server.

6. FTP and SFTP

Both transfer files, but they differ in security.

- FTP does not encrypt its standard communication.
- SFTP transfers files securely over SSH.

7. SSH

SSH allows you to securely access and control another computer remotely.

Example: Managing a Linux server from your laptop.

8. SMTP, IMAP and POP3

These protocols support email communication.

- SMTP: Sending emails.
- IMAP: Accessing and synchronizing emails.
- POP3: Retrieving emails, commonly by downloading them.

9. ICMP

ICMP helps devices report network errors and perform diagnostics.

Example: The `ping` command commonly uses ICMP Echo Request and Echo Reply messages.

10. TLS

TLS provides encryption, authentication and integrity protection for network communication.

Example: Protecting information exchanged with an HTTPS website.

11. SNMP

SNMP allows administrators to monitor network devices such as routers, switches and servers.

12. LDAP

LDAP provides access to directory information, such as user accounts and organizational information.

13. SMB

SMB allows computers to share files, folders and printers over a network.

Module 6: Network Security

Network security means protecting networks, devices and transmitted information against unauthorized access, attacks and misuse.

Guide to Network Security and Hardening
delinea.com

1. Packet Analysis

Packet analysis means examining network packets to understand communication between devices.

Example: Using Wireshark to inspect DNS requests and TCP connections.

Cybersecurity use: Identifying suspicious network activity.

2. Port Numbers

A port number identifies a specific communication endpoint associated with an application or service.

Some important ports:

Port	Protocol / service
20, 21	FTP
22	SSH / SFTP
23	Telnet
25	SMTP
53	DNS
67, 68	DHCP
80	HTTP
110	POP3
123	NTP
143	IMAP
161	SNMP
389	LDAP
443	HTTPS
445	SMB
3306	MySQL
3389	RDP

These are standard or commonly used ports; actual service configurations can differ.

3. Open and Closed Ports

- Open port: A service is listening and accepting connections on that port.
- Closed port: The destination is reachable but no service is listening on that port.
- Filtered port: A firewall or network device prevents the scanner from determining whether the port is open or closed.

Important: An open port is not automatically a vulnerability.

4. TCP Handshake

TCP uses a three-way handshake to establish a connection.

TCP 3-Way Handshake
blog.jargoniseasy.com

Three steps:

1. SYN: Client requests a connection.
2. SYN-ACK: Server acknowledges and responds.
3. ACK: Client confirms.

The connection is now established.

5. TCP Flags

TCP flags are control indicators inside TCP headers.

Important flags:

Flag	Meaning
SYN	Start connection
ACK	Acknowledge received data
FIN	Finish connection
RST	Reset connection
PSH	Request prompt delivery to application
URG	Indicates urgent data

You will encounter these flags frequently when analyzing packets in Wireshark and Nmap.

6. DNS Security

DNS security protects domain name resolution against manipulation and malicious activity.

Common threats:

- DNS spoofing.
- DNS cache poisoning.
- DNS hijacking.
- DNS tunneling.

7. VPN

VPN stands for Virtual Private Network.

It creates an encrypted tunnel between your device and a VPN endpoint.

Uses:

- Protecting traffic over untrusted networks.
- Remote access to private networks.
- Hiding your traffic from some local network observers.

A VPN does not make you completely anonymous.

8. Proxy

A proxy server acts as an intermediary between a client and another server.

Example: Your browser sends a request through a proxy rather than connecting directly to the destination.

9. Firewall Rules

Firewall rules define which network connections are allowed or blocked.

Example:

- Allow HTTPS traffic.
- Block unauthorized incoming connections.
- Allow SSH only from trusted IP addresses.

10. IDS and IPS

- IDS (Intrusion Detection System): Detects suspicious activities and generates alerts.
- IPS (Intrusion Prevention System): Detects and can automatically block malicious traffic.

11. Network Segmentation

Network segmentation divides a large network into smaller isolated sections.

Example: Separating employee computers, guest Wi-Fi and server networks.

This limits the spread of attacks.

12. VLAN

VLAN stands for Virtual Local Area Network.

It divides one physical switched network into multiple logical networks.

Example: Creating separate VLANs for students, teachers and administration.

13. Wireless Security

Wireless security protects Wi-Fi networks from unauthorized access and attacks.

Common threats:

- Unauthorized access.
- Rogue access points.
- Evil twin attacks.
- Wi-Fi password attacks.
- Packet interception.

14. WPA2 and WPA3

These are Wi-Fi security standards.

- WPA2: Uses strong encryption, commonly AES-CCMP.
- WPA3: Provides improved security, including stronger protection against password-guessing attacks through SAE in WPA3-Personal.

Prefer WPA3 when supported; otherwise, use properly configured WPA2-AES.

Module 7: Networking Tools

These tools are essential for cybersecurity professionals and ethical hackers.

Wireshark, Kali and nmap exercise
networkdatapedia.com

Tool	Purpose
Wireshark	Captures and analyzes network packets
Nmap	Discovers hosts and scans ports and services
tcpdump	Captures packets through the command line
Netcat	Reads and writes network connections
ping	Tests network connectivity
traceroute	Shows the network path to a destination
ipconfig	Displays network configuration on Windows
ip	Manages and displays Linux network configuration
nslookup	Queries DNS records

dig	Performs detailed DNS queries
curl	Sends requests to URLs and transfers data

Important tools explained

1. Wireshark

Wireshark is a network protocol analyzer.

It allows you to:

- Capture network packets.
- Inspect TCP communication.
- Identify DNS requests.
- Examine HTTP traffic.
- Investigate suspicious network activity.

2. Nmap

Nmap stands for Network Mapper.

It helps you:

- Discover devices.
- Identify open ports.
- Detect services.
- Identify service versions.
- Perform OS detection.

Example command:

```
nmap 192.168.1.1
```

Only scan systems you own or have explicit permission to test.

3. tcpdump

A command-line packet analyzer used to capture and inspect network traffic.

4. Netcat

A networking utility that can establish TCP or UDP connections and transfer data.

5. ping

Checks whether a destination responds to network diagnostic requests.

```
ping google.com
```

6. traceroute

Shows the network hops between your device and a destination.

Linux:

```
traceroute google.com
```

7. ipconfig and ip

These commands display and manage network configuration.

Windows:

```
ipconfig
```

Linux:

```
ip addr
```

8. nslookup and dig

Used to investigate DNS resolution.

```
nslookup google.com
```

```
dig google.com
```

9. curl

Used to communicate with web servers and inspect HTTP responses.

```
curl -I https://example.com
```

Practical Projects

Theory is not sufficient. You need to perform these five practical exercises.

Networking practical checklist

PROJECT PROGRESS

0/5 completed

1. Capture network packets using Wireshark

Capture traffic on your own network and identify different protocols.

2. Analyze TCP handshakes

Find SYN, SYN-ACK and ACK packets in Wireshark.

3. Identify DNS requests

Visit a website and inspect its DNS queries and responses.

4. Scan your own lab network using Nmap

Discover your own lab devices, open ports and running services.

5. Create a virtual network

Connect two virtual machines and test communication between them.

Recommended learning order

Module	Suggested time

Networking basics	2 days
OSI model	1 day
TCP/IP model	1 day
IP addressing and subnetting	3 days
Network protocols	2 days
Network security	3 days
Networking tools	3 days
Practical projects and revision	3 days
Total	18 days

Phase 3 — Linux and Windows Fundamentals

Essential for cybersecurity

Objective: Become comfortable managing, configuring, troubleshooting and securing Linux and Windows operating systems.

In cybersecurity, you must understand both environments because servers, personal computers, enterprise networks and cloud infrastructure use them extensively.

- Linux: Commonly used in servers, cloud infrastructure, security tools and penetration testing.
- Windows: Widely used in personal computers, business environments, enterprise networks and corporate infrastructure.

You should learn to administer both, not just use their graphical interfaces.

Part A — Linux Fundamentals

Home-Grown Red Team: Creating A Red Team Development Workstation | by assume-breach | Medium
assume-breach.medium.com

1. Linux Architecture

Linux architecture describes how different parts of the operating system work together.

Its major components are:

- Hardware: CPU, RAM, storage and other physical devices.
- Kernel: Manages hardware, memory, processes and devices.
- System libraries: Allow applications to interact with the kernel.
- Shell: Accepts commands from users.
- Utilities: Programs that perform different tasks.
- Applications: Software used by users.

Example: When you execute a Linux command, the shell interprets it and may request services from the kernel.

2. Linux Distributions

A Linux distribution (distro) is a complete operating system built around the Linux kernel with additional software and tools.

Popular distributions:

Ubuntu 22.04 LTS released with performance and security improvements

Ubuntu

Beginner-friendly general-purpose Linux distribution.

File:Kali 2020.4.png - Wikimedia Commons

Kali Linux

Security testing and penetration testing distribution.

Debial 13 "Trixie" Lands with RISC-V Support — But Prepares to Drop a Few Other Architectures - Hackster.io

Debian

Stable general-purpose Linux distribution.

Best Linux Distributions for AI and Machine Learning in 2026

Fedora

Modern Linux distribution with newer technologies.

3. Kali Linux

Kali Linux is a Debian-based Linux distribution designed for penetration testing, security auditing and digital forensics.

It includes tools such as:

- Nmap.
- Wireshark.
- Metasploit Framework.

- Burp Suite.
- Aircrack-ng.

Important: Kali Linux is not automatically more secure than other Linux distributions. It is a toolkit for security professionals.

4. Ubuntu

Ubuntu is a Debian-based Linux distribution used for desktops, servers, cloud infrastructure and software development.

It is useful for learning Linux administration.

5. Debian

Debian is a community-developed Linux distribution known for stability and its package management system.

Ubuntu and Kali Linux are both based on Debian.

6. Linux File System Hierarchy

Linux organizes files and directories in a hierarchical structure beginning with the root directory `/`.

Day 3: Understanding the Linux Filesystem | by Karthick Dk | devsecops-community | Medium
medium.com

Important directories:

Directory	Purpose
`/`	Root of the filesystem
`/home`	Regular users' personal directories
`/root`	Root user's home directory
`/etc`	System configuration files
`/bin`	Essential user commands
`/boot`	Boot-related files
`/dev`	Device files
`/var`	Logs and changing application data
`/tmp`	Temporary files
`/usr`	User applications and shared data
`/opt`	Optional application software

7. Terminal and Shell

- Terminal: An interface where you enter and view text commands.
- Shell: A program that interprets and executes commands.

Examples of shells:

- Bash.
- Zsh.
- Fish.

Example:

```
pwd
```

This command displays your current working directory.

8. File and Directory Management

Linux provides commands to create, delete, move, copy and inspect files and directories.

Command	Purpose
`ls`	List files
`cd`	Change directory
`pwd`	Show current directory
`cat`	Display file contents
`less`	Read files page by page
`mkdir`	Create directory
`touch`	Create an empty file
`cp`	Copy files
`mv`	Move or rename files
`rm`	Remove files
`find`	Search for files

Example:

```
mkdir cybersecurity
cd cybersecurity
touch notes.txt
ls
```

This creates a directory, enters it, creates a file and lists its contents.

9. File Permissions

Linux permissions determine who can read, modify or execute files.

Three permission types:

- `r` = Read.
- `w` = Write.
- `x` = Execute.

Three permission categories:

- `u` = Owner.
- `g` = Group.
- `o` = Others.

Example:

```
chmod 755 script.sh
```

This gives:

- Owner: Read, write and execute.
- Group: Read and execute.
- Others: Read and execute.

10. Users and Groups

Linux supports multiple users and groups.

A user represents an account, while a group allows multiple users to share permissions.

Important commands:

```
whoami
id
groups
```

- `whoami`: Shows the current username.
- `id`: Displays user and group IDs.
- `groups`: Displays group memberships.

11. Root Privileges

Root is the superuser account in Linux.

It has extensive administrative privileges over the system.

`sudo` allows authorized users to execute commands with elevated privileges.

Example:

```
sudo apt update
```

Security importance: Giving unnecessary root privileges increases the damage an attacker or compromised application can cause.

12. Package Management

Package management is the process of installing, updating and removing software.

Debian-based systems such as Ubuntu and Kali use APT.

Important commands:

```
sudo apt update
sudo apt upgrade
sudo apt install nmap
sudo apt remove nmap
```

- `update`: Refreshes package information.
- `upgrade`: Updates installed packages.
- `install`: Installs software.
- `remove`: Removes software.

13. Process Management

A process is a running instance of a program.

Linux assigns each process a Process ID (PID).

Important commands:

```
ps aux
top
kill PID
```

- `ps aux`: Displays running processes.
- `top`: Monitors processes and system resource usage.
- `kill`: Sends a signal to a process.

14. Service Management

A service is a background program that performs a specific function.

Examples: SSH server, web server and database server.

Linux commonly uses `systemd` to manage services.

```
systemctl status ssh
sudo systemctl start ssh
sudo systemctl stop ssh
```

```
sudo systemctl enable ssh
```

15. System Logs

Logs are records of events occurring inside an operating system.

They help administrators investigate errors, suspicious activities and security incidents.

Important commands:

```
journalctl
journalctl -b
journalctl -u ssh
```

Common log locations:

- `/var/log/auth.log`: Authentication events on many Debian-based systems.
- `/var/log/syslog`: General system messages on many Debian-based systems.
- `/var/log/`: Directory containing system and application logs.

16. Networking Commands

Linux includes commands for inspecting and troubleshooting networks.

Command	Purpose
`ip addr`	Display IP addresses
`ip route`	Display routing information
`ping`	Test connectivity
`ss`	Display socket information
`curl`	Send web requests
`ssh`	Connect to remote systems

Example:

```
ip addr
ss -tuln
```

17. SSH Configuration

SSH allows secure remote access to another computer.

The SSH server configuration is commonly stored in:

```
/etc/ssh/sshd_config
```

Important security settings include:

- Disable direct root login where appropriate.
- Use strong authentication.

- Prefer SSH keys over password authentication where practical.
- Restrict access to authorized users.

18. Cron Jobs

Cron is a Linux job scheduler that executes commands automatically at scheduled times.

Example:

```
crontab -e
```

A cron expression:

```
0 2 * * * command
```

Means execute the command every day at 2:00 AM.

19. Bash Fundamentals

Bash is a command-line shell and scripting language.

Bash scripting allows you to automate repetitive tasks.

Important concepts:

- Variables.
- Input and output.
- Conditions.
- Loops.
- Functions.
- Command-line arguments.

Example:

```
#!/bin/bash

echo "Welcome to Linux"
```

This prints a message in the terminal.

20. Linux Security and Hardening

Linux hardening means reducing security weaknesses and protecting the operating system against attacks.

Important practices:

1. Keep the operating system updated.
2. Use strong passwords.
3. Configure proper file permissions.
4. Avoid unnecessary root privileges.

5. Disable unused services.
6. Configure a firewall.
7. Secure SSH access.
8. Monitor system logs.
9. Remove unnecessary software.
10. Back up important data.

Part B — Windows Fundamentals

How to check device uptime on Windows 11 | Windows Central
windowscentral.com

Windows is an operating system developed by Microsoft. It is widely used in personal computers, businesses, corporate networks and enterprise environments.

In cybersecurity, Windows knowledge is particularly important because many organizations use Windows computers, Windows servers and Active Directory.

1. Windows Architecture

Windows architecture describes how its internal components work together.

Windows Kernel Exploitation Humla by Ashfaq Ansari – null - The Open Security Community – Official Blog for null community
blog.null.community

Main components:

- Hardware: CPU, RAM, storage and other physical devices.
- Kernel: Manages processes, memory, hardware and system operations.
- Executive: Provides important operating system services.
- Device drivers: Allow Windows to communicate with hardware.
- System services: Perform background operations.
- User applications: Programs such as browsers and editors.

2. Windows File System

A file system organizes and manages files and directories.

Windows commonly uses NTFS.

Important directories:

Directory	Purpose
`C:\Windows`	Windows operating system files
`C:\Users`	User accounts and personal data

`C:\Program Files`	Installed applications
`C:\Program Files (x86)`	Common location for 32-bit applications
`C:\ProgramData`	Application data shared across users
`C:\Temp`	May be used for temporary files

3. NTFS Permissions

NTFS permissions control which users can access files and folders.

Important permissions:

Permission	Meaning
Full Control	Complete access, including changing permissions
Modify	Read, write and delete
Read & Execute	Read and execute files
Read	View file contents
Write	Create or modify files

Cybersecurity importance: Incorrect permissions can allow unauthorized users to access sensitive information or modify important system files.

4. Users and Groups

Windows supports multiple user accounts and groups.

Important account types:

- Administrator: Can perform administrative operations.
- Standard User: Has restricted permissions.
- Guest: Limited access, where available and enabled.

Groups allow administrators to manage permissions for multiple users together.

Example: Adding employees to a particular group to provide access to shared folders.

5. Windows Registry

The Windows Registry is a hierarchical database that stores operating system and application configuration settings.

How to use the Windows Registry Editor
bleepingcomputer.com

Important registry root keys:

- `HKEY_LOCAL_MACHINE (HKLM)`
- `HKEY_CURRENT_USER (HKCU)`
- `HKEY_CLASSES_ROOT (HKCR)`
- `HKEY_USERS (HKU)`
- `HKEY_CURRENT_CONFIG (HKCC)`

Cybersecurity importance: Malware sometimes modifies registry settings to maintain persistence or change system behavior.

Be careful when editing the registry. Incorrect changes can damage Windows configuration.

6. Task Manager

Task Manager is a Windows utility used to monitor and manage running programs and system resources.

Particle: Microsoft Fixes Windows 11 Task Manager Quit Bug With KB5068861
particle.news

It helps you monitor:

- CPU usage.
- RAM usage.
- Disk activity.
- Network activity.
- Running processes.
- Startup applications.

Shortcut: `Ctrl + Shift + Esc`

7. Windows Services

Windows services are background programs that perform operating system and application tasks.

Examples:

- Windows Update.
- Windows Defender services.
- Print Spooler.
- Remote Procedure Call (RPC).

You can manage services through the Services application.

Run:

```
services.msc
```

8. Event Viewer

Event Viewer records events occurring inside Windows.

Windows Event Logging Explained: A SOC Analyst’s Guide to Log Analysis | by Lokesh Acharya | Dec, 2025 | Medium
medium.com

Important log categories:

- Application: Application-related events.
- Security: Login attempts, account activity and other audited security events.
- System: Operating system and driver events.
- Setup: Installation and setup events.

Run:

```
eventvwr.msc
```

Cybersecurity importance: Security professionals investigate Windows logs to identify suspicious logins, privilege changes and other security incidents.

9. Windows Defender

Microsoft Defender Antivirus is a built-in security solution that helps protect Windows against malware and other threats.

It provides:

- Real-time protection.
- Malware detection.
- Threat removal.
- Security scanning.
- Cloud-delivered protection.

It is an important part of Windows security, but it does not replace all other security controls.

10. Windows Firewall

Windows Defender Firewall controls incoming and outgoing network traffic according to configured rules.

Example:

- Allow trusted applications.
- Block unauthorized connections.
- Restrict access to particular ports.

Run:

`wf.msc`

11. PowerShell

PowerShell is a command-line shell and scripting language developed by Microsoft.

It is used for Windows administration, automation and security operations.

Examples:

`Get-Process`

Displays running processes.

`Get-Service`

Displays Windows services.

`Get-NetIPConfiguration`

Displays network configuration.

`Get-LocalUser`

Displays local user accounts.

PowerShell is particularly important for Windows cybersecurity and penetration testing.

12. Windows Networking

Windows provides commands to inspect and troubleshoot network connections.

Important commands:

Command	Purpose
`ipconfig`	Display IP configuration
`ipconfig /all`	Display detailed network information
`ping`	Test connectivity
`tracert`	Trace network route
`nslookup`	Query DNS
`netstat`	Display network connections
`arp -a`	Display ARP cache
`route print`	Display routing table

13. Group Policy

Group Policy allows administrators to manage and configure Windows computers and users centrally.

Examples:

- Enforce password policies.

- Restrict software installation.
- Configure security settings.
- Control access to system features.
- Manage Windows Defender settings.

Run:

```
gpedit.msc
```

Note: The Local Group Policy Editor is not available in every Windows edition.

14. Windows Authentication

Authentication means verifying the identity of a user or system.

Windows supports different authentication mechanisms.

Important concepts:

- Password authentication: Verifies users through passwords.
- NTLM: A Windows authentication protocol.
- Kerberos: A ticket-based network authentication protocol.
- Windows Hello: Supports biometric and PIN-based sign-in.
- Multi-factor authentication: Requires multiple verification factors.

Cybersecurity importance: Understanding authentication helps you understand password attacks, credential theft and identity security.

15. Windows Security Policies

Security policies define rules that control how Windows systems are protected.

Examples:

- Password complexity requirements.
- Account lockout policies.
- User rights assignments.
- Audit policies.
- Security restrictions.

You can inspect local security policies using:

```
secpol.msc
```

16. Windows Administration

Windows administration means managing and maintaining Windows computers and servers.

Common administrative tasks:

1. Create and manage user accounts.
2. Manage groups and permissions.
3. Install and update software.
4. Configure network settings.
5. Manage Windows services.
6. Monitor system performance.
7. Investigate system logs.
8. Configure firewalls.
9. Manage security policies.
10. Troubleshoot system problems.

Practical Projects

These projects will help you turn theory into administration skills.

Linux practical projects

LINUX PROGRESS

0/11

Install Ubuntu or Kali Linux in a virtual machine.
Create three Linux users.
Create a group and add users to it.
Configure different file permissions using chmod.
Install and remove software using APT.
Monitor processes using ps and top.
Start and stop services using systemctl.
Inspect authentication logs.
Configure and test SSH access.
Create a Bash script and a cron job.
Apply basic Linux security hardening.

Windows practical projects

WINDOWS PROGRESS

0/11

Create a Windows virtual machine.
Create multiple Windows user accounts.
Configure NTFS file permissions.
Explore Task Manager and Windows Services.
Investigate login events using Event Viewer.
Create Windows Firewall rules.
Practice basic PowerShell commands.

Explore Windows Registry.
Inspect Windows network configuration.
Configure basic Windows security policies.
Apply Windows security hardening.

Test yourself before Phase 4

Try answering these questions without looking at your notes.

1. Explain Linux architecture.
2. What is the difference between Kali Linux, Ubuntu and Debian?
3. What is the Linux root directory?
4. What is the difference between `chmod` and `chown`?
5. What is the difference between a normal user and root?
6. What is the purpose of `systemctl`?
7. What are Linux system logs?
8. What is Bash scripting?
9. What is Windows Registry?
10. What is NTFS?
11. What is the difference between Windows services and processes?
12. What is Event Viewer?
13. What is PowerShell?
14. What is the difference between NTLM and Kerberos?
15. What is Group Policy?

Phase 4 — Programming and Scripting Fundamentals

Essential for cybersecurity

Objective: Learn to write programs, automate security tasks, understand how software works internally, and identify programming-related vulnerabilities.

Programming is not optional if you want to become a strong cybersecurity professional.

You don't need to become an expert software developer before starting ethical hacking. However, you must understand code well enough to read it, write scripts, debug problems and identify insecure implementations.

1. Programming Languages — Learning Order

You have listed nine languages and technologies. You should not give all of them equal priority.

Language	Main purpose	Priority
Python	Automation and security scripts	Essential
Bash	Linux automation	Essential

PowerShell	Windows automation	Essential
SQL	Database security	Essential
JavaScript	Web security	Essential
HTML/CSS	Web fundamentals	Important
C	Memory and low-level security	Advanced
Assembly	Exploit development and reverse engineering	Advanced
Go	Security tooling	Optional

My recommendation: Start with Python, Bash and PowerShell. Then learn SQL, JavaScript and HTML. Learn C and Assembly when you reach low-level security and exploit development.

Go can wait.

2. Python Programming

[Python for Everyday Security Analysts: Simple Scripts that Save Time | by Raleigh Guevarra | Medium](medium.com)
medium.com

Python is a high-level programming language used extensively in cybersecurity because it is easy to read, has many libraries and supports automation.

Examples of cybersecurity applications:

- Automating repetitive tasks.
- Analyzing system logs.
- Processing network information.
- Interacting with APIs.
- Building security tools.
- Analyzing suspicious files.

Python syllabus

1. Variables and Data Types

Variables store information in a program.

Common data types:

- Integer: `10`
- Float: `10.5`
- String: `"Hello"`

- Boolean: `True` or `False`
- List: Collection of values.
- Dictionary: Key-value pairs.

2. Conditions

Conditions allow programs to make decisions.

Example: Checking whether a password meets a minimum length requirement.

3. Loops

Loops repeat instructions.

Two important types:

- `for` loop.
- `while` loop.

Example: Checking multiple files automatically.

4. Functions

Functions are reusable blocks of code that perform specific tasks.

Example: Creating a function that checks password strength.

5. Lists and Dictionaries

- List: Stores multiple values in an ordered collection.
- Dictionary: Stores information as key-value pairs.

Example: Storing usernames and their associated roles.

6. File Handling

File handling allows programs to create, read, write and modify files.

Cybersecurity use: Reading system logs and analyzing suspicious entries.

7. Exception Handling

Exception handling allows programs to manage errors without unexpectedly stopping.

Important keywords:

- `try`
- `except`
- `finally`
- `raise`

8. Modules and Packages

Modules allow you to reuse existing code.

Examples:

- `os`
- `sys`
- `json`
- `socket`
- `re`

9. Object-Oriented Programming (OOP)

OOP organizes programs using classes and objects.

Important concepts:

- Class.
- Object.
- Inheritance.
- Encapsulation.
- Polymorphism.
- Abstraction.

10. Regular Expressions

Regular expressions (Regex) identify patterns in text.

Example: Finding email addresses, IP addresses or suspicious strings in logs.

11. JSON

JSON is a data format used to exchange structured information.

Example: An API returning information about a user or security alert.

12. HTTP Requests

Python can send HTTP requests to web servers.

Common methods:

- GET.
- POST.
- PUT.
- DELETE.

Cybersecurity use: Testing web applications and interacting with APIs.

13. Socket Programming

Socket programming allows programs to communicate over a network.

You will learn:

- IP addresses.

- Ports.
- Client-server communication.
- TCP sockets.
- UDP sockets.

14. Multithreading

Multithreading allows a program to execute multiple threads concurrently.

Example: Processing multiple log files or network tasks efficiently.

15. API Interaction

An API allows different software applications to communicate.

Python can interact with APIs to retrieve information, submit requests and automate workflows.

16. Automation

Automation means using programs to perform repetitive tasks automatically.

Example: Automatically collecting system information and generating reports.

17. Logging

Logging records events that occur during program execution.

It helps with debugging, monitoring and security investigations.

3. Bash Scripting

[7 Practical Bash Script Examples for Automating Everyday Tasks
itslinuxfoss.com]

Bash is a command-line shell and scripting language commonly used in Linux.

It allows you to automate Linux administration and security tasks.

Bash syllabus

1. Shell basics.
2. Variables.
3. User input.
4. Conditions (`if`, `else`).
5. Loops (`for`, `while`).
6. Functions.
7. Command-line arguments.
8. File handling.
9. Pipes and redirection.

10. Environment variables.
11. Exit codes.
12. Process management.
13. Text processing using `grep`, `sed` and `awk`.
14. Cron automation.
15. Error handling.

Example:

```
#!/bin/bash

echo "Linux Security Learning"
```

This script displays a message in the terminal.

Cybersecurity applications:

- Automating log analysis.
- Collecting system information.
- Managing users.
- Checking file permissions.
- Automating system maintenance.

4. PowerShell

[10 PowerShell commands every sysadmin should know | PDQ
pdq.com]

PowerShell is Microsoft's command-line shell and scripting language.

It is especially useful for Windows administration and enterprise security.

PowerShell syllabus

1. PowerShell fundamentals.
2. Variables and data types.
3. Operators.
4. Conditions.
5. Loops.
6. Functions.
7. Arrays and hashtables.
8. Pipeline.
9. Objects and properties.

10. File handling.
11. Error handling.
12. Processes and services.
13. Windows Registry interaction.
14. Active Directory administration basics.
15. Event log analysis.
16. Automation and scripting.

Example:

```
Get-Process
```

Displays currently running processes.

Cybersecurity applications:

- Windows system monitoring.
- User and permission management.
- Event log analysis.
- Security automation.
- Incident response.

5. SQL

SQL Editor | Beekeeper Studio
beekeeperstudio.io

SQL stands for Structured Query Language.

It is used to communicate with relational databases.

SQL syllabus

1. Database fundamentals.
2. Tables, rows and columns.
3. Primary keys.
4. Foreign keys.
5. SELECT statements.
6. INSERT statements.
7. UPDATE statements.
8. DELETE statements.
9. WHERE conditions.

10. JOIN operations.
11. GROUP BY and ORDER BY.
12. Database permissions.
13. SQL injection.
14. Prepared statements.
15. Database security.

Cybersecurity importance:

SQL injection is a vulnerability that occurs when untrusted input is improperly included in database queries.

You must understand how insecure queries work and how parameterized queries prevent this vulnerability.

6. JavaScript

调试 JavaScript | Chrome DevTools | Chrome for Developers
developer.chrome.com

JavaScript is a programming language primarily used to make websites interactive.

It is extremely important for web application security.

JavaScript syllabus

1. Variables and data types.
2. Operators.
3. Conditions.
4. Loops.
5. Functions.
6. Arrays and objects.
7. DOM manipulation.
8. Events.
9. Asynchronous JavaScript.
10. Promises and async/await.
11. Fetch API.
12. HTTP requests.
13. Browser storage.
14. Cookies and sessions.
15. Same-Origin Policy.

16. Cross-Site Scripting (XSS).
17. Client-side security.

Cybersecurity applications:

- Understanding XSS vulnerabilities.
- Understanding how browsers execute JavaScript.
- Analyzing client-side authentication.
- Understanding cookies and sessions.
- Testing web application security.

7. HTML and CSS

Portfolio | Erin Bloxsidge
erinbloxsidge.com

HTML and CSS are not programming languages in the same sense as Python or JavaScript, but they are essential web technologies.

HTML

HTML stands for HyperText Markup Language.

It defines the structure and content of web pages.

Learn:

- HTML elements and tags.
- Headings and paragraphs.
- Forms.
- Input fields.
- Links.
- Tables.
- Attributes.
- HTML document structure.

Security importance: Understanding forms and input fields helps you understand how websites collect user information and handle input.

CSS

CSS stands for Cascading Style Sheets.

It controls the appearance and layout of web pages.

Learn:

- Selectors.

- Colors.
- Fonts.
- Margins and padding.
- Box model.
- Flexbox.
- Grid.
- Responsive design.

CSS has less direct importance in security testing than HTML and JavaScript, so don't spend excessive time mastering visual design.

8. C Programming

Setup Guide for C/C++ Programming on VSCode | by Usman Mehmood | Medium
medium.com

C is a low-level programming language that provides more direct control over memory and system resources than Python.

It is important for understanding how software interacts with computer memory.

C syllabus

1. Variables and data types.
2. Operators.
3. Conditions.
4. Loops.
5. Functions.
6. Arrays.
7. Strings.
8. Pointers.
9. Structures.
10. Dynamic memory allocation.
11. Memory management.
12. File handling.
13. Compilation.
14. Stack and heap.
15. Buffer overflow.
16. Memory corruption.

Cybersecurity applications:

- Understanding buffer overflow vulnerabilities.
- Understanding memory corruption.
- Studying operating system internals.
- Understanding exploit development.
- Analyzing low-level software vulnerabilities.

9. Assembly Language

GitHub - onegen-dev/VUT-FIT-ISU2022-cvicenia: Cvičení z předmětu Programování na strojové úrovni (ISU), druhý semestr bakalářského studia BIT na FIT VUT/BUT, ak.rok 2021/2022 · GitHub
github.com

Assembly language is a low-level programming language closely related to processor instructions.

It helps you understand what a computer actually executes.

Assembly syllabus

1. Binary and hexadecimal.
2. CPU registers.
3. Processor architecture.
4. Assembly instructions.
5. Memory addressing.
6. Stack operations.
7. Function calls.
8. Conditional jumps.
9. Loops.
10. System calls.
11. Machine code.
12. Disassembly.
13. Debugging.
14. x86 and x86-64 architecture.

Cybersecurity applications:

- Reverse engineering.
- Malware analysis.
- Binary analysis.
- Understanding exploit development.

- Analyzing executable files.

You don't need to master Assembly at this stage. Learn it when you reach reverse engineering and binary exploitation.

10. Go Programming

Close up of Go programming language source code displayed on dark screen with Hello World example in code editor. Software development, backend engineering, coding technology and developer workspace concept.

Go (Golang) is a compiled programming language developed by Google.

It is known for simplicity, performance and built-in concurrency support.

Go syllabus

1. Variables and data types.
2. Conditions and loops.
3. Functions.
4. Arrays and slices.
5. Maps.
6. Structures.
7. Interfaces.
8. Goroutines.
9. Channels.
10. Networking.
11. HTTP servers.
12. File handling.
13. Error handling.
14. Building command-line tools.

Cybersecurity applications:

- Building network scanners.
- Creating security utilities.
- Developing command-line tools.
- Writing network monitoring applications.

Go is optional. Don't learn it before mastering Python and the essential scripting skills.

11. Advanced Programming Concepts

These concepts connect programming with operating systems, networking and software vulnerabilities.

1. Memory Management

Memory management is the process of allocating, using and releasing computer memory.

You should understand:

- How programs use RAM.
- Memory allocation.
- Memory deallocation.
- Memory leaks.
- Memory corruption.

2. Stack and Heap

Stack and heap are two important memory regions used by programs.

Stack	Heap
Stores function call information and local variables in many implementations	Stores dynamically allocated memory
Usually managed automatically	Allocation and release depend on the language/runtime
Typically follows last-in, first-out behavior	Does not inherently follow LIFO
Often smaller	Often larger

3. Pointers

A pointer is a variable that stores a memory address.

Pointers are particularly important in C and low-level programming.

4. Buffer Overflow

A buffer overflow occurs when a program writes more data into a buffer than it can hold.

This can corrupt adjacent memory and potentially cause security vulnerabilities.

5. Data Structures

Data structures organize and store information efficiently.

Important examples:

- Arrays.

- Linked lists.
- Stacks.
- Queues.
- Hash tables.
- Trees.
- Graphs.

6. Algorithms

An algorithm is a step-by-step procedure for solving a problem.

Learn:

- Searching.
- Sorting.
- Recursion.
- Time complexity.
- Space complexity.

7. Operating System Interactions

Learn how programs interact with operating systems through system calls, processes, files, permissions and memory.

8. Networking Libraries

Learn how programming languages communicate over networks.

Python examples:

- `socket`
- `requests`
- `http.client`

9. Secure Coding

Secure coding means writing software in a way that prevents common security vulnerabilities.

Important concepts:

- Input validation.
- Output encoding.
- Authentication.
- Authorization.
- Secure password storage.

- Parameterized SQL queries.
- Secure session management.
- Error handling.
- Secrets management.

12. Practical Projects

You should build these six projects while learning programming. They will help you connect programming concepts with real cybersecurity tasks.

Project 1 — Log Analyzer

Python beginner

- Read Linux or Windows log files.
- Search for failed login attempts.
- Count suspicious events.
- Generate a summary report.

Skills: Python, file handling, loops, Regex.

Project 2 — Password Strength Checker

Python beginner

- Check password length.
- Check character variety.
- Identify common weak passwords.
- Provide improvement suggestions.

Skills: Conditions, functions, Regex, strings.

Project 3 — File Integrity Checker

Python intermediate

- Calculate file hashes using SHA-256.
- Store original file hashes.
- Compare current hashes against stored hashes.
- Detect unexpected file modifications.

Skills: Hashing, file handling, dictionaries, modules.

Project 4 — Network Information Collector

Python intermediate

- Collect local IP information.

- Display hostname.
- Display network interfaces.
- Display DNS configuration.
- Generate a report.

Skills: Python, networking libraries, operating system interaction.

Project 5 — Automated Security Report Generator

Python intermediate

- Collect system information.
- Check installed software.
- Identify running services.
- Collect relevant security events.
- Generate an HTML security report.

Skills: Python, automation, JSON, logging, HTML.

Project 6 — HTTP Security Header Analyzer

Python intermediate

- Accept a website URL.
- Send an HTTP request.
- Read response headers.
- Check for important security headers.
- Generate a security assessment.

Skills: HTTP, APIs, Python requests, web security.

13. Recommended Learning Schedule

Don't attempt to master all nine languages simultaneously. That is inefficient.

Learning stage	Topics	Suggested time
1	Python fundamentals	15 days
2	Python advanced concepts and projects	10 days
3	Bash scripting	7 days
4	PowerShell	7 days
5	SQL	7 days
6	HTML and CSS	4 days

7	JavaScript	12 days
8	C programming	15 days
9	Assembly fundamentals	7 days
10	Go fundamentals	5 days
11	Advanced programming concepts	10 days
	Total	99 days

These are suggested learning days, not guaranteed mastery times. You can learn Bash and PowerShell alongside your Linux and Windows administration practice.

14. What You Must Know Before Moving to Phase 5

Test yourself on these questions:

1. What is the difference between a compiler and an interpreter?
2. What are variables and data types?
3. What is the difference between a list and a dictionary in Python?
4. What is a function?
5. What is exception handling?
6. What is object-oriented programming?
7. What is Regex?
8. What is socket programming?
9. What is the difference between Bash and PowerShell?
10. What is SQL injection?
11. What is JavaScript DOM manipulation?
12. What is the difference between stack and heap?
13. What is a pointer?
14. What is a buffer overflow?
15. What is secure coding?

My recommendation: Learning priorities

Your priority order

1. Python: Become comfortable writing programs independently.

2. Bash and PowerShell: Automate Linux and Windows tasks.
3. SQL and JavaScript: Prepare for web application security.
4. HTML: Understand how web pages and forms work.
5. C: Understand memory, pointers and low-level vulnerabilities.
6. Assembly: Learn the fundamentals for reverse engineering.
7. Go: Learn later if you need to build advanced security tools.

PHASE 5 — CYBERSECURITY FUNDAMENTALS

Complete Study Notes | Beginner to Intermediate

Introduction to Cybersecurity Fundamentals

Cybersecurity is the practice of protecting computers, networks, applications, servers, and digital information from unauthorized access, attacks, damage, and theft.

Before learning advanced ethical hacking, penetration testing, or cloud security, you must understand the basic principles that make systems secure.

This phase covers five important areas:

1. Security principles
2. Cryptography
3. Authentication
4. Vulnerabilities and threats
5. Security frameworks

Learning objectives

After completing this phase, you should be able to:

- Explain the fundamental principles of information security.
- Understand how encryption, hashing, and digital signatures work.
- Understand how users and systems authenticate themselves.
- Identify common cyber threats and vulnerabilities.
- Understand the security frameworks used by organizations.
- Perform a basic security assessment of a fictional company.

MODULE 1 — SECURITY PRINCIPLES

1.1 What Are Security Principles?

Security principles are fundamental rules and guidelines used to protect information, computer systems, applications, and networks.

They help security professionals design, implement, and maintain secure systems.

For example, a company's employee should not be able to access confidential customer information unless their job requires it. This is an application of the principle of least privilege.

1.2 CIA Triad

The CIA Triad is one of the most important models in information security. It describes three main goals that every secure information system should maintain.

CIA stands for:

- C — Confidentiality
- I — Integrity
- A — Availability

Lesson 1: Cyber Security Fundamentals - Cyberseekh
cyberseekh.in

A. Confidentiality

Definition: Confidentiality means ensuring that information is accessible only to authorized people, systems, or processes.

Its purpose is to prevent unauthorized people from viewing or accessing sensitive information.

Example:

A bank stores customer details, account numbers, and transaction records. Only authorized employees and customers should be able to access the appropriate information.

Security methods:

- Encryption
- Password protection

- Access control
- Multi-factor authentication
- Data classification

Cybersecurity importance: Confidentiality protects sensitive information from data theft, unauthorized access, and information leakage.

B. Integrity

Definition: Integrity means ensuring that information remains accurate, complete, and unchanged unless an authorized modification is made.

It protects information from unauthorized modification, deletion, or corruption.

Example:

A student receives 85 marks in an examination. If someone changes the result to 95 without authorization, the integrity of the information has been violated.

Security methods:

- Cryptographic hashing
- Digital signatures
- File integrity monitoring
- Access permissions
- Audit logs

Cybersecurity importance: Integrity ensures that organizations can trust their data, software, transactions, and records.

C. Availability

Definition: Availability means ensuring that information, applications, and systems remain accessible to authorized users whenever they are needed.

Example:

An online banking application should remain accessible when customers want to transfer money. If attackers make the banking server unavailable, availability is affected.

Security methods:

- Regular backups
- Disaster recovery plans
- Redundant servers
- Load balancing
- DDoS protection
- System monitoring

Cybersecurity importance: Availability protects organizations from service interruptions, system failures, and attacks that prevent legitimate users from accessing services.

CIA Triad Comparison

Principle	Main purpose	Example of violation
Confidentiality	Protect information from unauthorized access	Customer data stolen
Integrity	Protect information from unauthorized changes	Bank balance modified
Availability	Keep systems accessible	Website taken offline

Important: A system is not fully secure just because it protects confidentiality. It must also maintain integrity and availability.

1.3 Authentication

Definition: Authentication is the process of verifying the identity of a user, device, or system.

It answers the question:

"Who are you?"

Example:

When you log in to your email account using your username and password, the email service checks whether the credentials belong to you.

Common authentication methods:

- Passwords
- PINs
- Security keys
- Fingerprints
- Facial recognition
- One-time passwords (OTP)

Cybersecurity importance: Authentication prevents unauthorized users from pretending to be legitimate users.

1.4 Authorization

Definition: Authorization is the process of deciding what an authenticated user is allowed to access or perform.

It answers the question:

"What are you allowed to do?"

Example:

In a company:

- An employee can view their own salary information.
- An HR manager can access employee salary records.
- A system administrator can manage server configurations.

Each user has different permissions.

Common authorization models:

- RBAC — Role-Based Access Control
- ABAC — Attribute-Based Access Control
- DAC — Discretionary Access Control
- MAC — Mandatory Access Control

Cybersecurity importance: Authorization prevents users from accessing resources or performing actions beyond their assigned permissions.

1.5 Accounting (Auditing)

Definition: Accounting is the process of recording and monitoring user activities, system events, and resource usage.

It helps organizations understand what happened, when it happened, and which account or system performed the action.

Example:

A company's server records:

- Who logged in.
- When a user accessed a file.
- Which administrator changed a system setting.
- Whether a login attempt failed.

These records are called logs or audit logs.

Common tools and technologies:

- SIEM systems
- System logs
- Audit logs
- Windows Event Viewer
- Linux journal logs

Cybersecurity importance: Accounting helps security teams investigate incidents, detect suspicious activities, and maintain accountability.

1.6 Non-Repudiation

Definition: Non-repudiation is a security principle that provides evidence that a particular action or transaction was performed by a specific party.

It makes it difficult for someone to deny an action they actually performed.

Example:

A company digitally signs an electronic contract. The digital signature can help verify the signer's identity and detect changes to the signed document.

Common methods:

- Digital signatures
- Secure audit logs
- Timestamping
- Digital certificates

Cybersecurity importance: Non-repudiation is important in digital contracts, financial transactions, legal records, and electronic communication.

1.7 Principle of Least Privilege

Definition: The principle of least privilege means giving users, applications, and systems only the minimum permissions necessary to perform their required tasks.

Example:

A normal employee does not need administrator access to install software, modify system configurations, or manage other employee accounts.

Therefore, the company should provide only the permissions required for that employee's job.

Benefits:

- Reduces unauthorized access.
- Limits damage caused by compromised accounts.
- Reduces privilege escalation opportunities.
- Protects sensitive information.

Cybersecurity importance: If an attacker compromises an ordinary employee account, limited permissions can prevent the attacker from immediately controlling the entire system.

1.8 Defense in Depth

Definition: Defense in depth is a security strategy that uses multiple layers of protection to defend systems, networks, and information.

Instead of depending on one security mechanism, an organization uses several independent or complementary security controls.

Seven Pillars of Cybersecurity Framework for Resilience | Sandeep Walia posted on the topic | LinkedIn
linkedin.com

Example:

A company protects its network using:

1. Firewall — controls network traffic.
2. Antivirus or endpoint protection — detects malicious software.
3. MFA — strengthens account security.
4. Encryption — protects sensitive information.
5. IDS/IPS — detects or blocks suspicious network activity.
6. Backups — help recover lost or encrypted data.
7. Monitoring — identifies suspicious activities.

Cybersecurity importance: If one security layer fails, other layers may still prevent or reduce the impact of an attack.

1.9 Zero Trust Security

Definition: Zero Trust is a security model based on the principle of "Never Trust, Always Verify."

It means that users, devices, and applications should not automatically be trusted just because they are inside an organization's network.

Every access request should be evaluated according to identity, permissions, device security, and other relevant conditions.

Example:

An employee working inside the company office wants to access a confidential cloud database.

Even though the employee is connected to the company's Wi-Fi, the system still checks:

- Is the employee's identity verified?
- Does the employee have permission?
- Is the device trusted and compliant?
- Is the requested access appropriate?

Core principles of Zero Trust:

- Verify explicitly.
- Use least privilege.
- Assume breach.
- Continuously evaluate access where appropriate.

Cybersecurity importance: Zero Trust helps protect organizations against compromised accounts, insider threats, stolen credentials, and unauthorized access.

1.10 Module 1 — Important Differences

Concept	Meaning
Authentication	Verifies identity
Authorization	Determines permissions
Accounting	Records activities
Confidentiality	Protects privacy of information
Integrity	Protects accuracy of information
Availability	Keeps systems accessible
Least privilege	Gives minimum required permissions
Defense in depth	Uses multiple security layers
Zero Trust	Verifies every access request

Module 1 — Practical Exercise

Task: Analyze a company login system.

Imagine a company has 50 employees and one internal database.

Answer these questions:

1. How will the company verify employee identities?
2. How will it prevent unauthorized employees from accessing confidential records?
3. How will it record employee activities?
4. How can it protect database information from unauthorized changes?
5. How can it keep the database available during server failure?
6. How can it reduce the damage if an employee account is compromised?

Expected learning: You should be able to apply the CIA Triad, AAA, least privilege, defense in depth, and Zero Trust to a real-world situation.

MODULE 2 — CRYPTOGRAPHY

2.1 What Is Cryptography?

Definition: Cryptography is the science of protecting information by using mathematical techniques to make data secure.

It helps protect information from unauthorized access, modification, and forgery.

Cryptography is used in:

- Online banking.
- Secure websites (HTTPS).
- Password storage.
- Digital payments.
- Secure messaging.
- Cloud storage.
- Digital certificates.
- VPN connections.

Example:

When you send a message through a secure messaging application, cryptography can help ensure that unauthorized people cannot read or modify the message while it travels across the network.

2.2 Encryption vs. Hashing

Encryption and hashing are two important cryptographic techniques, but they have different purposes.

A. Encryption

Definition: Encryption is the process of converting readable information (plaintext) into an unreadable format (ciphertext) using a cryptographic algorithm and a key.

The original information can be recovered through decryption using the appropriate key.

Example:

Original message:

```
HELLO
```

After encryption, it might look like:

```
X7#K9@P2
```

Only someone with the correct decryption key and algorithm can recover the original message.

Purpose: Protect the confidentiality of information.

B. Hashing

Definition: Hashing is the process of converting data of any size into a fixed-length value called a hash or digest using a hash function.

A secure cryptographic hash function is designed to be computationally infeasible to reverse.

Example:

Input:

```
Hello World
```

SHA-256 output:

```
a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e
```

Even a small change in the input normally produces a very different hash.

Purpose:

- Verify file integrity.
- Support digital signatures.
- Support secure password verification when used within a proper password-hashing scheme.

Encryption vs. Hashing Comparison

Feature	Encryption	Hashing
Purpose	Protect confidentiality	Produce a digest for verification
Reversible?	Yes, with the appropriate key	Designed to be one-way
Key required?	Yes, in standard encryption schemes	Not normally
Output	Ciphertext	Hash digest
Common use	Secure communication	Integrity checking

Important: Hashing is not encryption. A hash is not supposed to be decrypted.

2.3 Symmetric Encryption

Definition: Symmetric encryption is a cryptographic method in which the same secret key is used to encrypt and decrypt information.

Both communicating parties must have access to the secret key.

Integrating AES Encryption in C#: A Developer’s Guide | by Cemal Can Akgül | Medium
medium.com

Example:

Suppose Alice wants to send an encrypted file to Bob.

1. Alice encrypts the file using a secret key.
2. She sends the encrypted file to Bob.
3. Bob uses the same secret key to decrypt it.
4. Bob can now read the original file.

Advantages:

- Fast encryption and decryption.
- Suitable for large amounts of data.

- Efficient for storage and network communication.

Disadvantages:

- The secret key must be shared securely.
- If the key is stolen, an attacker may decrypt protected information.
- Managing keys becomes difficult when many users communicate.

Common Symmetric Encryption Algorithm

AES — Advanced Encryption Standard

AES is a widely used symmetric encryption algorithm.

It supports three key sizes:

AES version	Key size
AES-128	128 bits
AES-192	192 bits
AES-256	256 bits

AES is used in:

- File encryption.
- Database encryption.
- Disk encryption.
- VPN technologies.
- Secure network communication.

Important: AES is an encryption algorithm, not a hashing algorithm.

2.4 Asymmetric Encryption

Definition: Asymmetric cryptography is a cryptographic method that uses two mathematically related keys:

1. Public key — can be shared with other people.
2. Private key — must be kept secret by its owner.

The Cryptography Concept Journey — Asymmetric Encryption (Public Key Encryption) | by Shlomi Boutnaru, Ph.D. | Dec, 2025 | Medium
medium.com

Example:

Suppose Alice wants to send confidential information to Bob.

1. Bob shares his public key with Alice.
2. Alice encrypts the information using Bob's public key.

3. Alice sends the encrypted information to Bob.
4. Bob uses his private key to decrypt it.

This is a simplified example of public-key encryption.

Advantages:

- Public keys can be shared openly.
- It helps establish secure communication.
- It supports digital signatures.
- It can reduce the need to share a secret key beforehand.

Disadvantages:

- Generally slower than symmetric encryption.
- Requires secure private-key management.
- Public keys need trustworthy identity verification.

Common Asymmetric Cryptography Algorithms

A. RSA — Rivest–Shamir–Adleman

RSA is a public-key cryptographic algorithm based on the mathematical difficulty of factoring large integers.

It can be used for:

- Digital signatures.
- Public-key encryption.
- Certain key-establishment or key-transport systems.

RSA is commonly used with carefully selected key sizes and secure padding schemes.

B. ECC — Elliptic Curve Cryptography

ECC is a family of public-key cryptographic techniques based on mathematical operations involving elliptic curves.

It provides strong security with comparatively smaller keys than traditional RSA systems.

It is used in:

- Digital signatures.
- Secure communication.
- Mobile devices.
- TLS connections.
- Cryptographic key exchange.

RSA vs. ECC

Feature	RSA	ECC
Type	Asymmetric cryptography	Asymmetric cryptography
Mathematical basis	Integer factorization	Elliptic-curve mathematics
Key size	Generally larger	Generally smaller for comparable security
Common uses	Signatures, encryption	Signatures, key agreement
Efficiency	Depends on operation and key size	Often efficient with smaller keys

2.5 SHA-256

Definition: SHA-256 is a cryptographic hash function from the SHA-2 family that generates a fixed-length hash digest of 256 bits.

A 256-bit digest contains 64 hexadecimal characters.

Example:

Input:

```
Hello
```

SHA-256 produces a 256-bit digest.

If even one character changes, the resulting digest will normally look completely different.

Important properties:

1. Fixed output length — produces 256 bits regardless of input size.
2. One-way design — should be infeasible to recover the original input from the digest.
3. Collision resistance — it should be computationally difficult to find two different inputs with the same digest.
4. Avalanche effect — a small input change should cause a large change in the output.

Uses of SHA-256:

- File integrity verification.
- Digital signature systems.
- Security protocols.
- Data verification.
- Cryptographic applications.

Security warning: SHA-256 alone is not suitable for storing user passwords. Passwords should be stored using dedicated password-hashing algorithms such as Argon2id, scrypt, or bcrypt, with unique salts.

2.6 Salting

Definition: Salting is the process of adding a unique, randomly generated value to a password before processing it with a password-hashing algorithm.

The salt is stored alongside the resulting password hash.

Why is salting necessary?

Suppose two employees use the same password:

- Employee A: `password123`
- Employee B: `password123`

Without unique salts, a basic password-hashing system could produce identical stored hashes for both users.

With different salts, the stored password hashes will be different.

Benefits of salting:

- Makes precomputed rainbow-table attacks less useful.
- Prevents identical passwords from automatically producing identical stored hashes.
- Makes attackers perform separate guessing work for each salted password.

Important: A salt does not need to be secret. It should be unique and randomly generated. Salting does not make a weak password strong by itself.

2.7 Digital Signatures

Definition: A digital signature is a cryptographic mechanism that helps verify the authenticity and integrity of digital information.

A sender uses their private key to create a signature. A recipient uses the corresponding public key to verify it.

How it works:

1. The sender prepares a document.
2. A cryptographic hash of the document is calculated.
3. The sender's private key is used by a digital-signature algorithm to create a signature.
4. The document and signature are sent to the recipient.
5. The recipient verifies the signature using the sender's public key.

Example:

A company digitally signs an electronic contract. The recipient can verify that the signature matches the document and the public key associated with the signer.

Main benefits:

- Authenticity — helps verify the signer.

- Integrity — helps detect changes to signed data.
- Non-repudiation support — can provide evidence of signing, depending on key control, identity verification, and the surrounding process.

Important: A digital signature does not automatically encrypt a document. It verifies a signature; encryption is a separate process.

2.8 Digital Certificates

Definition: A digital certificate is an electronic document that connects an identity, such as a website domain or organization, with a public key.

It is generally issued and digitally signed by a trusted Certificate Authority (CA).

Example:

When you visit a secure website such as an online banking website, your browser checks its TLS certificate to help verify the website's identity.

A certificate commonly contains:

- Subject identity or domain name.
- Public key.
- Certificate issuer.
- Validity period.
- Serial number.
- Digital signature of the issuing CA.

Purpose of digital certificates:

- Verify identities.
- Support secure communication.
- Establish trust in public keys.
- Help prevent impersonation.

2.9 PKI — Public Key Infrastructure

Definition: Public Key Infrastructure (PKI) is a system of policies, procedures, people, and technologies used to create, issue, manage, validate, and revoke digital certificates and public keys.

PKI helps organizations establish trust between users, devices, applications, and services.

Main components of PKI:

1. Certificate Authority (CA): Issues and digitally signs certificates.
2. Registration Authority (RA): Helps verify certificate applicants before issuance.
3. Digital certificates: Connect identities with public keys.
4. Public and private keys: Used in public-key cryptography.

5. Certificate revocation: Allows certificates to be invalidated before their normal expiration.
6. Certificate repository: Stores certificates and related information.

Example:

A company uses PKI to issue certificates to employee laptops. These certificates help authenticate company devices when they connect to internal services.

2.10 TLS — Transport Layer Security

Definition: TLS is a cryptographic protocol used to protect communication between two systems over a network.

It provides:

- Confidentiality through encryption.
- Integrity protection.
- Authentication, commonly of the server through a digital certificate.

TLS is used by HTTPS, secure email systems, and many other network services.

Example:

When you open an HTTPS website:

1. Your browser connects to the web server.
2. The browser and server perform a TLS handshake.
3. The server presents its certificate.
4. The browser validates the certificate and the server's identity.
5. The parties establish shared session keys.
6. Communication is protected using the negotiated cryptographic algorithms.

SSL/TLS & Encryption: Symmetric vs Asymmetric keys, Certificate Authorities, Mutual TLS (mTLS) - DEV Community
dev.to

HTTP vs. HTTPS

HTTP	HTTPS
Does not itself provide TLS protection	HTTP communication protected by TLS
Data may be exposed or modified in transit	TLS protects data in transit
No TLS certificate validation	Uses TLS certificate validation
Usually uses port 80	Usually uses port 443

Important: HTTPS does not guarantee that a website is honest or free from malware. It protects the connection, not the trustworthiness of everything on the website.

2.11 Key Exchange

Definition: Key exchange is a cryptographic process that allows two parties to establish shared secret key material over a communication channel.

It is especially important when two systems need to communicate securely without sending a pre-shared secret key directly across the network.

Common key exchange technique: Diffie–Hellman

Diffie–Hellman is a key-agreement method that allows two parties to establish shared secret material even when communication can be observed by an attacker.

Its elliptic-curve form is called Elliptic-Curve Diffie–Hellman (ECDH).

Example:

1. Alice and Bob agree on public parameters.
2. Alice creates private key material.
3. Bob creates private key material.
4. They exchange corresponding public values.
5. Each calculates the same shared secret using their private value and the other party's public value.

An attacker who only observes the exchanged public values should not be able to feasibly calculate the shared secret.

Important: Key exchange alone does not necessarily authenticate the parties. Without authentication, an attacker may attempt a man-in-the-middle attack. Secure protocols such as TLS combine key establishment with authentication mechanisms.

2.12 Symmetric vs. Asymmetric Cryptography

Feature	Symmetric	Asymmetric
Keys	Same secret key	Public/private key pair
Speed	Generally faster	Generally slower
Key sharing	Secret key must be shared securely	Public key can be shared openly
Main uses	Bulk data encryption	Authentication, signatures, key establishment
Examples	AES	RSA, ECC

2.13 Module 2 — Practical Exercises

Exercise 1: Understand hashing

- Choose a sample text.
- Calculate its SHA-256 hash using a trusted local tool.

- Change one character.
- Calculate the hash again.
- Compare both outputs.

Exercise 2: Understand file integrity

- Create a sample text file.
- Calculate its SHA-256 hash.
- Modify the file.
- Calculate the hash again.
- Observe how the digest changes.

Exercise 3: Understand HTTPS

- Open a website using HTTPS.
- Inspect its certificate using your browser's security information.
- Identify the domain, issuer, and validity period.
- Understand why the certificate matters.

Exercise 4: Understand password storage

Research how Argon2id, bcrypt, and scrypt protect stored passwords. Explain why ordinary SHA-256 is not recommended for password storage.

Exercise 5: Understand key pairs

Study how a public key and private key are used in encryption and digital signatures. Clearly distinguish the two operations.

Module 2 — Key Takeaways

- Encryption protects confidentiality.
- Hashing produces a one-way digest.
- AES is symmetric encryption.
- RSA and ECC are asymmetric cryptographic techniques.
- SHA-256 produces a 256-bit hash.
- Salting helps protect password hashes against precomputed attacks.
- Digital signatures help verify authenticity and integrity.
- Certificates connect identities with public keys.
- PKI manages trust in certificates and keys.
- TLS protects network communication.
- Key exchange establishes shared cryptographic key material.

MODULE 3 — AUTHENTICATION

3.1 What Is Authentication?

Definition: Authentication is the process of verifying the identity of a person, device, application, or system before allowing it to access a resource.

Authentication is one of the first security checks performed when someone attempts to access a protected system.

Example:

When you log in to your university portal, the portal checks your username and password to verify your identity.

Authentication can use different types of evidence:

1. Something you know — password or PIN.
2. Something you have — security key, smartphone, or authentication token.
3. Something you are — fingerprint or facial recognition.
4. Somewhere you are — location information, when used as an additional security signal.
5. Something you do — behavioral characteristics, such as typing patterns.

The first three are the most commonly discussed authentication factor categories.

3.2 Password Authentication

Definition: Password authentication is a method of verifying a user's identity by checking a secret password associated with their account.

Example:

A user enters:

- Username: `student01`
- Password: Their secret password.

The application verifies the credentials before granting access.

Common password-related security problems:

- Weak passwords.
- Reused passwords.
- Stolen passwords.
- Password guessing.
- Phishing.
- Credential stuffing.
- Brute-force attacks.

Security best practices:

- Use long, unique passwords.
- Avoid predictable passwords.
- Use a password manager.
- Enable MFA.
- Never share passwords.
- Store passwords using secure password-hashing algorithms.

Cybersecurity importance: Passwords are a common target because attackers can use stolen or guessed credentials to access systems without exploiting a software vulnerability.

3.3 Two-Factor Authentication (2FA)

Definition: Two-Factor Authentication is a security method that requires two different authentication factors to verify a user's identity.

Example:

A banking application requires:

1. Password — something you know.
2. OTP from an authenticator application — something you have.

Even if an attacker steals the password, they may not be able to access the account without the second factor.

Common 2FA methods:

- Password + OTP.
- Password + security key.
- PIN + fingerprint.
- Password + hardware authentication device.

Benefits:

- Makes stolen passwords less useful.
- Reduces unauthorized account access.
- Adds another layer of security.

Limitation: Some methods, especially SMS-based OTP, can be vulnerable to phishing, SIM swapping, or interception.

3.4 Multi-Factor Authentication (MFA)

Definition: Multi-Factor Authentication is an authentication method that requires two or more independent authentication factors from different categories.

Example:

An employee logs in using:

1. Password — something they know.
2. Hardware security key — something they have.
3. Fingerprint — something they are.

This is an example of three-factor authentication.

Benefits:

- Strengthens account security.
- Reduces the impact of stolen passwords.
- Helps protect sensitive applications and systems.

Best practice: Phishing-resistant MFA, such as appropriately configured FIDO2 security keys or passkeys, is generally stronger against phishing than ordinary passwords and OTP codes.

2FA vs. MFA

Feature	2FA	MFA
Meaning	Two-factor authentication	Multi-factor authentication
Number of factors	Exactly two	Two or more
Example	Password + security key	Password + key + fingerprint
Relationship	A type of MFA	General category

Remember: Every properly implemented 2FA system is MFA, but not every MFA system uses exactly two factors.

3.5 Biometrics Authentication

Definition: Biometric authentication is a method of verifying identity using a person's physical or behavioral characteristics.

Types of biometrics:

A. Physical biometrics

- Fingerprint recognition.
- Facial recognition.
- Iris recognition.
- Retina recognition.

B. Behavioral biometrics

- Typing patterns.
- Voice characteristics.
- Mouse movement patterns.

- Walking patterns.

Example:

A smartphone allows its owner to unlock the device using fingerprint recognition.

Advantages:

- Convenient.
- Difficult to copy casually.
- Does not require remembering a password.

Disadvantages:

- Biometric systems can make mistakes.
- Biometric information is sensitive.
- A fingerprint or face cannot be changed as easily as a password.
- Presentation attacks and spoofing may be possible if the system is poorly designed.

Cybersecurity importance: Biometrics can strengthen authentication, but biometric systems still require secure storage, privacy protection, and additional safeguards.

3.6 Single Sign-On (SSO)

Definition: Single Sign-On is an authentication system that allows users to access multiple connected applications using one authentication session.

Instead of signing in separately to every application, users authenticate through a central identity provider.

Example:

A university student signs in once using the university account and then accesses:

- Student portal.
- Online learning platform.
- University email.
- Digital library.

How SSO works:

1. The user opens an application.
2. The application redirects the user to an identity provider.
3. The user authenticates.
4. The identity provider issues an authentication response or token.
5. The application validates the response.
6. The user receives access according to the application's permissions.

Advantages:

- Fewer passwords to manage.
- Convenient access.
- Centralized identity management.
- Easier account administration.

Disadvantages:

- A compromised central account can affect multiple applications.
- The identity provider becomes an important security dependency.
- Poor session management can create security problems.

3.7 OAuth 2.0

Definition: OAuth 2.0 is an authorization framework that allows an application to obtain limited access to a user's resources on another service without requiring the user to share their password with that application.

Example:

Suppose a photo-printing application wants permission to access selected photos from a cloud storage account.

Instead of asking for the user's cloud password, it can request an access token through an OAuth authorization process.

The user can approve or deny the requested permissions.

Important OAuth components:

1. Resource Owner — the user who owns the protected data.
2. Client — the application requesting access.
3. Authorization Server — issues access tokens after the required authorization process.
4. Resource Server — hosts the protected resources.
5. Access Token — represents granted access.
6. Scope — describes the permissions being requested or granted.

Common OAuth concepts:

- Authorization Code Flow.
- Access tokens.
- Refresh tokens.
- Scopes.
- Redirect URIs.
- Client authentication.

Cybersecurity importance: OAuth 2.0 is widely used in modern applications and APIs to provide controlled access to resources.

Important: OAuth 2.0 is primarily about authorization. It is not, by itself, a complete user authentication or login protocol.

3.8 OpenID Connect (OIDC)

Definition: OpenID Connect is an identity and authentication layer built on top of OAuth 2.0.

It allows applications to verify a user's identity and obtain standardized information about that user.

Example:

When a website offers a "Sign in with Google" option, it can use OpenID Connect to authenticate the user.

Important OIDC components:

- Identity Provider (IdP).
- ID Token.
- Access Token.
- User information endpoint.
- Authentication request.

ID Token: A security token containing claims about the authenticated user and the authentication event.

OIDC vs. OAuth 2.0

OAuth 2.0	OpenID Connect
Authorization framework	Authentication and identity layer
Grants access to resources	Verifies user identity
Uses access tokens	Uses ID tokens and may also use access tokens
Commonly used for delegated API access	Commonly used for application login

3.9 SAML — Security Assertion Markup Language

Definition: SAML is an XML-based standard used to exchange authentication and authorization-related information between an identity provider and a service provider.

It is commonly used for enterprise Single Sign-On.

Example:

An employee wants to access a company's internal HR application.

1. The employee opens the HR application.
2. The application redirects the employee to the company's identity provider.

3. The employee authenticates.
4. The identity provider sends a signed SAML assertion.
5. The HR application validates the assertion.
6. The employee receives access.

Important SAML terms:

- Identity Provider (IdP): Authenticates users.
- Service Provider (SP): Provides the application or service.
- SAML Assertion: Contains authentication or attribute information.
- SAML Response: Carries the assertion and related information.

Advantages:

- Supports enterprise SSO.
- Centralizes authentication.
- Enables identity federation between organizations and applications.

3.10 Kerberos

Definition: Kerberos is a network authentication protocol that uses tickets and secret-key cryptography to authenticate users and services over an insecure network.

It is commonly used in Microsoft Active Directory environments.

Main components:

1. Client — the user or system requesting access.
2. Key Distribution Center (KDC) — trusted service that manages authentication tickets.
3. Authentication Server (AS) — verifies the user's initial authentication and issues a Ticket-Granting Ticket.
4. Ticket-Granting Server (TGS) — issues service tickets.
5. Service Server — the system the user wants to access.

How Kerberos works — simplified:

1. The user attempts to log in.
2. The client communicates with the Authentication Server.
3. The user receives a Ticket-Granting Ticket (TGT).
4. The client requests a service ticket from the Ticket-Granting Server.
5. The client presents the service ticket to the target service.
6. The service verifies the ticket and grants access.

Understanding Kerberos: A Cybersecurity Protocol | Shreehari H A posted on the topic | LinkedIn

linkedin.com

Advantages:

- Uses ticket-based authentication.
- Passwords are not repeatedly sent to each service.
- Supports mutual authentication in appropriate configurations.
- Useful in centralized enterprise environments.

Cybersecurity importance: Understanding Kerberos is particularly important when studying Windows domain security, Active Directory, and enterprise penetration testing.

3.11 Authentication Technologies — Comparison

Technology	Main purpose	Common use
Password	Verify identity using a secret	Website login
2FA	Verify identity using two factors	Banking
MFA	Verify identity using multiple factors	Corporate systems
Biometrics	Verify physical or behavioral characteristics	Smartphone login
SSO	One authentication session for multiple applications	Enterprise applications
OAuth 2.0	Delegated authorization	API access
OIDC	Authentication and identity	Social login
SAML	Exchange identity assertions	Enterprise SSO
Kerberos	Ticket-based network authentication	Active Directory

3.12 Module 3 — Practical Exercises

Exercise 1: Compare authentication factors

Identify the authentication factors used in these situations:

- Logging in using only a password.
- Unlocking a phone using a fingerprint.
- Using a password and hardware security key.
- Using a PIN and facial recognition.

Exercise 2: Understand SSO

Research how a university or company uses one account to access multiple services. Identify its identity provider and service providers.

Exercise 3: Understand OAuth 2.0

Study the authorization process behind an application requesting permission to access a cloud account. Identify the client, resource owner, authorization server, and access token.

Exercise 4: Understand OIDC and SAML

Compare their purposes, token or assertion formats, and common use cases.

Exercise 5: Understand Kerberos

Draw the authentication flow showing the client, KDC, TGT, service ticket, and target service.

Module 3 — Key Takeaways

- Authentication verifies identity.
- Passwords are a common but often weak authentication method.
- 2FA uses exactly two factors.
- MFA uses two or more independent factors.
- Biometrics use physical or behavioral characteristics.
- SSO provides access to multiple applications through one authentication session.
- OAuth 2.0 provides delegated authorization.
- OIDC adds authentication and identity to OAuth 2.0.
- SAML is commonly used for enterprise SSO.
- Kerberos uses tickets for network authentication.

MODULE 4 — VULNERABILITIES AND THREATS

4.1 Understanding Threats, Vulnerabilities, and Attacks

Before learning individual cyberattacks, understand three important terms.

A. Threat

Definition: A threat is any potential event, action, or circumstance that could cause harm to a computer system, network, application, or information.

Example: A cybercriminal attempting to steal customer information is a security threat.

B. Vulnerability

Definition: A vulnerability is a weakness in software, hardware, configuration, security controls, or processes that could be exploited.

Example: A web application that does not properly validate user input may contain a vulnerability.

C. Attack

Definition: A cyberattack is an intentional attempt to exploit a weakness, gain unauthorized access, steal information, disrupt services, or cause damage.

Example: An attacker exploits a vulnerable website to access its database without permission.

Threat vs. Vulnerability vs. Attack

Term	Meaning	Example
Threat	Potential source of harm	Cybercriminal
Vulnerability	Weakness that can be exploited	Unpatched software
Attack	Attempt to cause harm	Exploiting the software weakness

4.2 Malware

Definition: Malware is malicious software designed to damage systems, steal information, disrupt operations, spy on users, or gain unauthorized access.

The word malware comes from "malicious software."

Common malware categories:

- Ransomware.
- Spyware.
- Trojan.
- Worm.
- Rootkit.
- Virus.
- Keylogger.
- Bot malware.

Common ways malware spreads:

- Malicious email attachments.
- Fake software downloads.
- Compromised websites.
- Infected removable devices.
- Exploitation of software vulnerabilities.
- Malicious links.
- Supply-chain compromises.

Cybersecurity importance: Security professionals must understand malware behavior to prevent infections, investigate incidents, and protect systems.

4.3 Ransomware

Definition: Ransomware is a type of malware that prevents users from accessing their data or systems, often by encrypting files, and demands payment for restoring access.

How ransomware attacks commonly work:

1. An attacker gains access to a system.
2. Malicious software is executed.
3. The malware may spread to other systems.
4. Important files are encrypted or access is blocked.
5. The attacker displays a ransom demand.

Some ransomware operations also steal data and threaten to publish it.

Example:

A company's file server becomes infected with ransomware. Employees can no longer access important documents, and the attackers demand payment.

Prevention methods:

- Maintain offline or immutable backups.
- Keep software updated.
- Use endpoint protection.
- Apply least privilege.
- Enable MFA.
- Restrict unnecessary network access.
- Train employees to recognize phishing.

Important: Paying a ransom does not guarantee that the attacker will restore the files or delete stolen data.

4.4 Spyware

Definition: Spyware is malicious software that secretly monitors user activities and collects information without proper authorization.

It may collect:

- Browsing history.
- Login credentials.
- Personal information.
- System information.
- Keystrokes.

- Other sensitive data.

Example:

A user installs a fake application. The application secretly monitors the user's activities and sends collected information to an attacker.

Prevention methods:

- Install software from trusted sources.
- Keep operating systems updated.
- Use endpoint security tools.
- Review application permissions.
- Avoid suspicious links and downloads.

4.5 Trojan

Definition: A Trojan, or Trojan horse, is malware disguised as legitimate or useful software to trick users into installing or executing it.

Unlike a worm, a Trojan does not inherently self-replicate and spread automatically.

Example:

An attacker creates a fake software installer that appears to be a legitimate application. When a user installs it, hidden malicious code runs.

Common Trojan purposes:

- Stealing credentials.
- Downloading additional malware.
- Providing unauthorized remote access.
- Spying on users.
- Destroying or modifying information.

Prevention methods:

- Download applications from trusted sources.
- Verify software publishers and signatures.
- Use endpoint protection.
- Avoid suspicious attachments.

4.6 Worm

Definition: A worm is a type of malware that can self-replicate and spread between computers or systems, often through networks, without requiring a user to manually copy it each time.

Example:

A network worm exploits a software vulnerability in one computer and spreads to other vulnerable computers on the same network.

Characteristics:

- Self-replication.
- Automatic spreading.
- Network propagation is common.
- Can consume system and network resources.
- May carry additional malicious payloads.

Prevention methods:

- Patch known vulnerabilities.
- Use firewalls.
- Segment networks.
- Monitor network traffic.
- Disable unnecessary services.

4.7 Rootkit

Definition: A rootkit is a collection of malicious techniques or software designed to hide unauthorized activity and maintain privileged access to a compromised system.

Rootkits may operate at different levels of a system.

Common types:

1. User-mode rootkit — operates at the application or user level.
2. Kernel-mode rootkit — operates within or modifies kernel-level behavior.
3. Bootkit — targets the boot process.
4. Firmware rootkit — targets device firmware.

Example:

An attacker compromises a server and installs a rootkit to hide malicious processes or files from normal system monitoring tools.

Prevention and detection:

- Secure boot where appropriate.
- Regular system updates.
- Endpoint detection tools.
- Integrity monitoring.
- Trusted recovery and reinstallation procedures.

4.8 Phishing

Definition: Phishing is a social engineering attack in which an attacker uses fake messages, websites, or communications to trick people into revealing sensitive information or performing unsafe actions.

Example:

A user receives an email that appears to come from their bank. The message claims that the account will be blocked and asks the user to log in through a fake website.

If the user enters their credentials, the attacker may steal them.

Common types of phishing:

- Email phishing — fraudulent emails.
- Spear phishing — targeted messages aimed at a specific person or organization.
- Whaling — targeted phishing against senior executives or other high-value individuals.
- Smishing — phishing through SMS messages.
- Vishing — phishing through voice calls.
- Clone phishing — a fake copy of a legitimate message containing a malicious replacement link or attachment.

Warning signs:

- Unexpected requests for passwords.
- Urgent threats or pressure.
- Suspicious links.
- Unexpected attachments.
- Unusual sender addresses.
- Requests to bypass normal company procedures.

Prevention methods:

- Verify the sender through another trusted channel.
- Inspect website domains carefully.
- Avoid unexpected attachments.
- Use phishing-resistant MFA.
- Report suspicious messages.

4.9 Social Engineering

Definition: Social engineering is the psychological manipulation of people to make them reveal confidential information, provide unauthorized access, or perform actions that compromise security.

Instead of directly attacking software, attackers exploit human behavior.

Common techniques:

1. Pretexting — creating a false story or identity to gain trust.
2. Baiting — offering something attractive to encourage unsafe behavior.
3. Quid pro quo — offering a service or benefit in exchange for information or access.
4. Tailgating — following an authorized person into a restricted physical area.
5. Impersonation — pretending to be a trusted person or organization.
6. Scareware — using frightening security warnings to trick users into taking unsafe actions.

Example:

An attacker calls an employee pretending to be an IT support technician and asks for the employee's password.

Prevention methods:

- Employee security awareness training.
- Identity verification procedures.
- Clear reporting processes.
- Physical access controls.
- Never sharing passwords.

Important distinction: Phishing is one type of social engineering. Social engineering includes many other techniques beyond phishing.

4.10 Brute-Force Attack

Definition: A brute-force attack is a password or credential attack in which an attacker repeatedly tries different possible combinations until a valid credential is found or the attempt is stopped.

Example:

An attacker attempts to guess a weak account password by trying many possible combinations.

Common types:

- Simple brute force.
- Dictionary attack.
- Credential stuffing.
- Hybrid password guessing.

Prevention methods:

- Strong, unique passwords.
- MFA.
- Login rate limiting.

- Account lockout or progressive delays, designed carefully.
- Detection of unusual login attempts.
- Secure password storage.

Ethical hacking note: Password-guessing exercises should only be performed against accounts and lab systems for which you have explicit authorization.

4.11 Password Spraying

Definition: Password spraying is an attack in which an attacker tries a small number of commonly used passwords against many different user accounts.

Unlike traditional brute force, it distributes attempts across accounts to reduce the chance of triggering account lockout rules on a single account.

Example:

An attacker attempts one weak password against many employee accounts rather than trying hundreds of passwords against one employee.

Prevention methods:

- Block common and compromised passwords.
- Enable MFA.
- Monitor authentication logs.
- Detect unusual login patterns across multiple accounts.
- Apply sensible rate limits.

Brute Force vs. Password Spraying

Brute force	Password spraying
Tries many passwords against one or more accounts	Tries a small number of passwords across many accounts
Can generate many failed attempts on one account	Distributes attempts across accounts
May trigger account lockouts	Attempts to avoid account-specific lockouts
Defended by rate limits and MFA	Defended by MFA, monitoring, and password controls

4.12 DoS — Denial of Service

Definition: A Denial of Service attack is an attempt to make a website, server, application, or network service unavailable to legitimate users.

How it works:

An attacker sends excessive requests or traffic, or exploits a resource-consuming weakness, causing a service to become overloaded or unavailable.

Example:

An online shopping website becomes unavailable because its server cannot handle a large volume of malicious requests.

Possible impacts:

- Website downtime.
- Business interruption.
- Financial losses.
- Loss of customer trust.
- Disruption of critical services.

Prevention methods:

- Rate limiting.
- Firewalls.
- Traffic filtering.
- Load balancing.
- DDoS protection services.
- Capacity planning.

4.13 DDoS — Distributed Denial of Service

Definition: A Distributed Denial of Service attack is a DoS attack launched using multiple systems or sources to overwhelm a target service.

Attackers often use compromised devices that form a botnet.

Example:

Thousands of compromised devices send malicious traffic to a company's website simultaneously. The website becomes unable to serve legitimate visitors.

Common DDoS categories:

1. Volumetric attacks — consume network bandwidth.
2. Protocol attacks — exhaust resources used by network protocols or infrastructure.
3. Application-layer attacks — overwhelm an application using requests that consume server resources.

Prevention methods:

- DDoS mitigation services.
- Traffic filtering.

- Rate limiting.
- Content Delivery Networks (CDNs).
- Web Application Firewalls (WAFs).
- Network monitoring.

DoS vs. DDoS

DoS	DDoS
Attack comes from one source or a limited number of sources	Attack comes from many distributed sources
Usually easier to identify the source	More difficult to distinguish malicious traffic from legitimate distributed traffic
Can exhaust service resources	Can overwhelm network, protocol, or application resources

4.14 Man-in-the-Middle (MitM) Attack

Definition: A Man-in-the-Middle attack occurs when an attacker positions themselves between two communicating parties to secretly observe, intercept, or modify their communication.

Example:

A user connects to a malicious or compromised public Wi-Fi network. An attacker attempts to intercept the user's network communication.

Possible impacts:

- Credential theft.
- Session hijacking.
- Data modification.
- Information disclosure.
- Communication impersonation.

Prevention methods:

- Use properly configured HTTPS.
- Validate TLS certificates.
- Avoid untrusted networks for sensitive activity.
- Use secure VPN connections where appropriate.
- Enable MFA.
- Protect wireless networks.

Important: Encryption and certificate validation make many MitM attacks much harder, but insecure applications and incorrect configurations can still create opportunities for interception.

4.15 Spoofing

Definition: Spoofing is an attack technique in which an attacker falsifies information or identity to make a system or person believe that communication comes from a trusted source.

Common types:

1. IP spoofing — falsifying the source IP address.
2. MAC spoofing — changing or impersonating a device's MAC address.
3. Email spoofing — falsifying email sender information.
4. DNS spoofing — providing false DNS information.
5. Website spoofing — creating a fake website that imitates a legitimate website.

Example:

An attacker sends an email that appears to come from a company's official email address and asks an employee to open a malicious attachment.

Prevention methods:

- Email authentication mechanisms such as SPF, DKIM, and DMARC.
- Secure DNS configuration.
- Network filtering.
- TLS certificate validation.
- User awareness training.

4.16 Privilege Escalation

Definition: Privilege escalation is a security attack in which an attacker gains permissions or access rights higher than those originally assigned to them.

Two main types:

A. Vertical Privilege Escalation

An attacker moves from a lower level of access to a higher level of access.

Example:

A normal user exploits a vulnerable system service to obtain administrator or root privileges.

B. Horizontal Privilege Escalation

An attacker gains access to another user's resources while remaining at a similar privilege level.

Example:

A user changes an account identifier in a poorly secured application and accesses another customer's private information.

Prevention methods:

- Least privilege.
- Secure application design.
- Proper access control.
- Regular patching.
- Permission reviews.
- Monitoring privileged actions.

Cybersecurity importance: Privilege escalation is a major topic in penetration testing and operating-system security because attackers often try to increase their access after obtaining an initial foothold.

4.17 Zero-Day Vulnerability

Definition: A zero-day vulnerability is a software or hardware weakness for which a security fix is not yet available to the affected users or defenders at the time it is exploited.

A zero-day exploit is the technique or code used to take advantage of such a vulnerability.

Example:

An attacker discovers a previously unknown weakness in a widely used application and exploits it before the vendor releases a security patch.

Why zero-days are dangerous:

- Defenders may not know the weakness exists.
- No patch may be available.
- Security tools may not recognize the attack.
- Affected systems may be exposed before mitigation is possible.

Protection methods:

- Defense in depth.
- Network segmentation.
- Application allowlisting.
- Endpoint monitoring.
- Secure configurations.
- Rapid threat intelligence and incident response.
- Virtual patching or other compensating controls where appropriate.

Important distinction:

A zero-day vulnerability is not simply an old vulnerability that has not been patched. The term concerns the vulnerability's disclosure and remediation status.

4.18 Module 4 — Threat Comparison Table

Threat	Main purpose or behavior
Malware	General term for malicious software
Ransomware	Blocks access to data and demands payment
Spyware	Secretly collects information
Trojan	Disguises malware as legitimate software
Worm	Self-replicates and spreads
Rootkit	Hides malicious activity and maintains privileged access
Phishing	Tricks users into revealing information or taking unsafe actions
Social engineering	Manipulates people
Brute force	Tries many possible passwords
Password spraying	Tries a few common passwords across many accounts
DoS	Makes a service unavailable
DDoS	Uses distributed sources to disrupt a service
MitM	Intercepts or modifies communication
Spoofing	Falsifies identity or source information
Privilege escalation	Gains higher or unauthorized permissions
Zero-day	Exploits a vulnerability without an available fix at the relevant time

4.19 Module 4 — Practical Exercises

Exercise 1: Identify phishing

Examine sample phishing emails from a trusted cybersecurity awareness resource. Identify suspicious links, sender addresses, attachments, and language.

Exercise 2: Analyze malware categories

Create a comparison chart for ransomware, spyware, Trojan, worm, and rootkit. Explain how their behaviors differ.

Exercise 3: Understand password attacks

Compare brute force, dictionary attacks, credential stuffing, and password spraying. Explain the defensive measures for each.

Exercise 4: Study network attacks

Draw simple diagrams showing:

- DoS.
- DDoS.
- Man-in-the-Middle.
- IP spoofing.

Exercise 5: Privilege escalation concepts

Study the difference between normal-user privileges, administrator privileges, and root privileges on Linux and Windows.

Exercise 6: Zero-day research

Find a publicly documented zero-day vulnerability and summarize:

- The affected software.
- The vulnerability.
- Its potential impact.
- How defenders responded.

Use only public reports and authorized lab environments.

Module 4 — Key Takeaways

- Malware is a general category of malicious software.
- Ransomware can encrypt data and demand payment.
- Trojans disguise themselves as legitimate software.
- Worms can spread automatically.
- Phishing and social engineering target human behavior.
- Brute force and password spraying are different credential attacks.
- DoS and DDoS attacks affect availability.
- MitM attacks target communication between parties.
- Spoofing involves falsifying identity or source information.
- Privilege escalation increases access permissions.
- Zero-day vulnerabilities may be exploited before a fix is available.

MODULE 5 — SECURITY FRAMEWORKS

5.1 What Are Security Frameworks?

Definition: A cybersecurity framework is a structured collection of guidelines, principles, practices, and controls that helps organizations manage cybersecurity risks and protect their information systems.

Frameworks help organizations answer questions such as:

- What information and systems must be protected?
- What security risks exist?
- Which security controls should be implemented?
- How should security incidents be detected?
- How should an organization respond to cyberattacks?
- How can security improve over time?

Example:

A bank wants to improve its cybersecurity. It can use recognized frameworks to establish security policies, manage risks, protect customer information, monitor systems, and respond to incidents.

The following are important frameworks and resources used in the cybersecurity industry.

5.2 NIST Cybersecurity Framework (NIST CSF)

Definition: The NIST Cybersecurity Framework is a framework developed by the National Institute of Standards and Technology to help organizations understand, assess, prioritize, and manage cybersecurity risks.

It can be used by organizations of different sizes and industries.

Six Core Functions of NIST CSF 2.0

A Practical Overview of the NIST Cybersecurity Framework 2.0
linkedin.com

1. Govern

Definition: Govern establishes the organization's cybersecurity strategy, policies, responsibilities, and risk-management approach.

Example:

A company defines who is responsible for cybersecurity, establishes security policies, and decides how security risks should be managed.

2. Identify

Definition: Identify means understanding the organization's assets, systems, data, business environment, and cybersecurity risks.

Example:

A company creates an inventory of its:

- Computers.
- Servers.
- Cloud services.
- Applications.

- Customer databases.
- Network devices.

The organization identifies which assets are most important and what risks affect them.

3. Protect

Definition: Protect involves implementing safeguards to reduce cybersecurity risks and prevent security incidents.

Example:

A company implements:

- MFA.
- Firewalls.
- Encryption.
- Access controls.
- Employee security training.
- Security updates.

4. Detect

Definition: Detect involves identifying potential cybersecurity attacks, unusual activities, and security incidents.

Example:

A security monitoring system detects multiple unusual login attempts from an unfamiliar location.

Common technologies:

- SIEM.
- IDS.
- IPS.
- Endpoint Detection and Response (EDR).
- Security monitoring systems.

5. Respond

Definition: Respond means taking appropriate action after detecting a cybersecurity incident.

Example:

When ransomware is detected, the security team isolates the affected systems, investigates the incident, and prevents further spread.

6. Recover

Definition: Recover involves restoring affected systems, services, and information after a cybersecurity incident.

Example:

A company restores its data from secure backups after a ransomware incident and returns its services to normal operation.

NIST CSF Summary

Function	Main purpose
Govern	Establish security strategy and responsibility
Identify	Understand assets and risks
Protect	Implement security controls
Detect	Find suspicious activity
Respond	Handle security incidents
Recover	Restore affected systems

Cybersecurity importance: NIST CSF helps organizations build a complete cybersecurity risk-management program rather than focusing only on individual security tools.

5.3 ISO/IEC 27001

Definition: ISO/IEC 27001 is an international standard that specifies requirements for establishing, implementing, maintaining, and continually improving an Information Security Management System (ISMS).

An ISMS is a structured approach to managing information-security risks within an organization.

What Is an ISMS?

An Information Security Management System is a collection of policies, procedures, responsibilities, risk assessments, and security controls used to protect organizational information.

Example:

A software company stores customer information, employee records, and business documents.

It establishes an ISMS to manage:

- Information access.
- Data protection.
- Security policies.
- Risk assessments.
- Employee responsibilities.
- Incident management.
- Business continuity.

Main Elements of ISO 27001

1. Risk assessment: Identify and evaluate information-security risks.
2. Risk treatment: Decide how identified risks should be managed.
3. Security policies: Establish organizational security requirements.
4. Security controls: Implement safeguards to reduce risks.
5. Monitoring: Check whether security measures are effective.
6. Continual improvement: Review and improve the ISMS over time.

What Is ISO 27001 Certification?

ISO 27001 certification is an independent assessment of whether an organization's ISMS meets the requirements of the standard.

Benefits:

- Improves information-security management.
- Helps organizations manage risks.
- Demonstrates commitment to information security.
- Supports customer and business trust.
- Provides a structured security-management approach.

Cybersecurity importance: Security professionals may work with ISO 27001 when conducting risk assessments, implementing security controls, preparing audits, and managing compliance.

5.4 CIS Controls

Definition: The CIS Critical Security Controls are a prioritized set of cybersecurity safeguards developed by the Center for Internet Security to help organizations defend against common cyber threats.

They provide practical recommendations that organizations can implement to improve their security.

Important CIS Control Areas

The CIS Controls v8.1 framework contains 18 controls.

Important areas include:

1. Inventory and Control of Enterprise Assets.
2. Inventory and Control of Software Assets.
3. Data Protection.
4. Secure Configuration of Enterprise Assets and Software.
5. Account Management.
6. Access Control Management.

7. Continuous Vulnerability Management.
8. Audit Log Management.
9. Email and Web Browser Protections.
10. Malware Defenses.
11. Data Recovery.
12. Network Infrastructure Management.
13. Network Monitoring and Defense.
14. Security Awareness and Skills Training.
15. Service Provider Management.
16. Application Software Security.
17. Incident Response Management.
18. Penetration Testing.

Example

A small company wants to reduce the risk of cyberattacks.

Using CIS Controls, it can:

- Maintain a list of computers and devices.
- Remove unnecessary software.
- Update vulnerable applications.
- Restrict user permissions.
- Protect important information.
- Monitor security logs.
- Maintain backups.
- Perform authorized penetration tests.

CIS Implementation Groups

CIS Controls use Implementation Groups (IGs) to help organizations prioritize safeguards according to their size, resources, and risk profile.

- IG1 — Essential safeguards for organizations with limited resources and lower complexity.
- IG2 — Additional safeguards for organizations with more complex environments.
- IG3 — More comprehensive safeguards for organizations with greater security needs and resources.

Cybersecurity importance: CIS Controls are useful for turning security recommendations into practical defensive actions.

5.5 MITRE ATT&CK Framework

Definition: MITRE ATT&CK is a globally accessible knowledge base of real-world adversary tactics, techniques, and procedures based on observed cyberattack behavior.

ATT&CK stands for:

Adversarial Tactics, Techniques, and Common Knowledge.

It helps security professionals understand how attackers operate and how defenders can detect and respond to their activities.

Important ATT&CK Concepts

1. Tactics

A tactic describes the attacker's objective or goal.

Examples:

- Initial Access.
- Execution.
- Persistence.
- Privilege Escalation.
- Defense Evasion.
- Credential Access.
- Discovery.
- Lateral Movement.
- Collection.
- Exfiltration.
- Impact.

2. Techniques

A technique describes how an attacker attempts to achieve a particular objective.

For example, an attacker may use phishing to obtain initial access to an organization's systems.

3. Sub-techniques

Sub-techniques provide more specific descriptions of how a technique can be performed.

4. Procedures

Procedures describe how a particular threat actor or malware family has actually implemented a technique.

Example

Suppose an attacker sends a phishing email to an employee.

The employee opens a malicious attachment, allowing malicious code to execute.

The attacker then attempts to obtain credentials and move to another system.

Security professionals can map these activities to relevant MITRE ATT&CK techniques and develop detections.

The Strategic Guide to the MITRE ATT&CK Framework
klogixsecurity.com

Uses of MITRE ATT&CK:

- Threat intelligence.
- Security monitoring.
- Detection engineering.
- Incident response.
- Red-team exercises.
- Blue-team operations.
- Security gap analysis.

Cybersecurity importance: MITRE ATT&CK helps defenders understand attacker behavior rather than relying only on lists of malware names or attack tools.

5.6 Cyber Kill Chain

Definition: The Cyber Kill Chain is a cybersecurity model developed by Lockheed Martin that describes common stages of a cyberattack.

It helps defenders understand how an attack may progress and where they can interrupt it.

Seven Stages of the Cyber Kill Chain

Packt+ | Advance your knowledge in tech
packtpub.com

1. Reconnaissance

Definition: Reconnaissance is the process of collecting information about a potential target before an attack.

Example:

An attacker gathers publicly available information about a company's website, employees, and technology.

2. Weaponization

Definition: Weaponization is the process of preparing a malicious payload or exploit for use against a target.

Example:

An attacker prepares a malicious document intended to exploit a software weakness or trick an employee.

3. Delivery

Definition: Delivery is the stage in which the attacker sends or places the malicious payload where it can reach the target.

Example:

The attacker sends the malicious document through an email attachment.

4. Exploitation

Definition: Exploitation occurs when the attacker triggers a vulnerability or successfully tricks a user into executing malicious content.

Example:

The malicious document exploits a vulnerable application.

5. Installation

Definition: Installation is the stage in which malicious software is installed on the compromised system to maintain access or perform malicious activities.

Example:

Malware installs itself on an employee's computer.

6. Command and Control (C2)

Definition: Command and Control is the communication channel through which an attacker remotely controls compromised systems.

Example:

The infected computer communicates with an attacker-controlled server to receive instructions.

7. Actions on Objectives

Definition: This is the final stage in which the attacker performs their intended activity.

Examples:

- Stealing confidential information.
- Encrypting company files.
- Disrupting services.
- Obtaining financial information.
- Destroying data.

Cyber Kill Chain Summary

Stage	Activity

Reconnaissance	Collect information
Weaponization	Prepare malicious payload
Delivery	Send payload to target
Exploitation	Trigger vulnerability or execute malicious content
Installation	Install malware or establish access
Command and Control	Control compromised system
Actions on Objectives	Achieve attacker's goal

Cybersecurity importance: Security teams can use the Cyber Kill Chain to identify opportunities to prevent, detect, and interrupt an attack before the attacker achieves their objective.

5.7 OWASP

Definition: OWASP (Open Worldwide Application Security Project) is a nonprofit organization and global community that provides resources, tools, documentation, and guidance to improve software and application security.

It is especially important for web application security.

What Is the OWASP Top 10?

The OWASP Top 10 is an awareness document that highlights major web application security risk categories.

It helps developers, testers, and security professionals understand common application-security problems.

The following list is the OWASP Top 10:2025.

1. Broken Access Control: Users can access information or perform actions beyond their permissions.
2. Security Misconfiguration: Applications or infrastructure have insecure configurations.
3. Software Supply Chain Failures: Risks arise from insecure dependencies, build systems, updates, or software suppliers.
4. Cryptographic Failures: Sensitive information is inadequately protected through cryptography.
5. Injection: Untrusted input is interpreted as commands or queries, potentially changing application behavior.
6. Insecure Design: Security weaknesses exist in the application's design or architecture.
7. Authentication Failures: Authentication mechanisms are weak or incorrectly implemented.
8. Software or Data Integrity Failures: Software or data cannot be trusted because integrity protections are insufficient.

9. Security Logging and Alerting Failures: Important security activities are not properly recorded, monitored, or reported.
10. Mishandling of Exceptional Conditions: Errors and unusual conditions are not handled securely.

Example: Broken Access Control

Imagine an online shopping website.

Customer A logs in and can view their order information.

If Customer A changes an order identifier and can access Customer B's private order information, the application has a broken access-control problem.

Example: Injection

Imagine a website accepts user input and sends it to a database without properly handling it.

An attacker may submit specially crafted input that changes the intended database query.

This can lead to SQL injection.

OWASP resources include:

- OWASP Top 10.
- OWASP Web Security Testing Guide (WSTG).
- OWASP Application Security Verification Standard (ASVS).
- OWASP Juice Shop.
- OWASP ZAP.

Cybersecurity importance: OWASP resources are particularly useful for web application penetration testing, secure coding, vulnerability assessment, and application security engineering.

5.8 Security Frameworks — Comparison

Framework	Main purpose	Main area
NIST CSF	Manage cybersecurity risk	Organization-wide cybersecurity
ISO 27001	Establish and maintain an ISMS	Information-security management
CIS Controls	Implement prioritized security safeguards	Practical defense
MITRE ATT&CK	Understand adversary behavior	Threat detection and analysis
Cyber Kill Chain	Understand stages of cyberattacks	Attack progression
OWASP	Improve application security	Web and software security

Important Differences

- NIST CSF provides a broad risk-management framework.

- ISO 27001 specifies requirements for an information-security management system.
- CIS Controls provide prioritized technical and organizational safeguards.
- MITRE ATT&CK documents attacker tactics and techniques.
- Cyber Kill Chain describes a common attack progression.
- OWASP provides application-security guidance, testing resources, and tools.

Module 5 — Practical Exercises

Exercise 1: NIST CSF

Create a table showing how a small company can apply Govern, Identify, Protect, Detect, Respond, and Recover.

Exercise 2: ISO 27001

Research what an ISMS contains and identify five security policies a software company should establish.

Exercise 3: CIS Controls

Create an inventory of the devices and software in your own authorized lab environment. Identify missing updates and unnecessary services.

Exercise 4: MITRE ATT&CK

Choose a publicly documented cyberattack and identify some of the tactics and techniques used by the attackers.

Exercise 5: Cyber Kill Chain

Create a diagram showing the seven stages of a hypothetical phishing-based attack. Add one defensive measure at each stage.

Exercise 6: OWASP

Study common web application vulnerabilities using OWASP Juice Shop or another intentionally vulnerable local lab.

Only test applications you own or have explicit authorization to assess.

Module 5 — Key Takeaways

- NIST CSF helps organizations manage cybersecurity risks.
- ISO 27001 provides requirements for an ISMS.
- CIS Controls provide practical security safeguards.
- MITRE ATT&CK describes real-world adversary behavior.
- Cyber Kill Chain explains stages of a common cyberattack.
- OWASP provides important application-security resources.

PRACTICAL PROJECT — SECURITY ASSESSMENT OF A FICTIONAL COMPANY

Phase 5 practical project

Project Title: Cybersecurity Risk Assessment of a Small Company

1. Project Objective

The objective of this project is to apply the security principles, cryptography concepts, authentication methods, threat knowledge, and security frameworks learned in Phase 5.

You will act as a junior cybersecurity analyst responsible for evaluating the security of a fictional company.

You must identify important assets, analyze possible threats, evaluate security weaknesses, and recommend appropriate security controls.

2. Company Background

Company Name: TechNova Solutions

Industry: Software Development

Company Size: 25 Employees

Infrastructure:

- 25 employee laptops.
- One internal file server.
- One company website.
- One customer database.
- Cloud-based email.
- Office Wi-Fi network.
- Remote employee access.
- Cloud storage.
- One administrator account shared by multiple IT employees.

The company stores customer information, employee records, software projects, business documents, and financial information.

Currently, the company has limited cybersecurity policies and no dedicated security team.

3. Identify Important Assets

An asset is anything valuable to an organization that needs protection.

Identify the company's important assets.

Asset	Importance	Security requirement
Customer database	Very high	Confidentiality, integrity, availability
Employee records	High	Confidentiality and integrity
File server	High	All three CIA principles
Company website	High	Integrity and availability
Employee laptops	High	Confidentiality and integrity
Cloud email	High	Confidentiality and availability
Administrator accounts	Very high	Strong authentication and access control
Backup files	Very high	Availability and integrity

4. Identify Possible Threats

Analyze the possible threats affecting TechNova Solutions.

Threat	Possible impact
Phishing	Employee account compromise
Ransomware	Loss of access to company files
Password spraying	Unauthorized account access
DDoS	Website unavailability
Insider threat	Unauthorized data disclosure
Malware	System compromise
MitM	Interception of insecure communication
Privilege escalation	Unauthorized administrator access
Data theft	Loss of confidential information

5. Identify Security Weaknesses

The company has several security weaknesses:

1. Some employees use weak passwords.
2. MFA is not enabled for all important accounts.

3. Administrator credentials are shared.
4. Software updates are not performed regularly.
5. Employees have more permissions than necessary.
6. Backups are not tested regularly.
7. Network activity is not monitored.
8. No formal incident-response plan exists.
9. Remote access is not sufficiently restricted.
10. Employee security awareness training is limited.

These are examples of vulnerabilities or security-management weaknesses that could increase organizational risk.

6. Perform Risk Assessment

Definition: Risk assessment is the process of identifying security risks, evaluating their likelihood and potential impact, and determining how they should be managed.

For this project, use a simple qualitative risk-rating method.

- High — serious likelihood or impact.
- Medium — moderate likelihood or impact.
- Low — limited likelihood and impact.

Risk	Likelihood	Impact	Overall priority
Phishing and account compromise	High	High	High
Ransomware infection	Medium	High	High
Data theft	Medium	High	High
Weak passwords	High	Medium	High
DDoS attack	Medium	Medium	Medium
Unauthorized administrator access	Medium	High	High
Data loss due to server failure	Medium	High	High

These ratings are illustrative. A real assessment must justify ratings using evidence, business impact, and the organization's actual environment.

7. Recommend Security Controls

Now recommend security measures to reduce the identified risks.

Security problem	Recommended solution
Weak passwords	Strong unique passwords and password manager

No MFA	Enable MFA for important accounts
Shared administrator account	Create separate administrator accounts
Outdated software	Establish regular patch management
Excessive permissions	Apply least privilege
No tested backups	Maintain and test secure backups
No monitoring	Implement centralized logging and monitoring
Phishing risk	Conduct employee awareness training
Insecure remote access	Use secure remote access with MFA
No incident plan	Create an incident-response plan

8. Apply Security Frameworks

Show how the company could use the frameworks from this phase.

Framework	How TechNova can use it
NIST CSF	Organize cybersecurity risk management
ISO 27001	Establish an information-security management system
CIS Controls	Implement practical security safeguards
MITRE ATT&CK	Understand attack techniques and improve detection
Cyber Kill Chain	Identify opportunities to interrupt attacks
OWASP	Improve website and application security

9. Final Security Recommendations

TechNova Solutions should prioritize the following actions:

Priority 1 — Immediate actions

- Enable MFA for email and administrator accounts.
- Stop sharing administrator credentials.
- Change weak and compromised passwords.
- Restrict unnecessary account permissions.
- Create secure backups.

Priority 2 — Security improvements

- Update operating systems and applications.

- Deploy endpoint protection.
- Configure firewalls.
- Secure remote access.
- Establish centralized logging.
- Conduct phishing awareness training.

Priority 3 — Long-term improvements

- Establish an ISMS.
- Conduct regular security assessments.
- Create an incident-response plan.
- Review access permissions periodically.
- Perform authorized penetration testing.
- Improve security monitoring and recovery procedures.

10. Expected Project Deliverable

Prepare a security assessment report containing:

1. Company introduction.
2. Scope of assessment.
3. Asset inventory.
4. Threat identification.
5. Vulnerability identification.
6. Risk assessment table.
7. Recommended security controls.
8. Framework mapping.
9. Final recommendations.
10. Conclusion.

Conclusion:

The assessment shows that TechNova Solutions faces several cybersecurity risks, including phishing, ransomware, weak authentication, excessive permissions, and insufficient monitoring.

By implementing strong authentication, least privilege, secure backups, patch management, network protection, and security monitoring, the company can significantly reduce its exposure to common cyber threats.

This project demonstrates how cybersecurity professionals apply security principles and frameworks to real organizational environments.

PHASE 5 — FINAL REVISION

1. Important Definitions

Term	Definition
Cybersecurity	Protection of systems, networks, applications, and information from cyber threats
CIA Triad	Confidentiality, Integrity, and Availability
Authentication	Verification of identity
Authorization	Determining what an authenticated user can access
Accounting	Recording user and system activities
Cryptography	Mathematical techniques for protecting information
Encryption	Converting readable data into protected ciphertext
Hashing	Producing a fixed-length digest from input data
AES	Symmetric encryption algorithm
RSA	Public-key cryptographic algorithm
ECC	Family of elliptic-curve cryptographic techniques
SHA-256	Cryptographic hash function producing a 256-bit digest
Salt	Random value added to a password before password hashing
Digital Signature	Cryptographic mechanism for verifying signed data
Digital Certificate	Electronic document connecting an identity with a public key
PKI	Infrastructure for managing public keys and certificates
TLS	Protocol for protecting communication over networks
MFA	Authentication using two or more independent factors
SSO	Single Sign-On
OAuth 2.0	Delegated authorization framework
OIDC	Authentication and identity layer built on OAuth 2.0
SAML	Standard for exchanging authentication and authorization-related information
Kerberos	Ticket-based network authentication protocol
Malware	Malicious software

Phishing	Deceptive communication designed to trick users
Vulnerability	A weakness that could be exploited
Threat	A potential cause of harm
Attack	An attempt to exploit a weakness
Privilege Escalation	Gaining higher or unauthorized permissions
Zero-Day	Vulnerability exploited before a fix is available to defenders
NIST CSF	Cybersecurity risk-management framework
ISO 27001	Standard for information-security management systems
CIS Controls	Prioritized cybersecurity safeguards
MITRE ATT&CK	Knowledge base of adversary tactics and techniques
OWASP	Organization providing application-security resources

2. Important Comparison Questions

Make sure you can explain these differences without looking at the notes:

1. Confidentiality vs. Integrity vs. Availability.
2. Authentication vs. Authorization.
3. Authentication vs. Accounting.
4. Encryption vs. Hashing.
5. Symmetric vs. Asymmetric Encryption.
6. AES vs. RSA.
7. RSA vs. ECC.
8. Hashing vs. Salting.
9. Digital Signature vs. Digital Certificate.
10. 2FA vs. MFA.
11. OAuth 2.0 vs. OpenID Connect.
12. SAML vs. Kerberos.
13. Virus vs. Worm vs. Trojan.
14. Brute Force vs. Password Spraying.
15. DoS vs. DDoS.
16. Threat vs. Vulnerability vs. Attack.
17. Vertical vs. Horizontal Privilege Escalation.

18.NIST CSF vs. ISO 27001.

19.MITRE ATT&CK vs. Cyber Kill Chain.

20.OWASP vs. CIS Controls.

3. Knowledge Assessment

Answer these questions without looking at the notes.

Basic Level

1. What is cybersecurity?
2. What are the three components of the CIA Triad?
3. What is confidentiality?
4. What is integrity?
5. What is availability?
6. What is authentication?
7. What is authorization?
8. What is the principle of least privilege?
9. What is encryption?
10. What is hashing?

Intermediate Level

11.Explain the difference between symmetric and asymmetric encryption.

12.What is AES?

13.What is RSA?

14.Why is salting important?

15.What is the purpose of digital signatures?

16.What is PKI?

17.How does TLS protect communication?

18.Explain the difference between 2FA and MFA.

19.What is the difference between OAuth 2.0 and OIDC?

20.How does Kerberos authentication work?

Advanced Foundation Level

21.Explain how ransomware can affect an organization.

22.How is password spraying different from brute force?

23.Explain a Man-in-the-Middle attack.

24.What is privilege escalation?

25.What makes a zero-day vulnerability dangerous?

26.Explain the six functions of NIST CSF 2.0.

27.How does MITRE ATT&CK help security professionals?

28.Explain the seven stages of the Cyber Kill Chain.

29.What is the purpose of OWASP Top 10?

30.How would you assess the cybersecurity risks of a small company?

PHASE 6 — SECURITY TOOLS AND PRACTICAL LABORATORY

CYBERSECURITY AND ETHICAL HACKING — STUDY NOTES

1. Introduction to Security Tools

What Are Cybersecurity Tools?

Definition: Cybersecurity tools are software applications or utilities used to identify security weaknesses, analyze networks, test applications, monitor systems, investigate incidents, and protect digital assets.

Security professionals use these tools to understand how systems behave, identify potential vulnerabilities, and improve security.

Objectives of This Phase

By completing this phase, you should be able to:

1. Install and configure a cybersecurity laboratory.
2. Understand the purpose of commonly used security tools.

3. Perform basic network discovery and port scanning.
4. Capture and analyze network packets.
5. Test web applications for common security weaknesses.
6. Understand vulnerability assessment and controlled exploitation.
7. Analyze passwords and password hashes in authorized labs.
8. Perform basic digital forensics.
9. Monitor systems and investigate security events.
10. Interpret tool output and verify findings manually.

Important: A security tool does not automatically make someone a cybersecurity professional. You must understand what the tool is doing, why it is doing it, and whether its findings are accurate.

2. Build Your Cybersecurity Laboratory

What Is a Cybersecurity Lab?

Definition: A cybersecurity laboratory is a controlled environment containing computers, virtual machines, networks, and intentionally vulnerable applications used for learning and authorized security testing.

A lab allows you to practice security techniques without affecting real organizations or systems.

Recommended Lab Components

Component	Purpose
Host operating system	Runs your main computer
Kali Linux	Provides security testing tools
Ubuntu Linux	Provides a normal Linux server environment
Windows VM	Provides a Windows testing environment
OWASP Juice Shop	Provides an intentionally vulnerable web application
Metasploitable	Provides an intentionally vulnerable practice machine
Virtual network	Allows controlled communication between lab machines
Snapshots	Allow restoration of virtual machines

New Project: Virtual Pentesting Lab | securityfox
codingfox86.github.io

Recommended Laboratory Network

A simple lab can contain:

- Kali Linux — security testing machine.

- Ubuntu Server — target server.
- Windows VM — Windows security practice.
- OWASP Juice Shop — web application testing target.

Keep intentionally vulnerable targets on an isolated virtual network. Do not expose them directly to the public internet.

Virtualization Software

Virtualization: Virtualization is the technology that allows multiple virtual computers to run on one physical computer.

Common virtualization platforms:

- VirtualBox.
- VMware Workstation.
- KVM/QEMU.
- Hyper-V.

Important Lab Safety Rules

1. Test only systems you own or have explicit authorization to test.
2. Use intentionally vulnerable applications for attack simulations.
3. Keep vulnerable virtual machines isolated.
4. Do not scan public IP addresses without authorization.
5. Take snapshots before experimenting.
6. Never use real passwords or confidential data in practice labs.
7. Restore the lab after destructive experiments.

MODULE 1 — KALI LINUX

1.1 What Is Kali Linux?

Definition: Kali Linux is a Debian-based Linux distribution designed for penetration testing, digital forensics, security research, and cybersecurity training.

It includes many security tools that help professionals assess and investigate computer systems.

DяA | Daniel Ruiz de Alegría
drasite.com

Main Uses of Kali Linux

- Network security testing.
- Vulnerability assessment.
- Web application security testing.

- Wireless security assessment.
- Password security testing.
- Digital forensics.
- Reverse engineering.
- Security research.

Important Features

1. Contains a large collection of security tools.
2. Supports multiple hardware platforms.
3. Can run as a virtual machine.
4. Supports live booting from USB.
5. Provides access to Linux command-line utilities.
6. Supports installation of additional security software.

1.2 Kali Linux vs. Ubuntu

Kali Linux	Ubuntu
Designed mainly for penetration testing and security work	Designed as a general-purpose operating system
Includes many security tools	Focuses on general computing and server use
Commonly used by security professionals	Commonly used by developers and system administrators
Provides specialized security-testing environments	Provides general Linux environments

Important: Kali Linux is not inherently more secure than Ubuntu. Its purpose is specialized security testing, not automatic protection from cyberattacks.

1.3 Important Kali Linux Commands

Command	Purpose
`pwd`	Displays the current directory
`ls`	Lists files and directories
`cd`	Changes directory
`mkdir`	Creates a directory
`touch`	Creates an empty file
`cat`	Displays file contents

`sudo`	Executes a command with elevated privileges
`whoami`	Displays the current username
`ip addr`	Displays network interface information
`ip route`	Displays routing information
`ps`	Displays running processes
`ss -tuln`	Displays listening network sockets
`man`	Displays a command's manual

1.4 Kali Linux Package Management

Kali uses the APT package management system.

Important commands:

- `sudo apt update` — refreshes package information.
- `sudo apt upgrade` — upgrades installed packages.
- `sudo apt install package-name` — installs a package.
- `sudo apt remove package-name` — removes a package.
- `apt search package-name` — searches for packages.

Cybersecurity importance: A security professional must understand Linux administration, software installation, permissions, services, and system maintenance.

1.5 Practical Exercises

1. Install Kali Linux in a virtual machine.
2. Explore its desktop and terminal.
3. Create files and directories.
4. Identify network interfaces.
5. Check running processes.
6. Install and update software.
7. Take a virtual machine snapshot.
8. Configure an isolated lab network.

MODULE 2 — NETWORK SCANNING AND DISCOVERY

2.1 Nmap

nmap Command Examples in Linux
linuxhandbook.com

Definition: Nmap (Network Mapper) is an open-source network discovery and security auditing tool used to identify hosts, open ports, services, and other network information.

Main Uses of Nmap

1. Discover active devices on a network.
2. Identify open, closed, and filtered ports.
3. Identify running services.
4. Detect service versions.
5. Perform operating-system detection.
6. Help identify potential security weaknesses.

Important Nmap Concepts

A. Host Discovery

Host discovery identifies which devices are reachable on a network.

Example: Finding computers, servers, and network devices in your isolated lab.

B. Port Scanning

Port scanning checks the communication ports of a target system to determine their status.

Common port states:

- Open — an application is accepting connections.
- Closed — the port is reachable but no application is listening.
- Filtered — a firewall or network filter prevents Nmap from determining the port state reliably.

C. Service Detection

Service detection identifies applications or services running on open ports.

Examples:

- HTTP.
- HTTPS.
- SSH.

- FTP.
- DNS.

D. Version Detection

Version detection attempts to identify the software and version associated with a network service.

E. OS Detection

OS detection attempts to identify the operating system of a target device using network behavior and responses.

Important Nmap Options

Option	Purpose
`-sn`	Host discovery without a port scan
`-sT`	TCP connect scan
`-sS`	TCP SYN scan
`-sU`	UDP scan
`-sV`	Service and version detection
`-O`	Operating-system detection
`-p`	Specify ports
`-p-`	Scan all TCP ports
`-A`	Enable several advanced detection features
`-oN`	Save output in normal text format

Basic Nmap Examples

Only run these examples against your own lab machines or explicitly authorized targets.

Check Nmap version:

```
nmap --version
```

Scan your own lab machine:

```
nmap 192.168.56.10
```

Check selected ports:

```
nmap -p 22,80,443 192.168.56.10
```

Identify services and versions:

```
nmap -sV 192.168.56.10
```

Save scan results:

```
nmap -oN scan-results.txt 192.168.56.10
```

How to Interpret Nmap Results

Suppose a lab scan reports:

Port	State	Service
22	Open	SSH
80	Open	HTTP
443	Closed	HTTPS
3306	Filtered	MySQL

This means:

- Port 22 is accepting SSH connections.
- Port 80 is accepting HTTP connections.
- Port 443 is reachable but no service is listening.
- Port 3306 has an uncertain state because filtering prevents a reliable determination.

Important: An open port does not automatically indicate a vulnerability. You must investigate the service, its configuration, version, exposure, and security controls.

2.2 Gobuster

[Academy001 Walk-thorough. Setting up the Box | by Syed Abeer Ahmed | Medium](syedabeerahmed.medium.com)

Definition: Gobuster is a command-line tool used for discovering hidden directories, files, DNS subdomains, and virtual hosts on authorized targets.

Main Uses

- Directory enumeration.
- File discovery.
- DNS subdomain discovery.
- Virtual host discovery.

Important Concepts

Directory Enumeration: Finding web directories that are not directly linked from a website's visible pages.

Subdomain Enumeration: Identifying subdomains associated with a domain.

Wordlist: A collection of possible names used by a discovery tool.

Example:

A website may contain an administrative directory that is not visible through its homepage. Gobuster can help identify such paths during an authorized assessment.

Practical Exercise

Use a deliberately vulnerable local web application to understand directory enumeration and interpret the returned HTTP status codes.

2.3 ffuf

ffuf - Fuzz The Web
rashahacks.com

Definition: ffuf (Fuzz Faster U Fool) is a fast web fuzzer used to discover hidden resources and test how web applications respond to different inputs.

Main Uses

- Directory discovery.
- File discovery.
- Virtual host discovery.
- Parameter discovery.
- Input fuzzing.
- Web application testing.

What Is Fuzzing?

Definition: Fuzzing is a software-testing technique in which an application receives many different inputs to identify unexpected behavior, errors, or potential security weaknesses.

Example:

A security tester sends different values to an authorized test application's input field and observes whether the application handles them correctly.

Gobuster vs. ffuf

Gobuster	ffuf
Focuses on resource discovery and enumeration	Supports flexible fuzzing and resource discovery
Simple command-line workflow	Highly configurable input-position fuzzing
Useful for directory and DNS discovery	Useful for directories, parameters, headers, and other inputs

2.4 Nikto

CTF - Challenges - Hackathon
blogoverflow.de

Definition: Nikto is an open-source web server scanner that checks for potentially dangerous files, outdated software, insecure configurations, and known server-related issues.

Main Uses

- Web server security assessment.
- Detection of risky server configurations.
- Identification of potentially exposed files.
- Checking for outdated server components.
- Finding known server weaknesses.

Important Limitation

Nikto can produce false positives. Its findings must be manually verified before reporting them as confirmed vulnerabilities.

Practical Exercise

Scan an intentionally vulnerable local web server and classify each finding as:

- Confirmed issue.
- Potential issue.
- False positive.
- Informational finding.

MODULE 3 — NETWORK PACKET ANALYSIS

3.1 Wireshark

Wireshark user interface - Packet Analysis with Wireshark [Book]
oreilly.com

Definition: Wireshark is an open-source network protocol analyzer used to capture and inspect network packets.

It helps security professionals understand communication between devices and investigate network problems or suspicious activity.

Main Uses

1. Capture network packets.
2. Analyze TCP and UDP traffic.
3. Examine DNS requests and responses.
4. Inspect HTTP communication.
5. Understand TCP handshakes.
6. Investigate unusual network activity.

7. Troubleshoot network communication.

Important Wireshark Concepts

A. Packet

A packet is a unit of data transmitted across a network.

B. Packet Capture

Packet capture is the process of collecting network traffic for analysis.

C. Network Interface

A network interface is the hardware or software connection through which a device communicates over a network.

Examples:

- Ethernet.
- Wi-Fi.
- Loopback.

D. Display Filter

A display filter is used to show only packets matching specific conditions.

Common Wireshark display filters:

Filter	Purpose
`ip`	Displays IP packets
`tcp`	Displays TCP packets
`udp`	Displays UDP packets
`dns`	Displays DNS traffic
`http`	Displays HTTP traffic
`icmp`	Displays ICMP packets
`tcp.port == 80`	Displays TCP packets using port 80
`ip.addr == 192.168.56.10`	Displays packets involving the specified IP address

Understanding the Wireshark Interface

The main interface contains three important sections:

1. Packet List — displays captured packets.
2. Packet Details — displays the selected packet's protocol information.
3. Packet Bytes — displays packet data in hexadecimal and ASCII representation where possible.

TCP Three-Way Handshake

Definition: The TCP three-way handshake is the process used to establish a TCP connection between two devices.

It contains three steps:

1. SYN — client requests a connection.
2. SYN-ACK — server acknowledges and responds.
3. ACK — client acknowledges the server's response.

TCP 3-Way Handshake
blog.jargoniseasy.com

Practical Exercises

1. Capture traffic from your own computer.
2. Identify TCP packets.
3. Find DNS queries.
4. Observe a TCP three-way handshake.
5. Compare HTTP and HTTPS traffic.
6. Identify source and destination IP addresses.
7. Export a packet capture for later analysis.

Security note: Packet captures may contain private or sensitive information. Capture only traffic you are authorized to inspect.

3.2 tcpdump

Definition: tcpdump is a command-line packet analyzer used to capture and inspect network traffic.

Main Uses

- Capture network packets.
- Monitor network communication.
- Filter traffic.
- Investigate network problems.
- Save packet captures for later analysis.

Important Commands

Capture packets on a network interface:

```
sudo tcpdump -i eth0
```

Capture only TCP traffic:

```
sudo tcpdump -i eth0 tcp
```

Capture DNS traffic:

```
sudo tcpdump -i eth0 port 53
```

Save captured packets:

```
sudo tcpdump -i eth0 -w capture.pcap
```

Read a saved capture:

```
tcpdump -r capture.pcap
```

The interface name may differ on your machine. Check it before using the commands.

Wireshark vs. tcpdump

Wireshark	tcpdump
Provides a graphical interface	Primarily command-line based
Detailed visual packet analysis	Lightweight packet capture and inspection
Useful for learning protocols visually	Useful for server monitoring and automation
Rich graphical filtering	Powerful command-line filters

MODULE 4 — WEB APPLICATION SECURITY TESTING

4.1 Burp Suite

Reissuing requests with Burp Repeater - PortSwigger
portswigger.net

Definition: Burp Suite is a web application security testing platform used to inspect, modify, and analyze HTTP and HTTPS communication between a browser and a web server.

It is widely used by web application penetration testers.

Main Uses

- Inspect HTTP requests and responses.
- Analyze cookies and sessions.
- Test application inputs.
- Identify potential web vulnerabilities.
- Understand authentication and authorization behavior.
- Test web application security.

Important Burp Suite Components

1. Proxy

Intercepts and displays communication between your browser and a web application.

2. Repeater

Allows you to modify and resend individual HTTP requests to study application responses.

3. Intruder

Automates customized request testing using defined payloads. Use it only within authorized testing limits.

4. Decoder

Encodes and decodes data in formats such as URL encoding and Base64.

5. Comparer

Compares two pieces of data, such as HTTP responses.

6. Sequencer

Analyzes the randomness and quality of tokens, such as session identifiers.

Important HTTP Concepts

- Request — message sent by a client to a server.
- Response — message returned by a server.
- Header — additional information about a request or response.
- Cookie — data stored by a browser and sent to a website.
- Session — mechanism used to maintain a user's authenticated state.
- Parameter — input value supplied to an application.

Practical Exercise

Use Burp Suite Community Edition with OWASP Juice Shop.

1. Configure your browser to use Burp Proxy.
2. Open the local test application.
3. Intercept a normal HTTP request.
4. Examine its method, URL, headers, and body.
5. Send the request to Repeater.
6. Change a harmless input value.
7. Compare the server's responses.
8. Document what you observed.

Important: Burp Suite Community Edition and Professional Edition have different capabilities. You can learn many fundamental web-testing concepts with the Community Edition.

4.2 OWASP ZAP

Top Dynamic Application Security Testing (DAST) tools [2026]
intruder.io

Definition: OWASP ZAP (Zed Attack Proxy) is an open-source web application security testing tool used to identify potential vulnerabilities in web applications.

It acts as an intermediary between the browser and web application, allowing testers to inspect and analyze requests and responses.

Main Uses

- Intercept HTTP requests.
- Inspect application responses.
- Discover web application resources.
- Perform passive security analysis.
- Perform authorized active security testing.
- Identify potential web vulnerabilities.
- Generate security reports.

Important ZAP Features

1. Automated Scanner

Tests an application for selected security weaknesses.

2. Passive Scanner

Analyzes observed traffic without actively sending attack payloads to the application.

3. Active Scanner

Sends specially constructed requests to test for vulnerabilities. This can affect the target application, so use it only on authorized test systems.

4. Spider

Explores an application by following links and discovering resources.

5. AJAX Spider

Uses browser-based exploration to discover resources in applications that rely heavily on JavaScript.

6. Alerts

Displays potential security issues identified during testing.

Burp Suite vs. OWASP ZAP

Burp Suite	OWASP ZAP

Commercial platform with a free Community Edition	Open-source security testing tool
Widely used in professional web penetration testing	Useful for learning and automated testing
Offers Proxy, Repeater, Intruder, and other tools	Offers proxy, spiders, scanners, and automation
Professional Edition includes additional capabilities	Provides extensive features without a paid license

Practical Exercise

1. Install OWASP ZAP.
2. Open OWASP Juice Shop locally.
3. Configure ZAP to observe browser traffic.
4. Explore the application's pages.
5. Review passive scan alerts.
6. Understand the evidence behind each alert.
7. Verify findings manually.
8. Generate a security report.

Important: Automated scanner alerts are not automatically confirmed vulnerabilities. Verify each finding before including it in a security report.

MODULE 5 — VULNERABILITY ASSESSMENT AND CONTROLLED EXPLOITATION

5.1 Metasploit Framework

Create a Budget-Friendly Virtual Private Server with a Metasploit Instance | Optiv
optiv.com

Definition: Metasploit Framework is an open-source penetration testing platform used to research, validate, and demonstrate security vulnerabilities in authorized environments.

It provides modules for vulnerability testing, controlled exploitation, payload handling, and post-exploitation assessment.

Main Uses

- Vulnerability validation.
- Controlled exploitation.

- Security testing.
- Payload testing in laboratories.
- Post-exploitation assessment.
- Penetration testing demonstrations.

Important Metasploit Concepts

1. Exploit

A method or code that takes advantage of a vulnerability.

2. Payload

Code or functionality delivered after an exploit succeeds.

3. Auxiliary Module

A module used for activities such as scanning, information gathering, or other supporting security tests.

4. Encoder

A module that transforms payload representations. Encoding does not automatically make malicious code undetectable or harmless.

5. Post Module

A module used for additional assessment after a system has been compromised in an authorized test.

6. Session

An active communication channel established with a target after a successful test.

Important Metasploit Commands

Command	Purpose
`msfconsole`	Opens the Metasploit console
`help`	Displays available commands
`search`	Searches for modules
`info`	Displays module information
`show options`	Displays module configuration options
`show payloads`	Displays compatible payloads
`back`	Returns to the previous context
`exit`	Exits the console

Practical Exercise

Use a deliberately vulnerable virtual machine, such as Metasploitable, on an isolated lab network.

1. Understand the target operating system.
2. Identify its exposed services.
3. Research a documented vulnerability affecting the lab target.
4. Read the corresponding Metasploit module documentation.
5. Review the module's requirements and expected effects.
6. Perform a controlled validation only in the isolated lab.
7. Record the result and explain the security impact.
8. Restore the target using a snapshot.

Important: Do not use Metasploit against public systems, university networks, or other people's devices without explicit authorization.

5.2 Vulnerability Validation vs. Exploitation

Vulnerability Validation: The process of checking whether a suspected vulnerability actually exists.

Exploitation: The process of taking advantage of a vulnerability to demonstrate its impact.

Vulnerability validation	Exploitation
Confirms whether a weakness exists	Demonstrates what can happen when a weakness is exploited
May use safe checks	May change the target's state
Usually lower operational risk	Can cause service disruption or data loss
Helps reduce false positives	Requires stricter authorization and safeguards

A professional penetration tester should use the least intrusive method that provides sufficient evidence.

MODULE 6 — PASSWORD SECURITY AND HASH ANALYSIS

6.1 John the Ripper

[Using John the Ripper — Get Certified Get Ahead
getcertifiedgetahead.com]

Definition: John the Ripper is an open-source password security auditing tool used to test password strength by attempting to recover passwords from supported password-hash formats.

Main Uses

- Password-strength auditing.
- Password-hash analysis.

- Identifying weak passwords.
- Testing password policies.
- Auditing password security in authorized environments.

Important Concepts

Password Hash: A fixed-length output produced by a password-hashing function.

Dictionary Attack: An attempt to recover a password by testing words and likely password combinations from a list.

Brute-Force Attack: An attempt to recover a password by trying many possible combinations.

Wordlist: A file containing words or candidate passwords used during password auditing.

Practical Exercise

Use a small collection of sample hashes created specifically for your lab.

1. Understand the hash format.
2. Identify the password-hashing algorithm where possible.
3. Use a small educational wordlist.
4. Observe how password complexity affects recovery.
5. Recommend stronger password policies.

6.2 Hashcat

Hashcat vs John the Ripper: A comparative benchmarking of password cracking tools 🚀 - DEV Community
dev.to

Definition: Hashcat is a password-recovery and auditing tool that uses CPU and supported GPU hardware to test candidate passwords against password hashes.

Main Uses

- Password-security auditing.
- Testing password strength.
- Analyzing supported hash formats.
- Measuring password recovery difficulty.
- Evaluating password policies.

Important Concepts

- Hash mode — identifies the hash format being tested.
- Attack mode — determines how password candidates are generated.
- Wordlist — provides candidate passwords.

- Rule-based attack — modifies candidate words according to defined rules.
- GPU acceleration — uses supported graphics hardware to speed up certain computations.

John the Ripper vs. Hashcat

John the Ripper	Hashcat
General-purpose password auditing tool	Specialized high-performance password recovery tool
Supports many hash formats	Supports many hash formats
Useful for flexible password auditing	Often used for GPU-accelerated testing
Strong command-line workflow	Strong command-line workflow

Practical Exercise

Create your own sample password hashes and compare the recovery difficulty of:

- A short common password.
- A predictable password.
- A longer random password.
- A unique passphrase.

Important: Never attempt to recover passwords from accounts, databases, or hash collections that you do not own or have permission to audit.

6.3 Hydra

Hydra Password-Cracking Tool: How to Download and Use it for Good
techtarget.com

Definition: Hydra is a network login auditing tool used to test authentication security across supported network services.

Main Uses

- Auditing password policies.
- Testing login security.
- Evaluating authentication defenses.
- Checking the effectiveness of account lockout and rate-limiting controls.

Important Concepts

Authentication Service: A service that verifies user credentials.

Rate Limiting: Restricting how frequently a user or client can make requests.

Account Lockout: Temporarily preventing additional login attempts after a defined number of failures.

Practical Exercise:

Study how an intentionally vulnerable local authentication service responds to failed login attempts. Evaluate its rate limiting, account lockout, and logging.

Security restriction: Do not use Hydra to attempt logins against real accounts or services without explicit permission and a defined testing scope.

MODULE 7 — NETWORK UTILITIES AND CRYPTOGRAPHIC TOOLS

7.1 Netcat

Power of Netcat command. Netcat, also known as “nc,” is a… | by Preritbgr | Medium
medium.com

Definition: Netcat is a command-line networking utility used to establish TCP or UDP connections, transfer data, and troubleshoot network communication.

It is sometimes called the networking Swiss Army knife.

Main Uses

- Testing TCP connectivity.
- Testing UDP communication.
- Checking whether a service is reachable.
- Sending and receiving data.
- Troubleshooting network connections.
- Creating simple client-server communication tests.

Important Concepts

- TCP connection.
- UDP communication.
- Listening port.
- Client.
- Server.
- Network socket.

Practical Exercise

Create a simple client-server communication test between two virtual machines in your isolated laboratory.

Observe which IP addresses and ports are involved.

7.2 OpenSSL

How to Check Certificate with OpenSSL
linuxhandbook.com

Definition: OpenSSL is an open-source cryptographic toolkit used to implement and inspect cryptographic operations, certificates, keys, and TLS-related functionality.

Main Uses

- Generating cryptographic keys.
- Creating test certificates.
- Inspecting certificates.
- Calculating hashes.
- Testing TLS connections.
- Understanding cryptographic operations.

Important OpenSSL Concepts

1. Private Key

A secret cryptographic key that must be protected.

2. Public Key

A cryptographic key that can be shared publicly.

3. Certificate

An electronic document that connects an identity to a public key.

4. Hashing

Generating a digest from data.

5. TLS

Protecting communication between networked systems.

Useful OpenSSL Commands

Check the installed version:

```
openssl version
```

Calculate a SHA-256 hash:

```
openssl dgst -sha256 example.txt
```

Display certificate information:

```
openssl x509 -in certificate.pem -text -noout
```

Inspect a TLS connection to a test server:

```
openssl s_client -connect example.com:443
```

Practical Exercise

1. Generate a test private key.
2. Create a self-signed test certificate.
3. Inspect its subject and issuer.
4. Examine its validity period.
5. Identify its public key.
6. Calculate a SHA-256 hash.
7. Understand the difference between a self-signed certificate and a certificate issued by a trusted Certificate Authority.

Cybersecurity importance: OpenSSL helps security professionals understand encryption, certificates, public-key infrastructure, and secure network communication.

MODULE 8 — REVERSE ENGINEERING AND DIGITAL FORENSICS

8.1 Ghidra

Install Ghidra on Linux | Flathub
flathub.org

Definition: Ghidra is a software reverse-engineering framework developed by the U.S. National Security Agency (NSA). It is used to analyze compiled programs and understand how software works.

What Is Reverse Engineering?

Definition: Reverse engineering is the process of examining a program or system to understand its internal structure, behavior, and functionality.

Main Uses of Ghidra

- Analyzing executable files.
- Understanding compiled programs.
- Examining suspicious software.
- Identifying potentially malicious functions.
- Studying program logic.
- Supporting malware analysis.

Important Ghidra Features

1. Disassembler

Converts machine-code instructions into assembly-language representations.

2. Decompiler

Attempts to convert compiled code into a higher-level representation that resembles source code.

3. Function Analysis

Identifies and examines functions inside a program.

4. Strings Analysis

Extracts readable text from executable files.

5. Cross-References

Shows where functions, variables, or other program elements are referenced.

Important Concepts

- Executable file.
- Machine code.
- Assembly language.
- Disassembly.
- Decompilation.
- Functions.
- Registers.
- Memory addresses.

Practical Exercise

1. Open a harmless program that you compiled yourself.
2. Import it into Ghidra.
3. Allow Ghidra to analyze the program.
4. Examine its functions.
5. Inspect readable strings.
6. View its assembly representation.
7. Compare the decompiled output with your original source code.

Cybersecurity importance: Reverse engineering helps security researchers understand malware, suspicious executables, software vulnerabilities, and program behavior.

8.2 Autopsy

Autopsy User Documentation: Quick Start Guide
sleuthkit.org

Definition: Autopsy is an open-source digital forensics platform used to investigate digital evidence from computers, storage devices, and other supported sources.

It provides a graphical interface for forensic analysis.

What Is Digital Forensics?

Definition: Digital forensics is the process of identifying, collecting, preserving, examining, and reporting digital evidence in a reliable and systematic manner.

Main Uses of Autopsy

- Investigating files and folders.
- Recovering deleted-file information where possible.
- Examining file metadata.
- Analyzing browser activity.
- Investigating digital evidence.
- Creating forensic reports.

Important Concepts

1. Digital Evidence

Information stored or transmitted in digital form that may be relevant to an investigation.

2. File Metadata

Information about a file, such as its name, size, timestamps, and other properties.

3. File Hash

A digest used to help verify whether a file has changed.

4. Chain of Custody

A documented record of who collected, handled, transferred, and examined evidence.

5. Disk Image

A forensic copy of a storage device or supported storage volume.

Practical Exercise

1. Create a test collection of files.
2. Generate a forensic image or use a supported sample evidence image.
3. Create a case in Autopsy.
4. Add the evidence source.
5. Examine files and metadata.
6. Identify deleted-file information where available.
7. Generate a report.
8. Document how evidence integrity was maintained.

Important: In real investigations, preserve original evidence and follow approved forensic procedures. Do not casually modify the original storage device.

8.3 Volatility

[How to Perform Memory Forensic Analysis in Windows 11 Using Volatility 3
cpuu.hashnode.dev]

Definition: Volatility is an open-source memory forensics framework used to investigate the contents of computer memory captures.

What Is Memory Forensics?

Definition: Memory forensics is the process of analyzing a computer's volatile memory, usually RAM, to investigate system activity and identify possible security incidents.

Main Uses

- Investigating running processes.
- Examining loaded modules.
- Identifying suspicious processes.
- Analyzing network connections recorded in memory.
- Investigating malware activity.
- Supporting incident response.

Important Concepts

1. RAM Dump

A captured copy of the contents of a computer's memory at a particular time.

2. Process

A running instance of a program.

3. Memory Artifact

Information left in memory that can help investigators understand system activity.

4. Volatile Data

Information that may disappear when a system loses power or restarts.

Practical Exercise

1. Obtain a legitimate sample memory image for training.
2. Identify its operating-system context.
3. Examine running processes.
4. Review available network-related artifacts.
5. Identify suspicious process behavior.

6. Write a short investigation report.

Cybersecurity importance: Memory forensics can reveal evidence that may not be available from ordinary files or disk analysis.

8.4 Ghidra vs. Autopsy vs. Volatility

Tool	Main purpose	Evidence or target
Ghidra	Reverse engineering	Executable programs
Autopsy	Digital forensics	Disk and file-system evidence
Volatility	Memory forensics	RAM captures

MODULE 9 — SECURITY MONITORING AND INCIDENT DETECTION

9.1 Sysmon

[Getting Started with Sysmon: Installation and Configuration | by Little_Sun4lower | Medium](medium.com)

Definition: Sysmon (System Monitor) is a Windows system service and device driver from Microsoft Sysinternals that records detailed information about system activity in Windows event logs.

It helps security professionals investigate suspicious activity and monitor endpoint behavior.

Main Uses

- Process creation monitoring.
- Network connection monitoring.
- File creation and modification monitoring.
- Detection of suspicious system activity.
- Security investigation.
- Supporting threat detection.

Important Sysmon Events

Event ID	Description
1	Process creation
3	Network connection
5	Process termination
7	Image loaded

10	Process access
11	File creation
12–14	Registry events
22	DNS query

Event generation depends on the Sysmon configuration.

Practical Exercise

1. Install Sysmon on a Windows lab VM.
2. Apply a suitable training configuration.
3. Open Event Viewer.
4. Locate Sysmon operational logs.
5. Identify process-creation events.
6. Examine recorded network connections.
7. Investigate unusual events.

Cybersecurity importance: Sysmon provides useful endpoint telemetry for security monitoring, incident investigation, and threat hunting.

9.2 Wazuh

Platform Overview: Open Source XDR & SIEM | Wazuh
wazuh.com

Definition: Wazuh is an open-source security platform used for security monitoring, threat detection, vulnerability detection, file-integrity monitoring, and security compliance.

Main Uses

- Security event monitoring.
- Log analysis.
- File-integrity monitoring.
- Vulnerability detection.
- Configuration assessment.
- Threat detection.
- Endpoint monitoring.

Important Wazuh Components

1. Wazuh Agent

Software installed on monitored endpoints to collect security information.

2. Wazuh Server

Receives and analyzes data from connected agents.

3. Wazuh Indexer

Stores and indexes security information for searching and analysis.

4. Wazuh Dashboard

Provides a graphical interface for viewing alerts, events, and security information.

Important Wazuh Features

- Security Configuration Assessment.
- File Integrity Monitoring.
- Vulnerability Detection.
- Log Data Analysis.
- Security Event Detection.
- Compliance Monitoring.

Practical Exercise

1. Set up a Wazuh laboratory.
2. Connect a Linux or Windows test machine.
3. Verify that the agent is communicating.
4. Review collected security events.
5. Examine file-integrity alerts.
6. Investigate a sample security alert.

9.3 Splunk

5 Best SIEM tools to consider in 2026 | CyberArrow
cyberarrow.io

Definition: Splunk is a data platform used to collect, search, analyze, and visualize machine-generated data, including logs and security events.

Splunk Enterprise Security provides SIEM capabilities for security monitoring and threat detection.

What Is SIEM?

Definition: Security Information and Event Management (SIEM) is a security solution that collects, centralizes, analyzes, and correlates security-related information from different systems to help identify suspicious activities.

Main Uses of Splunk

- Centralized log collection.
- Searching security events.
- Monitoring system activity.
- Detecting suspicious patterns.
- Investigating security incidents.
- Creating dashboards and reports.

Important Splunk Concepts

1. Event

A recorded activity or occurrence in a system.

2. Log

A record of system or application activity.

3. Index

A storage structure used by Splunk to organize ingested data.

4. Search Processing Language (SPL)

The language used to search and analyze data in Splunk.

5. Dashboard

A visual representation of selected information and search results.

6. Alert

A notification generated when defined search or detection conditions are met.

Practical Exercise

1. Explore a Splunk training environment.
2. Import sample security logs.
3. Search for failed login events.
4. Identify unusual activity.
5. Create a simple dashboard.
6. Configure a test alert.
7. Write a short security investigation report.

9.4 Elastic Security

Security Overview | Elastic
elastic.co

Definition: Elastic Security is a security solution built on the Elastic Stack that provides security monitoring, detection, investigation, and response capabilities.

Important Components

1. Elasticsearch

A distributed search and analytics engine used to store and search data.

2. Logstash

A data-processing pipeline used to collect and transform data.

3. Kibana

A graphical interface used to search, analyze, and visualize data.

4. Elastic Agent

An agent used to collect and forward data from endpoints and other sources.

Main Uses

- Security event monitoring.
- Threat detection.
- Log analysis.
- Endpoint security.
- Security investigations.
- Alert management.
- Security dashboards.

Practical Exercise

1. Explore an Elastic Security training environment.
2. Examine security alerts.
3. Review event timelines.
4. Investigate process and network activity.
5. Identify suspicious events.
6. Document your investigation.

9.5 Wazuh vs. Splunk vs. Elastic Security

Tool	Main focus	Typical use
Wazuh	Open-source security monitoring and detection	Endpoint monitoring and security operations
Splunk	Data analytics and SIEM	Enterprise log analysis and security

		monitoring
Elastic Security	Security analytics and detection	Threat detection and investigation

Important: These products have overlapping capabilities, but they are not identical. Their licensing, architecture, integrations, deployment requirements, and available features differ.

MODULE 10 — ADDITIONAL SECURITY UTILITIES

The following tools are also useful in cybersecurity. You should understand their purpose even if you do not use every tool immediately.

Tool	Category	Main purpose
`ping`	Network diagnostics	Checks network reachability
`traceroute`	Network diagnostics	Examines the route packets take
`dig`	DNS analysis	Queries DNS information
`nslookup`	DNS analysis	Looks up DNS records
`curl`	HTTP and networking	Sends requests to URLs and APIs
`ip`	Network administration	Manages and displays network configuration
`ss`	Network administration	Displays socket information
`grep`	Linux utility	Searches text patterns
`journalctl`	Linux administration	Reads system journal logs
`YARA`	Malware analysis	Identifies patterns in files and memory
`ClamAV`	Malware detection	Scans files for known malware signatures
`OSQuery`	Endpoint visibility	Queries operating-system information
`Velociraptor`	Digital forensics and response	Collects endpoint evidence and supports investigations

10.1 How to Choose the Right Tool

A professional does not select a tool simply because it is popular.

Choose tools according to the task:

Task	Suitable tools
Discover network devices	Nmap

Analyze network packets	Wireshark, tcpdump
Enumerate web resources	Gobuster, ffuf
Scan web servers	Nikto
Inspect web requests	Burp Suite, OWASP ZAP
Validate vulnerabilities	Metasploit in an authorized lab
Audit password strength	John the Ripper, Hashcat
Test network connectivity	Netcat
Inspect cryptographic certificates	OpenSSL
Reverse engineer executables	Ghidra
Analyze disk evidence	Autopsy
Analyze memory captures	Volatility
Monitor Windows activity	Sysmon
Monitor security events	Wazuh, Splunk, Elastic Security

PRACTICAL PROJECT — BUILD A COMPLETE CYBERSECURITY LAB

Phase 6 practical project

Project Title: Personal Cybersecurity Testing and Monitoring Laboratory

1. Project Objective

Build a small virtual cybersecurity environment and use professional security tools to perform basic security assessments, analyze network traffic, investigate security events, and prepare a technical report.

2. Laboratory Architecture

SOC Lab: Building a Mini Security Operations Center for Threat Detection, Analysis, and Incident Response
babatundeqodri.hashnode.dev

Required systems:

System	Role
Kali Linux	Security testing machine

Ubuntu Server	Network and server testing
Windows VM	Windows security monitoring
OWASP Juice Shop	Web application testing
Wazuh	Security monitoring

You do not need to install every tool at once. Build the laboratory gradually.

3. Project Tasks

Task 1 — Prepare the environment

- Install a virtualization platform.
- Create a Kali Linux VM.
- Create an Ubuntu Server VM.
- Create a Windows VM.
- Configure an isolated virtual network.
- Take snapshots of all machines.

Task 2 — Network discovery

Use Nmap to:

- Discover your lab machines.
- Identify open ports.
- Identify running services.
- Save scan results.
- Explain what each discovered service does.

Task 3 — Packet analysis

Use Wireshark to:

- Capture network traffic.
- Identify TCP connections.
- Examine DNS requests.
- Observe a TCP handshake.
- Identify source and destination addresses.

Task 4 — Web application testing

Use Burp Suite or OWASP ZAP with OWASP Juice Shop to:

- Inspect HTTP requests.
- Examine HTTP responses.

- Understand cookies and sessions.
- Review scanner alerts.
- Verify findings.
- Prepare a vulnerability report.

Task 5 — Vulnerability validation

Use Metasploit in an isolated vulnerable VM to:

- Research a known vulnerability.
- Understand its impact.
- Review the relevant testing module.
- Perform an authorized controlled validation.
- Document the result.

Task 6 — Password security

Use John the Ripper or Hashcat with sample hashes to:

- Understand password hashing.
- Compare weak and strong passwords.
- Study wordlist-based password auditing.
- Recommend better password policies.

Task 7 — Digital forensics

Use Autopsy or Volatility to:

- Examine a sample evidence image.
- Investigate files and metadata.
- Review available memory artifacts.
- Document important findings.

Task 8 — Security monitoring

Use Sysmon and Wazuh to:

- Collect endpoint events.
- Monitor process activity.
- Review security alerts.
- Investigate suspicious activity.
- Prepare a short incident report.

4. Final Project Report

Your final report should contain:

1. Project introduction.
2. Laboratory architecture.
3. Virtual machine configurations.
4. Network configuration.
5. Tools used.
6. Network discovery results.
7. Packet analysis findings.
8. Web application testing findings.
9. Vulnerability validation results.
10. Password security observations.
11. Forensic investigation findings.
12. Security monitoring results.
13. Recommendations.
14. Conclusion.

Expected outcome: You should be able to explain how different cybersecurity tools work together during a basic security assessment.

PHASE 6 — FINAL REVISION

1. Important Tool Definitions

Tool	Definition
Kali Linux	Linux distribution designed for penetration testing and security research
Nmap	Network discovery and port-scanning tool
Wireshark	Graphical network packet analyzer
Burp Suite	Web application security testing platform
Metasploit	Penetration testing and vulnerability validation framework
OWASP ZAP	Open-source web application security scanner
Gobuster	Directory, file, and DNS enumeration tool
ffuf	Web application fuzzing tool
Nikto	Web server security scanner
John the Ripper	Password security auditing tool
Hashcat	Password recovery and auditing tool

Hydra	Network login auditing tool
Netcat	Network connection and troubleshooting utility
OpenSSL	Cryptographic toolkit
Ghidra	Software reverse-engineering framework
Autopsy	Digital forensics platform
Volatility	Memory forensics framework
Sysmon	Windows system activity monitoring tool
Wazuh	Security monitoring and detection platform
Splunk	Data analytics and SIEM platform
Elastic Security	Security monitoring and detection solution

2. Important Comparison Questions

Make sure you can explain these differences:

1. Kali Linux vs. Ubuntu.
2. Nmap vs. Wireshark.
3. Wireshark vs. tcpdump.
4. Gobuster vs. ffuf.
5. Nikto vs. OWASP ZAP.
6. Burp Suite vs. OWASP ZAP.
7. Vulnerability scanning vs. vulnerability validation.
8. Vulnerability validation vs. exploitation.
9. John the Ripper vs. Hashcat.
10. Netcat vs. Nmap.
11. Ghidra vs. Autopsy.
12. Autopsy vs. Volatility.
13. Sysmon vs. Wazuh.
14. Wazuh vs. Splunk.
15. Splunk vs. Elastic Security.

3. Knowledge Assessment

Answer these questions without looking at the notes.

Basic Level

1. What is Kali Linux?
2. What is a cybersecurity laboratory?
3. What is Nmap?
4. What is a port scan?
5. What is an open port?
6. What is Wireshark?
7. What is a network packet?
8. What is Burp Suite?
9. What is Metasploit?
10. What is OWASP ZAP?

Intermediate Level

11. What is the difference between TCP and UDP scanning?
12. What is service detection in Nmap?
13. What is the purpose of Gobuster?
14. What is fuzzing?
15. What is the difference between passive and active scanning?
16. Explain the purpose of Burp Proxy and Repeater.
17. What is a Metasploit payload?
18. How do John the Ripper and Hashcat work?
19. What is the purpose of OpenSSL?
20. What is the difference between Wireshark and tcpdump?

Advanced Foundation Level

21. How would you investigate an unexpected open port?
22. How would you verify a web scanner's vulnerability finding?
23. How would you analyze a suspicious network connection?
24. What is reverse engineering?
25. What is digital forensics?
26. What is memory forensics?
27. How does Sysmon help security monitoring?
28. What is SIEM?

29.How would you investigate a suspicious login event using Wazuh or Splunk?

30.How would you build an isolated cybersecurity laboratory?

PHASE 7 — ETHICAL HACKING AND PENETRATION TESTING

Complete Study Notes | Beginner to Advanced Foundations

Phase objective

Understand how authorized security professionals identify, analyze, validate, and report security weaknesses in computer systems, networks, and applications.

By completing this phase, you should be able to:

1. Explain the professional penetration testing process.
2. Define the scope and rules of engagement for an authorized test.
3. Perform passive and controlled active reconnaissance.
4. Discover hosts, ports, services, and potential vulnerabilities.

5. Evaluate vulnerabilities using CVE and CVSS.
6. Understand exploitation and privilege escalation concepts.
7. Validate security weaknesses in an isolated lab.
8. Write a professional penetration testing report.
9. Recommend practical remediation and verify fixes.

Important rule: Only test systems you own or have explicit permission to assess. A publicly accessible system is not automatically authorized for testing. Keep vulnerable virtual machines isolated from public networks.

MODULE 1 — PENETRATION TESTING FUNDAMENTALS

1.1 What is ethical hacking?

Definition: Ethical hacking is the authorized process of examining computer systems, networks, or applications to identify security weaknesses before malicious attackers can exploit them.

Ethical hackers use many of the same techniques as attackers, but they operate within an approved scope and follow legal and professional rules.

Example:

A company hires a security professional to test its website. The professional discovers that one user can access another user's private information. They document the weakness and recommend a fix.

The professional does not misuse or publish the information.

1.2 What is penetration testing?

Definition: Penetration testing is a controlled security assessment in which an authorized tester attempts to validate whether identified weaknesses can be exploited and what their potential impact could be.

A penetration test answers questions such as:

- Can an attacker access a restricted system?
- Can a normal user gain administrator privileges?
- Can sensitive information be accessed without permission?
- Can a weakness allow unauthorized code execution?
- How serious would a successful attack be?

Penetration testing vs ethical hacking

Ethical hacking	Penetration testing
Broad term for authorized security activities	A structured type of security assessment

Can include research, auditing, and testing	Follows a defined scope and methodology
May involve many security techniques	Focuses on validating security weaknesses
Can be ongoing	Usually performed during an agreed testing period

1.3 Penetration testing methodology

A methodology is a structured process used to conduct a penetration test consistently.

Seven stages of a penetration test

1. Planning and authorization

   Agree on objectives, scope, permissions, timelines, and restrictions.

2. Reconnaissance

   Collect information about the authorized target.

3. Scanning and enumeration

   Identify live hosts, ports, services, and system information.

4. Vulnerability assessment

   Identify and evaluate possible security weaknesses.

5. Controlled exploitation

   Safely validate selected vulnerabilities within the agreed scope.

6. Analysis and reporting

   Document evidence, business impact, severity, and recommendations.

7. Remediation and retesting

   Verify whether the organization has successfully fixed the reported weaknesses.

These stages are not always strictly sequential. A professional tester may return to an earlier stage when new information is discovered.

1.4 Scope definition

Definition: Scope defines exactly which systems, networks, applications, and activities are included in a penetration test.

A scope document prevents misunderstandings and accidental testing of unauthorized systems.

Example scope

Item	Example
Target	Company test website

IP address	192.168.56.10
Environment	Isolated virtual lab
Testing period	Approved dates and hours
Allowed activities	Reconnaissance, scanning, controlled validation
Restricted activities	Destructive testing, denial-of-service
Data handling	Do not access or retain unnecessary personal data

In-scope vs out-of-scope

- In-scope: Systems that the organization explicitly permits you to test.
- Out-of-scope: Systems or activities that are not included in the authorization.

Never assume that permission to test one website also gives permission to test its hosting provider, connected services, employees, or other customers.

1.5 Rules of engagement

Definition: Rules of engagement are the written instructions that establish how a penetration test must be performed.

They normally specify:

- Authorized targets.
- Testing dates and working hours.
- Approved techniques.
- Prohibited techniques.
- Emergency contacts.
- Data protection requirements.
- Reporting requirements.
- Conditions for stopping the test.

Example: A company permits vulnerability scanning but prohibits denial-of-service testing and destructive exploitation.

1.6 Authorization

Definition: Authorization is explicit permission from the system owner or an authorized representative to conduct specified security testing.

Authorization should be documented before testing begins.

It should clearly identify:

- Who is giving permission.

- Who is conducting the test.
- Which systems are authorized.
- What testing activities are allowed.
- When permission is valid.

1.7 Legal and ethical boundaries

An ethical hacker must:

1. Obtain permission before testing.
2. Stay within the approved scope.
3. Avoid unnecessary access to sensitive information.
4. Avoid disrupting business operations.
5. Protect collected evidence.
6. Report vulnerabilities responsibly.
7. Stop testing when required by the rules of engagement.

Important: Calling an activity educational, experimental, or ethical does not make unauthorized access legal.

1.8 Vulnerability assessment vs penetration testing

Vulnerability assessment	Penetration testing
Identifies potential vulnerabilities	Validates whether selected weaknesses can be exploited
Often uses automated scanners	Uses manual analysis and selected tools
Usually produces a list of weaknesses	Examines the practical impact of weaknesses
Focuses on identifying and prioritizing risks	Focuses on validating attack paths and impact
May cover a large number of systems	Often has a defined target and testing objective

Neither replaces the other. Organizations commonly use both.

1.9 Penetration testing reporting

Definition: A penetration testing report is a formal document describing the assessment, identified vulnerabilities, supporting evidence, risks, and recommended fixes.

A professional report should be:

- Accurate.
- Reproducible.

- Clear.
- Evidence-based.
- Understandable to both technical and non-technical readers.

Practical exercise

Create a fictional company named SecureTech.

Write a one-page testing authorization document containing:

1. Company name.
2. Tester name or testing team.
3. Target systems.
4. Approved testing activities.
5. Prohibited activities.
6. Testing period.
7. Emergency contact.
8. Reporting deadline.

MODULE 2 — RECONNAISSANCE

2.1 What is reconnaissance?

Definition: Reconnaissance is the process of collecting information about a target before performing deeper security testing.

It helps a penetration tester understand the target's infrastructure, technologies, public-facing assets, and possible entry points.

Example:

Before testing an authorized company's website, a tester collects information about:

- Domain names.
- Public DNS records.
- Subdomains.
- Web technologies.
- Public IP addresses.
- Exposed services.

2.2 Passive reconnaissance

Definition: Passive reconnaissance involves collecting information without directly interacting with the target's systems in a way that generates testing traffic against them.

Examples include:

- Public search engines.
- Public company websites.
- Public DNS records.
- Public certificate transparency records.
- Public WHOIS information.
- Public documentation and repositories.

Purpose: Understand the target while minimizing direct interaction.

2.3 Active reconnaissance

Definition: Active reconnaissance involves directly interacting with authorized target infrastructure to collect information.

Examples include:

- Host discovery.
- Port scanning.
- Service detection.
- Web server identification.
- Authorized DNS queries.

Active reconnaissance can generate logs and trigger security alerts. It requires authorization and must follow the rules of engagement.

Passive vs active reconnaissance

Passive	Active
Uses publicly available information	Interacts directly with target infrastructure
Usually generates no direct test traffic to the target	Generates network traffic
Generally lower operational risk	Can trigger alerts or disrupt fragile systems
Example: public DNS information	Example: authorized port scanning

2.4 OSINT

Full form: Open-Source Intelligence.

Definition: OSINT is the process of collecting and analyzing information from publicly available sources.

OSINT does not mean hacking into systems. It means gathering information that is already publicly accessible.

Common OSINT sources

Source	Information
Search engines	Public websites and documents
Company websites	Departments, services, technology references
WHOIS services	Domain registration information
Certificate Transparency	Publicly logged TLS certificates
Public DNS records	Domain infrastructure information
GitHub	Publicly exposed code and configuration
Job listings	Technologies and software used by an organization
Public IP databases	Internet-facing infrastructure information

Public information can still contain sensitive data. Do not misuse it or attempt to access private accounts or systems.

2.5 DNS enumeration

Definition: DNS enumeration is the process of collecting information about a domain's DNS records and associated infrastructure.

DNS stands for Domain Name System. It translates domain names into information such as IP addresses and mail server destinations.

Important DNS records

Record	Purpose
A	Maps a domain to an IPv4 address
AAAA	Maps a domain to an IPv6 address
CNAME	Defines an alias for another domain
MX	Identifies mail servers
NS	Identifies authoritative name servers
TXT	Stores text information, often used for verification and email security
SOA	Contains administrative information about a DNS zone
PTR	Supports reverse DNS lookups

Useful DNS commands

```
dig example.com
```

Queries DNS information.

```
dig example.com A
```

Requests the IPv4 address record.

```
dig example.com MX
```

Requests mail server records.

```
dig example.com NS
```

Requests name server records.

```
nslookup example.com
```

Performs a DNS lookup.

Use these commands against domains you own or are authorized to assess.

2.6 WHOIS

Definition: WHOIS is a protocol and associated lookup service used to retrieve registration information about domain names and IP address resources.

It may reveal:

- Registrar.
- Registration and expiration dates.
- Name servers.
- Registration status.
- Registrant details, when publicly available.

Example:

```
whois example.com
```

Privacy protection and registry policies may hide personal registration details. WHOIS information is not always complete or current.

2.7 Subdomain discovery

Definition: Subdomain discovery is the process of identifying subdomains associated with a primary domain.

Example:

- `example.com`
- `mail.example.com`
- `dev.example.com`
- `api.example.com`
- `shop.example.com`

Subdomains may expose additional services or applications.

Why subdomains matter

An organization may secure its main website while leaving an older development application or forgotten subdomain poorly maintained.

Finding a subdomain does not prove that it is vulnerable.

2.8 Technology fingerprinting

Definition: Technology fingerprinting is the process of identifying technologies used by a website, server, or application.

It may identify:

- Web server software.
- Programming languages.
- Content management systems.
- Frameworks.
- Operating system clues.
- JavaScript libraries.
- Cloud or hosting services.

Example

A website might use:

- Nginx as its web server.
- PHP as its server-side language.
- WordPress as its content management system.
- MySQL as its database.

This information helps a tester identify which technologies and known security issues may be relevant.

Limitation: Technology fingerprints can be incomplete or misleading. A server header may be hidden, modified, or inaccurate.

2.9 Attack surface mapping

Definition: Attack surface mapping is the process of identifying all possible points through which an attacker might interact with an organization's systems.

Common attack surface components

- Websites.
- APIs.
- Public IP addresses.

- Open network services.
- Remote access systems.
- Cloud resources.
- Email services.
- Authentication systems.
- Third-party integrations.

Example attack surface inventory

Asset	Type	Identified technology	Security concern
Main website	Web application	Nginx	Application vulnerabilities
API	Application interface	REST API	Broken access control
Mail server	Network service	Mail service	Authentication weaknesses
Admin portal	Web application	Unknown	Restricted access verification

2.10 Practical reconnaissance exercise

Use a domain that you own or a purpose-built training domain.

1. Identify its public DNS records.
2. Record its name servers.
3. Identify mail server records.
4. Research publicly available registration information.
5. Identify technologies using non-invasive public information.
6. Create an asset inventory.
7. Separate confirmed facts from assumptions.

Deliverable: A reconnaissance report with sources and timestamps for each observation.

MODULE 3 — SCANNING AND ENUMERATION

3.1 What is scanning?

Definition: Scanning is the process of examining authorized systems and networks to identify active devices, open ports, running services, and possible security weaknesses.

Scanning helps a tester understand what is running on a target.

3.2 Host discovery

Definition: Host discovery is the process of identifying which devices are active on a network.

A host may be:

- A laptop.
- A desktop.
- A server.
- A router.
- A printer.
- A virtual machine.

Nmap host discovery example

```
nmap -sn 192.168.56.0/24
```

This performs host discovery across the specified private lab subnet.

The `-sn` option disables port scanning and focuses on host discovery.

3.3 Port scanning

Definition: Port scanning is the process of checking network ports on a system to determine which are open, closed, or filtered.

A port is a logical communication endpoint used by network applications.

Common ports

Port	Common service
20/21	FTP
22	SSH
23	Telnet
25	SMTP
53	DNS
80	HTTP
110	POP3
143	IMAP
443	HTTPS
445	SMB
3306	MySQL

3389 RDP

5432 PostgreSQL

8080 Alternative HTTP

A port number does not guarantee which application is actually running on it. Services can use non-standard ports.

Nmap port scanning example

```
nmap -p 22,80,443 192.168.56.10
```

Checks the listed ports on the lab target.

Important port states

State	Meaning
Open	A service is accepting connections
Closed	The port is reachable, but no service is listening
Filtered	Nmap cannot determine whether the port is open because packet filtering prevents a clear result
Open|Filtered	Nmap cannot distinguish between open and filtered
Closed| Filtered	Nmap cannot distinguish between closed and filtered

Remember: An open port is not automatically a vulnerability.

3.4 Service detection

Definition: Service detection is the process of identifying the application or network service running on an open port.

Example:

A scan may identify:

- Port 22: SSH.
- Port 80: HTTP.
- Port 443: HTTPS.

Nmap service detection

```
nmap -sV 192.168.56.10
```

The `-sV` option attempts to identify service names and versions.

Why version information matters:

An outdated version may contain known vulnerabilities. However, version detection alone does not prove that a vulnerability exists.

3.5 Operating system detection

Definition: Operating system detection attempts to identify the operating system running on a target by analyzing network behavior and responses.

Example:

- Linux.
- Windows.
- FreeBSD.

Nmap example

```
sudo nmap -O 192.168.56.10
```

The `-O` option enables OS detection.

Limitation: OS detection is an estimate. Firewalls, network devices, and customized configurations can affect accuracy.

3.6 Vulnerability scanning

Definition: Vulnerability scanning is the automated or semi-automated process of checking systems for known security weaknesses.

It may identify:

- Outdated software.
- Missing patches.
- Weak configurations.
- Known vulnerable services.
- Exposed management interfaces.
- Missing security controls.

Common vulnerability scanning tools

Tool	Purpose
Nessus	Vulnerability assessment
OpenVAS / Greenbone	Vulnerability scanning and management
Nmap	Host, port, service, and selected vulnerability checks
OWASP ZAP	Web application security testing
Nikto	Web server configuration and known issue checks

A scanner report is not a final security conclusion. Findings must be verified.

3.7 Service enumeration

Definition: Service enumeration is the process of collecting detailed information about services running on a target.

It may reveal:

- Service version.
- Configuration.
- Supported protocols.
- Available features.
- Accessible resources.
- Authentication requirements.

Examples include authorized enumeration of:

- SSH.
- SMB.
- DNS.
- HTTP.
- FTP.
- SMTP.

3.8 Network mapping

Definition: Network mapping is the process of creating a visual or structured representation of network devices, connections, services, and network boundaries.

A network map can show:

- Routers.
- Switches.
- Servers.
- Workstations.
- Firewalls.
- Subnets.
- Network services.

Example network map

Lab Network

192.168.56.0/24

Kali Linux

192.168.56.5

Tester

Target VM

192.168.56.10

Authorized target

3.9 Important Nmap options revision

Option	Meaning
`-sn`	Host discovery without port scanning
`-sT`	TCP connect scan
`-sS`	TCP SYN scan
`-sU`	UDP scan
`-sV`	Service/version detection
`-O`	OS detection
`-p`	Specify ports
`-p-`	Scan all TCP ports
`-A`	Enable several advanced detection features
`-oN`	Save output in normal text format

Only use these options within an authorized testing scope.

Practical exercise

On your isolated virtual network:

1. Identify the target VM's IP address.
2. Perform host discovery.
3. Scan selected ports.
4. Identify services.
5. Attempt OS detection.
6. Record the results.
7. Create a simple network map.

Deliverable: A network discovery report.

MODULE 4 — VULNERABILITY ASSESSMENT

4.1 What is a vulnerability?

Definition: A vulnerability is a weakness in software, hardware, configuration, or security procedures that could be exploited to affect confidentiality, integrity, or availability.

Examples:

- Outdated software.
- Weak passwords.
- Incorrect permissions.
- Missing security updates.
- Insecure authentication.
- Broken access control.

4.2 CVE

Full form: Common Vulnerabilities and Exposures.

Definition: CVE is a publicly recognized identification system used to assign unique identifiers to publicly disclosed cybersecurity vulnerabilities.

Example format:

`CVE-2021-44228`

A CVE identifier helps security professionals refer to the same disclosed vulnerability.

Important points

- A CVE identifier is not a severity score.
- A CVE entry describes an identified vulnerability.
- Not every vulnerability has a CVE identifier.
- A system's presence of a vulnerable software version does not automatically prove exploitability.

4.3 CVSS

Full form: Common Vulnerability Scoring System.

Definition: CVSS is a standardized framework for assessing the technical severity of security vulnerabilities.

It produces a score from 0.0 to 10.0.

CVSS severity ratings

Score	Severity
0.0	None
0.1–3.9	Low
4.0–6.9	Medium
7.0–8.9	High
9.0–10.0	Critical

CVSS version 4.0 is a current version of the framework. When evaluating a real vulnerability, always record which CVSS version and vector are being used.

What affects CVSS?

The assessment considers factors such as:

- Attack Vector: How the vulnerability can be reached.
- Attack Complexity: How difficult exploitation is.
- Privileges Required: Whether the attacker needs existing privileges.
- User Interaction: Whether another user must take an action.
- Impact: Effects on confidentiality, integrity, and availability.

Important: CVSS measures technical severity. It does not fully represent the business risk of a vulnerability in a particular organization.

4.4 Misconfiguration

Definition: Misconfiguration occurs when a system or application is configured insecurely or incorrectly.

Examples:

- Publicly accessible administrative interfaces.
- Unnecessary services enabled.
- Weak file permissions.
- Exposed storage.
- Insecure TLS settings.
- Excessive user privileges.

A properly installed application can still be insecure because of misconfiguration.

4.5 Outdated software

Definition: Outdated software is software that is no longer at the latest supported or secure version, potentially leaving known weaknesses unpatched.

Examples:

- Unsupported operating systems.
- Old web servers.
- Unpatched applications.
- Outdated libraries.

Why outdated software is dangerous

Attackers may already know how to exploit publicly disclosed weaknesses in old versions.

However, version age alone does not establish that a system is vulnerable. Check vendor advisories, affected version ranges, patches, and configuration.

4.6 Default credentials

Definition: Default credentials are usernames and passwords supplied by a manufacturer or software provider.

Example:

A newly installed device may use a default administrator password that the owner never changes.

Security risks

- Unauthorized access.
- Administrative account compromise.
- Exposure of private information.
- Unauthorized system configuration changes.

Prevention: Change default credentials during installation and enforce strong authentication.

4.7 Authentication weaknesses

Definition: Authentication weaknesses occur when a system does not properly verify the identity of users or services.

Examples:

- Weak password policies.
- Missing MFA.
- Insecure password recovery.
- Poor session management.
- Predictable authentication tokens.

- Lack of rate limiting.

4.8 Access control weaknesses

Definition: Access control weaknesses occur when users can access resources or perform actions beyond their authorized permissions.

Examples:

- Normal users accessing administrator functions.
- One customer viewing another customer's records.
- Users modifying records they do not own.
- Missing server-side permission checks.

Authentication vs authorization

Authentication	Authorization
Verifies identity	Determines permitted actions
Answers: Who are you?	Answers: What can you access?
Example: Logging in	Example: Accessing an admin dashboard

4.9 Vulnerability prioritization

Not every vulnerability deserves the same urgency.

A professional tester considers:

1. Technical severity.
2. Exploitability.
3. Business impact.
4. Exposure to attackers.
5. Asset importance.
6. Existing security controls.
7. Availability of a reliable fix.

Example vulnerability priority table

Finding	Technical severity	Business context	Priority
Public admin interface with weak authentication	High	Critical business system	Critical
Outdated library in an isolated test application	Medium	Low exposure	Medium
Missing security header	Low	Public website	Low

Unrestricted access to customer records	Critical	Sensitive customer data	Critical	

These are illustrative judgments, not universal CVSS scores.

4.10 Practical exercise

Create a vulnerability register for your lab.

ID	Asset	Vulnerability	Evidence	Severity	Recommended fix
V-01	Lab web server	Outdated component	Version evidence	To assess	Update component
V-02	Lab application	Weak access control	Authorized test evidence	To assess	Enforce server-side checks
V-03	Lab account	Weak password policy	Configuration evidence	To assess	Improve password policy

Do not assign severity without evidence and reasoning.

MODULE 5 — EXPLOITATION FUNDAMENTALS

5.1 What is an exploit?

Definition: An exploit is a technique, code, or sequence of actions that takes advantage of a vulnerability to cause an unintended security outcome.

Examples of possible outcomes:

- Unauthorized access.
- Information disclosure.
- Privilege escalation.
- Remote code execution.
- Service disruption.

A vulnerability is a weakness. An exploit is a method of taking advantage of that weakness.

5.2 Exploitation

Definition: Exploitation is the process of using an exploit to validate or take advantage of a vulnerability.

In professional penetration testing, exploitation must be controlled and limited to what is necessary to demonstrate risk.

Example

Suppose a test application has an access control vulnerability.

A tester creates two authorized test accounts and demonstrates that one account can access information belonging to the other.

This validates the weakness without using real customer data.

5.3 Proof of concept (PoC)

Definition: A Proof of Concept is a demonstration showing that a security weakness exists or can produce a particular security impact.

A PoC can be:

- A controlled test.
- A minimal code example.
- A screenshot.
- A log entry.
- A carefully documented sequence of steps.

Why PoCs matter

A PoC helps the organization understand that a finding is real rather than merely a theoretical possibility.

A PoC should demonstrate the minimum necessary impact. It should not cause unnecessary damage.

5.4 Payload concepts

Definition: A payload is the part of an exploit or security test that performs the intended action after a vulnerability is triggered.

Depending on context, a payload may:

- Display a harmless test message.
- Demonstrate controlled code execution.
- Create a test file.
- Return proof of access.
- Establish a controlled test session.

A payload is not always malicious. In authorized testing, harmless payloads can be used to validate a security weakness.

5.5 Remote code execution (RCE)

Definition: Remote Code Execution is a vulnerability that allows an attacker to execute code on a remote system.

RCE can be extremely serious because it may allow:

- Unauthorized command execution.
- Access to application data.
- Installation of unwanted software.
- Further compromise of the system.

Example

A vulnerable application processes user input unsafely. An attacker may be able to cause the server to execute unintended commands.

Prevention:

- Validate input.
- Avoid unsafe command execution.
- Apply security patches.
- Use least privilege.
- Isolate application processes.
- Monitor suspicious activity.

5.6 Local privilege escalation

Definition: Local privilege escalation occurs when a user who already has access to a system gains higher permissions than intended.

Example:

A normal user discovers a weakness in a service configuration and gains administrator-level access.

Privilege escalation is discussed in greater detail in Module 6.

5.7 Authentication bypass

Definition: Authentication bypass occurs when an application allows access without correctly verifying a user's identity.

Examples:

- Incorrect authentication logic.
- Insecure password reset.
- Improper session validation.
- Misconfigured identity checks.

Prevention:

- Enforce authentication on the server.
- Validate every sensitive request.

- Use secure session management.
- Implement MFA where appropriate.
- Test authentication and recovery workflows.

5.8 Session attacks

Definition: Session attacks target the mechanisms used by applications to maintain a user's authenticated state.

Important session-related weaknesses

Weakness	Explanation
Session hijacking	Unauthorized use of another user's session
Session fixation	Attacker causes a victim to use a known session identifier
Session prediction	Guessing or predicting a weak session identifier
Insecure cookies	Cookies lack important security protections
Session expiration weakness	Sessions remain valid longer than appropriate

Cookie security attributes

- `Secure`: Cookie is sent over HTTPS.
- `HttpOnly`: Client-side JavaScript cannot normally read the cookie.
- `SameSite`: Helps control cross-site cookie sending.

These settings reduce risk but do not fix every session security problem.

5.9 Post-exploitation concepts

Definition: Post-exploitation refers to activities performed after a vulnerability has been successfully exploited to understand the level of access and potential impact.

In authorized testing, these activities should be limited and documented.

Common concepts include:

- Identifying the privileges of the test account.
- Checking what resources are accessible.
- Understanding the affected system.
- Determining whether sensitive data could be exposed.
- Evaluating possible lateral movement risks.
- Recording evidence.
- Cleaning up test artifacts.

Important limitation

A penetration tester should not automatically perform every possible post-exploitation activity. The scope and rules of engagement determine what is permitted.

5.10 Practical exercise

Use an intentionally vulnerable application such as OWASP Juice Shop in an isolated lab.

1. Identify a documented training vulnerability.
2. Understand the root cause.
3. Create test accounts.
4. Reproduce the issue using harmless test data.
5. Record the evidence.
6. Explain the potential impact.
7. Recommend a fix.
8. Retest after applying the fix.

MODULE 6 — PRIVILEGE ESCALATION

6.1 What is privilege escalation?

Definition: Privilege escalation is the process of obtaining permissions greater than those originally assigned to a user or process.

Example:

A normal user gains administrator or root privileges because of a security weakness.

Two main types

Vertical privilege escalation

A user gains a higher level of access.

Normal user
Administrator

Horizontal privilege escalation

A user gains access to another user's resources at a similar permission level.

User A
User B's data

6.2 Linux privilege escalation

Linux systems use users, groups, file permissions, capabilities, and privilege-management mechanisms to control access.

Important Linux privilege concepts

Concept	Meaning
Root	The highest-privilege administrative account
User	An account that accesses the system
Group	A collection of users
sudo	Allows permitted users to execute commands with elevated privileges
SUID	Permission bit that allows a program to run with its file owner's effective user ID
SGID	Permission bit that can apply group privileges
File permissions	Control read, write, and execute access
Linux capabilities	Divide certain administrative privileges into smaller units

6.3 Linux user and permission review

Useful commands for inspecting your own lab system:

`whoami`

Displays the current username.

`id`

Displays user and group identity information.

`groups`

Displays group memberships.

`ls -l`

Displays file permissions and ownership.

`sudo -l`

Displays the commands the current user is permitted to run through sudo.

`getcap -r /usr/bin 2>/dev/null`

Lists file capabilities in the specified directory, where permitted.

Understanding Linux permissions

Example:

`-rwxr-xr--`

This represents:

Part	Meaning
-	Regular file

`rwx` Owner can read, write, execute

`r-x` Group can read and execute

`r--` Others can only read

A permissions mistake can allow users to access or modify files they should not control.

6.4 Sudo misconfiguration

Definition: Sudo misconfiguration occurs when sudo permissions allow users to perform administrative actions that exceed their intended responsibilities.

Examples include:

- Unnecessarily broad sudo permissions.
- Permission to execute unsafe administrative utilities.
- Poorly controlled command arguments.
- Incorrect privilege delegation.

Prevention:

- Grant only necessary sudo permissions.
- Avoid unrestricted administrative access.
- Review sudoers configuration.
- Monitor privileged activity.
- Test permissions after configuration changes.

6.5 Linux scheduled tasks

Linux supports scheduled tasks through mechanisms such as:

- Cron.
- Systemd timers.
- Other service schedulers.

Definition: A scheduled task is an operation configured to execute automatically at a particular time or interval.

Security concerns

- A scheduled script is writable by an untrusted user.
- A privileged task runs unsafe files.
- Scripts use insecure paths.
- Task configuration grants excessive permissions.

Prevention: Ensure scheduled scripts and their parent directories have appropriate ownership and permissions.

6.6 Windows privilege escalation

Windows uses accounts, groups, access control lists, services, tokens, and security policies to manage permissions.

Important Windows security concepts

Concept	Meaning
Administrator	Account with administrative privileges
Standard user	Account with limited permissions
UAC	User Account Control
ACL	Access Control List
Windows service	Background program managed by Windows
Access token	Contains a user's security identity and privileges
SYSTEM	Highly privileged Windows service account

6.7 Windows misconfigurations

Common areas that may introduce privilege escalation risks include:

- Incorrect service permissions.
- Weak file and directory permissions.
- Excessive local administrator membership.
- Insecure scheduled tasks.
- Unprotected credentials.
- Poorly configured registry permissions.
- Unpatched operating systems and applications.

Useful Windows inspection commands

Run these on your own Windows lab machine.

```
whoami
```

Displays the current user.

```
whoami /priv
```

Displays the privileges associated with the current security token.

```
whoami /groups
```

Displays group memberships.

```
Get-Service
```

Lists Windows services.

`Get-ScheduledTask`

Lists scheduled tasks.

`Get-LocalGroup`

Lists local groups.

6.8 Vulnerable services

Definition: A vulnerable service is a system service or application containing a security weakness that may allow unauthorized actions.

Potential causes:

- Outdated software.
- Incorrect permissions.
- Weak authentication.
- Unsafe configuration.
- Excessive service privileges.

6.9 Credential security

Definition: Credential security refers to protecting usernames, passwords, tokens, keys, and other authentication information from unauthorized access.

Common credential risks:

- Passwords stored in plaintext.
- Password reuse.
- Hardcoded credentials.
- Exposed API keys.
- Insecure configuration files.
- Credentials stored in scripts.
- Weak secret management.

Prevention

1. Use unique passwords.
2. Enable MFA.
3. Store passwords using secure password hashing mechanisms.
4. Use approved secret-management solutions.
5. Rotate exposed credentials.
6. Restrict access to sensitive configuration files.
7. Avoid committing secrets to public repositories.

6.10 Practical exercise

Create two test accounts on your own Linux and Windows virtual machines:

- Standard user.
- Administrator/root-controlled account.

Inspect:

1. User permissions.
2. Group memberships.
3. File ownership.
4. Service configurations.
5. Scheduled tasks.
6. Privileged execution permissions.

Document any unsafe configuration you find and explain how to fix it.

Do not attempt to escalate privileges on a real system without explicit authorization.

MODULE 7 — PENETRATION TESTING REPORTS

7.1 What is a penetration testing report?

Definition: A penetration testing report is a professional document that communicates the results of a security assessment, including vulnerabilities, evidence, severity, business impact, and recommended remediation.

A test without a clear report has limited value. Organizations need to understand what is wrong, why it matters, and how to fix it.

7.2 Executive summary

Definition: An executive summary is a short, non-technical overview of the penetration test and its most important findings.

It is written for:

- Business owners.
- Managers.
- Executives.
- Decision-makers.

It should explain:

- What was tested.

- Why it was tested.
- The overall security condition.
- The most serious risks.
- The most important recommendations.

Example executive summary

> A security assessment was performed on the authorized test environment of SecureTech Solutions. The assessment identified weaknesses in access control, software maintenance, and system configuration. The most significant finding involved unauthorized access to restricted test records. The organization should prioritize fixing access control, reviewing permissions, and applying security updates. All findings were documented with supporting evidence and remediation recommendations.

7.3 Technical findings

Definition: Technical findings are detailed descriptions of individual vulnerabilities discovered during an assessment.

Each finding should contain:

1. Finding title.
2. Affected asset.
3. Description.
4. Evidence.
5. Severity.
6. Potential impact.
7. Reproduction steps.
8. Recommended remediation.
9. Retest status.

7.4 Evidence

Definition: Evidence is information that supports a reported security finding.

Examples:

- Screenshots.
- Application responses.
- Scan results.
- Relevant log entries.
- Configuration records.
- Test account results.

Evidence must be:

- Relevant.
- Accurate.
- Reproducible where possible.
- Stored securely.
- Free of unnecessary sensitive data.

Do not include passwords, private keys, personal information, or unrelated confidential data in reports.

7.5 Severity assessment

Definition: Severity assessment is the process of estimating how serious a vulnerability is based on its technical characteristics and potential impact.

Typical reporting categories

Severity	Meaning	Typical response
Critical	Severe compromise or major business impact	Immediate attention
High	Significant security risk	Urgent remediation
Medium	Meaningful but more limited risk	Planned remediation
Low	Limited impact	Routine remediation
Informational	Observation or improvement opportunity	Consider improvement

These categories should be based on a documented rating methodology. Do not assign severity merely because a tool labels something high.

7.6 Reproduction steps

Definition: Reproduction steps are instructions that allow an authorized reader to understand and verify the reported vulnerability.

Good reproduction steps include:

- Required test environment.
- Test account or permissions.
- Actions performed.
- Expected result.
- Actual result.
- Relevant evidence.

Keep the steps safe and limited to what is necessary to validate the finding.

7.7 Remediation

Definition: Remediation is the process of fixing or reducing the risk associated with a security vulnerability.

Examples:

Vulnerability	Recommended remediation
Weak passwords	Enforce stronger password policies and MFA
Outdated software	Upgrade to a supported, patched version
Broken access control	Enforce server-side authorization checks
Insecure file permissions	Apply least-privilege permissions
Exposed administration panel	Restrict access and strengthen authentication
Insecure session handling	Use secure session management and cookie attributes

A useful recommendation identifies what needs to change, not merely that something is insecure.

7.8 Retesting

Definition: Retesting is the process of checking a previously reported vulnerability after remediation to determine whether the fix is effective.

Possible retest outcomes

- Fixed: The original weakness is no longer reproducible.
- Partially fixed: The risk has been reduced but remains.
- Not fixed: The original weakness still exists.
- Unable to verify: Testing could not establish whether the issue was resolved.

Retesting should also check whether the fix introduced new security problems.

7.9 Professional report structure

Use this structure when writing your report:

Penetration Testing Report

SecureTech Solutions — Sample Report Structure

1. Cover page
2. Document control and confidentiality
3. Executive summary
4. Assessment objectives
5. Scope and authorization

6. Rules of engagement
7. Testing methodology
8. Tools and environment
9. Summary of findings
10. Detailed technical findings
11. Risk and severity assessment
12. Remediation recommendations
13. Retesting results
14. Limitations
15. Conclusion

PRACTICAL PROJECTS

The following projects combine the concepts from all seven modules.

Project 1 — Penetration test on an intentionally vulnerable VM

Required practical project

Objective: Conduct a controlled penetration test against a vulnerable virtual machine in an isolated lab.

Recommended lab environment

Component	Purpose
Kali Linux	Testing workstation
Metasploitable 2 or another training VM	Intentionally vulnerable target
VirtualBox or VMware	Virtualization
Host-only network	Isolated lab communication

You may also use a purpose-built application such as OWASP Juice Shop for web application testing.

Tasks

1. Build the isolated virtual network.
2. Confirm that the target belongs to your lab.
3. Record the authorized scope.
4. Discover the target host.

5. Scan ports.
6. Identify running services.
7. Enumerate relevant services.
8. Identify potential vulnerabilities.
9. Validate selected findings safely.
10. Document results.

Expected outcome: A structured record of the test, not merely a collection of screenshots or tool outputs.

Project 2 — Discover and document vulnerabilities

Objective: Create a vulnerability register.

For each finding, record:

- Vulnerability ID.
- Affected system.
- Vulnerability description.
- Evidence.
- CVE, if applicable.
- CVSS score and version, if applicable.
- Business impact.
- Severity.
- Recommended remediation.

Expected outcome: A vulnerability assessment spreadsheet or document.

Project 3 — Demonstrate controlled exploitation

Objective: Demonstrate how a verified vulnerability can affect a system without causing unnecessary damage.

Tasks:

1. Select a documented vulnerability in your training environment.
2. Understand its root cause.
3. Review the applicable security advisory or training documentation.
4. Use a harmless, approved proof of concept.
5. Record the outcome.
6. Stop once sufficient evidence has been collected.
7. Explain how the weakness could be abused.

8. Recommend remediation.

Expected outcome: A safe proof-of-concept demonstration with supporting evidence.

Project 4 — Recommend fixes

Objective: Understand the defensive side of penetration testing.

For each identified weakness:

1. Identify its root cause.
2. Recommend a technical fix.
3. Explain the business benefit.
4. Identify any configuration or patch changes.
5. Define how the fix will be verified.
6. Retest after remediation.

Expected outcome: A remediation plan prioritized by risk.

Project 5 — Produce a professional penetration testing report

Objective: Combine all previous projects into one professional report.

Your report must include:

- Executive summary.
- Scope and authorization.
- Methodology.
- Tools used.
- Asset inventory.
- Vulnerability findings.
- Evidence.
- Severity assessment.
- Reproduction steps.
- Remediation.
- Retesting results.
- Limitations and conclusion.

Expected outcome: A complete penetration testing report that another security professional can understand and review.

IMPORTANT TOOLS FOR PHASE 7

These tools should be learned according to their purpose, not memorized as a list.

Tool	Primary purpose
Nmap	Host discovery, port scanning, service detection
Wireshark	Network traffic analysis
Burp Suite	Web application security testing
OWASP ZAP	Web application security testing
Metasploit Framework	Controlled exploit validation
Gobuster	Directory and DNS enumeration
ffuf	Web fuzzing
Nikto	Web server checks
Greenbone / OpenVAS	Vulnerability assessment
Nessus	Vulnerability assessment
Netcat	Network connection testing
John the Ripper	Password security auditing
Hashcat	Password hash auditing
LinPEAS	Linux privilege-escalation assessment assistance
WinPEAS	Windows privilege-escalation assessment assistance

Tools such as LinPEAS and WinPEAS help identify potentially unsafe configurations. Their output is not proof that privilege escalation is possible. Manually verify findings.

FINAL REVISION NOTES

1. Important definitions

Term	Simple definition
Ethical hacking	Authorized security testing
Penetration testing	Controlled testing to validate security weaknesses
Scope	Systems and activities permitted for testing
Rules of engagement	Instructions governing the test
Reconnaissance	Collecting target information
OSINT	Intelligence from publicly available sources
DNS enumeration	Collecting DNS-related information

Scanning	Examining systems and services
Enumeration	Collecting detailed service information
Vulnerability	A security weakness
CVE	Identifier for a publicly disclosed vulnerability
CVSS	Vulnerability severity scoring framework
Exploit	Technique that takes advantage of a vulnerability
PoC	Demonstration that a vulnerability exists or has an effect
Payload	Component that performs an intended action
RCE	Remote code execution
Privilege escalation	Gaining higher permissions than intended
Session hijacking	Unauthorized use of another user's session
Remediation	Fixing or reducing a security weakness
Retesting	Checking whether a fix works

2. Important comparisons

Comparison	Main difference
Passive vs active reconnaissance	Public information collection vs direct target interaction
Vulnerability assessment vs penetration testing	Identifying weaknesses vs validating selected weaknesses
CVE vs CVSS	Vulnerability identifier vs severity scoring system
Authentication vs authorization	Identity verification vs permission checking
Vertical vs horizontal escalation	Higher privileges vs another user's resources
Vulnerability vs exploit	Weakness vs method of exploiting it
PoC vs payload	Demonstration vs action performed by exploit code
Remediation vs retesting	Fixing the issue vs verifying the fix

3. Knowledge assessment — 30 questions

Answer these questions without looking at the notes.

Fundamentals

1. What is ethical hacking?

2. What is penetration testing?
3. What are the main stages of a penetration test?
4. What is the purpose of scope definition?
5. Why are rules of engagement important?

Reconnaissance

6. What is reconnaissance?
7. What is the difference between passive and active reconnaissance?
8. What is OSINT?
9. What is DNS enumeration?
10. What is the purpose of WHOIS?

Scanning and enumeration

11. What is host discovery?
12. What is port scanning?
13. What does an open port indicate?
14. What is service detection?
15. What is network mapping?

Vulnerability assessment

16. What is a vulnerability?
17. What is CVE?
18. What is CVSS?
19. What is the difference between authentication and authorization?
20. How should vulnerabilities be prioritized?

Exploitation and privilege escalation

21. What is an exploit?
22. What is a proof of concept?
23. What is remote code execution?
24. What is privilege escalation?
25. What is the difference between vertical and horizontal privilege escalation?

Reporting

26. What is an executive summary?
27. Why is evidence important?
28. What should a technical finding contain?

29.What is remediation?

30.What is retesting?

Self-assessment standard

Score	Interpretation
27–30	Strong understanding of the phase's theory
23–26	Good understanding; revise weak areas
18–22	Partial understanding; repeat practical exercises
12–17	Significant gaps; revise modules
Below 12	Relearn the fundamentals before moving forward

A high score does not prove practical competence. You must also complete the projects.

PHASE 7 COMPLETION CHECKLIST

Your progress

0 of 27 completed

I understand the penetration testing methodology.
I can explain scope, authorization, and rules of engagement.
I understand legal and ethical boundaries.
I can distinguish vulnerability assessment from penetration testing.
I understand passive and active reconnaissance.
I can explain OSINT, DNS enumeration, and WHOIS.
I understand subdomain discovery and technology fingerprinting.
I can create an attack surface inventory.
I can perform authorized host discovery using Nmap.
I can identify open, closed, and filtered ports.
I can identify services and understand OS detection limitations.
I can explain vulnerability scanning and service enumeration.
I understand CVE and CVSS.
I can identify common misconfigurations and authentication weaknesses.
I can prioritize vulnerabilities using technical and business risk.
I understand exploits, PoCs, and payload concepts.
I can explain RCE and authentication bypass.
I understand session attacks and post-exploitation boundaries.
I understand Linux users, groups, permissions, and sudo.
I understand Windows users, services, privileges, and scheduled tasks.
I can identify credential security risks.
I can document findings with evidence.
I can write an executive summary and technical findings.
I can recommend remediation.
I can perform authorized retesting.

I completed a penetration test in an isolated training lab.
I produced a professional penetration testing report.
Reset checklist

PHASE 8 — WEB APPLICATION AND API SECURITY

Phase 8
Beginner to Advanced

Module 1 — Web Fundamentals

What Is A Web Application? | Ramotion Agency
ramotion.com

1. Learning objectives

After completing this module, you should be able to:

1. Understand how websites and web applications work internally.
2. Explain communication between clients and servers.
3. Understand HTTP requests and responses.
4. Identify and understand different HTTP methods.
5. Read and analyze HTTP headers.
6. Understand cookies and session management.
7. Understand authentication and authorization.
8. Explain REST APIs and JSON.
9. Understand WebSockets and real-time communication.
10. Understand the Same-Origin Policy (SOP).
11. Understand Cross-Origin Resource Sharing (CORS).

12. Use these concepts when performing web application penetration testing.

2. Topic-wise explanations

Topic 1: Client-Server Communication

1.1 What is client-server communication?

Client-server communication is a process in which a client requests information or services from a server, and the server processes the request and sends back a response.

Client: A device or application that requests a service.

Server: A computer or software application that receives requests, processes them, and provides responses.

1.2 How does it work?

Understanding Client–Server Architecture | by Sudha Rani Maddala | Mar, 2026 | Medium
sudhamsr.medium.com

Consider opening a website such as `https://example.com`.

The following process happens:

1. You enter the website address in your browser.
2. The browser identifies the server's IP address using DNS.
3. The browser establishes a network connection with the server.
4. The browser sends an HTTP request.
5. The server receives and processes the request.
6. The server may communicate with a database.
7. The server sends an HTTP response.
8. The browser displays the website.

1.3 Real-world example

Suppose you open an online banking website.

- Client: Your web browser.
- Server: The bank's web server.
- Database: Stores account information and transaction records.
- Request: You request your account balance.
- Response: The server returns your account information after checking your permissions.

1.4 Cybersecurity importance

Penetration testers need to understand client-server communication because:

- Attackers may intercept HTTP traffic.
- Sensitive information may be exposed during transmission.
- Servers may incorrectly trust client-provided information.
- Poorly secured communication may allow unauthorized access.
- Server-side vulnerabilities may expose databases.

Example: If a banking application trusts the account number supplied by the browser without checking whether the logged-in user owns that account, an attacker may access another customer's information.

Topic 2: HTTP Requests and Responses

2.1 What is HTTP?

HTTP stands for Hypertext Transfer Protocol.

It is an application-layer protocol used for communication between web clients and web servers.

HTTP defines how clients request resources and how servers respond.

2.2 What is an HTTP request?

An HTTP request is a message sent by a client to a server asking for a resource or requesting an action.

Example:

```
GET /products HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0
Accept: text/html
```

Explanation:

Component	Meaning
GET	HTTP method
/products	Requested resource
HTTP/1.1	HTTP version
Host	Target website
User-Agent	Information about the client
Accept	Expected response format

2.3 What is an HTTP response?

An HTTP response is a message sent by a server to a client after processing its request.

Example:

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 35

<h1>Welcome to Example</h1>
```

Explanation:

Component	Meaning
HTTP/1.1	HTTP version
200 OK	Status code
Content-Type	Type of returned content
Content-Length	Length of response body
Response body	Actual returned content

2.4 HTTP status codes

Status codes tell the client what happened when the server processed a request.

Code	Name	Meaning
200	OK	Request successful
201	Created	Resource successfully created
301	Moved Permanently	Resource permanently redirected
302	Found	Temporary redirect
400	Bad Request	Invalid request
401	Unauthorized	Authentication is required or failed
403	Forbidden	Access is not permitted
404	Not Found	Resource does not exist or is not available
405	Method Not Allowed	HTTP method is not supported for that resource
429	Too Many Requests	Too many requests received
500	Internal Server Error	Server encountered an error
502	Bad Gateway	Invalid response from an upstream server
503	Service Unavailable	Server temporarily unavailable

2.5 HTTP vs HTTPS

HTTP	HTTPS

Hypertext Transfer Protocol	Hypertext Transfer Protocol Secure
No TLS protection	Uses TLS protection
Traffic is not encrypted by HTTP itself	HTTP traffic is protected by TLS
Usually port 80	Usually port 443
Vulnerable to interception and modification on untrusted networks	Provides confidentiality and integrity in transit when correctly configured

Important: HTTPS does not automatically make a website secure. An application can still contain SQL injection, XSS, authentication vulnerabilities, and other security problems.

2.6 Cybersecurity importance

Security professionals inspect HTTP requests and responses to identify:

- Exposed passwords or sensitive information.
- Insecure cookies.
- Missing security headers.
- Unauthorized data access.
- Unexpected server responses.
- Vulnerable application endpoints.

Practical tool: Burp Suite Proxy allows you to intercept and inspect HTTP requests and responses in an authorized testing environment.

Topic 3: HTTP Methods

3.1 What are HTTP methods?

HTTP methods define the action that a client wants the server to perform on a particular resource.

For example, a client may request information, create a resource, update information, or delete a resource.

3.2 Important HTTP methods

Method	Purpose	Example
GET	Retrieve information	View a user profile
POST	Submit data or create a resource	Register a new account
PUT	Replace a resource	Replace a complete user profile
PATCH	Partially update a resource	Change a user's email
DELETE	Delete a resource	Delete a user account

HEAD	Retrieve headers without the response body	Check resource metadata
OPTIONS	Discover supported communication options	Check allowed HTTP methods
TRACE	Diagnostic method that reflects the received request	Diagnose request handling

3.3 GET example

```
GET /users/101 HTTP/1.1
Host: example.com
```

This requests information about user 101.

3.4 POST example

```
POST /users HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "name": "Rahul",
  "email": "rahul@example.com"
}
```

This sends information to create a new user.

3.5 Important security concepts

1. Safe methods

A safe method is intended not to change the server's application state.

GET, HEAD and OPTIONS are defined as safe methods.

2. Idempotent methods

An idempotent method produces the same intended server state when the same request is performed multiple times.

GET, HEAD, PUT, DELETE and OPTIONS are defined as idempotent.

POST is not generally idempotent.

3.6 Cybersecurity applications

Penetration testers examine HTTP methods to identify:

- Unnecessary methods enabled on servers.
- Unauthorized DELETE operations.
- Incorrect access restrictions on PUT and PATCH.

- Sensitive information exposed through GET parameters.
- Dangerous server configurations.

Example: If a normal user can send a DELETE request to remove another user's account, the application may have a broken access control vulnerability.

Topic 4: HTTP Headers

4.1 What are HTTP headers?

HTTP headers are additional information included in HTTP requests and responses.

They communicate information about the client, server, requested content, authentication, caching, and security policies.

4.2 Types of HTTP headers

There are two major categories.

A. Request headers

Sent by the client to provide information about the request.

Example:

```
GET /dashboard HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0
Accept: application/json
Authorization: Bearer example_token
```

B. Response headers

Sent by the server to provide information about its response.

Example:

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: no-store
X-Content-Type-Options: nosniff
```

4.3 Important HTTP headers

Header	Purpose
Host	Identifies the target host
User-Agent	Identifies the client software
Accept	Specifies acceptable response formats
Content-Type	Specifies the media type of the message body

Authorization	Carries authentication credentials
Cookie	Sends cookies to the server
Set-Cookie	Instructs the browser to store a cookie
Origin	Identifies the origin of a request
Referer	Identifies the referring URL when provided
Cache-Control	Controls caching behavior
Location	Identifies a redirect destination
Access-Control-Allow-Origin	Specifies origins allowed by CORS

4.4 Important security headers

Header	Security purpose
Content-Security-Policy	Helps reduce the impact of content injection attacks
Strict-Transport-Security	Enforces HTTPS for future connections
X-Content-Type-Options	Helps prevent MIME-type sniffing
Referrer-Policy	Controls referrer information disclosure
Permissions-Policy	Controls access to selected browser features
X-Frame-Options	Controls whether a page can be embedded in frames

4.5 Practical example

Suppose a website returns this response:

```
HTTP/1.1 200 OK
Content-Type: text/html
Server: ExampleServer/1.0
```

A security tester may investigate whether the server header reveals unnecessary technology information.

However, exposing a server header alone does not prove that the server is vulnerable.

4.6 Cybersecurity importance

HTTP headers help professionals:

- Identify application technologies.
- Examine authentication mechanisms.
- Analyze cookie security.
- Detect missing security policies.

- Investigate CORS misconfigurations.
- Identify unnecessary information disclosure.

Topic 5: Cookies

5.1 What are cookies?

Cookies are small pieces of data stored by a web browser and associated with a website.

They allow websites to maintain information between HTTP requests.

HTTP is stateless, meaning individual HTTP requests do not automatically remember previous requests.

Cookies help applications maintain continuity.

5.2 How do cookies work?

HTTP Cookies Explained: A Complete Guide | by Code Wiz | Medium
medium.com

Example:

1. A user logs into a website.
2. The server generates a session identifier.
3. The server sends a cookie through the `Set-Cookie` header.
4. The browser stores the cookie.
5. The browser sends the cookie with subsequent matching requests.
6. The server uses the session identifier to identify the user's session.

Example response:

```
HTTP/1.1 200 OK
Set-Cookie: sessionid=abc123; Secure; HttpOnly; SameSite=Lax
```

5.3 Important cookie attributes

Attribute	Purpose
Secure	Sends the cookie only over secure connections
HttpOnly	Prevents JavaScript from accessing the cookie through browser cookie APIs
SameSite	Controls when cookies are sent with cross-site requests
Expires	Sets a specific cookie expiration date
Max-Age	Sets the cookie's lifetime
Domain	Controls which hosts can receive the cookie
Path	Controls the URL paths for which the cookie is sent

5.4 Types of cookies

Type	Explanation
Session cookie	Normally expires when the browser session ends
Persistent cookie	Remains until its expiration or deletion
First-party cookie	Set in the context of the website being visited
Third-party cookie	Associated with a different site or context

5.5 Cybersecurity risks

Cookies can become security risks when they contain sensitive information or are incorrectly configured.

Examples:

- Session hijacking.
- Cookie theft.
- Session fixation.
- Cross-site scripting leading to account compromise.
- Insecure transmission of session cookies.

5.6 Practical example

Suppose a website stores its authentication session in a cookie without the Secure attribute.

If the browser sends that cookie over an unencrypted HTTP connection, an attacker on the network may be able to intercept it.

Using HTTPS and the Secure attribute helps prevent this problem.

Topic 6: Sessions

6.1 What is a session?

A session is a mechanism used by a web application to maintain information about a user's interaction across multiple requests.

Because HTTP is stateless, sessions help applications recognize users between requests.

6.2 How do sessions work?

Testing Authentication and Session Handling in Modern Web Applications · PacketWanderer
packetwanderer.com

Example:

1. A user logs into an application.
2. The server verifies the username and password.

3. The server creates a session.
4. The server generates a unique session identifier.
5. The identifier is sent to the browser, commonly through a cookie.
6. The browser sends that identifier with subsequent requests.
7. The server identifies the session and checks the user's authentication state.

6.3 Types of session storage

Type	Explanation
Server-side session	Session information is maintained on the server
Client-side session	Session-related information is maintained on the client, often in a signed or encrypted token

6.4 Session security

Important security practices include:

- Generate unpredictable session identifiers.
- Use HTTPS.
- Apply Secure, HttpOnly and appropriate SameSite cookie attributes.
- Regenerate session IDs after login or privilege changes.
- Expire inactive sessions.
- Invalidate sessions after logout.
- Prevent session fixation.

6.5 Cybersecurity example

Suppose an attacker obtains another user's valid session identifier.

If the server accepts that identifier, the attacker may be able to impersonate that user.

This attack is known as session hijacking.

Topic 7: Authentication

7.1 What is authentication?

Authentication is the process of verifying the identity of a user, device, or system.

In simple words:

Authentication answers: Who are you?

7.2 Common authentication methods

Method	Explanation

Password authentication	User provides a password
Multi-factor authentication	Uses multiple independent authentication factors
OTP authentication	Uses a one-time password
Biometric authentication	Uses fingerprints or facial recognition
Token-based authentication	Uses an authentication token
Certificate-based authentication	Uses digital certificates

7.3 Authentication factors

Authentication factors are commonly divided into three categories:

1. Something you know — Password or PIN.
2. Something you have — Security key or registered authentication device.
3. Something you are — Fingerprint or facial characteristics.

Using two different passwords does not count as two independent authentication factors.

7.4 Authentication vs Authorization

This distinction is extremely important in cybersecurity.

Authentication	Authorization
Verifies identity	Determines access permissions
Answers: Who are you?	Answers: What can you access?
Happens before access decisions	Usually follows authentication
Example: Logging in	Example: Accessing an admin dashboard

7.5 Cybersecurity applications

Security professionals test authentication systems for:

- Weak passwords.
- Brute-force vulnerabilities.
- Credential stuffing.
- Missing multi-factor authentication.
- Weak password recovery mechanisms.
- Authentication bypass.
- Improper account lockout policies.

Topic 8: REST APIs

8.1 What is an API?

API stands for Application Programming Interface.

An API is a defined interface that allows different software applications to communicate with each other.

Example:

A mobile banking application communicates with a banking server through APIs to retrieve account information.

8.2 What is REST?

REST stands for Representational State Transfer.

REST is an architectural style for designing networked applications.

A RESTful API commonly uses HTTP methods to perform operations on resources.

8.3 How does a REST API work?

NumPy Deep Dive: Real Interview Questions with Practical Answers | by Rupa Gupta | May, 2026 | Medium
medium.com

Example:

A client requests information about user 101.

```
GET /api/users/101 HTTP/1.1
Host: example.com
Accept: application/json
```

The server responds:

```
{
  "id": 101,
  "name": "Rahul",
  "role": "student"
}
```

8.4 Common REST API operations

Operation	HTTP method	Example endpoint
Retrieve users	GET	/api/users
Retrieve one user	GET	/api/users/101
Create a user	POST	/api/users
Replace user information	PUT	/api/users/101

Partially update a user	PATCH	/api/users/101
Delete a user	DELETE	/api/users/101

8.5 Cybersecurity importance

REST APIs can expose sensitive application functionality.

Security testers investigate:

- Weak API authentication.
- Broken authorization.
- Insecure endpoints.
- Excessive data exposure.
- Missing rate limits.
- Improper input validation.
- API key leakage.

Topic 9: JSON

9.1 What is JSON?

JSON stands for JavaScript Object Notation.

It is a lightweight data-interchange format used to store and exchange structured information between applications.

JSON is language-independent, even though its syntax is derived from JavaScript object notation.

9.2 JSON example

```
{
  "id": 101,
  "name": "Bikash",
  "email": "bikash@example.com",
  "isAdmin": false,
  "skills": ["Linux", "Networking", "Python"]
}
```

9.3 Important JSON elements

Element	Meaning
Object	Collection of key-value pairs
Key	Name used to identify a value

Value	Actual information
Array	Ordered collection of values
String	Text value
Number	Numeric value
Boolean	True or false
Null	Represents an empty or absent value

9.4 JSON in cybersecurity

Security professionals inspect JSON because APIs commonly use it to exchange information.

Example:

```
{
  "userId": 101,
  "role": "user"
}
```

Suppose an application accepts a client-supplied `role` field and uses it to decide user permissions without validating it on the server.

An attacker might attempt to change the value to:

```
{
  "userId": 101,
  "role": "admin"
}
```

If the server accepts the unauthorized privilege change, this may indicate a mass assignment or broken access control vulnerability.

Important: Changing a JSON value does not automatically grant privileges. The vulnerability exists when the server improperly trusts or processes that value.

Topic 10: WebSockets

10.1 What are WebSockets?

WebSockets are a communication protocol that provides full-duplex communication between a client and a server over a single long-lived connection.

Full-duplex means both sides can send and receive messages independently.

10.2 How do WebSockets work?

The ultimate guide to websockets | Everything you need to know | Engati
engati.com

1. The client sends an HTTP request to initiate a WebSocket connection.

2. The server accepts the WebSocket handshake.
3. The connection is upgraded to WebSocket.
4. Both client and server can exchange messages.
5. The connection remains open until it is closed.

Example handshake:

```
GET /chat HTTP/1.1
Host: example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Version: 13
Sec-WebSocket-Key: example-key
```

10.3 Where are WebSockets used?

- Live chat applications.
- Online multiplayer games.
- Live stock market updates.
- Real-time notifications.
- Collaborative editing applications.
- Live dashboards.

10.4 Cybersecurity risks

Security professionals test WebSocket implementations for:

- Missing authentication.
- Missing authorization.
- Cross-site WebSocket hijacking.
- Improper origin validation.
- Insecure message handling.
- Injection vulnerabilities.
- Sensitive information exposure.

10.5 HTTP vs WebSockets

HTTP	WebSockets
Request-response communication	Full-duplex communication
Typically independent requests	Long-lived connection
Client generally initiates each request	Both sides can send messages

Commonly used for websites and APIs	Commonly used for real-time applications

Topic 11: Same-Origin Policy (SOP)

11.1 What is the Same-Origin Policy?

The Same-Origin Policy is a browser security mechanism that restricts how documents and scripts from one origin can interact with resources from another origin.

Its main purpose is to prevent one website from freely accessing sensitive information belonging to another website through the browser.

11.2 What is an origin?

An origin consists of three components:

1. Scheme (protocol).
2. Hostname.
3. Port.

Example:

`https://example.com:443`

Here:

- Scheme: HTTPS
- Hostname: example.com
- Port: 443

11.3 Same-origin examples

Assume the original website is:

`https://example.com`

URL	Same origin?	Reason
https://example.com/about	Yes	Same scheme, hostname and port
https://example.com/contact	Yes	Same origin
http://example.com	No	Different scheme
https://api.example.com	No	Different hostname
https://example.com:8443	No	Different port

11.4 How SOP protects users

Suppose you are logged into your banking website.

At the same time, you visit a malicious website.

Without browser security restrictions, the malicious website might try to read sensitive information from your banking website using your browser.

SOP restricts unauthorized cross-origin access to that information.

11.5 Cybersecurity importance

Understanding SOP helps professionals understand:

- Cross-origin data access.
- Browser security boundaries.
- Cross-site scripting.
- CORS.
- Cross-site request forgery.
- Web application isolation.

Important: SOP does not prevent every cross-site request. It primarily restricts cross-origin access to responses and certain browser resources. Other browser security mechanisms control when requests can be sent.

Topic 12: Cross-Origin Resource Sharing (CORS)

12.1 What is CORS?

CORS stands for Cross-Origin Resource Sharing.

It is a browser-enforced mechanism that allows servers to specify which origins are permitted to access their resources through cross-origin browser requests.

CORS works through HTTP headers.

12.2 Why is CORS needed?

Suppose a frontend application runs at:

`https://frontend.example.com`

Its API runs at:

`https://api.example.com`

These are different origins.

The frontend needs permission to access the API response through the browser.

The API server can provide that permission using CORS headers.

12.3 How CORS works

Common CORS errors and how to fix them — WorkOS
workos.com

Example server response:

```
HTTP/1.1 200 OK
```

```
Access-Control-Allow-Origin: https://frontend.example.com
Access-Control-Allow-Methods: GET, POST
Access-Control-Allow-Headers: Content-Type, Authorization
```

These headers tell the browser which origin, methods, and request headers are permitted.

12.4 CORS preflight request

For certain cross-origin requests, the browser first sends an OPTIONS request called a preflight request.

Example:

```
OPTIONS /api/users HTTP/1.1
Host: api.example.com
Origin: https://frontend.example.com
Access-Control-Request-Method: POST
Access-Control-Request-Headers: content-type
```

The server responds with its permitted origin, methods, and headers.

If the preflight checks succeed, the browser proceeds with the actual request.

Not every cross-origin request requires a preflight.

12.5 Important CORS headers

Header	Purpose
Access-Control-Allow-Origin	Specifies permitted origins
Access-Control-Allow-Methods	Specifies permitted HTTP methods
Access-Control-Allow-Headers	Specifies permitted request headers
Access-Control-Allow-Credentials	Allows credentials in qualifying cross-origin requests
Access-Control-Expose-Headers	Specifies response headers accessible to browser scripts
Access-Control-Max-Age	Controls how long preflight results can be cached

12.6 CORS security misconfiguration

Example of an overly permissive response:

```
Access-Control-Allow-Origin: *
```

This allows any origin to read qualifying non-credentialed cross-origin responses.

Another dangerous configuration is dynamically reflecting arbitrary origins while allowing credentials.

Such a configuration may expose sensitive information to malicious websites when the browser includes credentials and the server authorizes those origins.

12.7 SOP vs CORS

SOP	CORS
Browser security policy	Mechanism for controlled cross-origin access
Restricts cross-origin access	Allows servers to authorize certain cross-origin access
Provides browser security boundaries	Uses HTTP response headers
Enforced by browsers	Enforced by browsers using server-provided policies

12.8 Cybersecurity importance

Penetration testers examine CORS configurations to identify:

- Arbitrary origin reflection.
- Unsafe credential sharing.
- Trust in untrusted subdomains.
- Incorrect wildcard configurations.
- Exposure of sensitive API responses.

Important: CORS is not authentication or authorization. It does not prevent attackers from directly sending requests using tools such as curl or Burp Suite.

3. Important definitions — Revision notes

Term	Definition
Client	Application or device that requests services from a server.
Server	System that receives and processes client requests.
HTTP	Application-layer protocol used for web communication.
HTTPS	HTTP communication protected using TLS.
HTTP Request	Message sent by a client to a server.
HTTP Response	Message returned by a server to a client.
HTTP Method	Defines the intended operation on a resource.
HTTP Header	Additional information included in an HTTP message.
Cookie	Small piece of data stored by a browser for a website.
Session	Mechanism used to maintain user interaction state across requests.
Authentication	Process of verifying identity.
Authorization	Process of determining permitted access.

API	Interface that allows software applications to communicate.
REST	Architectural style for designing networked applications.
JSON	Lightweight format for exchanging structured data.
WebSocket	Protocol supporting full-duplex communication over a persistent connection.
SOP	Browser policy restricting cross-origin access.
CORS	Mechanism that allows servers to authorize specified cross-origin browser access.

4. Practical examples — Real cybersecurity scenarios

Concept	Real-world security scenario
Client-server communication	Intercepting traffic between a browser and a web server
HTTP requests	Modifying request parameters during authorized testing
HTTP methods	Testing whether unauthorized DELETE operations are possible
HTTP headers	Identifying missing security headers
Cookies	Checking whether session cookies have Secure and HttpOnly attributes
Sessions	Testing session fixation and session hijacking protections
Authentication	Testing weak login and password recovery mechanisms
REST APIs	Testing API endpoints for unauthorized access
JSON	Checking whether unauthorized fields are accepted
WebSockets	Testing real-time message authorization
SOP	Understanding browser cross-origin restrictions
CORS	Identifying unsafe cross-origin response access

5. Cybersecurity tools used in this module

Como o Burp Suite pode levar seu pentest para o próximo nível

1. Burp Suite

Used to intercept, inspect and modify HTTP requests and responses.

File:OWASP ZAP logo.svg - Wikimedia Commons

2. OWASP ZAP

Used for web application security testing and vulnerability analysis.

Wireshark • Go Deep

3. Wireshark

Used to capture and analyze network traffic.

Visionco | services

4. Postman

Used to send API requests and inspect API responses.

Open the Inspector - Firefox Developer Tools

5. Firefox Developer Tools

Used to inspect requests, responses, cookies, storage and browser security behavior.

6. Practical exercises — Your cybersecurity lab

Hands-on practice

Use your own applications or intentionally vulnerable training applications. Do not test websites or APIs without permission.

Exercise 1: Inspect HTTP requests

Tool: Firefox Developer Tools

Steps:

1. Open Firefox.
2. Open Developer Tools using `F12`.
3. Select the Network tab.
4. Visit a website you control or a local test application.
5. Select a network request.
6. Examine the request method.
7. Examine the request headers.
8. Examine the response headers.
9. Identify the HTTP status code.

Expected result: You should be able to understand the structure of an HTTP request and response.

Exercise 2: Understand cookies

Steps:

1. Open a local web application.
2. Open Developer Tools.

3. Navigate to Storage or Application.
4. Open Cookies.
5. Inspect available cookies.
6. Identify their Domain, Path, Secure, HttpOnly and SameSite attributes.

Expected result: You should understand how browsers store and transmit cookies.

Exercise 3: Understand REST APIs

Tool: Postman

Use a local API or a public API specifically intended for testing.

1. Send a GET request.
2. Observe the response status.
3. Inspect the response headers.
4. Inspect the JSON response.
5. Send a POST request to a test endpoint.
6. Compare GET and POST behavior.

Expected result: You should understand API request-response communication.

Exercise 4: Understand WebSockets

Steps:

1. Open a local WebSocket demonstration application.
2. Open Developer Tools.
3. Select Network.
4. Filter requests by WS.
5. Establish a WebSocket connection.
6. Inspect the handshake.
7. Observe messages sent and received.

Expected result: You should understand persistent, bidirectional communication.

Exercise 5: Understand SOP and CORS

Create two local test origins, for example:

- Frontend: `http://localhost:3000`
- Backend: `http://localhost:4000`

Steps:

1. Make a frontend request to the backend.

2. Observe the browser's CORS error.
3. Configure the backend to allow the frontend origin.
4. Repeat the request.
5. Observe the difference.
6. Test what happens when the backend allows a different origin.

Expected result: You should understand how browser cross-origin restrictions work.

7. Key takeaways

1. Client-server communication is the foundation of web applications.
2. HTTP is used for communication between clients and servers.
3. HTTPS protects HTTP communication using TLS.
4. HTTP methods define the intended operation.
5. Headers provide additional request and response information.
6. Cookies help browsers maintain information between requests.
7. Sessions maintain user interaction state.
8. Authentication verifies identity, while authorization determines permissions.
9. REST APIs allow applications to communicate through resources and HTTP operations.
10. JSON is a common format for exchanging structured data.
11. WebSockets support real-time, bidirectional communication.
12. SOP restricts unauthorized cross-origin access in browsers.
13. CORS allows servers to authorize specific cross-origin browser access.
14. Understanding these concepts is essential before studying web vulnerabilities.

Next module: Phase 8 — Module 2: OWASP Web Vulnerabilities.

1. Learning objectives

By the end of this module, you should be able to:

1. Understand common web application vulnerabilities.
2. Explain how attackers exploit weaknesses in web applications.
3. Identify SQL Injection vulnerabilities.
4. Understand different types of Cross-Site Scripting (XSS).
5. Understand Cross-Site Request Forgery (CSRF).
6. Identify Broken Access Control vulnerabilities.
7. Understand authentication failures.
8. Identify security misconfigurations.
9. Understand Server-Side Request Forgery (SSRF).
10. Identify insecure file upload mechanisms.
11. Understand Path Traversal attacks.
12. Understand Insecure Deserialization.
13. Identify Cryptographic Failures.
14. Understand Business Logic Vulnerabilities.
15. Learn how to test, document and prevent these vulnerabilities.

2. Introduction to OWASP

2.1 What is OWASP?

OWASP stands for Open Worldwide Application Security Project.

It is a nonprofit organization that works to improve software and application security.

OWASP provides security resources, documentation, testing guides and awareness projects for developers and cybersecurity professionals.

Official website: OWASP Official Website

2.2 What is the OWASP Top 10?

OWASP Top 10 is an awareness document that identifies important security risks affecting web applications.

It helps developers, penetration testers and security professionals understand common application security problems.

OWASP Top 10 2025 Update: What's New and What Changed
zerothreat.ai

Important: The current OWASP Top 10 edition is 2025. The syllabus below follows your requested vulnerability topics, which are not necessarily one-to-one with the ten official categories.

The following are the official OWASP Top 10:2025 categories:

No.	Category
A01	Broken Access Control
A02	Security Misconfiguration
A03	Software Supply Chain Failures
A04	Cryptographic Failures
A05	Injection
A06	Insecure Design
A07	Authentication Failures
A08	Software or Data Integrity Failures
A09	Security Logging and Alerting Failures
A10	Mishandling of Exceptional Conditions

Source: OWASP Top 10:2025.

3. Topic-wise explanations

Topic 1: SQL Injection (SQLi)

Manual Security Testing - How To Run It & Best Practices
globalapptesting.com

1.1 What is SQL Injection?

SQL Injection is a web application vulnerability in which an attacker manipulates SQL queries through improperly handled user input.

SQL stands for Structured Query Language.

It is used to communicate with relational databases.

SQL Injection can allow attackers to:

- Read sensitive database information.
- Bypass application login mechanisms.
- Modify database records.
- Delete database information.
- In some circumstances, execute database-level administrative operations.

1.2 How does SQL Injection work?

Consider a website with a login form.

The user enters:

- Username
- Password

The application sends a database query to verify the credentials.

A vulnerable application might directly combine user input with an SQL query.

For example, conceptually:

```
SELECT * FROM users
WHERE username = 'USER_INPUT'
AND password = 'PASSWORD_INPUT';
```

If the application inserts untrusted input directly into the query, an attacker may manipulate the query's intended logic.

The database may then execute a different query than the developer intended.

1.3 Types of SQL Injection

Type	Explanation
In-band SQLi	Attacker receives results through the same communication channel
Error-based SQLi	Database errors reveal useful information
UNION-based SQLi	Uses SQL UNION functionality to retrieve additional query results
Blind SQLi	Database results are inferred indirectly
Boolean-based SQLi	Uses true/false conditions to infer information
Time-based SQLi	Uses response delays to infer information

1.4 Real-world example

Suppose an online shopping application has a product search feature.

The application constructs database queries using untrusted search input.

An attacker may manipulate the input to access database information that should not be exposed.

Potentially exposed information includes:

- Customer names.
- Email addresses.
- Password hashes.
- Order information.
- Payment-related records.

The actual impact depends on database permissions and application design.

1.5 How to identify SQL Injection

A penetration tester can:

1. Identify input fields and URL parameters.
2. Observe how the application responds to different inputs.
3. Check whether database errors are exposed.
4. Test input handling in an authorized environment.
5. Use automated tools to help detect possible injection points.
6. Manually verify findings to eliminate false positives.

1.6 Prevention

- Use parameterized queries.
- Use prepared statements.
- Validate input appropriately.
- Avoid constructing SQL queries through unsafe string concatenation.
- Apply least-privilege database permissions.
- Avoid exposing detailed database errors.
- Keep database software updated.

Important: Input validation alone is not a reliable SQL Injection defense. Parameterized queries are the primary defense.

Topic 2: Cross-Site Scripting (XSS)

XSS (Cross-Site Scripting) Explained: How It Works & How to Stop It
plexicus.ai

2.1 What is XSS?

Cross-Site Scripting (XSS) is a web security vulnerability in which an attacker causes malicious client-side code, commonly JavaScript, to execute in another user's browser within the security context of a vulnerable website.

XSS can potentially allow attackers to:

- Perform actions as the victim within the website.
- Access sensitive information available to scripts.
- Modify website content.
- Steal accessible session information when protections are insufficient.
- Redirect users to malicious destinations.

2.2 How does XSS work?

Suppose a website allows users to submit comments.

A vulnerable application displays submitted content without properly handling it.

An attacker submits content containing executable JavaScript.

When another user visits the affected page, the browser may execute the injected script.

The problem is not simply that the website accepts HTML. The problem is that untrusted content is interpreted as executable code in an unsafe context.

2.3 Types of XSS

1. Stored XSS

XSS vs CSRF - What Is The Difference? Comparison of attacks ⚔
wallarm.com

Malicious content is stored on the server, such as in a database, and executes when another user views the affected content.

Example: A malicious comment on a public discussion forum.

2. Reflected XSS

XSStrike — A tool to detect XSS. Introduction: | by Aswin Chandran | Medium
medium.com

Malicious input is included in a request and reflected unsafely in the server's response.

Example: A vulnerable search page displays a user's search input as executable HTML.

3. DOM-Based XSS

Mastering DOM-based XSS: Understanding Sources, Sinks, and Secure DOM Practices | by Ali El-Beltagy | Medium
medium.com

The vulnerability occurs in client-side JavaScript when untrusted data is processed and inserted into the DOM in an unsafe way.

Example: A page reads data from a URL fragment and inserts it into the page using an unsafe HTML operation.

2.4 Stored vs Reflected vs DOM-Based XSS

Feature	Stored XSS	Reflected XSS	DOM-Based XSS
Main location	Server-side stored content	Request and server response	Client-side code
Persistence	Usually persistent	Usually temporary	Depends on client-side behavior
Victim interaction	Visits affected content	Usually follows a crafted request or link	Visits a page with malicious input
Main investigation	Stored data and output	Request-response flow	JavaScript and DOM operations

2.5 Prevention

- Apply context-aware output encoding.
- Use safe DOM manipulation methods.
- Avoid unsafe HTML insertion.
- Sanitize HTML when HTML input is genuinely required.
- Implement Content Security Policy (CSP) as an additional security layer.
- Use HttpOnly cookies to reduce direct JavaScript access to session cookies.

Important: HttpOnly does not prevent XSS itself. It only limits one potential consequence: direct access to protected cookies through JavaScript.

Topic 3: Cross-Site Request Forgery (CSRF)

Spring Security - Hacker
klarciel.net

3.1 What is CSRF?

Cross-Site Request Forgery is an attack in which an attacker tricks an authenticated user's browser into sending an unwanted request to a website where the user is already authenticated.

The application mistakenly accepts the request as a legitimate action.

3.2 How does CSRF work?

Consider this situation:

1. A user logs into an online banking application.

2. The browser stores an authentication cookie.
3. The user visits a malicious website in another tab.
4. The malicious website causes the browser to send a request to the banking application.
5. The browser may automatically attach the banking session cookie.
6. If the banking application lacks effective CSRF protections, it may process the unwanted request.

3.3 Real-world example

Suppose a banking application allows money transfers through a request that relies only on the user's session cookie.

An attacker may try to make the victim's browser submit an unwanted transfer request.

The vulnerability occurs because the application cannot reliably distinguish a legitimate user action from a forged cross-site request.

3.4 Prevention

- Use unpredictable CSRF tokens.
- Validate CSRF tokens on state-changing requests.
- Configure appropriate SameSite cookie attributes.
- Validate request origins where appropriate.
- Avoid using GET requests for state-changing operations.
- Require reauthentication or additional verification for sensitive actions.

3.5 CSRF vs XSS

CSRF	XSS
Tricks an authenticated browser into sending unwanted requests	Causes malicious script execution in a vulnerable website's context
Exploits trust in the user's authenticated session	Exploits trust in website-generated content and scripts
Often uses automatically attached credentials	Executes code in the victim's browser
CSRF tokens and SameSite help prevent it	Output encoding and safe DOM handling help prevent it

Topic 4: Broken Access Control

Broken Access Control. Broken access control: -occurs when the… | by Mahmoud Fawzy | Medium
medium.com

4.1 What is Broken Access Control?

Broken Access Control occurs when an application fails to properly enforce restrictions on what authenticated or unauthenticated users are allowed to access or modify.

It can allow users to perform actions beyond their assigned permissions.

4.2 Types of access control failures

1. Horizontal privilege escalation

A user accesses another user's resources at the same privilege level.

Example:

- Rahul has user ID 101.
- Amit has user ID 102.
- Rahul changes a request from `/profile/101` to `/profile/102`.
- The application returns Amit's private information without checking ownership.

This is an example of an Insecure Direct Object Reference (IDOR).

2. Vertical privilege escalation

A user gains access to functionality intended for a higher privilege level.

Example:

A normal user accesses an administrative function without authorization.

3. Missing function-level access control

An application does not correctly restrict access to sensitive functions.

Example:

A normal account can access an administrative API endpoint.

4.3 Important definition: IDOR

IDOR stands for Insecure Direct Object Reference.

It occurs when an application exposes an object identifier and fails to verify whether the requesting user is authorized to access that object.

4.4 Prevention

- Enforce authorization on the server.
- Check ownership of requested resources.
- Use role-based or attribute-based access control where appropriate.
- Apply least privilege.
- Verify authorization on every sensitive operation.
- Never rely on hiding buttons or modifying frontend interfaces.

Important: Changing a user ID in a URL is not automatically a vulnerability. It becomes a vulnerability when the server fails to enforce access permissions.

Topic 5: Authentication Failures

Improve Your Cybersecurity with Password MFA
defense.com

5.1 What are Authentication Failures?

Authentication Failures occur when an application does not properly verify user identities or securely manage authentication processes.

They may allow attackers to access accounts without the legitimate user's permission.

5.2 Common authentication vulnerabilities

Vulnerability	Explanation
Weak passwords	Application permits easily guessable passwords
Brute-force attacks	Repeatedly attempts different passwords
Credential stuffing	Uses leaked username-password combinations
Password spraying	Tries a small number of common passwords against many accounts
Weak password recovery	Allows attackers to take over accounts through recovery mechanisms
Missing MFA	Accounts lack an additional authentication factor
Session fixation	Attacker causes a victim to authenticate using a session identifier known to the attacker
Improper session expiration	Sessions remain valid longer than intended
Authentication bypass	Attacker accesses protected functionality without valid authentication

5.3 How does a brute-force attack work?

Suppose a website has a login form.

An attacker repeatedly attempts different passwords against an account.

If the application has no effective rate limiting, monitoring or other protections, the attacker may eventually discover a valid password.

5.4 Credential stuffing vs brute force

Brute force	Credential stuffing
Attempts many possible passwords	Uses previously leaked credentials
May generate combinations	Uses known username-password pairs

Can be computationally expensive	Can succeed quickly when users reuse passwords
Password unpredictability helps	Unique passwords and MFA help

5.5 Prevention

- Enforce strong password policies.
- Use multi-factor authentication.
- Implement rate limiting.
- Detect suspicious login activity.
- Use secure password hashing algorithms such as Argon2id or bcrypt.
- Implement secure password recovery.
- Regenerate session identifiers after authentication.
- Provide secure session expiration and logout.

Important: Passwords should never be stored as plaintext or protected using ordinary fast hashing algorithms such as unsalted MD5 or SHA-1.

Topic 6: Security Misconfiguration

Website Vulnerabilities: How to Disable Directory Listing and Why It’s Crucial for Security
improved-move.com

6.1 What is Security Misconfiguration?

Security Misconfiguration occurs when an application, server, database, cloud service or security component is configured incorrectly or insecurely.

These mistakes may expose sensitive information or create unauthorized access opportunities.

6.2 Common examples

Misconfiguration	Security risk
Default passwords	Unauthorized access
Debug mode enabled in production	Exposure of internal information
Directory listing enabled	Exposure of files
Unnecessary open ports	Increased attack surface
Exposed configuration files	Leakage of credentials
Incorrect cloud storage permissions	Public exposure of private data
Missing security headers	Reduced browser-side protection
Unnecessary services enabled	Additional attack surface
Verbose error messages	Disclosure of internal application details

6.3 Real-world example

Imagine a company deploys a web application with debug mode enabled.

When an error occurs, the application exposes:

- Internal file paths.
- Database connection information.
- Application framework details.
- Environment variables.

If sensitive secrets are included in the error output, attackers may use them to compromise the application.

6.4 Cloud security example

Suppose a company stores customer documents in cloud object storage.

Due to incorrect access permissions, the storage bucket allows public access.

An unauthorized person may be able to retrieve private documents.

This is a cloud security misconfiguration.

6.5 Prevention

- Follow secure configuration standards.
- Disable debug mode in production.
- Remove default credentials.
- Disable unnecessary services.
- Restrict access to sensitive files.
- Review cloud storage permissions.
- Use automated configuration scanning.
- Regularly update software and dependencies.

Topic 7: Server-Side Request Forgery (SSRF)

Misconfiguration Spotlight: Securing the EC2 Instance Metadata Service | Datadog Security Labs
securitylabs.datadoghq.com

7.1 What is SSRF?

SSRF stands for Server-Side Request Forgery.

It is a vulnerability in which an attacker tricks a server into making requests to unintended destinations.

The server may have access to internal resources that are not directly accessible from the internet.

7.2 How does SSRF work?

Consider a website that allows users to enter a URL to retrieve an image.

For example:

```
https://example.com/fetch-image?url=https://images.example.org/photo.jpg
```

The application receives the URL and requests the image from that location.

If the application does not properly validate the destination, an attacker may attempt to make the server request internal resources.

7.3 SSRF attack targets

Potential targets include:

1. Internal web servers.
2. Private network services.
3. Internal administrative panels.
4. Database services.
5. Cloud metadata endpoints.
6. Internal APIs.

7.4 SSRF and cloud security

This vulnerability is particularly important in cloud environments.

Some cloud platforms expose instance metadata services containing information about the running virtual machine.

If an application has SSRF and the metadata service is insufficiently protected, an attacker may attempt to access sensitive metadata or temporary credentials.

For example, cloud environments may use metadata services to provide workload identity information.

7.5 Types of SSRF

Type	Explanation
Basic SSRF	Server makes an unintended request and the result may be visible to the attacker
Blind SSRF	Server makes the request, but the response is not directly returned
Blind SSRF with out-of-band detection	Tester identifies the request through an external interaction monitoring service

7.6 Prevention

- Use strict destination allowlists.

- Validate URLs after parsing them.
- Restrict access to private and internal IP ranges.
- Disable unnecessary URL schemes.
- Prevent DNS rebinding and DNS resolution bypasses.
- Use network-level egress restrictions.
- Protect cloud metadata services.
- Do not rely only on checking the hostname before connecting.

Important: SSRF is not limited to cloud environments. Any server that makes requests to attacker-influenced destinations may be vulnerable.

Topic 8: File Upload Vulnerabilities

Understanding File Upload Vulnerabilities Through Hands-On Practice | by Vedant Girawale | Medium
medium.com

8.1 What are File Upload Vulnerabilities?

File Upload Vulnerabilities occur when a web application does not properly validate, store or handle files uploaded by users.

Attackers may upload files that cause unintended behavior or expose sensitive resources.

8.2 Common file upload weaknesses

Weakness	Explanation
Extension validation failure	Application accepts dangerous file extensions
MIME type trust	Application trusts a client-provided content type
Insufficient content validation	Application does not inspect actual file contents
Unsafe file storage	Files are stored in executable web directories
Predictable filenames	Attackers can guess uploaded file locations
Excessive file size	Large uploads can cause resource exhaustion
Unsafe file processing	Image or document processing introduces vulnerabilities

8.3 Real-world example

Suppose a website allows users to upload profile pictures.

The application checks only the filename extension.

An attacker may attempt to upload a file containing executable content disguised as an image.

If the server stores the file in an executable location and processes it unsafely, this may lead to serious compromise.

8.4 Possible impacts

- Remote code execution.
- Stored XSS.
- Malware distribution.
- Denial of service.
- Sensitive file exposure.
- Unauthorized file access.

8.5 Prevention

- Use an allowlist of permitted file types.
- Validate actual file content, not only extension or MIME type.
- Generate random server-side filenames.
- Store uploaded files outside the web root.
- Disable execution in upload directories.
- Apply file size restrictions.
- Scan files where appropriate.
- Restrict access to uploaded content.

Important: Renaming a dangerous file to `.jpg` does not make it safe.

Topic 9: Path Traversal

Back to Basics: Directory Traversal | Fastly
fastly.com

9.1 What is Path Traversal?

Path Traversal is a vulnerability that allows attackers to access files or directories outside the intended application directory by manipulating file paths.

It is also called Directory Traversal.

9.2 How does it work?

Consider an application that allows users to download documents.

The application may accept a filename as a parameter.

Example:

```
https://example.com/download?file=report.pdf
```

The application uses the supplied filename to locate the document.

If the application does not properly restrict file paths, an attacker may attempt to navigate outside the intended directory.

The common path traversal sequence is:

`../`

It represents moving one directory level upward in a relative filesystem path.

9.3 Real-world example

Suppose a website stores documents in:

`/var/www/app/documents/`

The application should allow users to access only files inside this directory.

A vulnerable application might allow a manipulated path to reach files outside the intended directory, potentially exposing configuration files or other sensitive resources.

9.4 Possible impacts

- Reading sensitive files.
- Exposing application configuration.
- Accessing source code.
- Revealing credentials.
- Modifying files if unsafe write functionality is also present.

9.5 Prevention

- Avoid directly trusting user-supplied file paths.
- Use server-side file identifiers instead of arbitrary paths.
- Canonicalize and normalize paths.
- Verify that resolved paths remain inside the permitted directory.
- Apply least-privilege filesystem permissions.
- Restrict file access to approved locations.

Topic 10: Insecure Deserialization

[React2Shell Explained: How It Works & How to Fix It Now | Medium
medium.com]

10.1 What is Serialization?

Serialization is the process of converting an object or data structure into a format that can be stored or transmitted.

10.2 What is Deserialization?

Deserialization is the process of converting serialized data back into an object or data structure.

Example:

An application converts a user object into serialized data before storing it.

Later, the application deserializes that data to recreate the original object.

10.3 What is Insecure Deserialization?

Insecure Deserialization occurs when an application processes untrusted serialized data without adequate security controls.

An attacker may manipulate serialized data to influence application behavior.

10.4 How does it work?

1. An application receives serialized data.
2. The application trusts the data without sufficient validation.
3. The application deserializes the data.
4. During deserialization, unsafe object behavior may be triggered.
5. Depending on the technology and vulnerability, the attacker may achieve unauthorized actions or code execution.

10.5 Real-world example

Imagine an application that stores user session information as a serialized object.

If the application accepts client-controlled serialized data and deserializes it unsafely, an attacker may attempt to manipulate the object's properties or behavior.

10.6 Possible impacts

- Remote code execution.
- Authentication bypass.
- Privilege escalation.
- Data manipulation.
- Denial of service.

10.7 Prevention

- Avoid deserializing untrusted data.
- Prefer simple data formats such as JSON when suitable.
- Validate data structures and values.
- Use safe deserialization mechanisms.
- Restrict the types of objects that can be instantiated.
- Protect serialized data with appropriate integrity controls.
- Keep frameworks and libraries updated.

Important: JSON is not automatically safe. A JSON-based application can still suffer from unsafe object binding, mass assignment and other input-handling vulnerabilities.

Topic 11: Cryptographic Failures

[What are Public and Private Keys?
coingecko.com]

11.1 What are Cryptographic Failures?

Cryptographic Failures occur when an application fails to properly protect sensitive information through appropriate cryptographic mechanisms.

They were previously commonly discussed under the term Sensitive Data Exposure.

11.2 Common cryptographic weaknesses

Weakness	Explanation
Weak encryption algorithms	Uses outdated or insecure cryptographic algorithms
Weak key management	Encryption keys are stored or managed insecurely
Plaintext passwords	Passwords are stored without secure password hashing
Hardcoded secrets	Cryptographic keys are embedded in source code
Missing encryption in transit	Sensitive information is transmitted without adequate protection
Insecure random numbers	Predictable values are used for security-sensitive operations
Weak password hashing	Uses fast or outdated hashing algorithms for password storage

11.3 Encryption vs Hashing

These concepts are different and must not be confused.

Encryption	Hashing
Converts readable data into ciphertext	Converts input into a fixed-length digest
Designed to be reversible with the correct key	Designed to be one-way
Requires an encryption/decryption key	Does not require a decryption key
Protects confidentiality	Commonly supports integrity checks and password verification

11.4 Symmetric vs Asymmetric Encryption

Symmetric encryption	Asymmetric encryption
Uses the same secret key for encryption and decryption	Uses a public-private key pair
Generally faster	Generally more computationally expensive
Example: AES	Example: RSA

Key distribution must be secured	Public key can be shared openly

11.5 Real-world example

Suppose an e-commerce website stores customer passwords in plaintext.

If an attacker obtains database access, all stored passwords become immediately readable.

A secure application should use a dedicated password hashing algorithm such as Argon2id, with appropriate parameters and unique salts.

11.6 Prevention

- Use modern, trusted cryptographic algorithms.
- Use TLS to protect data in transit.
- Use Argon2id or another appropriate password hashing algorithm.
- Securely store and rotate cryptographic keys.
- Never hardcode sensitive secrets.
- Use cryptographically secure random number generators.
- Avoid outdated protocols and algorithms.
- Encrypt sensitive information at rest where appropriate.

Topic 12: Business Logic Vulnerabilities

Manual vs Automated Penetration Testing: The 2025 Guide
deepstrike.io

12.1 What are Business Logic Vulnerabilities?

Business Logic Vulnerabilities occur when an application allows users to misuse legitimate features because the application's business rules or workflow restrictions are incorrectly designed or enforced.

These vulnerabilities are different from traditional technical vulnerabilities.

They often exploit the way an application is intended to operate.

12.2 Real-world examples

Example 1: E-commerce price manipulation

Suppose an online shopping website allows customers to submit product prices through the browser.

If the server trusts the submitted price instead of calculating the price independently, an attacker may manipulate the purchase amount.

Example 2: Coupon abuse

Suppose an application allows a discount coupon to be used multiple times even though the business rules permit only one use per customer.

An attacker may repeatedly exploit the discount.

Example 3: Payment workflow bypass

Suppose an application allows an order to be marked as completed without verifying successful payment.

A user may obtain a product without completing the required payment process.

Example 4: Quantity manipulation

Suppose a promotional offer allows only one discounted item per customer.

If the server does not enforce that restriction, users may purchase multiple discounted items.

12.3 Common business logic vulnerabilities

Vulnerability	Explanation
Price manipulation	Changing product or transaction prices
Coupon abuse	Using discounts beyond permitted limits
Workflow bypass	Skipping required application steps
Race conditions	Exploiting simultaneous operations to violate business rules
Limit bypass	Circumventing purchase, withdrawal or usage limits
Trusting client-side values	Accepting sensitive values from the browser without server verification

12.4 Prevention

- Enforce business rules on the server.
- Recalculate prices and totals server-side.
- Validate transaction states.
- Use atomic database operations where appropriate.
- Apply transaction limits.
- Prevent repeated use of one-time benefits.
- Test unusual application workflows.
- Review business requirements for security weaknesses.

Important: Business logic vulnerabilities frequently require manual testing. Automated vulnerability scanners may not understand the intended business rules.

4. Important definitions — Revision notes

These definitions are suitable for exams, interviews and quick revision.

Term	Definition

OWASP	An organization that provides resources and guidance to improve software security.
SQL Injection	A vulnerability that allows attackers to manipulate database queries through unsafe input handling.
XSS	A vulnerability that allows malicious scripts to execute in a victim's browser within a website's context.
Stored XSS	XSS in which malicious content is stored and later executed when users view it.
Reflected XSS	XSS in which malicious input is reflected unsafely in a server response.
DOM-Based XSS	XSS caused by unsafe client-side JavaScript handling of data in the DOM.
CSRF	An attack that tricks an authenticated user's browser into sending an unwanted request.
Broken Access Control	A vulnerability that allows users to perform actions beyond their permissions.
IDOR	A vulnerability in which users can access other users' objects because ownership checks are missing.
Authentication Failure	A weakness in verifying user identity or managing authentication securely.
Security Misconfiguration	An insecure configuration of an application, server or infrastructure.
SSRF	A vulnerability that tricks a server into making unintended requests.
File Upload Vulnerability	A weakness in how applications validate, store or process uploaded files.
Path Traversal	A vulnerability that allows access to files outside an intended directory.
Serialization	Converting an object or data structure into a transferable or storable format.
Deserialization	Reconstructing an object or data structure from serialized data.
Insecure Deserialization	Unsafe processing of untrusted serialized data.
Cryptographic Failure	Improper use or implementation of cryptography that exposes sensitive information.
Business Logic Vulnerability	A weakness in application rules or workflows that allows unintended behavior.
Privilege Escalation	Gaining permissions beyond those originally assigned.

5. Important differences — Comparison tables

These comparisons are particularly important for cybersecurity interviews and penetration-testing assessments.

5.1 SQL Injection vs XSS

SQL Injection	XSS
Targets database query handling	Targets browser-side script execution
Exploits unsafe SQL construction	Exploits unsafe handling of untrusted content
Can expose or modify database information	Can execute malicious scripts in users' browsers
Primary defense: parameterized queries	Primary defense: context-aware output encoding

5.2 XSS vs CSRF

XSS	CSRF
Executes malicious scripts in a website's context	Causes an authenticated browser to send unwanted requests
Exploits unsafe content handling	Exploits trust in authenticated browser requests
Can affect confidentiality and integrity	Primarily targets unauthorized actions
Output encoding and safe DOM APIs help prevent it	CSRF tokens and SameSite cookies help prevent it

5.3 SSRF vs CSRF

SSRF	CSRF
Tricks the server into making requests	Tricks the user's browser into making requests
Server is the request-making system	Victim's browser is the request-making system
Can target internal services	Can target websites where the victim is authenticated
Important in cloud security	Important in web authentication security

5.4 Authentication vs Access Control

Authentication	Access Control
Verifies identity	Enforces permissions
Checks who is accessing	Checks what they can access
Example: Login verification	Example: Restricting admin functions
Weakness may cause account compromise	Weakness may cause unauthorized access

5.5 Path Traversal vs File Upload Vulnerability

Path Traversal	File Upload
Manipulates file paths	Manipulates uploaded files
May expose files outside permitted directories	May introduce malicious or unsafe files
Targets file access restrictions	Targets upload and storage restrictions
Prevention: path normalization and access restrictions	Prevention: content validation and safe storage

5.6 Technical vs Business Logic Vulnerabilities

Technical vulnerability	Business logic vulnerability
Caused by technical implementation weaknesses	Caused by incorrect business rules or workflow enforcement
Often follows recognizable vulnerability patterns	Often specific to an application
Can sometimes be detected by automated scanners	Often requires manual investigation
Example: SQL Injection	Example: Coupon abuse

6. Cybersecurity tools used in this module

These tools are commonly used in web application penetration testing.

Como o Burp Suite pode levar seu pentest para o próximo nível

1. Burp Suite

- Intercept HTTP requests.
- Modify request parameters.
- Analyze authentication and authorization.
- Test web application vulnerabilities.

Website: PortSwigger Burp Suite

File:OWASP ZAP logo.svg - Wikimedia Commons

2. OWASP ZAP

- Automated web vulnerability scanning.
- Passive security analysis.
- Manual request testing.

Website: OWASP ZAP

Sqlmap Logo & Brand Assets (SVG, PNG and vector) - Brandfetch

3. SQLmap

- Helps detect and test SQL Injection.
- Supports multiple database technologies.
- Automates certain SQL Injection testing tasks.

Website: SQLmap

Nmap Download - Netzwerkscanner für Profis

4. Nmap

- Network discovery.
- Port scanning.
- Service identification.
- Security assessment of authorized infrastructure.

Website: Nmap

Интеграция Test IT с Postman | Test IT

5. Postman

- Send API requests.
- Inspect JSON responses.
- Test API authentication.
- Examine API behavior.

Website: Postman

7. Practical exercises — Your cybersecurity lab

Hands-on learning

You should not learn these vulnerabilities only through definitions. You need to observe how they work.

Use intentionally vulnerable applications or applications you own and have permission to test.

7.1 Recommended vulnerable practice environments

OWASP Juice Shop | Probably the most modern and sophisticated insecure web application for security trainings, awareness demos and CTFs.

1. OWASP Juice Shop

Recommended first lab

- SQL Injection

- XSS
- Broken Access Control
- Authentication problems
- Business logic vulnerabilities

Official project
Damn Vulnerable Web App download | SourceForge.net

2. DVWA

- SQL Injection
- XSS
- CSRF
- File Upload
- Brute Force

GitHub repository
Web Security Academy: Free Online Training from PortSwigger

3. PortSwigger Web Security Academy

- Guided vulnerability labs.
- SQL Injection labs.
- XSS labs.
- CSRF labs.
- SSRF labs.
- Access control labs.
- Advanced web security labs.

Free training
Web Security Academy
What is WebGoat and How to setup WebGoat in Linux? | by Vishnu Shivalal P | Medium

4. OWASP WebGoat

- Interactive web security lessons.
- Authentication vulnerabilities.
- Injection attacks.
- Access control.
- Secure coding exercises.

Official project

7.2 Practical assignment

Complete these exercises in the order given.

No.	Exercise	Expected learning
1	Identify SQL Injection in a vulnerable lab	Understand unsafe database queries
2	Identify Stored and Reflected XSS	Understand script injection
3	Compare XSS and CSRF	Understand browser-based attacks
4	Test user-to-user access restrictions	Identify IDOR
5	Examine login protections	Understand authentication weaknesses
6	Identify insecure server configurations	Understand security misconfiguration
7	Study a controlled SSRF lab	Understand server-side request behavior
8	Test file upload restrictions	Understand upload validation
9	Explore a Path Traversal lab	Understand filesystem access restrictions
10	Study unsafe deserialization	Understand serialized object risks
11	Examine password storage	Understand hashing and encryption
12	Test an e-commerce workflow	Identify business logic weaknesses

8. Practical project — Web Vulnerability Assessment

Module project

Project title: Web Application Vulnerability Assessment Report

Objective

Perform an authorized security assessment of OWASP Juice Shop or another deliberately vulnerable application.

Project stages

Stage 1 — Reconnaissance

- Identify application pages.
- Identify input fields.
- Identify authentication mechanisms.
- Identify available endpoints.

Stage 2 — Vulnerability testing

Test the application for:

- SQL Injection.
- XSS.
- CSRF.
- Broken Access Control.
- Authentication Failures.
- Security Misconfiguration.
- SSRF.
- File Upload Vulnerabilities.
- Path Traversal.
- Insecure Deserialization.
- Cryptographic Failures.
- Business Logic Vulnerabilities.

Stage 3 — Documentation

For every confirmed vulnerability, record:

Field	Information
Vulnerability name	Name of the vulnerability
Location	Affected URL or endpoint
Severity	Critical, High, Medium or Low
Description	What the weakness is
Evidence	Screenshots or sanitized request/response
Impact	What an attacker could achieve
Root cause	Why the vulnerability exists
Recommendation	How to fix it

Stage 4 — Remediation

- Fix the identified vulnerabilities.
- Repeat the tests.
- Confirm whether the vulnerabilities are resolved.
- Prepare the final report.

A penetration tester must distinguish between a suspected vulnerability and a verified finding. Do not report a vulnerability as confirmed without sufficient evidence.

9. Key takeaways

1. SQL Injection targets unsafe database query construction.
2. XSS targets unsafe handling of content in a user's browser.
3. CSRF abuses authenticated browser requests.
4. Broken Access Control allows unauthorized actions or data access.
5. IDOR is an important example of broken object-level access control.
6. Authentication Failures can lead to account compromise.
7. Security Misconfiguration exposes weaknesses caused by incorrect settings.
8. SSRF makes a server perform unintended requests.
9. File Upload vulnerabilities arise from unsafe file handling.
10. Path Traversal allows access outside intended filesystem locations.
11. Insecure Deserialization can cause serious application compromise.
12. Cryptographic Failures expose sensitive information.
13. Business Logic vulnerabilities exploit weaknesses in application workflows.
14. Manual testing is essential because not every vulnerability can be reliably identified through automated scanners.
15. Every security finding must have evidence, impact analysis and remediation guidance.

10. Revision questions — Test your understanding

These questions progress from basic definitions to practical penetration-testing scenarios.

Module 2 examination

0/16 answered

Q1. What does OWASP stand for?

Open Web Application Security Project
Open Worldwide Application Security Project
Online Web Application Safety Program
Open Website Access Security Protocol

Q2. What is the primary cause of SQL Injection?

Weak internet connection
Unsafe construction of SQL queries using untrusted input
Missing HTTPS certificate
Weak Wi-Fi password

Q3. Which SQL Injection type uses response delays to infer database information?

Stored SQLi
Union-based SQLi
Time-based blind SQLi
Error-based SQLi

Q4. Which XSS type stores malicious content on the server?

Reflected XSS
Stored XSS
DOM-Based XSS
Blind XSS

Q5. What is the primary purpose of a CSRF token?

Encrypt database passwords
Help verify that a state-changing request is legitimate
Hide the server IP
Prevent SQL Injection

Q6. A user accesses another user's private profile by changing an ID in the URL. What vulnerability may exist?

SSRF
IDOR
SQL Injection
Cryptographic Failure

Q7. What is vertical privilege escalation?

Accessing another user's account at the same permission level
Gaining access to higher-privilege functionality
Changing an IP address
Reading browser cookies

Q8. What is a common security misconfiguration?

Using parameterized queries
Enabling debug mode in production
Using MFA
Disabling unnecessary services

Q9. What does SSRF exploit?

The victim's browser making requests
A server being tricked into making unintended requests
A weak Wi-Fi password
An incorrectly hashed password

Q10. Which cloud-related resource may be exposed through SSRF?

Keyboard settings
Cloud instance metadata service

Monitor brightness
Browser bookmarks

Q11. Why is checking only a file extension insufficient for secure uploads?

Extensions are always encrypted
Attackers may disguise the actual file content
Extensions cannot be changed
Browsers do not support extensions

Q12. What does ../ commonly represent in Path Traversal?

Move one directory upward
Delete a directory
Encrypt a directory
Create a directory

Q13. What is deserialization?

Encrypting a password
Converting serialized data back into an object or data structure
Deleting database records
Creating an HTTP request

Q14. Which algorithm is suitable for password hashing?

MD5
Plain SHA-1
Argon2id
Base64

Q15. An e-commerce application allows users to purchase products without verifying payment completion. What vulnerability may exist?

Business Logic Vulnerability
DNS spoofing
Path Traversal
Reflected XSS

Q16. Which is the primary defense against SQL Injection?

Parameterized queries
Hiding the login button
Disabling JavaScript
Changing the website logo
Submit examination

11. Module completion checklist

Your progress

0/12

Understand all 12 OWASP vulnerability topics
Understand the differences between SQL Injection, XSS and CSRF
Understand IDOR and privilege escalation
Understand authentication and security misconfiguration
Understand SSRF and cloud metadata risks
Understand file upload and Path Traversal
Understand insecure deserialization
Understand cryptographic failures
Understand business logic vulnerabilities
Complete at least 5 practical vulnerability labs
Prepare one vulnerability assessment report
Pass the revision examination

12. Recommended learning sequence

Do not rush into advanced exploitation before understanding the underlying vulnerability.

Order	Study topic	Recommended lab
1	SQL Injection	PortSwigger SQLi
2	XSS	PortSwigger XSS
3	CSRF	PortSwigger CSRF
4	Broken Access Control	PortSwigger Access Control
5	Authentication Failures	PortSwigger Authentication
6	Security Misconfiguration	OWASP Juice Shop
7	SSRF	PortSwigger SSRF
8	File Upload	PortSwigger File Upload
9	Path Traversal	PortSwigger Path Traversal
10	Insecure Deserialization	PortSwigger Deserialization
11	Cryptographic Failures	OWASP Juice Shop
12	Business Logic	PortSwigger Business Logic

My recommendation: Complete the PortSwigger labs alongside these notes. Reading about a vulnerability is not the same as being able to identify, reproduce, explain and remediate it.

Next module

Phase 8 — Module 3: Advanced Web Security

Topics:

1. DOM-Based XSS
2. Blind SQL Injection
3. Authentication Bypass
4. JWT Vulnerabilities
5. OAuth Security
6. API Authorization
7. GraphQL Security
8. Race Conditions
9. HTTP Request Smuggling
10. Web Cache Poisoning
11. Web Cache Deception

phase 8- module 3: advanced web security

1. Learning objectives

By the end of this module, you should be able to:

1. Understand advanced DOM-Based XSS vulnerabilities.
2. Identify and understand Blind SQL Injection.
3. Understand authentication bypass techniques.
4. Explain JSON Web Tokens (JWT) and their security weaknesses.
5. Understand OAuth 2.0 security.
6. Identify API authorization vulnerabilities.
7. Understand GraphQL architecture and security risks.
8. Understand race conditions and concurrent request vulnerabilities.
9. Explain HTTP Request Smuggling.
10. Understand Web Cache Poisoning.
11. Understand Web Cache Deception.
12. Perform advanced security assessments using Burp Suite and specialized testing labs.
13. Analyze complex vulnerabilities and recommend appropriate mitigations.

2. Topic-wise explanations

Topic 1: DOM-Based XSS

Mastering DOM-based XSS: Understanding Sources, Sinks, and Secure DOM Practices | by Ali El-Beltagy | Medium
medium.com

2.1 What is DOM-Based XSS?

DOM-Based Cross-Site Scripting is a type of XSS vulnerability that occurs when client-side JavaScript processes untrusted data and inserts or executes it in an unsafe way.

DOM stands for Document Object Model.

The DOM represents a web page as a structured document that JavaScript can access and modify.

2.2 How does DOM-Based XSS work?

Consider a website that reads information from a URL and displays it on the page.

For example:

```
https://example.com/search#keyword=security
```

The website's JavaScript reads the URL fragment and inserts the value into the page.

If the application uses an unsafe operation to insert that value as HTML, an attacker may be able to introduce executable content.

The browser may execute the injected script.

2.3 Important DOM concepts

Two important concepts are:

A. Sources

Sources are locations from which JavaScript obtains potentially untrusted data.

Examples:

- `location.search`
- `location.hash`
- `document.referrer`
- `window.name`
- `postMessage` event data

B. Sinks

Sinks are functions or properties where data is used in a way that may create a security risk.

Examples:

- `innerHTML`
- `outerHTML`
- `document.write()`
- `eval()`

Not every use of these APIs is automatically vulnerable. The risk depends on the data and how it is handled.

2.4 DOM-Based XSS example

Unsafe JavaScript:

```
const input = location.hash.substring(1);

document.getElementById("output").innerHTML = input;
```

The application reads data from the URL and inserts it as HTML.

A safer approach is:

```
const input = location.hash.substring(1);
document.getElementById("output").textContent = input;
```

`textContent` treats the supplied value as text rather than interpreting it as HTML.

2.5 Prevention

- Avoid unsafe DOM manipulation.
- Prefer `textContent` over `innerHTML` when displaying plain text.
- Use context-aware output encoding.
- Sanitize HTML when rich HTML content is required.
- Validate messages received through `postMessage`.
- Apply a suitable Content Security Policy.

2.6 Cybersecurity applications

Penetration testers examine:

- JavaScript source code.
- URL parameters and fragments.
- DOM sources and sinks.
- Unsafe HTML insertion.
- Client-side data handling.
- Browser-side security controls.

Topic 2: Blind SQL Injection

Blind SQL injection with conditional responses | by Kanav Arora | Jan, 2026 | Medium
medium.com

2.1 What is Blind SQL Injection?

Blind SQL Injection is a type of SQL Injection in which the application does not directly display database query results or useful database errors.

Instead, attackers infer information by observing changes in application behavior.

For example:

- Different responses.
- Different page content.
- Different status codes.
- Different response times.

2.2 Why is it called blind?

In ordinary SQL Injection, database information or error messages may appear directly in the application's response.

In Blind SQL Injection, the application hides those results.

The attacker therefore has to infer information indirectly.

2.3 Types of Blind SQL Injection

1. Boolean-Based Blind SQL Injection

This technique uses conditions that evaluate to true or false.

The tester observes whether the application responds differently depending on the result.

Example concept:

- Condition A: True.
- Condition B: False.
- Compare the application's responses.

If the responses differ consistently, the application may reveal information about how its database query is evaluated.

2. Time-Based Blind SQL Injection

This technique uses database response delays to infer whether a particular condition is true or false.

Example concept:

- One condition produces a normal response.
- Another condition causes a deliberate database delay.
- The tester compares response times.

2.4 Comparison

Boolean-based SQLi	Time-based SQLi
Observes true/false response differences	Observes response-time differences
Depends on distinguishable application behavior	Depends on measurable timing differences
May be faster when responses are clear	Can be slower and noisier
Network latency can affect interpretation	Network latency can complicate measurements

2.5 Real-world example

Imagine an online shopping website with a product filter.

The application accepts a category parameter and uses it in a database query.

The website does not display database errors or query results.

However, manipulated inputs cause consistent changes in the returned page.

A tester may investigate whether these differences indicate a blind SQL Injection vulnerability.

2.6 Prevention

- Use parameterized queries.
- Use prepared statements.
- Avoid dynamic SQL construction with untrusted input.
- Apply least-privilege database permissions.
- Avoid exposing database errors.
- Validate and constrain inputs appropriately.

Important: Hiding error messages does not fix SQL Injection. It only reduces one way of observing the vulnerability.

Topic 3: Authentication Bypass

CakePHP Application Cybersecurity Research - Forgotten Endpoint: Authentication bypass with /open prefix | Web Application Security Testing
zigrin.com

3.1 What is Authentication Bypass?

Authentication Bypass is a security vulnerability that allows someone to access protected functionality without successfully completing the required authentication process.

In simple terms, the application allows access without correctly verifying identity.

3.2 Common authentication bypass techniques

Technique	Explanation
SQL Injection	Manipulates authentication database queries
Weak session validation	Accepts invalid or improperly validated sessions
Default credentials	Uses unchanged default login credentials
Authentication logic flaws	Exploits incorrect login verification logic
JWT validation failure	Accepts improperly validated tokens
Password reset weakness	Exploits insecure account recovery
Insecure direct access	Protected pages or APIs fail to enforce authentication

3.3 Real-world example

Suppose an application has:

- A login page.

- A user dashboard.
- An administrator dashboard.

The developer hides the administrator dashboard link from normal users.

However, the server does not verify whether the user has administrative permissions when accessing the administrator endpoint directly.

A normal user may access administrative functionality.

This is an authentication or access-control design failure, depending on the precise cause.

3.4 Authentication bypass vs authorization bypass

Authentication bypass	Authorization bypass
Avoids or defeats identity verification	Defeats permission checks
Attacker may access an application without valid authentication	An authenticated user accesses unauthorized functionality
Example: Invalid session accepted	Example: Normal user accesses admin functions

3.5 Prevention

- Enforce authentication on the server.
- Validate sessions on every protected request.
- Apply authorization checks separately.
- Secure password recovery.
- Use MFA for sensitive accounts.
- Validate tokens correctly.
- Avoid relying on frontend restrictions.

3.6 Cybersecurity applications

Security professionals test:

- Login mechanisms.
- Password reset workflows.
- Session management.
- Protected API endpoints.
- Administrative functions.
- Token validation.

Topic 4: JWT Vulnerabilities

[Mastering API Security: JWT (JSON Web Tokens) Authentication in .NET Core | by Md Julakadar | Medium
medium.com]

4.1 What is JWT?

JWT stands for JSON Web Token.

It is a compact format used to represent claims that can be transmitted between parties.

JWTs are commonly used in authentication and authorization systems.

4.2 JWT structure

A typical signed JWT consists of three parts separated by dots.

`HEADER.PAYLOAD.SIGNATURE`

The three components are:

1. Header.
2. Payload.
3. Signature.

4.3 JWT example

A simplified JWT header:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

A simplified payload:

```
{
  "userId": 101,
  "role": "user",
  "exp": 1799999999
}
```

Explanation:

Field	Meaning
alg	Signing algorithm
typ	Token type
userId	User identifier

role	User role claim
exp	Token expiration time

The signature helps the receiving system verify the integrity and authenticity of the token.

4.4 Important JWT vulnerabilities

1. Algorithm confusion

A system may incorrectly handle the token's declared algorithm or accept an unexpected algorithm.

2. None algorithm vulnerability

A vulnerable implementation may accept unsigned tokens where a valid signature should be required.

3. Weak signing secret

An attacker may be able to guess or crack a weak HMAC signing secret.

4. Signature validation failure

An application may accept tokens without correctly verifying their signatures.

5. Expiration validation failure

The application may fail to enforce token expiration.

6. Sensitive information exposure

Developers may place passwords, secrets or sensitive personal information in the JWT payload.

7. Key confusion

An incorrectly implemented verifier may confuse symmetric and asymmetric cryptographic key handling.

4.5 Important: JWT encoding vs encryption

JWT payloads are commonly Base64URL-encoded, not encrypted.

Anyone who obtains an ordinary signed JWT can generally decode and read its header and payload.

A signed JWT protects integrity when correctly validated. It does not automatically provide confidentiality.

Encrypted JWTs use a different format called JWE (JSON Web Encryption).

4.6 Prevention

- Use trusted JWT libraries.
- Explicitly allow only expected algorithms.
- Verify signatures.
- Validate issuer and audience claims.
- Check expiration and not-before claims.

- Use strong signing keys.
- Protect signing keys from exposure.
- Avoid placing sensitive secrets in ordinary JWT payloads.
- Implement appropriate token expiration and revocation strategies.

4.7 Cybersecurity applications

JWT security testing is essential when assessing:

- Single-page applications.
- Mobile application backends.
- REST APIs.
- Cloud-based authentication systems.
- Microservices.
- Identity providers.

Topic 5: OAuth Security

OAuth 2.0 for Identity Governance: A 2026 Security Guide
credentialgovernance.avatier.com

5.1 What is OAuth?

OAuth 2.0 is an authorization framework that allows an application to obtain limited access to resources on another service without directly receiving the user's password.

For example:

You visit a website and select Continue with Google.

The website can request permission to access certain information from your Google account without asking you to give it your Google password.

Important: OAuth is primarily an authorization framework, not an authentication protocol. OpenID Connect (OIDC) adds an identity layer on top of OAuth 2.0.

5.2 Main components of OAuth

Component	Meaning
Resource Owner	User who owns the protected information
Client	Application requesting access
Authorization Server	Server that authenticates the user and issues authorization tokens
Resource Server	Server hosting protected resources
Access Token	Credential used to access permitted resources

Refresh Token	Credential used to obtain new access tokens

5.3 How OAuth works

Consider a user connecting a third-party application to a Google account.

1. The application redirects the user to the authorization server.
2. The user authenticates with the provider.
3. The user is asked to approve requested permissions.
4. The authorization server redirects the browser with an authorization code.
5. The client exchanges the code for tokens through the token endpoint.
6. The application uses the access token to request permitted resources.
7. The resource server validates the token and permissions.

5.4 Important OAuth security vulnerabilities

Vulnerability	Explanation
Redirect URI manipulation	Attacker abuses incorrect redirect URI validation
CSRF in authorization flow	Authorization requests are not properly bound to the initiating user session
Missing PKCE	Public clients lack protection against authorization code interception
Authorization code leakage	Authorization codes are exposed to unauthorized parties
Token leakage	Access tokens are exposed through insecure storage or transmission
Scope abuse	Application requests or receives excessive permissions
State parameter weakness	Application fails to validate the authorization response against the initiating request

5.5 What is PKCE?

PKCE stands for Proof Key for Code Exchange.

It is an OAuth security extension that protects the authorization code flow, particularly for public clients such as mobile and browser-based applications.

It uses:

- Code verifier: A secret value generated by the client.
- Code challenge: A derived value sent during the authorization request.

The authorization server checks the verifier during the token exchange.

This makes it harder for an attacker who intercepts an authorization code to exchange it successfully.

5.6 Prevention

- Use Authorization Code Flow with PKCE.
- Validate redirect URIs exactly.
- Validate the state parameter.
- Use appropriate OAuth scopes.
- Protect access and refresh tokens.
- Use HTTPS.
- Follow current OAuth security best practices.

Reference: OAuth 2.0 documentation.

Topic 6: API Authorization

What Is BOLA? | Akamai
akamai.com

6.1 What is API Authorization?

API Authorization is the process of checking whether a user, application or service has permission to perform a particular operation on an API resource.

Authentication identifies the caller.

Authorization determines what the caller is allowed to do.

6.2 Types of API authorization

1. Role-Based Access Control (RBAC)

Permissions are assigned according to user roles.

Example:

Role	Permission
Normal user	View own profile
Moderator	Manage reported content
Administrator	Manage users and system settings

2. Attribute-Based Access Control (ABAC)

Access decisions are based on attributes such as:

- User identity.
- Resource ownership.
- Location.

- Time.
- Department.
- Resource classification.

3. Object-Level Authorization

Checks whether a particular user is allowed to access a particular object.

Example:

A user should only retrieve orders belonging to their own account unless additional permissions exist.

6.3 Common API authorization vulnerabilities

Vulnerability	Explanation
BOLA	Accessing another user's object without authorization
BFLA	Accessing functions without the required permission
Privilege escalation	Obtaining higher permissions
Missing ownership checks	Server fails to verify resource ownership
Excessive data exposure	API returns information beyond what the caller needs

6.4 Real-world example

Suppose a banking API has:

```
GET /api/accounts/101
```

```
GET /api/accounts/102
```

A user is authorized to access account 101.

If the server allows that user to retrieve account 102 simply by changing the account ID, the API has a potential Broken Object Level Authorization vulnerability.

6.5 Prevention

- Verify permissions on every API operation.
- Check object ownership on the server.
- Use least-privilege permissions.
- Separate authentication from authorization.
- Validate user roles and attributes.
- Avoid trusting user IDs or roles supplied by the client.
- Test every sensitive endpoint with different user roles.

Topic 7: GraphQL Security

The Ultimate Guide to GraphQL: From Zero to Production Mastery | by Harshavardhan Mamidipaka | Feb, 2026 | Medium
medium.com

7.1 What is GraphQL?

GraphQL is a query language for APIs and a runtime for executing those queries against a defined data schema.

It allows clients to request specific data rather than receiving a fixed response structure.

GraphQL was developed by Meta.

Official website: GraphQL Official Website.

7.2 How GraphQL works

A typical GraphQL API exposes a schema that defines:

- Available objects.
- Fields.
- Relationships.
- Queries.
- Mutations.
- Subscriptions.

7.3 Three important GraphQL operations

Operation	Purpose	Example
Query	Retrieve data	Get user information
Mutation	Modify data	Create or update a user
Subscription	Receive ongoing updates	Live notifications

7.4 Example GraphQL query

```
query {
  user(id: 101) {
    id
    name
    email
  }
}
```

This requests selected fields about user 101.

Example response:

```
{
  "data": {
    "user": {
      "id": 101,
      "name": "Rahul",
      "email": "rahul@example.com"
    }
  }
}
```

7.5 Common GraphQL vulnerabilities

Vulnerability	Explanation
Introspection exposure	Schema information reveals application structure
Broken authorization	Users access unauthorized fields or objects
Excessive query depth	Deeply nested queries consume server resources
Query complexity abuse	Expensive queries overload the server
Batching abuse	Multiple operations are submitted to exploit weak controls
Injection	Unsafe resolver or database handling introduces injection vulnerabilities
Sensitive field exposure	API returns confidential information

7.6 Real-world example

Suppose a GraphQL API allows a user to request:

```
query {
  user(id: 102) {
    name
    email
    privateDocuments
  }
}
```

If the server does not check whether the requesting user is authorized to access user 102's private documents, the application has a broken authorization vulnerability.

7.7 Prevention

- Enforce authorization at resolver or service level.
- Apply query depth restrictions.

- Implement query complexity limits.
- Use rate limiting.
- Restrict sensitive fields.
- Validate inputs.
- Monitor expensive operations.
- Disable or restrict introspection in production when appropriate.

Important: Disabling introspection does not replace proper authorization or other security controls.

Topic 8: Race Conditions

Race conditions | Web Security Academy
portswigger.net

8.1 What is a Race Condition?

A Race Condition occurs when the outcome of an operation depends on the timing or ordering of multiple concurrent operations.

In cybersecurity, attackers may exploit race conditions to cause an application to behave incorrectly.

8.2 How does it work?

Consider an online shopping application.

A user has one coupon that can only be used once.

The application performs these operations:

1. Check whether the coupon has already been used.
2. Apply the discount.
3. Mark the coupon as used.

If two requests execute simultaneously, both may check the coupon before either request marks it as used.

Both requests may receive the discount.

This is a race condition.

8.3 Common race condition scenarios

- Double spending.
- Duplicate transactions.
- Reusing one-time coupons.
- Multiple withdrawals.
- Duplicate account operations.
- Circumventing purchase limits.

- Incorrect inventory management.

8.4 Important concept: TOCTOU

TOCTOU stands for Time-of-Check to Time-of-Use.

It occurs when an application checks a condition and later uses a resource, but the condition changes between those two operations.

Example:

A system checks whether a file is safe, but the file is replaced before the system uses it.

8.5 Prevention

- Use database transactions.
- Apply appropriate locking mechanisms.
- Use atomic operations.
- Enforce unique constraints.
- Implement idempotency keys where appropriate.
- Recheck important conditions during the actual transaction.
- Avoid relying on separate, unprotected check-and-update operations.

8.6 Cybersecurity applications

Race condition testing is especially important in:

- Banking applications.
- Payment gateways.
- E-commerce systems.
- Ticket booking systems.
- Digital wallets.
- Subscription platforms.

Topic 9: HTTP Request Smuggling

HTTP Request Smuggling: как особенности в обработке HTTP-заголовков приводят к атакам CL.TE и TE.CL / Хабр
habr.com

9.1 What is HTTP Request Smuggling?

HTTP Request Smuggling is a vulnerability that occurs when two HTTP processing systems interpret the boundaries of HTTP requests differently.

For example, a frontend proxy and a backend server may disagree about where one request ends and the next begins.

This disagreement can allow an attacker to interfere with how subsequent requests are processed.

9.2 How does it happen?

Many web applications use multiple server components:

1. Client.
2. Reverse proxy or load balancer.
3. Backend web server.
4. Application server.

If these components interpret HTTP message boundaries differently, request desynchronization can occur.

9.3 Important HTTP headers

Two headers are particularly important:

Content-Length (CL)

Specifies the length of the message body in bytes.

Transfer-Encoding (TE)

Specifies the transfer coding applied to the message body, commonly including chunked transfer encoding.

9.4 Types of request smuggling

Type	General explanation
CL.TE	Frontend prioritizes Content-Length; backend prioritizes Transfer-Encoding
TE.CL	Frontend prioritizes Transfer-Encoding; backend prioritizes Content-Length
TE.TE	Both systems process Transfer-Encoding differently because of implementation differences

These labels describe different parser disagreement scenarios.

9.5 Potential impacts

- Request desynchronization.
- Unauthorized request interference.
- Bypassing security controls.
- Accessing unintended responses.
- Cache poisoning in certain architectures.
- Interference with other users' requests.

9.6 Prevention

- Standardize HTTP parsing across infrastructure.

- Reject ambiguous or malformed HTTP requests.
- Normalize requests at the frontend.
- Configure proxies and backend servers consistently.
- Patch affected server components.
- Avoid inconsistent handling of Content-Length and Transfer-Encoding.

Important: Request smuggling is an advanced topic. Testing it against shared or production systems can affect other users' requests. Use isolated training environments.

Topic 10: Web Cache Poisoning

Is Your Application Vulnerable to Web Cache Poisoning? – Hackmanit
hackmanit.de

10.1 What is Web Cache Poisoning?

Web Cache Poisoning is a vulnerability in which an attacker causes a shared web cache to store a manipulated response that may later be delivered to other users.

10.2 What is a web cache?

A web cache temporarily stores responses so that future requests can be served faster.

Examples include:

- CDN caches.
- Reverse proxy caches.
- Browser caches.
- Application caches.

10.3 How does cache poisoning work?

1. An attacker identifies a request parameter or header that influences the server response.
2. The attacker sends a manipulated request.
3. The server produces a response influenced by that input.
4. The cache stores the response under a cache key that does not adequately distinguish the manipulated request.
5. Another user requests the same cached resource.
6. The user may receive the poisoned response.

10.4 Real-world example

Suppose a website generates a response using a particular HTTP header.

However, the caching system does not include that header in its cache key.

An attacker manipulates the header and causes the cache to store an unwanted response.

Other users may then receive that response.

10.5 Potential impacts

- Content injection.
- Malicious redirects.
- Website defacement.
- Delivery of unwanted content.
- Exposure of cached sensitive information.

10.6 Prevention

- Design cache keys correctly.
- Avoid allowing untrusted input to influence cached responses unsafely.
- Use appropriate Cache-Control policies.
- Separate personalized responses from shared cache content.
- Validate request headers and parameters.
- Test cache behavior with different request variations.

Topic 11: Web Cache Deception

Web cache deception | Web Security Academy
portswigger.net

11.1 What is Web Cache Deception?

Web Cache Deception is an attack in which an attacker tricks a caching system into storing a response containing private or sensitive user information.

Unlike cache poisoning, the main objective is often to make private information accessible through a shared cache.

11.2 How does it work?

Consider a website with a personalized account page.

The user visits their account while authenticated.

The application returns private information.

An attacker tricks the user into visiting a specially crafted URL that the application interprets as an authenticated account page.

However, the cache interprets the URL as a static resource that can be cached.

If the cache stores the private response and later serves it to another user, sensitive information may be exposed.

11.3 Potential impacts

- Personal information exposure.
- Account information leakage.
- Sensitive cached responses.
- Privacy violations.
- Unauthorized disclosure of authenticated content.

11.4 Prevention

- Configure cache rules carefully.
- Do not cache sensitive authenticated responses in shared caches.
- Use appropriate `Cache-Control: private` or `no-store` directives.
- Ensure origin routing and cache behavior are consistent.
- Test unusual URL paths and file extensions.
- Separate personalized content from publicly cacheable resources.

11.5 Web Cache Poisoning vs Web Cache Deception

Web Cache Poisoning	Web Cache Deception
Attacker manipulates a response stored in the cache	Attacker causes a private response to be stored in the cache
Main objective: distribute a malicious or unwanted response	Main objective: expose private information
Focuses on response manipulation	Focuses on cache misclassification
Can affect many cache users	Can expose an authenticated user's information

3. Important definitions — Revision notes

Term	Definition
DOM	A structured representation of a web document that scripts can access and modify.
DOM-Based XSS	XSS caused by unsafe client-side handling of untrusted data.
Blind SQL Injection	SQL Injection in which information is inferred indirectly from application behavior.
Boolean-Based SQLi	Blind SQL Injection that uses true/false response differences.
Time-Based SQLi	Blind SQL Injection that uses response delays to infer database

	information.
Authentication Bypass	Unauthorized access caused by failure of authentication controls.
JWT	JSON Web Token, a compact format for representing claims between parties.
JWT Signature	Cryptographic value used to verify token integrity and authenticity.
OAuth 2.0	Authorization framework for obtaining limited access to protected resources.
PKCE	OAuth extension that protects authorization code exchanges.
API Authorization	Process of determining what an API caller is permitted to access or perform.
GraphQL	Query language and runtime for APIs.
Race Condition	Vulnerability caused by unsafe timing or ordering of concurrent operations.
TOCTOU	Time-of-Check to Time-of-Use vulnerability.
HTTP Request Smuggling	Vulnerability caused by inconsistent HTTP request parsing between systems.
Web Cache Poisoning	Attack that causes a cache to store an unintended or manipulated response.
Web Cache Deception	Attack that causes private responses to be incorrectly stored in a shared cache.
Cache Key	Information used by a caching system to identify a cached response.

4. Important differences — Comparison tables

4.1 Stored XSS vs DOM-Based XSS

Stored XSS	DOM-Based XSS
Malicious content is stored and later displayed	Client-side JavaScript handles untrusted input unsafely
Often involves server-side storage	Primarily involves browser-side processing
May affect many users	Affects users who trigger the vulnerable client-side behavior
Output encoding is important	Safe DOM operations and output handling are important

4.2 Normal SQL Injection vs Blind SQL Injection

Normal SQL Injection	Blind SQL Injection
Results may be directly visible	Results are inferred indirectly
Errors may expose information	Errors may be hidden
Often easier to recognize	Often requires more careful analysis
Can use UNION-based techniques	Commonly uses Boolean or timing behavior

4.3 JWT vs OAuth

JWT	OAuth 2.0
Token format	Authorization framework
Represents claims	Defines authorization flows
Can be signed or encrypted	Can use different token formats
Does not automatically establish authorization correctness	Provides a framework for delegated access

4.4 Authentication vs OAuth

Authentication	OAuth 2.0
Verifies identity	Grants delegated access
Answers who the user is	Answers what access an application has been granted
Example: Logging into an account	Example: Allowing an application to access selected account data
OIDC can provide identity information over OAuth	OAuth alone is not an authentication protocol

4.5 REST vs GraphQL

REST	GraphQL
Commonly uses multiple resource endpoints	Commonly uses one GraphQL endpoint
Response structure is defined by endpoints	Client selects requested fields
Uses HTTP methods for operations	Uses queries, mutations and subscriptions
May return more data than required	Can request specific fields
Authorization is commonly enforced at endpoint and resource level	Authorization must also be enforced at field and resolver level where necessary

4.6 Race Condition vs TOCTOU

Race Condition	TOCTOU
Broad category of timing-related problems	Specific type of race condition
Involves unsafe concurrent or ordering behavior	Involves a gap between checking and using a resource
Example: Duplicate coupon redemption	Example: File changes after a security check

4.7 Cache Poisoning vs Cache Deception

Cache Poisoning	Cache Deception
Manipulates cached responses	Misleads the cache into storing private responses
Can distribute attacker-controlled content	Can expose private user information
Focuses on integrity	Often focuses on confidentiality

5. Cybersecurity tools used in this module

These tools are useful for advanced web application security testing.

Burp Suite Professional

1. Burp Suite

- Advanced request interception.
- JWT inspection.
- Authentication testing.
- Request smuggling research.
- Cache behavior analysis.

Official website
Turbo Intruder: Embracing the billion-request attack | PortSwigger Research

2. Turbo Intruder

- Specialized HTTP request testing.
- High-volume request experiments.
- Race condition research in controlled environments.

GitHub repository
Five Myths About JWTs Debunked

3. JWT.io

- Decode JWTs.
- Inspect token headers and payloads.

- Understand JWT structure and signatures.

JWT debugger
GraphQL from the perspective of a Delphi developer | landgraf.dev

4. GraphQL Voyager

- Visualize GraphQL schemas.
- Understand relationships between types.
- Explore API structures.

GitHub repository
Visionco | services

5. Postman

- Test OAuth flows.
- Inspect API authorization.
- Test GraphQL requests.
- Analyze API responses.

Official website

6. Practical exercises — Advanced security lab

Hands-on practice

For this module, use PortSwigger Web Security Academy as your primary training platform.

PortSwigger Web Security Academy — Free labs

Exercise 1: DOM-Based XSS

Objective: Understand how JavaScript sources and sinks create XSS vulnerabilities.

Tasks:

1. Open a DOM XSS training lab.
2. Identify the source of untrusted input.
3. Identify the JavaScript sink.
4. Observe how the browser processes the input.
5. Compare unsafe and safe DOM handling.

Expected result: Understand how client-side JavaScript can introduce XSS.

Exercise 2: Blind SQL Injection

Tasks:

1. Open a Boolean-based SQL Injection lab.

2. Identify the parameter interacting with the database.
3. Observe differences between controlled true and false conditions.
4. Study a time-based SQL Injection lab.
5. Compare response behavior.

Expected result: Understand how database information can be inferred without direct query output.

Exercise 3: JWT Security

Tasks:

1. Obtain a test JWT from a training application.
2. Inspect its header.
3. Decode its payload.
4. Identify the algorithm.
5. Examine expiration and other claims.
6. Study how the server validates the signature.
7. Review the application's authorization checks.

Expected result: Understand token structure and common JWT security weaknesses.

Exercise 4: OAuth Security

Tasks:

1. Study an authorization code flow.
2. Identify the client and authorization server.
3. Observe the redirect URI.
4. Identify the state parameter.
5. Understand PKCE.
6. Study token exchange and token validation.

Expected result: Understand how OAuth authorization works and where security failures can occur.

Exercise 5: GraphQL Security

Tasks:

1. Open a GraphQL training application.
2. Inspect its schema.
3. Identify queries and mutations.
4. Examine access restrictions.
5. Test whether users can access unauthorized objects.

6. Study query depth and complexity restrictions.

Expected result: Understand GraphQL structure and authorization weaknesses.

Exercise 6: Race Conditions

Tasks:

1. Use a dedicated race condition lab.
2. Identify a business operation with a one-time restriction.
3. Understand how concurrent requests are processed.
4. Compare sequential and concurrent behavior.
5. Observe whether the business rule is enforced correctly.

Expected result: Understand how concurrent requests can violate application rules.

Exercise 7: HTTP Request Smuggling

Tasks:

1. Study frontend and backend HTTP parsing.
2. Understand Content-Length.
3. Understand Transfer-Encoding.
4. Compare CL.TE and TE.CL.
5. Complete a PortSwigger request smuggling lab.
6. Identify how parser disagreement creates desynchronization.

Expected result: Understand HTTP request boundary disagreements.

Exercise 8: Cache Security

Tasks:

1. Study cache keys.
2. Identify which request components influence caching.
3. Examine Cache-Control headers.
4. Complete a cache poisoning lab.
5. Complete a cache deception lab.
6. Compare their security impacts.

Expected result: Understand how caching systems can introduce confidentiality and integrity vulnerabilities.

7. Practical project — Advanced Web Application Security Assessment

Major module project

Project title: Advanced Web Application Penetration Testing and Security Report

Objective

Perform an authorized assessment of a deliberately vulnerable web application and document advanced security findings.

Project stages

Stage	Work
1. Reconnaissance	Identify application structure and endpoints
2. Client-side testing	Examine DOM behavior and JavaScript
3. Database testing	Investigate blind SQL Injection
4. Authentication testing	Examine login and session mechanisms
5. Token testing	Analyze JWT validation
6. OAuth testing	Examine authorization flows
7. API testing	Test object and function-level authorization
8. GraphQL testing	Examine schema and resolver security
9. Concurrency testing	Investigate race conditions
10. HTTP testing	Study request smuggling
11. Cache testing	Investigate poisoning and deception
12. Reporting	Document findings and remediation

Required project report

Your final report should contain:

1. Executive summary.
2. Application architecture.
3. Testing methodology.
4. Vulnerabilities identified.
5. Evidence and reproduction steps.
6. Severity classification.

7. Business impact.
8. Root cause.
9. Recommended remediation.
10. Retesting results.

8. Key takeaways

1. DOM-Based XSS originates from unsafe client-side data handling.
2. Blind SQL Injection reveals information indirectly through response behavior.
3. Authentication bypass defeats identity verification mechanisms.
4. JWTs contain a header, payload and signature in their ordinary signed form.
5. JWT payload encoding is not encryption.
6. OAuth 2.0 is an authorization framework.
7. PKCE protects authorization code exchanges.
8. API authorization ensures users access only permitted resources.
9. GraphQL requires careful schema, resolver and field-level authorization.
10. Race conditions exploit unsafe timing and concurrency.
11. TOCTOU is a specific type of race condition.
12. HTTP Request Smuggling exploits inconsistent HTTP parsing.
13. Cache Poisoning manipulates cached responses.
14. Cache Deception can expose private information through incorrect caching.
15. Advanced penetration testing requires understanding application architecture, not just running automated scanners.

10. Module completion checklist

Module progress

0/14

Understand DOM-Based XSS and JavaScript sources and sinks
Understand Boolean-Based and Time-Based Blind SQL Injection
Understand Authentication Bypass
Understand JWT structure and vulnerabilities
Understand OAuth 2.0 and PKCE
Understand API authorization and BOLA

Understand GraphQL queries, mutations and security
Understand Race Conditions and TOCTOU
Understand HTTP Request Smuggling
Understand Web Cache Poisoning
Understand Web Cache Deception
Complete practical training labs
Prepare the advanced penetration-testing report
Pass the module examination

Phase 8 — Web Application and API Security

Module 4: API Security

Beginner to advanced
8 topics

1. Learning Objectives

By the end of this module, you should be able to:

1. Understand how REST APIs work and where their security weaknesses occur.
2. Explain API authentication and authorization.
3. Implement and test rate limiting.
4. Validate and sanitize API input correctly.
5. Understand mass assignment vulnerabilities.
6. Identify and prevent Broken Object-Level Authorization (BOLA).
7. Manage API keys securely.
8. Test deliberately vulnerable APIs.
9. Build and security-test your own REST API.

Important: API security is not just about adding login functionality. An API can authenticate a user correctly and still expose another user's private data because its authorization checks are defective.

2. Topic-Wise Explanations

Topic 1: REST API Security

1.1 What is an API?

API stands for Application Programming Interface.

An API allows two software applications to communicate and exchange information.

For example:

- A mobile banking application communicates with a bank's server.
- A shopping website communicates with its product database.
- A weather application retrieves weather information from a weather service.

The API acts as the communication interface between these applications.

1.2 What is REST?

REST stands for Representational State Transfer.

It is an architectural style used to design web APIs.

A REST API generally communicates using HTTP and represents resources through URLs.

Example:

```
GET /api/users
GET /api/users/101
POST /api/users
PUT /api/users/101
PATCH /api/users/101
DELETE /api/users/101
```

Each URL identifies a resource or collection of resources.

1.3 How a REST API works

Hand drawn flat design api infographic | Free Vector
freepik.com

Consider a shopping application.

1. A user opens the application.
2. The application sends an HTTP request to the API.
3. The API receives the request.
4. The API authenticates the user, if required.
5. The API checks the user's permissions.

6. The API processes the request.
7. The API communicates with the database.
8. The API returns an HTTP response.

Example request:

```
GET /api/products/101 HTTP/1.1
Host: example.com
Accept: application/json
Authorization: Bearer <access_token>
```

Example response:

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "id": 101,
  "name": "Laptop",
  "price": 55000
}
```

1.4 Common REST API security risks

Security risk	Explanation
Broken authentication	Attackers can bypass or abuse authentication mechanisms.
Broken authorization	Users can access resources they are not permitted to access.
BOLA	Users can access another user's objects by manipulating identifiers.
Excessive data exposure	APIs return more information than the client needs.
Mass assignment	Attackers modify protected fields through submitted data.
Injection	Untrusted input affects database queries or other operations.
Missing rate limiting	Attackers can send excessive requests.
Security misconfiguration	Incorrect API settings create vulnerabilities.

1.5 Security best practices

- Use HTTPS for API communication.
- Authenticate users where authentication is required.
- Check authorization on every protected operation.
- Validate all incoming data.
- Return only necessary information.

- Apply rate limits.
- Avoid exposing internal error messages.
- Keep API dependencies updated.
- Log important security events.

Important definition

REST API Security: The process of protecting REST APIs against unauthorized access, data exposure, abuse, and attacks through appropriate authentication, authorization, validation, encryption, and monitoring.

Practical example

Imagine an API:

```
GET /api/accounts/101
```

It returns account information.

A user changes the account ID:

```
GET /api/accounts/102
```

If the API returns another person's account information without checking ownership, it has an authorization vulnerability.

The problem is not the URL or the ID itself. The problem is the missing permission check.

Cybersecurity application

Penetration testers inspect REST APIs to identify:

- Unauthorized access.
- Data leakage.
- Weak authentication.
- Broken authorization.
- Injection vulnerabilities.
- Improper request handling.

Topic 2: API Authentication

2.1 What is API authentication?

API authentication is the process of verifying the identity of a user, application, or service making an API request.

In simple words:

Authentication answers: Who are you?

For example, when you log in to a banking application, the bank verifies your identity before allowing access to your account.

2.2 Common API authentication methods

A. Username and password

The user provides a username and password.

The server verifies the credentials.

Example:

```
{
  "username": "user123",
  "password": "example-password"
}
```

The server should verify the password against a securely stored password hash, not a plaintext password.

Security risks:

- Weak passwords.
- Credential stuffing.
- Brute-force attacks.
- Insecure password storage.

B. API key authentication

An API key is a unique string used to identify or authorize an application or client.

Example:

```
GET /api/products
X-API-Key: <your_api_key>
```

API keys are commonly used for application-to-service communication.

They should not automatically be treated as proof of an individual user's identity.

C. Bearer token authentication

A client sends an access token in the HTTP Authorization header.

Example:

```
Authorization: Bearer <access_token>
```

The server validates the token before processing the request.

Bearer tokens are commonly used with OAuth 2.0 and other token-based authentication systems.

D. JWT authentication

JWT stands for JSON Web Token.

A JWT is a compact token format that can carry claims about a user or application.

A JWT commonly contains:

1. Header
2. Payload
3. Signature

Example structure:

`xxxxx.yyyyy.zzzzz`

The server must validate the signature, allowed algorithm, expiration, issuer, audience, and other relevant claims.

Important: A signed JWT is not automatically encrypted. Its payload can generally be decoded and read.

2.3 Authentication comparison

Method	Common use	Main concern
Username/password	User login	Password security
API key	Application identification	Key leakage
Bearer token	API access	Token theft
JWT	Token-based access	Incorrect validation

2.4 Authentication security best practices

- Use HTTPS.
- Store passwords using strong password hashing algorithms such as Argon2id or bcrypt.
- Use short-lived access tokens where appropriate.
- Validate tokens on the server.
- Implement secure refresh-token handling.
- Use multi-factor authentication for sensitive accounts.
- Protect against brute-force and credential-stuffing attacks.
- Never expose credentials in URLs or logs.

Important definition

API Authentication: The process of verifying the identity of a user, application, or service before allowing it to access an API.

Cybersecurity application

A penetration tester checks whether an API:

- Accepts invalid credentials.
- Uses weak token validation.
- Allows expired tokens.

- Exposes authentication tokens.
- Has insufficient protection against repeated login attempts.

Topic 3: API Authorization

3.1 What is API authorization?

API authorization determines what an authenticated user or application is allowed to access or perform.

In simple words:

Authorization answers: What are you allowed to do?

Example:

A shopping application has three users:

- Customer
- Seller
- Administrator

Their permissions are different.

Action	Customer	Seller	Administrator
View products	Yes	Yes	Yes
Buy products	Yes	No	Yes
Add own products	No	Yes	Yes
Manage all users	No	No	Yes

The API must enforce these permissions on the server.

3.2 Common authorization models

A. Role-Based Access Control (RBAC)

RBAC grants permissions based on assigned roles.

Example:

- Admin: manage users.
- Editor: modify content.
- Viewer: read content.

A user receives permissions through their assigned role.

B. Attribute-Based Access Control (ABAC)

ABAC determines access using attributes.

These attributes may include:

- User identity.
- User role.
- Resource ownership.
- Location.
- Time.
- Resource sensitivity.

Example:

An employee can access a document only if:

- They belong to the correct department.
- They have the required clearance.
- The document is within their permitted scope.

C. Relationship-Based Access Control (ReBAC)

ReBAC determines access based on relationships between users and resources.

Example:

A user can access a document because they are its owner or a member of the project that contains it.

3.3 Authentication vs authorization

Authentication	Authorization
Verifies identity	Checks permissions
Answers who you are	Answers what you can do
Usually happens before authorization	Follows identity verification
Example: logging in	Example: accessing admin settings

3.4 Common API authorization mistakes

- Checking whether a user is logged in but not checking permissions.
- Trusting a role supplied by the client.
- Allowing users to modify other users' records.
- Checking permissions only on the frontend.
- Applying authorization to some endpoints but forgetting others.

Important definition

API Authorization: The process of determining whether an authenticated user or application has permission to access a particular resource or perform a specific operation.

Cybersecurity application

Security professionals test authorization by checking whether different user roles can access resources outside their permitted scope.

Topic 4: Rate Limiting

4.1 What is rate limiting?

Rate limiting is a security mechanism that controls how many requests a client can make to an API within a specified period.

Example:

An API allows:

- 100 requests per minute per authenticated user.

If a client exceeds the limit, the API may temporarily reject additional requests.

4.2 Why is rate limiting necessary?

Without rate limiting, an attacker or malfunctioning client could send excessive requests.

Possible consequences include:

1. API resource exhaustion.
2. Denial-of-service conditions.
3. Brute-force login attempts.
4. OTP abuse.
5. Password-reset abuse.
6. Excessive scraping.
7. Unexpected infrastructure costs.

4.3 How rate limiting works

Rate Limiter System Design (Token Bucket Explained) · InterviewCrafted
interviewcrafted.com

Example:

Suppose an API allows 5 login attempts per minute.

Attempt	Result
1	Allowed
2	Allowed
3	Allowed
4	Allowed

5 Allowed

6 Rate limited

A rate-limited response commonly uses:

```
HTTP/1.1 429 Too Many Requests
```

The response may include:

```
Retry-After: 60
```

This indicates when the client can retry, in seconds.

4.4 Common rate-limiting algorithms

Algorithm	Explanation
Fixed Window	Counts requests within fixed time intervals.
Sliding Window	Tracks requests over a moving time interval.
Token Bucket	Allows requests using tokens that are replenished over time.
Leaky Bucket	Processes requests at a controlled rate.

4.5 Rate-limiting best practices

- Apply limits to sensitive endpoints.
- Consider user identity, API key, IP address, and other relevant signals.
- Use stricter limits for login, OTP, and password-reset operations.
- Return HTTP 429 when appropriate.
- Monitor repeated limit violations.
- Avoid relying exclusively on IP addresses.
- Use distributed rate limiting when an API runs across multiple servers.

Important definition

Rate Limiting: A technique used to restrict the number of API requests a client can make within a defined time period to prevent abuse and protect system resources.

Cybersecurity application

A penetration tester evaluates whether sensitive API endpoints are protected against excessive requests and automated abuse.

Topic 5: Input Validation

5.1 What is input validation?

Input validation is the process of checking whether data received from a user or external system meets the expected rules before the application processes it.

Example:

An API expects an age between 18 and 100.

Valid input:

```
{
  "age": 22
}
```

Invalid input:

```
{
  "age": -500
}
```

The API should reject invalid input.

5.2 Types of input validation

A. Type validation

Checks whether the data has the correct data type.

Example:

Expected: integer.

Received: string.

```
{
  "age": "twenty"
}
```

The API should reject it if the field requires an integer.

B. Length validation

Checks the length of submitted data.

Example:

- Username must contain 3–30 characters.
- Password must meet the application's password policy.

C. Range validation

Checks whether a value falls within an allowed range.

Example:

```
Age: 18–100
Quantity: 1–50
```

D. Format validation

Checks whether data follows a required format.

Examples:

- Email address.
- Phone number.
- Date.
- Postal code.

E. Allowlist validation

Allows only predefined acceptable values.

Example:

A user can select one of:

`pending`

`approved`

`rejected`

Any other status should be rejected.

5.3 What happens without input validation?

Poor input validation can lead to:

- SQL injection.
- Command injection.
- Cross-site scripting.
- Unexpected application behavior.
- Data corruption.
- Business logic abuse.

5.4 Input validation vs output encoding

These are different security controls.

Input validation	Output encoding
Checks incoming data	Makes output safe for its destination context
Rejects invalid values	Prevents data from being interpreted as executable code
Happens when data enters a system	Happens when data is displayed or inserted into an output context

Important: Input validation alone does not prevent every injection vulnerability. Use parameterized database queries, context-aware output encoding, and safe APIs as appropriate.

5.5 Security best practices

- Validate data on the server.

- Never trust frontend validation alone.
- Use allowlists where possible.
- Validate data types, length, and ranges.
- Use parameterized SQL queries.
- Apply output encoding.
- Reject unexpected fields when appropriate.
- Limit request body size.

Important definition

Input Validation: The process of checking incoming data against defined rules to ensure it is valid, expected, and safe to process.

Cybersecurity application

Penetration testers send unexpected input to API endpoints to identify weak validation, injection risks, and business logic flaws.

Topic 6: Mass Assignment

6.1 What is mass assignment?

Mass assignment is a vulnerability that occurs when an application automatically maps user-submitted data to internal object properties without properly restricting which properties the user can modify.

In simple words:

The application allows users to change fields they should not control.

6.2 Practical example

Imagine a shopping application with a user profile.

The database stores:

```
{
  "id": 101,
  "username": "bikash",
  "email": "user@example.com",
  "role": "customer",
  "is_verified": false
}
```

The application allows users to update their profile.

The intended request is:

```
{
```

```
  "username": "bikash_new",
  "email": "new@example.com"
}
```

However, the backend automatically accepts all submitted fields.

An attacker might try submitting:

```
{
  "username": "bikash",
  "role": "admin",
  "is_verified": true
}
```

If the application accepts these protected fields, the attacker may gain unauthorized privileges.

Security vulnerability

6.3 Why does mass assignment happen?

Common causes include:

- Automatically mapping request data to database objects.
- Accepting all fields from the client.
- Failing to distinguish editable and protected fields.
- Missing server-side authorization checks.
- Using unsafe object-binding frameworks.

6.4 How to prevent mass assignment

A. Use an allowlist of editable fields

Only permit fields that users are supposed to modify.

Example:

```
Allowed:
- username
- email
- profile_picture

Not allowed:
- role
- account_balance
- is_admin
- is_verified
```

B. Use Data Transfer Objects (DTOs)

A DTO defines which fields an API accepts for a particular operation.

For example, a profile-update DTO should not contain the administrator role.

C. Enforce server-side authorization

Sensitive fields should be modified only through properly authorized operations.

D. Test unexpected fields

Security testing should check whether the API accepts fields that are not part of its documented request schema.

Important definition

Mass Assignment: A vulnerability in which an application automatically assigns user-supplied input to internal object properties, allowing attackers to modify fields they should not control.

Cybersecurity application

During API penetration testing, professionals inspect create and update endpoints for hidden or undocumented fields that could change roles, permissions, balances, or account status.

Topic 7: Broken Object-Level Authorization (BOLA)

High-priority API vulnerability

7.1 What is BOLA?

BOLA stands for Broken Object-Level Authorization.

It occurs when an API fails to verify whether the requesting user has permission to access a specific object.

An object could be:

- User account.
- Bank transaction.
- Order.
- Invoice.
- Medical record.
- Private document.
- Customer profile.

7.2 Practical example

Imagine an application with two users.

User	User ID	Account
User A	101	Account A
User B	102	Account B

User A sends:

```
GET /api/accounts/101
```

The API returns User A's account information.

Now User A changes the ID:

```
GET /api/accounts/102
```

If the API returns User B's private information without checking ownership, the API has a BOLA vulnerability.

7.3 How BOLA works

보안 격차의 허점: 애플리케이션 공격과 API 공격의 증가 | Akamai
akamai.com

The vulnerability occurs because the application trusts the object ID without checking whether the requesting user is authorized to access that object.

Important: Changing an ID is not itself a vulnerability. The vulnerability exists when the server fails to enforce object-level authorization.

7.4 Why BOLA is dangerous

BOLA can expose:

- Personal information.
- Financial records.
- Customer data.
- Private files.
- Medical information.
- Business documents.

It can also allow unauthorized modification or deletion of another user's data.

7.5 How to prevent BOLA

1. Authenticate the requesting user.
2. Identify the requested object.
3. Check whether the user is authorized to access that specific object.
4. Allow the operation only after the permission check succeeds.
5. Apply the same checks to reading, updating, and deleting objects.

Example of a secure authorization rule:

```
User requests account 102.
```

```
Server checks:
Is this account owned by the authenticated user?
OR
Does the user have an explicitly authorized administrative permission?

If yes:
    Allow the request.
Otherwise:
    Deny the request.
```

The server must perform this check for every relevant request.

7.6 BOLA vs BFLA

BFLA stands for Broken Function-Level Authorization.

BOLA	BFLA
Object-level permission failure	Function-level permission failure
User accesses another user's object	User accesses a function they are not permitted to use
Example: viewing another user's invoice	Example: accessing an admin-only endpoint

7.7 BOLA vs authentication failure

BOLA	Authentication failure
User may be properly authenticated	Identity verification is defective
Access to a particular object is not checked correctly	Unauthorized identity or credentials may be accepted
Example: reading another user's account	Example: bypassing login

Important definition

BOLA: An API authorization vulnerability that occurs when the server fails to verify whether a user has permission to access or modify a specific object.

Cybersecurity application

BOLA testing is essential in applications that manage user-specific resources, such as banking, e-commerce, healthcare, education, and cloud platforms.

Topic 8: API Key Management

8.1 What is an API key?

An API key is a unique credential used to identify an application or client when communicating with an API.

Example:

```
GET /api/weather
X-API-Key: <secret_api_key>
```

An API key may be used for:

- Identifying an application.
- Controlling API access.
- Tracking API usage.
- Applying usage limits.
- Associating requests with a particular project.

Important: An API key is not automatically equivalent to user authentication. A key that identifies an application does not necessarily prove which human user is making a request.

8.2 Why is API key management important?

A leaked API key can allow unauthorized parties to use an API under the key owner's identity or permissions.

Possible consequences:

- Unauthorized API usage.
- Financial losses.
- Data exposure.
- Service abuse.
- Quota exhaustion.
- Unauthorized operations.

8.3 Common API key security mistakes

Mistake	Risk
Hardcoding keys in source code	Keys may leak through repositories
Publishing keys on GitHub	Public exposure
Storing keys in frontend JavaScript	Users can inspect them
Using the same key everywhere	Larger impact if leaked
Never rotating keys	Compromised keys remain usable
Giving keys excessive permissions	Unnecessary access
Logging complete keys	Accidental credential exposure

8.4 API key security best practices

A. Never expose secret keys

Do not put secret API keys in:

- Public GitHub repositories.
- Frontend JavaScript.
- Mobile application source code.
- Public configuration files.
- URLs.

B. Use environment variables or secret managers

Store secrets outside source code.

Example:

```
API_KEY=<secret_value>
```

C. Apply least privilege

Give each key only the permissions it requires.

D. Rotate keys

Replace keys periodically and immediately when they are suspected to be compromised.

E. Monitor API usage

Watch for unusual request patterns, unexpected locations, and unexpected usage volume.

F. Revoke compromised keys

A leaked key should be disabled or revoked promptly.

8.5 Environment variables vs secret managers

Environment variables	Secret managers
Store configuration outside source code	Dedicated services for storing and controlling secrets
Useful for simple applications	Useful for larger and production systems
Security depends on deployment and access controls	Can provide access policies, audit logs, and rotation support
May be sufficient for local development	Often preferred for production secrets

Examples of secret-management services:

- AWS Secrets Manager
- Google Cloud Secret Manager
- Azure Key Vault

- HashiCorp Vault

Important definition

API Key Management: The process of securely creating, storing, distributing, monitoring, rotating, and revoking API keys.

Cybersecurity application

Security professionals inspect applications and repositories for exposed API keys, excessive permissions, weak storage practices, and missing rotation procedures.

3. Important API Security Differences

Concept	Main purpose
Authentication	Verify identity
Authorization	Check permissions
Rate limiting	Control request frequency
Input validation	Check incoming data
Mass assignment	Unsafe modification of object properties
BOLA	Unauthorized access to specific objects
API key management	Protect API credentials

REST API vs API authentication vs API authorization

These terms are related but not interchangeable.

- REST API describes an architectural style.
- Authentication verifies the requester.
- Authorization determines what the requester can do.

An API can use REST, JWT authentication, and RBAC authorization simultaneously.

4. Practical Tools for API Security

Visionco | services

1. Postman

Send API requests, inspect responses, test authentication, and validate endpoints.

Official website
Como o Burp Suite pode levar seu pentest para o próximo nível

2. Burp Suite

Intercept and inspect HTTP traffic, modify requests, and test API security.

Official website
File:OWASP ZAP logo.svg - Wikimedia Commons

3. OWASP ZAP

Find common web and API security issues through authorized testing.

Official website
crAPI/services/web/src/containers/mechanicDashboard/mechanicDashboard.tsx at develop · OWASP/crAPI · GitHub

4. OWASP crAPI

A deliberately vulnerable API application designed for learning API security testing.

GitHub repository
OWASP Juice Shop | Probably the most modern and sophisticated insecure web application for security trainings, awareness demos and CTFs.

5. OWASP Juice Shop

A vulnerable application for practicing web and API security concepts.

Official project

5. Practical Exercises

Perform these exercises only against your own applications or deliberately vulnerable training environments.

Exercise 1: Test a vulnerable API

Target: OWASP crAPI.

Tasks:

1. Set up the application locally.
2. Register two test accounts.
3. Log in using both accounts.
4. Inspect the API requests.
5. Identify endpoints that access user-specific information.
6. Check whether one account can access the other account's data.
7. Document any authorization failures.

Expected learning: Authentication, authorization, and BOLA.

Exercise 2: Identify and document API vulnerabilities

Test these categories:

Test	What to investigate

Authentication	Invalid, expired, or missing tokens
Authorization	Unauthorized endpoint access
BOLA	Access to another user's objects
Rate limiting	Excessive requests to sensitive endpoints
Input validation	Invalid types, lengths, and values
Mass assignment	Unexpected or protected fields
API keys	Exposure and excessive permissions

For each finding, document:

- Vulnerability name.
- Affected endpoint.
- Required access level.
- Steps to reproduce safely.
- Expected behavior.
- Actual behavior.
- Security impact.
- Evidence.
- Recommended fix.

Exercise 3: Build a secure REST API

Build a small user-management API.

Suggested endpoints:

Method	Endpoint	Purpose
POST	`/api/register`	Register a user
POST	`/api/login`	Authenticate a user
GET	`/api/profile`	View own profile
PATCH	`/api/profile`	Update own profile
GET	`/api/users/{id}`	View a permitted user
DELETE	`/api/users/{id}`	Delete a permitted user

Suggested technology:

- Python.
- FastAPI.

- PostgreSQL.
- SQLAlchemy.
- Pydantic.
- JWT or an appropriate session/token authentication mechanism.

Security requirements:

- Hash passwords securely.
- Require authentication for protected endpoints.
- Enforce object-level authorization.
- Validate request data.
- Prevent mass assignment.
- Apply rate limiting.
- Store secrets outside source code.
- Use HTTPS in deployment.
- Log security-relevant events without recording passwords or complete tokens.

Exercise 4: Security-test your own API

After building your API, test it with Postman and Burp Suite.

Checklist:

- Can an unauthenticated user access protected endpoints?
- Can User A access User B's profile?
- Can a normal user change their role to admin?
- Does the API reject invalid data types?
- Are oversized requests rejected?
- Are login attempts rate-limited?
- Are expired tokens rejected?
- Are API keys absent from frontend code and public repositories?
- Does the API return only necessary information?
- Are errors handled without exposing internal details?

6. Practical Project — Secure REST API Development and Penetration Testing

Module project

Your main project is to build a secure REST API and then attempt to find vulnerabilities in it.

[API Driven Development: The Necessary Prerequisite to MCPs | by Devansh | Medium
machine-learning-made-simple.medium.com]

Project architecture

```
Client Application
       |
       | HTTPS
       |
API Gateway / Rate Limiter
       |
       v
REST API Server
       |
       +---- Authentication
       |
       +---- Authorization
       |
       +---- Input Validation
       |
       +---- Business Logic
       |
       v
Database
```

Development stages

Stage	Work
1	Design API endpoints
2	Build user registration and login
3	Implement authentication
4	Implement authorization
5	Add input validation
6	Prevent mass assignment
7	Implement BOLA protection
8	Add rate limiting
9	Secure API keys and secrets
10	Perform penetration testing

11 Fix vulnerabilities

12 Prepare a security report

Final deliverables

1. Working REST API.
2. API documentation.
3. Database design.
4. Authentication and authorization implementation.
5. Security test cases.
6. Vulnerability report.
7. Fixed and retested application.

7. Important Definitions — Revision Notes

Term	Definition
API	An interface that allows software applications to communicate.
REST	An architectural style for designing network-based APIs.
REST API Security	Protecting REST APIs against unauthorized access, abuse, and attacks.
API Authentication	Verifying the identity of an API requester.
API Authorization	Determining what an authenticated requester is allowed to do.
Rate Limiting	Restricting the number of requests allowed within a time period.
Input Validation	Checking incoming data against defined rules.
Mass Assignment	Automatically assigning user-submitted values to internal object properties without sufficient restrictions.
BOLA	An authorization vulnerability involving unauthorized access to specific objects.
API Key	A credential used to identify or authorize an application or client.
API Key Management	Securely creating, storing, monitoring, rotating, and revoking API keys.
RBAC	An authorization model based on user roles.
ABAC	An authorization model based on attributes.
DTO	A Data Transfer Object that defines data transferred between software components.

HTTP 429	The HTTP status code indicating too many requests.

Phase 9 — Network and System Security

Beginner to advanced
18 core topics

1. Learning Objectives

By the end of this phase, you should be able to:

1. Understand how networks and IT infrastructure are designed and secured.
2. Configure and evaluate firewall security.
3. Understand intrusion detection and prevention systems.
4. Secure wired, wireless, and remote network access.
5. Protect DNS, email, endpoints, Linux systems, and Windows systems.
6. Apply server hardening and network segmentation.
7. Identify, prioritize, and manage vulnerabilities.
8. Understand patch management, backups, and disaster recovery.
9. Monitor infrastructure and investigate suspicious activity.
10. Understand advanced network security concepts, including network forensics, lateral movement detection, and Zero Trust architecture.

2. Core Topics — Topic-Wise Explanations

Topic 1: Network Architecture Security

1.1 What is network architecture?

Network architecture is the overall design of a computer network. It defines how devices, servers, applications, and network components communicate.

A typical network contains:

- Client computers.
- Servers.
- Routers.
- Switches.
- Firewalls.
- Wireless access points.
- Network storage.
- Internet connections.

3. DMZ, NAT & PAT
choimungu.tistory.com

1.2 What is network architecture security?

Network architecture security means designing a network so that unauthorized users cannot easily access devices, systems, applications, or sensitive information.

1.3 Important network components

Component	Purpose
Router	Connects different networks
Switch	Connects devices within a local network
Firewall	Controls network traffic
IDS	Detects suspicious network activity
IPS	Detects and blocks malicious traffic
Access Point	Provides wireless network access
Proxy Server	Acts as an intermediary for network requests
VPN Gateway	Provides secure VPN connectivity
DNS Server	Translates domain names into IP addresses

1.4 Common network architecture security principles

A. Defense in Depth

Use multiple security layers instead of relying on one security product.

Example:

```
Internet
   |
Firewall
   |
IDS / IPS
   |
Network Segmentation
   |
Endpoint Security
   |
Protected Servers
```

B. Least Privilege

Give users and systems only the access they need.

C. Secure by Design

Include security requirements when designing the network rather than adding security after deployment.

D. Minimize Attack Surface

Disable unnecessary services, ports, accounts, and network connections.

Important definition

Network Architecture Security: The practice of designing and protecting network infrastructure to prevent unauthorized access, data exposure, and network attacks.

Cybersecurity application

Security architects use these principles when designing corporate networks, cloud environments, data centers, and enterprise infrastructure.

Topic 2: Firewall Architecture

2.1 What is a firewall?

A firewall is a security system that monitors and controls incoming and outgoing network traffic according to defined security rules.

In simple words, a firewall acts as a security gate between networks.

Firewall vs. Antivirus: Breaking Down the Differences - Palo Alto Networks

paloaltonetworks.co.uk

2.2 How does a firewall work?

A firewall examines network traffic and compares it against configured rules.

Example:

Traffic	Firewall action
HTTPS traffic on TCP port 443	Allow
Unapproved inbound SSH	Block
Traffic from a blocked IP address	Block
Authorized internal DNS requests	Allow

The actual decision depends on the firewall policy.

2.3 Types of firewalls

A. Packet Filtering Firewall

Checks packet information such as:

- Source IP.
- Destination IP.
- Source port.
- Destination port.
- Protocol.

B. Stateful Firewall

Tracks active network connections and uses connection state when making decisions.

C. Proxy Firewall

Acts as an intermediary between clients and servers. It can inspect application-level traffic.

D. Next-Generation Firewall (NGFW)

May combine traditional firewall functions with features such as application awareness, intrusion prevention, and advanced traffic inspection.

E. Web Application Firewall (WAF)

Protects web applications by inspecting HTTP and HTTPS requests for suspicious application-layer activity.

2.4 Firewall vs WAF

Firewall	WAF
Protects network traffic	Focuses on web application traffic

Can filter IPs, ports, and protocols	Inspects HTTP requests and responses
Protects network boundaries and segments	Helps defend web applications against attacks
Example: blocking unauthorized inbound connections	Example: detecting SQL injection patterns

2.5 Firewall security best practices

- Use default-deny policies for inbound traffic where practical.
- Allow only required ports and services.
- Remove unused firewall rules.
- Review rules periodically.
- Log important traffic and blocked connections.
- Restrict firewall administration.
- Keep firewall software and firmware updated.

Important definition

Firewall: A security system that monitors and controls network traffic according to predefined security policies.

Cybersecurity application

Penetration testers examine firewall rules to determine whether unnecessary services or network paths are exposed.

Topic 3: IDS and IPS

3.1 What is IDS?

IDS stands for Intrusion Detection System.

An IDS monitors network or system activity to identify suspicious behavior and potential security attacks.

Its primary purpose is detection and alerting.

3.2 What is IPS?

IPS stands for Intrusion Prevention System.

An IPS detects suspicious activity and can automatically take action to block or prevent it.

3.3 How IDS and IPS work

File:Ips-vs-ids-short.png - Wikimedia Commons
commons.wikimedia.org

An IDS or IPS may examine traffic for:

- Known attack signatures.
- Suspicious connection patterns.
- Malicious payloads.
- Unexpected protocol behavior.
- Unusual traffic volume.

3.4 Types of IDS and IPS

Type	Meaning
NIDS	Network-based Intrusion Detection System
HIDS	Host-based Intrusion Detection System
NIPS	Network-based Intrusion Prevention System
HIPS	Host-based Intrusion Prevention System

NIDS: Monitors network traffic.

HIDS: Monitors activity on an individual computer or server.

3.5 Detection methods

A. Signature-Based Detection

Compares activity against known attack patterns.

Advantage: Effective for known threats.

Limitation: May miss previously unknown attacks.

B. Anomaly-Based Detection

Identifies activity that differs from an established baseline.

Advantage: Can detect unusual or previously unseen behavior.

Limitation: Can generate false positives.

C. Behavior-Based Detection

Identifies suspicious activity based on behavior patterns.

3.6 IDS vs IPS

IDS	IPS
Detects suspicious activity	Detects and can block suspicious activity
Usually generates alerts	Can actively prevent traffic
Often deployed for monitoring	Often placed inline
Does not normally block traffic itself	May block, reset, or drop traffic

Important definitions

- IDS: A system that monitors activity to detect potential intrusions.
- IPS: A system that detects and takes action to prevent potential intrusions.

Cybersecurity application

SOC analysts use IDS and IPS alerts to investigate attacks, identify suspicious connections, and detect compromised systems.

Topic 4: Network Access Control (NAC)

4.1 What is NAC?

NAC stands for Network Access Control.

NAC controls which devices can connect to an organization's network and what access they receive.

Example:

A company may require every employee laptop to have:

- Updated operating system.
- Active endpoint protection.
- Enabled firewall.
- Required security configuration.

A device that does not meet these requirements may be denied access or placed in a restricted network.

4.2 How NAC works

NAC (Network Access Control) Explained | Technical Guides | Purple
purple.ai

Typical process:

1. A device attempts to connect.
2. NAC identifies the device or user.
3. The device may be checked for security compliance.
4. NAC evaluates access policies.
5. The device receives appropriate access or is restricted.

4.3 Important NAC components

- Authentication server.
- Policy server.
- Network switches or access points.
- Device posture assessment.

- Enforcement mechanism.

4.4 Common NAC technologies

- IEEE 802.1X.
- RADIUS.
- EAP.
- Network access policies.

Important definition

Network Access Control: A security mechanism that controls access to a network based on user identity, device identity, and security policies.

Cybersecurity application

NAC helps organizations prevent unknown, unauthorized, or non-compliant devices from accessing internal systems.

Topic 5: VPN Security

5.1 What is a VPN?

VPN stands for Virtual Private Network.

A VPN creates a protected communication tunnel between a device and another network or VPN endpoint.

It commonly uses encryption and authentication to protect traffic across untrusted networks.

Qu'est-ce qu'un VPN d'accès à distance ? | VPN d'accès à distance expliqué - Palo Alto Networks paloaltonetworks.fr

5.2 Why are VPNs used?

- Secure remote access.
- Protect traffic over untrusted networks.
- Connect separate office networks.
- Allow employees to access internal services.
- Protect data while it travels across a network.

5.3 Types of VPN

A. Remote Access VPN

Connects an individual device to an organization's network.

Example: An employee working from home connects to the company network.

B. Site-to-Site VPN

Connects two or more networks.

Example: A company's Mumbai office connects securely to its Delhi office.

5.4 Common VPN protocols

Protocol	Description
WireGuard	Modern VPN protocol with a relatively simple design
OpenVPN	VPN technology using TLS-based security
IPsec	A suite of protocols used to secure IP communications
IKEv2	Protocol used to establish and manage IPsec security associations

Avoid obsolete or weak configurations such as PPTP.

5.5 VPN security risks

- Weak authentication.
- Stolen VPN credentials.
- Outdated VPN software.
- Incorrect encryption configuration.
- Exposed VPN management interfaces.
- Excessive access after connection.
- Unpatched VPN appliances.

5.6 VPN security best practices

- Use strong authentication and MFA.
- Keep VPN software updated.
- Restrict access to necessary internal resources.
- Monitor VPN login activity.
- Disable unused VPN accounts.
- Use modern, secure protocols and configurations.
- Review remote access permissions.

Important definition

VPN: A technology that creates a protected network connection over another network, commonly using encryption and authentication.

Cybersecurity application

Security professionals evaluate VPN configurations, authentication mechanisms, exposed gateways, and remote-access permissions.

Topic 6: Wireless Network Security

6.1 What is wireless network security?

Wireless network security protects Wi-Fi networks, access points, connected devices, and wireless communication against unauthorized access and attacks.

6.2 Common wireless security protocols

Protocol	Security status
WEP	Insecure and obsolete
WPA	Legacy; no longer recommended
WPA2	Widely used; security depends on configuration
WPA3	Newer and generally stronger security

6.3 Common wireless attacks

A. Evil Twin Attack

An attacker creates a fake wireless access point that imitates a legitimate network.

Users may connect to the fake network and expose sensitive information.

B. Rogue Access Point

An unauthorized wireless access point is connected to an organization's network.

C. Deauthentication Attack

An attacker sends forged deauthentication frames to disrupt wireless connections. Management frame protection helps defend against certain forms of this attack.

D. Weak Password Attack

An attacker attempts to discover a Wi-Fi password when weak credentials or insecure configurations are used.

E. Packet Sniffing

An attacker captures wireless network traffic to inspect information that is exposed or insufficiently protected.

6.4 Wireless security best practices

- Prefer WPA3 where supported.
- Use WPA2-AES when WPA3 is unavailable and the configuration is secure.
- Use strong Wi-Fi passwords.
- Disable WPS when unnecessary.
- Change default router administrator credentials.
- Update router firmware.

- Use a separate guest network.
- Enable protected management frames where supported.
- Monitor unknown access points.

Important definition

Wireless Network Security: The practice of protecting wireless communication, access points, and connected devices from unauthorized access and attacks.

Cybersecurity application

Wireless penetration testers evaluate Wi-Fi authentication, encryption, access-point configuration, and exposure to rogue access points.

Topic 7: DNS Security

7.1 What is DNS?

DNS stands for Domain Name System.

DNS translates human-readable domain names into IP addresses.

Example:

`google.com → IP address`

Without DNS, users would generally need to remember IP addresses instead of domain names.

7.2 How DNS works

Understanding DNS and Web Browsing: A Deep Dive
graphitedge.com.au

Simplified process:

1. A user enters a domain name.
2. The device checks its local DNS cache.
3. It sends a query to a DNS resolver if needed.
4. The resolver obtains the necessary DNS information.
5. The IP address is returned to the device.
6. The device connects to the destination.

7.3 Common DNS attacks

A. DNS Spoofing

An attacker supplies false DNS information so users are directed to an unintended destination.

B. DNS Cache Poisoning

An attacker causes incorrect DNS information to be stored in a resolver's cache.

C. DNS Tunneling

DNS queries and responses are abused to carry data or communicate with external infrastructure.

D. DNS Amplification

An attacker abuses DNS servers to amplify traffic in a distributed denial-of-service attack.

E. Domain Hijacking

An attacker gains unauthorized control of a domain registration or its DNS configuration.

7.4 DNS security technologies

Technology	Purpose
DNSSEC	Provides origin authentication and integrity protection for DNS data
DoH	DNS over HTTPS
DoT	DNS over TLS
DNS filtering	Blocks or restricts access to malicious domains

Important: DNSSEC does not encrypt DNS queries. DoH and DoT protect DNS communication between a client and resolver, but do not by themselves guarantee that a returned domain is safe.

7.5 DNS security best practices

- Use trusted DNS resolvers.
- Enable DNSSEC validation where supported.
- Protect domain registrar accounts with MFA.
- Monitor DNS records for unexpected changes.
- Restrict DNS zone transfers.
- Monitor unusual DNS traffic.
- Use DNS filtering where appropriate.

Important definition

DNS Security: The practice of protecting DNS infrastructure, records, and queries against manipulation, unauthorized changes, and malicious use.

Cybersecurity application

Security analysts inspect DNS traffic to identify suspicious domains, malware communication, DNS tunneling, and possible command-and-control activity.

Topic 8: Email Security

8.1 What is email security?

Email security protects email accounts, mail servers, messages, and attachments from phishing, malware, spam, spoofing, and unauthorized access.

8.2 Common email attacks

A. Phishing

An attacker sends a deceptive message to trick a person into revealing credentials or performing an unsafe action.

B. Spear Phishing

A targeted phishing attack designed for a particular individual or organization.

C. Email Spoofing

An attacker forges email sender information to make a message appear to come from another address or domain.

D. Business Email Compromise (BEC)

An attacker impersonates a trusted business contact to trick employees into transferring money or sharing sensitive information.

E. Malicious Attachments

An attacker sends files designed to execute malicious code or exploit software vulnerabilities.

8.3 Email security technologies

A. SPF — Sender Policy Framework

Specifies which mail servers are authorized to send email for a domain.

B. DKIM — DomainKeys Identified Mail

Uses a digital signature to help verify that a message is associated with a domain and has not been modified in a way that invalidates the signature.

C. DMARC — Domain-based Message Authentication, Reporting, and Conformance

Allows domain owners to specify how receivers should handle messages that fail applicable SPF and DKIM checks, with reporting support.

8.4 SPF vs DKIM vs DMARC

Technology	Main purpose
SPF	Authorizes sending mail servers
DKIM	Verifies a domain-associated message signature
DMARC	Defines handling and reporting policies for authentication failures

8.5 Email security best practices

- Enable MFA.
- Use SPF, DKIM, and DMARC.
- Train users to identify phishing.

- Scan attachments and links.
- Apply anti-spam and anti-malware protection.
- Monitor unusual login activity.
- Restrict email forwarding rules.
- Protect administrative mail accounts.

Important definition

Email Security: The process of protecting email systems and communication against phishing, spoofing, malware, unauthorized access, and data theft.

Cybersecurity application

SOC teams investigate suspicious messages, email headers, malicious attachments, phishing domains, and account-compromise indicators.

Topic 9: Endpoint Security

9.1 What is an endpoint?

An endpoint is a device that connects to a network.

Examples:

- Desktop computers.
- Laptops.
- Smartphones.
- Servers.
- Virtual machines.
- Workstations.

9.2 What is endpoint security?

Endpoint security protects devices connected to a network against malware, unauthorized access, exploitation, and other threats.

Firewall vs. Antivirus: Breaking Down the Differences - Palo Alto Networks
origin-www.paloaltonetworks.co.uk

9.3 Common endpoint threats

- Malware.
- Ransomware.
- Spyware.
- Credential theft.
- Unauthorized software.

- Exploitation of unpatched vulnerabilities.
- Malicious USB devices.
- Privilege escalation.

9.4 Endpoint security technologies

A. Antivirus

Detects, blocks, and removes malicious software.

B. EDR — Endpoint Detection and Response

Collects endpoint activity and helps security teams detect, investigate, and respond to threats.

C. XDR — Extended Detection and Response

Correlates security data across multiple sources, potentially including endpoints, networks, email, and cloud environments.

D. Application Control

Restricts which applications are allowed to execute.

E. Disk Encryption

Protects stored information if a device is lost or stolen.

9.5 Endpoint security best practices

- Keep operating systems updated.
- Use endpoint protection.
- Enable full-disk encryption.
- Restrict administrator privileges.
- Enable host firewalls.
- Monitor suspicious processes.
- Remove unnecessary software.
- Use application control where appropriate.
- Maintain secure backups.

Important definition

Endpoint Security: The practice of protecting network-connected devices from cyberattacks, malware, unauthorized access, and data loss.

Cybersecurity application

Security analysts use endpoint telemetry to investigate malware infections, suspicious processes, persistence mechanisms, and compromised accounts.

Topic 10: Server Hardening

10.1 What is server hardening?

Server hardening is the process of reducing a server's attack surface by removing unnecessary services, applying secure configurations, and restricting access.

10.2 Why is server hardening important?

Servers often store or process:

- User information.
- Application data.
- Business records.
- Authentication credentials.
- Databases.
- Financial information.

An insecure server can become an entry point for attackers.

10.3 Server hardening techniques

A. Remove unnecessary services

Disable services that are not required.

B. Restrict open ports

Only expose necessary network services.

C. Secure administrator access

Use strong authentication and restrict privileged accounts.

D. Update software

Apply security patches regularly.

E. Enable logging

Record authentication attempts, administrative changes, and security events.

F. Protect data

Use encryption and appropriate access controls.

G. Configure firewalls

Restrict network traffic to required sources and destinations.

10.4 Server hardening checklist

- Remove unnecessary applications.
- Disable unused services.
- Restrict administrative access.

- Use strong authentication.
- Apply security updates.
- Configure host firewalls.
- Protect stored secrets.
- Enable security logging.
- Secure file permissions.
- Review exposed ports.

Important definition

Server Hardening: The process of securing a server by reducing its attack surface and applying appropriate security configurations.

Cybersecurity application

Infrastructure security engineers harden web servers, database servers, application servers, and cloud virtual machines.

Topic 11: Linux Hardening

11.1 What is Linux hardening?

Linux hardening means securing a Linux operating system against unauthorized access, exploitation, and misuse.

Since you are learning Linux and cybersecurity, this topic is particularly important for understanding how real servers are protected.

11.2 Linux hardening techniques

A. User and permission security

Linux uses file permissions and ownership to control access.

Important commands:

`ls -l`

`chmod`

`chown`

`id`

`whoami`

B. SSH security

SSH provides secure remote administration.

Recommended practices:

- Use SSH keys.
- Disable direct root login where practical.
- Disable password authentication when key-based access is properly configured and recovery is available.

- Restrict SSH access.
- Use MFA where supported.

C. Firewall configuration

Common Linux firewall tools:

- UFW.
- nftables.
- firewalld.

D. User privilege management

Use `sudo` with appropriate restrictions instead of sharing the root account.

E. Software updates

Install security updates regularly.

F. Audit and logging

Review logs for suspicious activity.

Useful commands:

```
journalctl
last
lastb
ss -tulpn
```

G. File permissions

Protect sensitive files such as:

- `/etc/shadow`
- SSH private keys.
- Application configuration files.
- Database credentials.

11.3 Important Linux hardening tools

Tool	Purpose
UFW	Simplified firewall management
nftables	Linux packet filtering framework
Fail2ban	Blocks repeated abusive authentication attempts based on configured log monitoring
AppArmor	Mandatory access control
SELinux	Mandatory access control

auditd	Linux audit framework
Lynis	Security auditing and hardening assessment

11.4 Linux hardening practical example

Check listening network services:

```
ss -tulpn
```

This helps identify services listening on network ports.

Check file permissions:

```
ls -l /etc/shadow
```

Check active users:

```
who
```

Warning: Do not blindly copy hardening commands from tutorials. Incorrect SSH or firewall changes can lock you out of your system. Test configuration changes locally and maintain a recovery route.

Important definition

Linux Hardening: The process of securing a Linux system through proper user permissions, service configuration, firewall rules, software updates, logging, and access controls.

Cybersecurity application

Linux hardening is used in web hosting, cloud infrastructure, penetration-testing environments, enterprise servers, and security appliances.

Topic 12: Windows Hardening

12.1 What is Windows hardening?

Windows hardening is the process of securing Windows computers and servers by reducing vulnerabilities and enforcing security controls.

12.2 Windows hardening techniques

A. Windows Defender

Provides built-in antivirus and security protection.

B. Windows Firewall

Controls incoming and outgoing network connections.

C. User Account Control (UAC)

Helps prevent unauthorized administrative changes by requiring approval or elevation.

D. BitLocker

Encrypts storage drives to protect data if a device is lost or stolen.

E. Windows Update

Provides security patches and system updates.

F. Group Policy

Allows administrators to configure security settings across Windows systems.

G. Microsoft Defender for Endpoint

Provides enterprise endpoint detection and response capabilities.

12.3 Windows hardening checklist

- Enable Microsoft Defender protection.
- Enable Windows Firewall.
- Keep Windows updated.
- Enable BitLocker where supported.
- Use standard accounts for daily work.
- Enable MFA for supported accounts.
- Restrict Remote Desktop access.
- Disable unnecessary services.
- Review startup applications.
- Monitor security events.

12.4 Important Windows security tools

Tool	Purpose
Windows Defender	Malware protection
Windows Firewall	Network traffic control
Event Viewer	Review system and security events
PowerShell	System administration and security automation
Group Policy	Centralized security configuration
Sysmon	Detailed system activity logging
Microsoft Defender for Endpoint	Enterprise endpoint security

Important definition

Windows Hardening: The process of protecting Windows systems by applying security configurations, restricting privileges, updating software, and monitoring system activity.

Cybersecurity application

Windows hardening is essential in enterprise networks because Windows workstations and servers commonly manage business applications, user accounts, and sensitive data.

Topic 13: Network Segmentation

13.1 What is network segmentation?

Network segmentation divides a large network into smaller, separate network sections.

Each segment can have different security rules and access permissions.

Free Network Diagram Maker | Create Network Topology Online - ConceptViz
conceptviz.app

13.2 Why is segmentation necessary?

Imagine an organization has:

- Employee computers.
- Database servers.
- Public web servers.
- Guest Wi-Fi.
- Administrative systems.

If all these devices belong to one unrestricted network, compromising one device may provide a path to other systems.

Segmentation limits unnecessary communication between these systems.

13.3 Types of network segmentation

A. Physical Segmentation

Uses separate physical network infrastructure.

B. Logical Segmentation

Uses technologies such as VLANs to separate network traffic.

C. Subnet Segmentation

Divides networks into smaller IP address ranges.

D. Microsegmentation

Applies fine-grained security controls between individual workloads, applications, or services.

13.4 Example

Network segment	Access policy
Guest Wi-Fi	Internet only
Employee network	Approved business services
Database network	Only approved application servers
Management network	Authorized administrators

DMZ	Public-facing services with restricted internal access

13.5 Network segmentation benefits

- Limits lateral movement.
- Reduces attack impact.
- Protects sensitive systems.
- Improves traffic control.
- Makes monitoring easier.
- Supports compliance requirements.

Important definition

Network Segmentation: The process of dividing a network into smaller isolated sections to control communication and reduce security risks.

Cybersecurity application

Security architects use segmentation to isolate critical servers, databases, production environments, and administrative systems.

Topic 14: Secure Remote Access

14.1 What is remote access?

Remote access allows users or administrators to connect to systems from another location.

Examples:

- An employee accessing a company application from home.
- An administrator managing a remote Linux server.
- An engineer accessing a cloud virtual machine.

14.2 Common remote access technologies

Technology	Purpose
SSH	Secure command-line access
RDP	Remote desktop access
VPN	Secure network connectivity
ZTNA	Identity- and policy-based access to applications
Bastion Host	Controlled gateway to internal systems

14.3 Remote access security risks

- Stolen credentials.

- Exposed remote desktop services.
- Weak passwords.
- Unpatched remote access software.
- Excessive permissions.
- Unauthorized remote connections.

14.4 Secure remote access best practices

- Require MFA.
- Use strong authentication.
- Restrict access by identity and device posture.
- Avoid exposing administrative services directly to the public internet.
- Monitor login activity.
- Apply least privilege.
- Patch remote access systems.
- Use approved VPN, ZTNA, or bastion architectures.

Important definition

Secure Remote Access: A method of providing remote connectivity to systems while enforcing authentication, authorization, encryption, and monitoring.

Cybersecurity application

Cloud security engineers use secure remote access to administer servers and infrastructure without unnecessarily exposing management interfaces.

Topic 15: Vulnerability Management

15.1 What is vulnerability management?

Vulnerability management is the continuous process of identifying, assessing, prioritizing, treating, and monitoring security vulnerabilities in systems and applications.

15.2 What is a vulnerability?

A vulnerability is a weakness in software, hardware, configuration, or processes that could be exploited to affect confidentiality, integrity, or availability.

Examples:

- Outdated software.
- Weak configurations.
- Unpatched operating systems.
- Exposed services.

- Weak access controls.

15.3 Vulnerability management lifecycle

[What Is Vulnerability Management: A 2026 Guide | AuditYourApp](auditYourApp)
auditYour.app

1. Discover: Identify assets and systems.
2. Scan: Search for known vulnerabilities.
3. Assess: Determine the severity and context.
4. Prioritize: Decide which weaknesses need attention first.
5. Remediate: Fix or reduce the risk.
6. Verify: Confirm the vulnerability is addressed.
7. Monitor: Continue monitoring for new issues.

15.4 CVE and CVSS

CVE — Common Vulnerabilities and Exposures

CVE provides standardized identifiers for publicly disclosed cybersecurity vulnerabilities.

Example format:

`CVE-2024-XXXX`

CVSS — Common Vulnerability Scoring System

CVSS provides a standardized method for describing the technical severity of vulnerabilities.

CVSS v3.1 and v4.0 use severity categories such as:

Score	Severity
0.0	None
0.1–3.9	Low
4.0–6.9	Medium
7.0–8.9	High
9.0–10.0	Critical

These ranges apply to both versions, although their scoring methods differ.

15.5 Vulnerability management tools

- Nessus.
- OpenVAS.
- Nmap.
- Qualys.

- Greenbone.

Important definition

Vulnerability Management: A continuous process of discovering, assessing, prioritizing, fixing, and monitoring security weaknesses.

Cybersecurity application

Vulnerability management teams use scanning and risk assessment to identify weaknesses before attackers exploit them.

Topic 16: Patch Management

16.1 What is patch management?

Patch management is the process of identifying, testing, deploying, and verifying software updates that fix bugs, security vulnerabilities, or stability problems.

16.2 What is a security patch?

A security patch is a software update that addresses a security weakness.

Example:

A Linux server uses an outdated web server version with a known vulnerability.

The administrator applies an appropriate vendor security update to fix the issue.

16.3 Patch management lifecycle

1. Identify affected systems.
2. Review available patches.
3. Assess urgency and compatibility.
4. Test patches in a suitable environment.
5. Deploy patches.
6. Verify successful installation.
7. Monitor for unexpected problems.

16.4 Types of updates

Update type	Purpose
Security update	Fixes security weaknesses
Bug fix	Corrects software defects
Feature update	Adds or changes functionality
Firmware update	Updates device firmware

16.5 Patch management best practices

- Maintain an asset inventory.
- Prioritize actively exploited vulnerabilities.
- Test important updates before broad deployment.
- Maintain rollback procedures.
- Track patch compliance.
- Document exceptions.
- Avoid leaving critical systems unpatched.

Important definition

Patch Management: The process of acquiring, testing, deploying, and verifying software updates to maintain security and stability.

Cybersecurity application

System administrators and security teams use patch management to reduce the number of known exploitable weaknesses in their infrastructure.

Topic 17: Backup and Disaster Recovery

17.1 What is a backup?

A backup is a separate copy of data that can be used to restore information after deletion, corruption, hardware failure, or a cyberattack.

17.2 What is disaster recovery?

Disaster recovery is the process of restoring IT systems, services, and data after a major disruption.

Examples:

- Ransomware attack.
- Server failure.
- Data center outage.
- Accidental deletion.
- Natural disaster.

17.3 Backup types

Type	Explanation
Full Backup	Copies all selected data
Incremental Backup	Copies data changed since the last backup
Differential Backup	Copies data changed since the last full backup

17.4 The 3-2-1 backup rule

A commonly used backup strategy:

- 3: Maintain three copies of important data.
- 2: Store them on two different types of storage media.
- 1: Keep one copy offsite.

An additional improvement is to keep an isolated or immutable copy that attackers cannot easily alter or delete.

17.5 RPO and RTO

RPO — Recovery Point Objective

The maximum acceptable amount of data loss measured in time.

Example:

An RPO of 1 hour means the organization aims to restore data to a point no more than one hour before the disruption.

RTO — Recovery Time Objective

The target maximum time required to restore a service after disruption.

Example:

An RTO of 4 hours means the organization aims to restore the service within four hours.

17.6 RPO vs RTO

RPO	RTO
Focuses on data loss	Focuses on service downtime
Measured as a time interval	Measured as a recovery time
Example: 1 hour of data loss	Example: 4 hours to restore service

17.7 Backup and disaster recovery best practices

- Schedule backups.
- Encrypt backup data.
- Restrict backup access.
- Maintain offline or immutable copies.
- Test restoration regularly.
- Document recovery procedures.
- Monitor backup failures.
- Define RPO and RTO requirements.

Important definitions

- Backup: A separate copy of data used for restoration.
- Disaster Recovery: The process of restoring systems and services after a major disruption.
- RPO: The acceptable data-loss period.
- RTO: The target time for restoring a service.

Cybersecurity application

Backup and disaster recovery are essential defenses against ransomware, data corruption, and infrastructure failures.

Topic 18: Infrastructure Monitoring

18.1 What is infrastructure monitoring?

Infrastructure monitoring is the continuous observation of systems, servers, networks, applications, and services to identify performance issues and security problems.

18.2 What should be monitored?

Component	What to monitor
Servers	CPU, RAM, disk, processes
Network	Traffic, connections, packet loss
Firewall	Allowed and blocked connections
Endpoints	Processes, malware alerts, logins
Applications	Errors, response times, failures
Cloud infrastructure	Access activity, configuration changes
Databases	Queries, availability, unusual access
Authentication	Failed logins, unusual login patterns

18.3 Types of monitoring

A. System Monitoring

Checks CPU, memory, storage, and process activity.

B. Network Monitoring

Tracks traffic, connections, bandwidth, and unusual communication.

C. Security Monitoring

Detects threats, suspicious behavior, and unauthorized activity.

D. Log Monitoring

Collects and analyzes event logs from multiple systems.

18.4 Important monitoring tools

Tool	Purpose
Wireshark	Network packet analysis
Zeek	Network security monitoring
Suricata	Network threat detection and prevention
Wazuh	Security monitoring and endpoint detection capabilities
Splunk	Log analysis and security information management
Prometheus	Metrics collection and monitoring
Grafana	Monitoring dashboards and visualization

18.5 SIEM

SIEM stands for Security Information and Event Management.

A SIEM collects and analyzes security logs from multiple sources to help detect suspicious activity.

Example:

```
Firewall Logs
      |
Server Logs
      |
Endpoint Logs
      |
Authentication Logs
      |
      v
     SIEM
      |
      v
Correlation and Detection
      |
      v
Security Alerts
```

Important definition

Infrastructure Monitoring: The continuous observation and analysis of IT infrastructure to identify performance problems, security threats, and service disruptions.

Cybersecurity application

SOC analysts use monitoring tools and SIEM platforms to identify unusual activity, investigate security alerts, and respond to incidents.

3. Advanced Network Security Topics

These topics move beyond basic network protection into enterprise security architecture, threat detection, and incident investigation.

Advanced Topic 1: Network Intrusion Detection

1. What is network intrusion detection?

Network intrusion detection is the process of monitoring network traffic to identify suspicious connections, attack patterns, and unauthorized activity.

It helps security teams detect attacks that may not be visible through endpoint monitoring alone.

2. What activities can be detected?

- Port scanning.
- Brute-force attempts.
- Suspicious DNS queries.
- Malware communication.
- Unauthorized connections.
- Unusual data transfers.
- Network exploitation attempts.
- Command-and-control communication.

3. How network intrusion detection works

A Scalable Framework for Real-Time Network Security Traffic Analysis and Attack Detection Using Machine and Deep Learning
mdpi.com

1. Network sensors collect traffic or metadata.
2. Detection engines analyze the collected information.
3. Suspicious patterns trigger alerts.
4. Analysts investigate the alerts.
5. The security team responds to confirmed threats.

4. Important tools

- Zeek.
- Suricata.

- Snort.
- Wireshark.

Important definition

Network Intrusion Detection: The process of monitoring network activity to identify potential attacks, suspicious traffic, and unauthorized access.

Cybersecurity application

Network security analysts use intrusion detection to discover attacks, investigate suspicious connections, and identify compromised systems.

Advanced Topic 2: Network Forensics

1. What is network forensics?

Network forensics is the process of collecting, preserving, examining, and analyzing network-related evidence to investigate cybersecurity incidents.

In simple words:

It helps investigators understand what happened on a network during an attack.

2. What evidence is examined?

- Packet captures.
- Firewall logs.
- DNS records.
- Proxy logs.
- Network flow records.
- VPN connection logs.
- IDS/IPS alerts.
- Server connection logs.

3. Network forensic investigation process

[What Is Network Forensics? Basics, Importance, And Tools
g2.com]

1. Identification: Identify the incident and relevant evidence.
2. Collection: Collect available network data.
3. Preservation: Protect evidence from alteration.
4. Examination: Extract relevant information.
5. Analysis: Reconstruct events and identify suspicious activity.
6. Reporting: Document findings and conclusions.

4. Important forensic concepts

Packet Capture (PCAP): A recorded collection of network packets.

Network Flow: A summarized record of communication between network endpoints, often including source, destination, ports, protocol, and traffic volume.

Chain of Custody: A documented record of who collected, handled, transferred, and accessed evidence.

Timeline Analysis: Arranging network events chronologically to reconstruct an incident.

5. Important tools

- Wireshark.
- Zeek.
- tcpdump.
- NetworkMiner.
- Arkime.

Important definition

Network Forensics: The investigation of network evidence to reconstruct security incidents, identify attack activity, and establish what occurred.

Cybersecurity application

Incident responders use network forensics to investigate malware infections, data exfiltration, unauthorized access, and suspicious external communication.

Advanced Topic 3: Lateral Movement Detection

1. What is lateral movement?

Lateral movement is a technique attackers use to move from one compromised system to other systems inside a network.

For example:

An attacker compromises an employee's laptop and then attempts to access a file server or administrator workstation.

Lateral Movement, Dwell Time, and the Role of Microsegmentation
pagonetworks.com

2. How lateral movement happens

A simplified attack sequence:

1. Attacker compromises an endpoint.
2. Attacker obtains or abuses credentials.
3. Attacker discovers other network systems.

4. Attacker attempts remote access.
5. Attacker moves to additional systems.
6. Attacker attempts to access sensitive information.

3. Common lateral movement techniques

- Remote service abuse.
- Credential theft.
- Pass-the-Hash.
- Pass-the-Ticket.
- Remote Desktop Protocol abuse.
- SSH credential abuse.
- Exploiting vulnerable internal services.
- Abusing administrative tools.

4. How to detect lateral movement

Monitor for:

- Unusual remote logins.
- Unexpected administrative activity.
- Authentication from unfamiliar devices.
- Unusual SMB connections.
- Unexpected RDP connections.
- Abnormal use of PowerShell or remote administration tools.
- Connections between network segments that normally do not communicate.
- Multiple failed logins followed by successful access.

5. Prevention techniques

- Use network segmentation.
- Apply least privilege.
- Protect privileged credentials.
- Use MFA.
- Monitor endpoint activity.
- Restrict remote administration.
- Deploy EDR.
- Monitor authentication events.

Important definition

Lateral Movement: An attack technique in which an attacker moves from one compromised system to other systems within a network to gain additional access.

Cybersecurity application

SOC analysts and threat hunters investigate lateral movement to identify how attackers spread through an organization.

Advanced Topic 4: Network Access Architecture

1. What is network access architecture?

Network access architecture defines how users, devices, applications, and services connect to a network and how their access is controlled.

It establishes:

- Who can connect.
- Which devices can connect.
- Which resources are accessible.
- How identity is verified.
- How access is monitored.

2. Traditional network access architecture

Traditional enterprise networks often use a perimeter-based approach.

Free Network Diagram Maker | Create Network Topology Online - ConceptViz
conceptviz.app

The organization protects its internal network using:

- Firewalls.
- VPN gateways.
- Network access controls.
- Internal network segmentation.

A common weakness is assuming that everything inside the network is trustworthy.

That assumption is unsafe.

3. Modern network access architecture

Modern organizations increasingly combine:

- Identity-based access.
- Device security checks.
- Network segmentation.

- Application-level access controls.
- Continuous monitoring.
- Zero Trust principles.

4. Important technologies

Technology	Purpose
NAC	Controls device access
VPN	Provides protected network connectivity
ZTNA	Provides identity- and context-based access to applications
SASE	Combines networking and security services through a cloud-delivered architecture
SD-WAN	Software-defined wide-area networking
802.1X	Controls network access through port-based authentication

Important definition

Network Access Architecture: The design of network connectivity and access-control mechanisms that determine how users and devices securely access organizational resources.

Cybersecurity application

Network architects use access architecture to protect enterprise environments, cloud infrastructure, remote workers, and distributed organizations.

Advanced Topic 5: Zero Trust Network Architecture

Important for cloud security

1. What is Zero Trust?

Zero Trust is a security approach based on the principle:

Never trust automatically; always verify access.

It means users, devices, and systems should not receive implicit trust simply because they are inside an organization's network.

Every access request must be evaluated using appropriate identity, device, policy, and contextual information.

2. Why is Zero Trust necessary?

Traditional network security often assumes that internal network users are trustworthy.

But attackers may:

- Steal employee credentials.
- Compromise employee laptops.

- Gain access through a VPN.
- Exploit internal servers.
- Move between network segments.

Zero Trust reduces reliance on the traditional trusted network perimeter.

3. Core Zero Trust principles

A. Verify Explicitly

Authenticate and authorize access using available identity and contextual signals.

B. Least Privilege Access

Grant only the permissions needed for a specific task.

C. Assume Breach

Design systems with the expectation that an attacker may already have compromised part of the environment.

D. Continuous Evaluation

Reassess access based on relevant changes in identity, device status, risk, or policy.

4. Zero Trust architecture

1 Summary — NIST SP 1800-27 documentation
nccoe.nist.gov

Important components:

Component	Purpose
Policy Engine	Decides whether access should be permitted
Policy Administrator	Establishes or terminates access
Policy Enforcement Point	Enforces the access decision
Identity Provider	Verifies user identity
Device Security	Evaluates device compliance and health
Monitoring	Tracks access and security events

5. Example

Imagine an employee wants to access a company's cloud database.

A Zero Trust system may evaluate:

1. Is the employee's identity verified?
2. Does the employee have permission?
3. Is MFA completed?

4. Is the device compliant?
5. Is the requested resource appropriate for the employee?
6. Does current risk information allow access?

If the conditions are not satisfied, access can be denied or restricted.

6. Zero Trust vs traditional security

Traditional perimeter model	Zero Trust
Trust often depends heavily on network location	Trust is not automatically granted based on location
Strong focus on network boundaries	Strong focus on identity, resources, and policy
Internal access may receive broader trust	Access is explicitly evaluated
Network perimeter is central	Continuous verification and enforcement are central

7. Zero Trust and cloud security

Zero Trust is particularly relevant to:

- AWS.
- Microsoft Azure.
- Google Cloud.
- Kubernetes environments.
- Remote workforces.
- SaaS applications.
- Hybrid cloud infrastructure.

Important: Zero Trust is not a single product, firewall, or VPN replacement. It is an architectural security approach that requires coordinated identity, device, network, application, and monitoring controls.

Important definition

Zero Trust Network Architecture (ZTNA): An approach to network and application access in which access is explicitly verified and granted according to identity, device status, policy, and context rather than assumed from network location.

Cybersecurity application

Cloud security engineers use Zero Trust principles to secure remote access, cloud workloads, internal applications, and privileged administrative operations.

4. Important Differences — Quick Comparison

Concepts	Main difference
Firewall vs IDS	Firewall controls traffic; IDS detects suspicious activity
IDS vs IPS	IDS detects; IPS can actively block
VPN vs ZTNA	VPN commonly provides network connectivity; ZTNA provides policy-controlled access to specific resources
NAC vs Firewall	NAC controls which devices can connect; firewall controls network traffic
Antivirus vs EDR	Antivirus focuses on malware protection; EDR provides broader endpoint detection and response
Vulnerability management vs patch management	Vulnerability management handles weaknesses broadly; patch management handles software updates
Backup vs Disaster Recovery	Backup copies data; disaster recovery restores systems and services
Network segmentation vs Microsegmentation	Segmentation divides networks; microsegmentation applies finer-grained workload or application controls
Network monitoring vs Network forensics	Monitoring observes ongoing activity; forensics investigates evidence to reconstruct events
Zero Trust vs Traditional perimeter	Zero Trust explicitly evaluates access instead of assuming internal network trust

5. Practical Tools You Should Learn

These tools are relevant to the topics in this phase. Learn what each does before trying to master every tool.

File:Wireshark icon.svg - Wikimedia Commons

1. Wireshark

Inspect network packets and investigate communication.

Official website
Nmap Project Logo & Brand Assets (SVG, PNG and vector) - Brandfetch

2. Nmap

Discover network hosts, ports, and services.

Official website

Suricata Installation and IDS/IPS Mode Configuration on Ubuntu | by Hacızadə Fidan | Medium

3. Suricata

Network intrusion detection and prevention.

Official website
TryHackMe | Zeek

4. Zeek

Analyze network activity and produce detailed connection logs.

Official website
Bringing AI to Wazuh SIEM: How MCP Server Enables Natural Language Security Queries | by Syed Jawad | Medium

5. Wazuh

Security monitoring, endpoint visibility, and configuration assessment.

Official website
OpenVAS Pricing & Reviews 2026 | Techjockey.com

6. Greenbone OpenVAS

Scan systems for known vulnerabilities.

Official website
Automate CIS Compliance on Ubuntu with Ansible

7. Lynis

Audit Linux security configurations and identify hardening opportunities.

Official website

6. Practical Exercises

Use your own Linux system, Windows system, or isolated virtual machines. Do not scan or test networks without authorization.

Exercise 1: Network discovery and analysis

Tools: Nmap and Wireshark.

Tasks:

1. Identify your own machine's IP address.
2. Identify its active network interfaces.
3. Identify listening services on your own machine.
4. Capture traffic generated by your own test activities.
5. Identify DNS, TCP, and HTTPS-related traffic.
6. Record the services and ports you discovered.

Expected learning: Network architecture, network discovery, ports, protocols, and packet analysis.

Exercise 2: Firewall security

Tools: UFW or Windows Firewall.

Tasks:

1. Review current firewall status.
2. Identify allowed and blocked rules.
3. Identify unnecessary exposed services.
4. Configure a test firewall policy.
5. Verify that only intended services are accessible.
6. Document the changes.

Expected learning: Firewall architecture and network access control.

Exercise 3: IDS and network monitoring

Tools: Suricata or Zeek.

Tasks:

1. Install the tool in an isolated lab.
2. Capture traffic from your lab network.
3. Analyze generated network logs.
4. Identify DNS requests and connection attempts.
5. Examine alerts generated by safe test traffic.
6. Document suspicious and normal activity.

Expected learning: Network intrusion detection and monitoring.

Exercise 4: Linux hardening

Tasks:

1. Review user accounts.
2. Inspect file permissions.
3. Review running services.
4. Inspect listening ports.
5. Review SSH configuration.
6. Review firewall rules.
7. Apply available security updates.
8. Run a security audit using Lynis.

Expected learning: Linux system hardening.

Exercise 5: Windows hardening

Tasks:

1. Review Windows Defender status.
2. Inspect Windows Firewall.
3. Review local user accounts.
4. Check Windows updates.
5. Review startup applications.
6. Inspect security events in Event Viewer.
7. Check whether BitLocker is available and enabled.
8. Document security improvements.

Expected learning: Windows system hardening.

Exercise 6: Vulnerability management

Tools: Greenbone/OpenVAS or Nessus Essentials, where available.

Tasks:

1. Set up an isolated vulnerable virtual machine.
2. Identify its IP address.
3. Scan only that authorized lab target.
4. Review discovered vulnerabilities.
5. Identify CVE references where available.
6. Understand severity ratings.
7. Prioritize remediation.
8. Apply a fix.
9. Rescan and verify.

Expected learning: Vulnerability assessment and remediation.

7. Major Practical Project — Enterprise Network Security Lab

Phase 9 project

Build a small enterprise network in a virtual lab and secure it.

7.1 Proposed architecture

Build a Realistic SOC Home Lab with pfSense & Wazuh | Gude Venkata Chaithanya posted on the topic | LinkedIn
linkedin.com

Your lab should contain:

System	Purpose
Kali Linux VM	Authorized security testing
Ubuntu Server VM	Linux server
Windows VM	Windows endpoint
Firewall VM	Network traffic control
IDS/IPS	Network threat detection
Monitoring system	Log collection and analysis

Your laptop has 16 GB RAM, so do not run every virtual machine simultaneously. Start with two or three lightweight VMs and add systems gradually.

7.2 Project stages

Stage	Task
1	Design the network topology
2	Configure network interfaces
3	Configure firewall rules
4	Create network segments
5	Harden Linux
6	Harden Windows
7	Configure IDS monitoring
8	Implement secure remote access
9	Perform vulnerability scanning
10	Collect and analyze logs
11	Investigate suspicious activity
12	Document findings and improvements

7.3 Final project deliverables

1. Network architecture diagram.
2. IP address and subnet plan.
3. Firewall configuration report.
4. Linux hardening report.
5. Windows hardening report.
6. IDS/IPS monitoring evidence.
7. Vulnerability assessment report.
8. Network monitoring evidence.
9. Backup and recovery plan.
10. Final security assessment.

8. Important Definitions — Revision Notes

Term	Definition
Network Architecture	The design of network components and their communication.
Firewall	A system that controls network traffic using security rules.
IDS	A system that detects suspicious activity.
IPS	A system that detects and can block malicious activity.
NAC	A system that controls which devices can access a network.
VPN	A protected network connection over another network.
Wireless Security	Protection of wireless networks and communications.
DNS	A system that translates domain names into IP addresses.
Email Security	Protection against email-based threats and unauthorized access.
Endpoint Security	Protection of network-connected devices.
Server Hardening	Reducing server attack surfaces through secure configuration.
Linux Hardening	Securing Linux systems through permissions, updates, and configuration.
Windows Hardening	Securing Windows systems through security controls and configuration.
Network Segmentation	Dividing a network into separate sections with controlled communication.

Secure Remote Access	Protected remote connectivity with authentication and access controls.
Vulnerability Management	Identifying, prioritizing, fixing, and monitoring security weaknesses.
Patch Management	Testing and deploying software updates.
Backup	A separate copy of data used for restoration.
Disaster Recovery	Restoring systems and services after a major disruption.
Infrastructure Monitoring	Continuously observing infrastructure for problems and threats.
Network Forensics	Investigating network evidence to reconstruct security incidents.
Lateral Movement	Moving from one compromised system to other systems in a network.
Zero Trust	A security approach that requires explicit verification before granting access.
SIEM	A platform that collects and analyzes security events from multiple sources.

9. Revision Questions

Try answering these without looking at the notes.

Phase 9 knowledge test

/ answered

10. Key Takeaways

1. Network architecture determines how systems communicate and where security controls are applied.
2. Firewalls control traffic, while IDS and IPS detect and help prevent intrusions.
3. NAC controls device access to organizational networks.
4. VPNs protect remote network connectivity, but VPN access does not automatically make a user trustworthy.
5. DNS and email require dedicated security controls.
6. Endpoint, Linux, Windows, and server hardening reduce attack surfaces.
7. Network segmentation limits unauthorized communication and lateral movement.
8. Vulnerability management identifies and prioritizes weaknesses; patch management helps fix software vulnerabilities.
9. Backups and disaster recovery protect business continuity.
10. Monitoring and network forensics help security teams detect and investigate attacks.

11. Zero Trust requires explicit access verification instead of assuming that internal network users are trustworthy.

11. Phase 9 Completion Checklist

Phase 9 progress

0/22

I understand network architecture security.
I understand firewall types and rules.
I can explain IDS and IPS.
I understand NAC and 802.1X.
I understand VPN technologies and risks.
I understand wireless network security.
I understand DNS attacks and DNSSEC.
I understand SPF, DKIM, and DMARC.
I understand endpoint security and EDR.
I can explain server hardening.
I can perform basic Linux hardening checks.
I can perform basic Windows hardening checks.
I understand network segmentation.
I understand secure remote access.
I understand vulnerability management and CVSS.
I understand patch management.
I understand backup, RPO, and RTO.
I understand infrastructure monitoring and SIEM.
I understand network intrusion detection and forensics.
I understand lateral movement detection.

I understand Zero Trust architecture.
I have completed my practical network security lab.

Phase 10 — Defensive Security and SOC

Beginner to advanced
6 modules

1. Learning Objectives

The objective of this phase is to develop the skills required to detect, investigate, analyze, and respond to cyberattacks.

By the end of this phase, you should be able to:

1. Understand how a Security Operations Center (SOC) operates.
2. Understand the responsibilities of a SOC analyst.
3. Collect and analyze Windows, Linux, network, and security logs.
4. Understand how SIEM platforms collect and analyze security events.
5. Configure Wazuh and investigate security alerts.
6. Understand antivirus, EDR, XDR, and endpoint telemetry.
7. Use threat intelligence and MITRE ATT&CK.
8. Perform basic threat hunting and detection engineering.
9. Follow an incident response process.
10. Build a home SOC lab and investigate simulated attacks.

What you are learning: Offensive security teaches you how attacks happen. Defensive security teaches you how to recognize those attacks, understand their impact, and stop them.

2. Module 1 — SOC Fundamentals

2.1 What is a SOC?

SOC stands for Security Operations Center.

A SOC is a centralized team or function responsible for continuously monitoring an organization's systems, detecting cybersecurity threats, investigating security alerts, and coordinating incident response.

In simple words:

A SOC is the security control center of an organization.

How does a SOC work daily? - Euvic
euvic.com

A SOC may operate 24 hours a day, 7 days a week, depending on the organization's requirements.

2.2 Why do organizations need a SOC?

Organizations use computers, servers, networks, cloud platforms, and applications.

These systems can be targeted by:

- Malware.
- Ransomware.
- Phishing.
- Credential theft.
- Unauthorized access.
- Data theft.
- Insider threats.
- Network attacks.

A SOC helps organizations detect these threats and respond before damage becomes worse.

2.3 SOC architecture

A basic SOC architecture looks like this:

What is SOC? Security Operations Center Explained
ctrlaltnod.com

Main components:

Component	Purpose
Data Sources	Generate security-related information
Log Collectors	Collect logs from different systems
SIEM	Centralizes and analyzes security events
Detection Rules	Identify suspicious activity
Alert Management	Organizes and prioritizes alerts
SOC Analysts	Investigate suspicious activity
Incident Response	Contains and resolves confirmed incidents
Threat Intelligence	Provides information about known threats

2.4 SOC analyst responsibilities

A SOC analyst monitors an organization's security environment and investigates suspicious activity.

Main responsibilities:

1. Monitor security dashboards.
2. Review security alerts.
3. Investigate suspicious login attempts.
4. Analyze system and network logs.
5. Identify malware activity.
6. Investigate unusual network connections.
7. Determine the severity of security incidents.
8. Escalate serious incidents.
9. Document investigations.
10. Support incident response.

2.5 SOC analyst levels

Entry level

L1 — SOC Analyst

- Monitor security alerts.
- Perform initial alert triage.
- Identify false positives.
- Collect basic evidence.
- Escalate suspicious incidents.

Intermediate

L2 — SOC Analyst

- Investigate complex alerts.
- Analyze endpoint and network activity.
- Correlate multiple security events.
- Determine incident scope.
- Recommend containment actions.

Advanced

L3 — Senior SOC Analyst / Threat Hunter

- Investigate advanced threats.
- Develop detection rules.

- Perform threat hunting.
- Analyze sophisticated attack techniques.
- Improve security monitoring.

Important: These levels are common industry conventions, not a universal standard. Job responsibilities vary between organizations.

2.6 Security monitoring

Security monitoring is the continuous observation of systems, networks, applications, and user activity to identify suspicious behavior.

Examples:

- Multiple failed login attempts.
- Login from an unusual location.
- Unexpected administrator activity.
- Malware detection.
- Unusual outbound network traffic.
- Unexpected changes to important files.

2.7 Alert triage

Alert triage is the process of reviewing a security alert to decide whether it requires further investigation.

Example:

A SIEM generates this alert:

```
Alert: Multiple Failed Login Attempts

Username: admin
Source IP: 192.168.1.50
Failed Attempts: 25
Time: 10:30 AM
```

The SOC analyst investigates:

1. Is the source IP known?
2. Is the target account important?
3. Were the attempts successful?
4. Are there other suspicious events?
5. Is this normal user behavior or potential brute-force activity?

The analyst then decides whether to close, investigate, or escalate the alert.

2.8 Incident classification

Security incidents are classified according to their type, severity, and impact.

Classification	Example
Informational	Routine security event
Low	Suspicious activity with limited impact
Medium	Potential unauthorized access
High	Confirmed compromise of an important endpoint
Critical	Major ransomware incident affecting business operations

Severity classifications vary between organizations.

2.9 Escalation procedures

Escalation means transferring an alert or incident to a more experienced analyst or the appropriate response team.

Example:

```
L1 SOC Analyst
      |
      v
Initial investigation
      |
      v
Suspicious activity confirmed
      |
      v
L2 SOC Analyst
      |
      v
Incident investigation
      |
      v
Incident Response Team
```

Important definitions

- SOC: A centralized security function responsible for monitoring, detecting, investigating, and responding to cybersecurity threats.
- SOC Analyst: A security professional who investigates alerts and monitors systems for suspicious activity.

- Alert Triage: The initial evaluation and prioritization of security alerts.
- Incident Classification: Categorizing a security incident according to its type and severity.
- Escalation: Transferring an alert or incident to a higher-level analyst or response team.

Cybersecurity application

SOC analysts use these processes to monitor enterprise networks, identify compromised accounts, investigate suspicious endpoints, and coordinate incident response.

3. Module 2 — Security Monitoring

3.1 What is security monitoring?

Security monitoring is the process of collecting and examining activity records from computer systems, applications, networks, and security devices to identify suspicious behavior.

The most important source of information for security monitoring is logs.

3.2 What is a log?

A log is a record of an event that occurred in a computer system or application.

Examples:

- A user logged in.
- A user entered an incorrect password.
- A file was modified.
- A service started.
- A firewall blocked a connection.
- An application encountered an error.

Example log:

```
Time: 2026-11-05 10:30:00
User: admin
Event: Failed Login
Source IP: 192.168.1.50
Status: Failure
```

A log helps security analysts understand what happened, when it happened, and which system or user was involved.

3.3 Types of security logs

A. System logs

System logs record operating system and system-level activities.

Examples:

- System startup.
- Shutdown.
- Service failures.
- Hardware-related events.
- System errors.
- Configuration changes.

Why important?

System logs help identify unexpected system behavior, failures, and suspicious changes.

B. Network logs

Network logs record information about network communications.

They may contain:

- Source IP address.
- Destination IP address.
- Source port.
- Destination port.
- Protocol.
- Connection status.
- Data volume.
- Connection timestamps.

Example:

```
Source IP: 192.168.1.10
Destination IP: 8.8.8.8
Protocol: UDP
Destination Port: 53
Status: Allowed
```

This represents a DNS-related network connection.

C. Firewall logs

Firewall logs record network traffic that a firewall allows, blocks, or otherwise processes.

Example:

```
Time: 11:00 AM
Source IP: 203.0.113.10
Destination IP: 192.168.1.20
```

```
Port: 22
Action: Blocked
```

A SOC analyst may investigate repeated attempts to connect to an exposed SSH service.

D. Authentication logs

Authentication logs record user login and identity-verification activities.

Examples:

- Successful login.
- Failed login.
- Account lockout.
- Password change.
- MFA failure.
- Privileged account activity.

These logs are particularly important for detecting brute-force attacks and compromised accounts.

E. Windows Event Logs

Windows records system, application, and security events through its event logging system.

Important Windows Event Log categories:

Log	Purpose
Application	Application-related events
Security	Login, authentication, and security events
System	Operating system and service events
Setup	Installation and setup events
Forwarded Events	Events received from other computers

Important Windows Event IDs:

Event ID	Meaning
4624	Successful logon
4625	Failed logon
4634	Logoff
4648	Logon using explicit credentials
4672	Special privileges assigned to a new logon
4688	Process creation, when auditing is enabled

4720	User account created
4726	User account deleted
4740	User account locked out
1102	Security audit log cleared

Important: Event IDs provide context, not proof of an attack. For example, Event ID 4625 can occur because a legitimate user entered an incorrect password.

F. Linux system logs

Linux systems use logging services such as:

- systemd-journald.
- rsyslog.
- syslog-ng.

Common log locations include:

File or location	Purpose
`/var/log/syslog`	General system messages on some distributions
`/var/log/auth.log`	Authentication events on Debian-based systems
`/var/log/secure`	Authentication events on some Red Hat-based systems
`/var/log/kern.log`	Kernel-related messages on some distributions
`journalctl`	Reads systemd journal entries

Useful commands:

```
journalctl
```

Displays system journal entries.

```
journalctl -b
```

Displays logs from the current boot.

```
journalctl -p warning
```

Displays warning-level and higher-priority messages.

```
sudo tail -f /var/log/auth.log
```

Follows authentication logs on systems that use this file.

Note: Log file names and locations vary by Linux distribution and configuration.

G. DNS logs

DNS logs record DNS queries and related responses.

Example:

```
Time: 12:00:00
Client: 192.168.1.10
Query: example.com
Type: A
Response: 93.184.216.34
```

SOC analysts examine DNS activity to identify:

- Connections to malicious domains.
- DNS tunneling.
- Suspicious domain lookups.
- Malware command-and-control communication.
- Unusual DNS query patterns.

H. Endpoint telemetry

Endpoint telemetry is security-related information collected from computers, servers, and other endpoint devices.

It can include:

- Running processes.
- Parent-child process relationships.
- File modifications.
- Network connections.
- User logins.
- Registry changes on Windows.
- Command-line activity.
- Security tool alerts.

Example:

```
Process: powershell.exe
Parent Process: winword.exe
User: employee
Command Line: powershell.exe ...
```

This may deserve investigation because an Office application launching PowerShell can be suspicious, although context determines whether it is malicious.

3.4 Why logs matter in SOC operations

Consider this attack scenario:

1. An attacker obtains a user's password.

2. The attacker logs into a workstation.
3. The attacker runs suspicious commands.
4. The attacker connects to another server.
5. The attacker attempts to access sensitive files.

Different logs provide different parts of the story.

Log source	Evidence
Authentication logs	Unusual login
Endpoint logs	Suspicious process
Network logs	Connection to another server
File activity logs	Sensitive file access
Firewall logs	Unusual outbound connection

Combining these events gives analysts a more complete picture than examining one log in isolation.

Important definitions

- Log: A recorded event from a system, application, or network device.
- System Log: A record of operating-system activities.
- Authentication Log: A record of identity-verification activities.
- Firewall Log: A record of traffic processed by a firewall.
- Endpoint Telemetry: Security-related information collected from endpoint devices.

Cybersecurity application

SOC analysts use logs to reconstruct attacks, identify suspicious activity, establish timelines, and determine which systems may be compromised.

4. Module 3 — SIEM

Core SOC skill

4.1 What is SIEM?

SIEM stands for Security Information and Event Management.

A SIEM platform collects security logs from different systems, analyzes the events, identifies suspicious patterns, and generates security alerts.

In simple words:

A SIEM brings security information from different parts of an organization into one central platform so analysts can investigate it.

4.2 Why is SIEM necessary?

Imagine an organization has:

- 500 employee computers.
- 30 servers.
- 5 firewalls.
- 2,000 user accounts.
- Multiple cloud applications.

Every system generates logs.

Manually checking every log is inefficient. A SIEM centralizes this information and helps analysts detect suspicious activity across the environment.

4.3 SIEM architecture

Open Source SIEM Architectures Scale Beyond Single-Server Deployments
linuxsecurity.com

A typical SIEM architecture contains:

Component	Purpose
Log Sources	Generate event data
Log Collectors	Gather logs from systems
Ingestion Pipeline	Receive and process events
Normalization	Convert events into a consistent structure
Storage	Store searchable event data
Correlation Engine	Identify relationships between events
Detection Rules	Identify suspicious patterns
Dashboards	Display security information
Alerting	Notify analysts about potential threats

4.4 Log collection

Log collection is the process of gathering event records from different sources.

Common log sources include:

- Windows computers.
- Linux servers.
- Firewalls.

- Routers.
- DNS servers.
- Applications.
- Cloud platforms.
- Endpoint protection systems.

There are different ways to collect logs:

A. Agent-based collection

An agent runs on the monitored system and forwards relevant information to the SIEM.

B. Agentless collection

Logs are collected without installing a dedicated agent on every monitored system. Examples include syslog forwarding, APIs, and Windows event forwarding.

C. Cloud-based collection

Cloud services send their security events to a SIEM using supported integrations, APIs, or connectors.

4.5 Log normalization

Different systems record events in different formats.

Example:

Windows might record a failed login as:

```
EventID: 4625
AccountName: admin
Status: Failure
```

Another application might record it as:

```
event_type: authentication_failure
username: admin
result: denied
```

Log normalization converts these different formats into a common structure.

A normalized event might look like:

```
{
  "event.category": "authentication",
  "event.action": "login",
  "user.name": "admin",
  "event.outcome": "failure",
  "source.ip": "192.168.1.50"
}
```

Why is normalization important?

It allows analysts and detection rules to search and correlate events from different systems consistently.

4.6 Correlation rules

Correlation rules identify relationships between multiple events.

Example:

An account experiences:

- 20 failed login attempts.
- A successful login shortly afterward.
- Login from an unfamiliar source.
- Unusual administrative activity.

Individually, these events might not prove an attack. Together, they may indicate a compromised account.

A simplified detection rule:

```
IF
    failed_login_count >= 10
AND
    successful_login occurs shortly afterward
THEN
    generate a suspicious_login alert
```

This is an illustrative rule. A production detection should account for time windows, account type, source context, expected behavior, and false positives.

4.7 Alert generation

An alert is a notification generated when a detection rule or security analytic identifies suspicious activity.

Example:

```
Alert Name: Possible Brute-Force Attack


Username: admin

Source IP: 192.168.1.50

Failed Attempts: 25

Time Window: 5 minutes

Severity: High

Status: New
```

The analyst investigates the alert to determine whether it represents a real threat.

Important: An alert is not automatically a confirmed security incident.

4.8 SIEM dashboards

Dashboards present security information in a visual format.

They may display:

- Number of security alerts.
- Failed login attempts.
- Successful logins.
- High-severity incidents.
- Top source IP addresses.
- Malware detections.
- Endpoint health.
- Network traffic trends.

Wazuh SIEM. Introduction | by Cristiano Soares | Medium
medium.com

Dashboards help SOC analysts understand the security condition of an environment.

4.9 Threat hunting

Threat hunting is the proactive process of searching an organization's systems and data for suspicious activity that may not have triggered existing security alerts.

Traditional detection:

```
Security tool detects suspicious activity
             |
             v
         Alert created
             |
             v
        Analyst investigates
```

Threat hunting:

```
Analyst develops a hypothesis
             |
             v
Searches logs and telemetry
             |
```

```
             v
Investigates suspicious patterns
             |
             v
Finds evidence or rejects hypothesis
             |
             v
Improves detection
```

Example hypothesis:

"An attacker may be using PowerShell to execute suspicious commands on employee computers."

The analyst searches endpoint telemetry for unusual PowerShell execution, parent processes, command lines, and related network connections.

4.10 SIEM tools

Homepage

1. Splunk

Collects, searches, analyzes, and visualizes machine-generated data. Splunk Enterprise Security adds security analytics and SIEM capabilities.

Official website
Elastic Security Price, Features and Reviews in 2026 | Techjockey US

2. Elastic Security

Uses the Elastic Stack for log analysis, security detection, investigation, and monitoring.

Official website
Bringing AI to Wazuh SIEM: How MCP Server Enables Natural Language Security Queries | by Syed Jawad | Medium

3. Wazuh

Provides log analysis, endpoint monitoring, file integrity monitoring, vulnerability detection, and security configuration assessment.

Official website
Day 10: Advent of Cyber 2025 — TryHackMe (Updated) | by ms.chalo_cy | Medium

4. Microsoft Sentinel

A cloud-native SIEM and security operations platform that collects data, detects threats, supports investigation, and automates response workflows.

Official website

4.11 Important differences

Term	Meaning

Log collection	Gathering event records
Log normalization	Converting records into a consistent structure
Correlation	Connecting related events
Detection rule	Logic that identifies suspicious activity
Alert	Notification of potential suspicious activity
Dashboard	Visual representation of security information
Threat hunting	Proactively searching for threats

Important definition

SIEM: A security platform that collects, centralizes, analyzes, and correlates event data to identify and investigate potential cybersecurity threats.

Cybersecurity application

SOC analysts use SIEM platforms to investigate authentication attacks, detect suspicious processes, correlate network activity, monitor endpoints, and prioritize incidents.

5. Module 4 — Endpoint Detection

5.1 What is endpoint detection?

Endpoint detection involves monitoring computers, servers, and other endpoint devices to identify suspicious activities, malware, and security threats.

An endpoint can be:

- Windows laptop.
- Linux server.
- Desktop computer.
- Cloud virtual machine.
- Workstation.

Endpoint security is important because attackers frequently target individual devices to obtain credentials, execute malware, and gain access to internal networks.

5.2 Antivirus

Antivirus is security software designed to detect, block, quarantine, and remove malicious software.

Common threats include:

- Viruses.

- Trojans.
- Worms.
- Spyware.
- Ransomware.
- Malicious scripts.

How antivirus works

An antivirus product may use:

A. Signature-Based Detection

Compares files against known malware signatures.

B. Heuristic Detection

Examines suspicious characteristics of files and programs.

C. Behavior-Based Detection

Monitors program activity for malicious behavior.

Limitations

Traditional antivirus protection may fail to detect certain new, modified, or fileless attacks. This is one reason organizations use broader endpoint detection capabilities.

5.3 EDR

EDR stands for Endpoint Detection and Response.

EDR collects endpoint activity, detects suspicious behavior, supports investigations, and provides response capabilities.

What Is Endpoint Detection and Response (EDR)? | CloudSEK
cloudsek.com

EDR may monitor:

- Process creation.
- Command execution.
- File changes.
- Network connections.
- Registry changes.
- User activity.
- Suspicious scripts.

Example:

A Word document launches PowerShell, which downloads an executable and attempts to run it.

An EDR system may correlate these actions and generate an alert.

EDR response capabilities

Depending on the product and configuration, EDR may allow analysts to:

- Isolate a compromised endpoint.
- Terminate a suspicious process.
- Quarantine a malicious file.
- Collect investigation evidence.
- Search activity across endpoints.
- Investigate the attack timeline.

5.4 XDR

XDR stands for Extended Detection and Response.

XDR expands security detection and response beyond individual endpoints by correlating information from multiple security domains.

These may include:

- Endpoints.
- Network devices.
- Email systems.
- Identity platforms.
- Cloud environments.
- Security applications.

XDR Solution | Microsoft Security
microsoft.com

Example:

1. An employee receives a phishing email.
2. The employee clicks a malicious link.
3. A suspicious login occurs.
4. A suspicious process starts on the employee's computer.
5. The device connects to a malicious destination.

XDR can correlate these events to provide a more complete incident picture.

5.5 Sysmon

Sysmon stands for System Monitor.

It is a Windows system service and device driver from Microsoft Sysinternals that records detailed system activity in Windows Event Logs.

Sysmon can record:

- Process creation.
- Network connections.
- File creation.
- File deletion.
- Driver loading.
- Registry changes.
- DNS queries.
- Process access.

Example

Suppose an unusual PowerShell process starts on a Windows computer.

Sysmon may record:

- Process image.
- Command line.
- Parent process.
- User.
- Timestamp.
- Process ID.

This information helps analysts investigate what happened.

Important: Sysmon collects telemetry. It is not itself a complete antivirus or EDR solution. It requires appropriate configuration and analysis.

Official resource: Microsoft Sysmon documentation.

5.6 Endpoint monitoring

Endpoint monitoring is the continuous collection and analysis of endpoint activity.

A SOC analyst may monitor:

Activity	What to investigate
New process	Unexpected executable
PowerShell activity	Suspicious commands
New user account	Unauthorized account creation

File modification	Unexpected changes to important files
Network connection	Unknown external destination
Privilege changes	Unexpected administrator privileges
Scheduled task	Suspicious persistence mechanism

5.7 Behavioral detection

Behavioral detection identifies suspicious activities based on what a program, user, or device does rather than relying exclusively on known malware signatures.

Example:

A user normally logs in from one device during business hours.

Suddenly:

- The user logs in from an unusual location.
- Multiple systems are accessed.
- Large amounts of data are downloaded.
- New administrative permissions are assigned.

These behaviors may indicate account compromise.

Antivirus vs EDR vs XDR

Feature	Antivirus	EDR	XDR
Malware detection	Yes	Yes	Yes, through integrated sources
Process monitoring	Varies	Yes	Yes, through endpoint data
Endpoint investigation	Limited to product capabilities	Strong	Strong
Network visibility	Usually limited	Endpoint-related connections	Broader network integration
Email visibility	Usually limited	Varies	May integrate email telemetry
Cross-domain correlation	Limited	Primarily endpoint-focused	Yes

Important definitions

- Antivirus: Software designed to detect, block, and remove malicious programs.

- EDR: A technology that monitors endpoints, detects threats, supports investigation, and enables response.
- XDR: A security approach that correlates detection and response data across multiple security domains.
- Sysmon: A Windows system monitoring service that records detailed system activity in Event Logs.
- Behavioral Detection: Identifying threats based on suspicious patterns of activity.

Cybersecurity application

SOC analysts use endpoint telemetry and EDR to investigate suspicious processes, malware execution, credential theft, persistence, and lateral movement.

6. Module 5 — Threat Intelligence

6.1 What is threat intelligence?

Threat intelligence is information about cyber threats that has been collected, analyzed, and organized to help organizations make security decisions.

It helps answer:

- Who may be attacking?
- What systems are being targeted?
- What techniques are being used?
- What indicators should security teams monitor?
- How can the organization defend itself?

Threat intelligence is more than collecting malicious IP addresses. Useful intelligence connects indicators with context, attack methods, confidence, and relevance to the organization.

6.2 IOC — Indicator of Compromise

IOC stands for Indicator of Compromise.

An IOC is an observable piece of evidence that may indicate a system has been compromised.

Examples:

- Malicious IP address.
- Malicious domain.
- File hash associated with malware.
- Suspicious file path.
- Known malicious email address.
- Malware-related registry entry.

Example:

```
Suspicious Domain: malware-example.test
File Hash: Example hash value
Source IP: 203.0.113.50
```

These are illustrative values, not verified indicators of a real threat.

Important: An IOC does not prove an attack by itself. Indicators can be outdated, shared, or associated with legitimate infrastructure.

6.3 IOA — Indicator of Attack

IOA stands for Indicator of Attack.

An IOA is a behavior or activity pattern that suggests an attack may be occurring.

Examples:

- An Office application launches an unusual script interpreter.
- A user attempts to disable security software.
- An account accesses many unrelated systems.
- A process attempts to steal credentials.
- A system makes suspicious connections to external infrastructure.

IOC vs IOA

IOC	IOA
Focuses on observable evidence	Focuses on suspicious behavior
Often identifies known indicators	Can identify attack activity without a known malware signature
Example: known malicious file hash	Example: suspicious credential-dumping behavior

6.4 Threat feeds

A threat feed is a source of threat-related information that can be updated and consumed by security systems.

A feed may provide:

- Malicious IP addresses.
- Suspicious domains.
- Malware hashes.
- Phishing URLs.
- Threat actor information.

- Vulnerability information.

Examples of threat intelligence resources:

- AlienVault OTX — Official website.
- AbuseIPDB — Official website.
- VirusTotal — Official website.
- URLhaus — Official website.
- MalwareBazaar — Official website.

Privacy caution: Do not upload confidential files, private documents, or proprietary data to public analysis services without authorization. Submitted samples may become accessible to other researchers or service users, depending on the service.

6.5 MITRE ATT&CK mapping

MITRE ATT&CK is a knowledge base describing tactics and techniques used by real-world adversaries.

It helps security teams understand how attackers operate and organize detection strategies.

Important ATT&CK concepts

Tactic: The attacker's objective.

Technique: The method used to achieve that objective.

Sub-technique: A more specific variation of a technique.

Examples of tactics

Tactic	Purpose
Initial Access	Enter an environment
Execution	Run malicious code
Persistence	Maintain access
Privilege Escalation	Gain higher permissions
Defense Evasion	Avoid security controls
Credential Access	Obtain credentials
Discovery	Learn about the environment
Lateral Movement	Move between systems
Collection	Gather information
Exfiltration	Steal or transfer data
Impact	Disrupt or damage systems

Example attack sequence

Exploring MITRE ATT&CK Framework: Enhancing IT Audit and Cybersecurity
linkedin.com

Consider a simulated attack:

1. An employee opens a malicious attachment.
2. A script executes.
3. The attacker establishes persistence.
4. Credentials are accessed.
5. The attacker moves to another system.
6. Sensitive data is collected.
7. Data is transferred outside the organization.

A SOC analyst maps observed behaviors to relevant MITRE ATT&CK techniques.

Official resource: MITRE ATT&CK knowledge base.

6.6 Threat hunting

Threat hunting is the proactive search for hidden threats in systems, networks, and security data.

Example hypothesis:

"An attacker may be attempting to steal credentials from Windows endpoints."

The analyst searches for:

- Suspicious access to credential-related processes.
- Unusual process execution.
- Abnormal account activity.
- Related network connections.
- Unexpected privilege changes.

A threat hunt should have a hypothesis, relevant data sources, a defined search method, and documented results.

6.7 Detection engineering

Detection engineering is the process of designing, testing, deploying, and improving security detections.

A detection engineer develops rules to identify suspicious behavior.

Example:

```
Detection:
```

```
Repeated failed login attempts followed by
a successful login from the same source.

Data source:
Authentication logs.

Purpose:
Identify possible brute-force activity.

Response:
Generate an alert for analyst investigation.
```

Detection engineering involves:

1. Understanding attack behavior.
2. Identifying required telemetry.
3. Writing detection logic.
4. Testing against known activity.
5. Measuring false positives and false negatives.
6. Deploying the detection.
7. Reviewing and improving it.

Important definitions

- Threat Intelligence: Analyzed information about cyber threats used to support security decisions.
- IOC: Observable evidence that may indicate a system compromise.
- IOA: A behavior or activity pattern that may indicate an attack.
- Threat Feed: A regularly updated source of threat information.
- MITRE ATT&CK: A knowledge base describing adversary tactics and techniques.
- Threat Hunting: Proactively searching for hidden or undetected threats.
- Detection Engineering: Developing and improving rules and analytics to detect malicious activity.

Cybersecurity application

SOC teams use threat intelligence to enrich alerts, map attacker behavior, improve detection rules, and identify threats that may affect their organization.

7. Module 6 — Incident Response

Essential defensive skill

7.1 What is incident response?

Incident response is the organized process of identifying, containing, investigating, and recovering from cybersecurity incidents.

Examples:

- Ransomware infection.
- Data breach.
- Compromised account.
- Malware infection.
- Unauthorized server access.
- Suspicious data transfer.

The goal is to reduce damage, restore normal operations, and prevent similar incidents.

7.2 Incident response lifecycle

Your syllabus uses six stages:

Incident Response for Security+: Key Procedures Overview
infosecinstitute.com

Stage 1: Preparation

Preparation means establishing the people, procedures, tools, and resources required to handle security incidents.

Activities:

- Create an incident response plan.
- Assign responsibilities.
- Configure monitoring.
- Prepare backup systems.
- Maintain contact information.
- Conduct response exercises.
- Establish evidence-handling procedures.

Example:

An organization prepares a response plan for ransomware incidents before one occurs.

Stage 2: Identification

Identification means determining whether suspicious activity represents a security incident.

Activities:

- Review alerts.
- Analyze logs.
- Examine affected systems.
- Identify suspicious processes.
- Determine possible attack methods.
- Estimate the incident's scope.

Example:

A SOC analyst investigates an alert showing suspicious PowerShell activity and unusual outbound network connections.

Stage 3: Containment

Containment means limiting the spread or impact of an active security incident.

Possible actions:

- Isolate a compromised endpoint.
- Disable a compromised account.
- Block malicious network communication.
- Restrict affected services.
- Apply temporary access restrictions.

Important: Containment should be planned carefully. Disconnecting or shutting down a system can destroy volatile evidence or disrupt critical services.

Stage 4: Eradication

Eradication means removing the attacker's presence and the underlying cause of the incident.

Activities:

- Remove malware.
- Eliminate persistence mechanisms.
- Remove unauthorized accounts.
- Patch exploited vulnerabilities.
- Reset compromised credentials.
- Correct insecure configurations.

Stage 5: Recovery

Recovery means restoring affected systems and services to normal operation.

Activities:

- Restore clean data.
- Rebuild compromised systems.
- Reconnect systems safely.
- Monitor for reinfection.
- Verify system integrity.
- Confirm that required services work correctly.

Stage 6: Lessons Learned

Lessons learned means reviewing the incident to understand what happened and improve security.

Questions to investigate:

- How did the attacker enter?
- Which systems were affected?
- Which detection controls worked?
- What was missed?
- How long did detection take?
- How can the organization prevent recurrence?

The team should document findings and update security controls and procedures.

7.3 Incident response example — Ransomware

Imagine a company detects ransomware on an employee workstation.

Stage	Example action
Preparation	Existing response plan and backups are available
Identification	EDR detects suspicious file encryption
Containment	Isolate the affected workstation
Eradication	Remove malicious software and fix the entry point
Recovery	Rebuild the workstation and restore clean data
Lessons Learned	Improve email security, endpoint controls, and backup procedures

7.4 Important incident response terms

False Positive: An alert that indicates suspicious activity when the activity is actually legitimate.

False Negative: Malicious activity that a detection system fails to identify.

Containment: Limiting the impact or spread of an incident.

Eradication: Removing the attacker's access, malware, and underlying causes.

Recovery: Restoring affected systems and operations.

Root Cause Analysis: Identifying the underlying reason an incident occurred.

Important definition

Incident Response: The organized process of detecting, investigating, containing, removing, and recovering from cybersecurity incidents.

Cybersecurity application

Incident responders use this lifecycle to manage malware infections, ransomware, compromised accounts, data breaches, and attacks against enterprise infrastructure.

8. Practical Projects — Build Your Home SOC Lab

Main Phase 10 project

The objective is to create a small security operations environment where you can collect logs, generate security events, investigate alerts, and document simulated attacks.

Project 1: Build a Home SOC Lab

8.1 Required architecture

Proof of Concept guide · Wazuh documentation
documentation.wazuh.com

Your lab will contain:

Component	Purpose
Wazuh Server	Central log collection, analysis, and alerting
Windows VM	Generate Windows security events
Linux VM	Generate Linux system and authentication events
Kali Linux VM	Perform controlled security testing
SOC Dashboard	View alerts and investigate activity

8.2 Recommended software

Software	Purpose
VirtualBox or VMware	Run virtual machines

Wazuh	SIEM/XDR-style security monitoring
Windows 10/11 evaluation VM	Windows log generation
Ubuntu Server	Linux log generation
Kali Linux	Authorized security testing
Sysmon	Detailed Windows telemetry
Wireshark	Network traffic analysis

Official resources:

- Wazuh documentation
- VirtualBox
- Microsoft Evaluation Center
- Ubuntu Server
- Kali Linux

8.3 Hardware planning

Your laptop has 16 GB RAM. That is enough to start learning SOC fundamentals, but running a full SIEM, Windows, Linux, and Kali simultaneously can become resource-intensive.

Start with a smaller setup:

Machine	Suggested RAM
Wazuh server	4–6 GB
Windows endpoint	4 GB
Ubuntu endpoint	1–2 GB
Host operating system	Remaining memory

Run only the machines you need for each exercise. Start with the Wazuh server and one endpoint. Add the second endpoint later.

A full Wazuh installation includes several services, and its resource requirements depend on the number of monitored endpoints and event volume. Check the official sizing documentation before deployment.

8.4 Build sequence

1. Install a virtualization platform.
2. Create an isolated virtual network.
3. Install Ubuntu Server.
4. Deploy Wazuh using its documented installation method.

5. Install a Wazuh agent on Ubuntu.
6. Install a Windows virtual machine.
7. Install a Wazuh agent on Windows.
8. Configure Windows event collection.
9. Install Sysmon on Windows.
10. Generate safe test events.
11. Investigate alerts in the Wazuh dashboard.

Network safety: Keep the lab isolated from your personal devices and production networks. Do not expose the Wazuh dashboard or agent-management services directly to the public internet.

Project 2: Collect Windows and Linux Logs

Windows log collection

Tasks:

1. Open Event Viewer.
2. Navigate to Windows Logs.
3. Open the Security log.
4. Find successful login events.
5. Find failed login events.
6. Inspect the event details.
7. Forward the relevant events to Wazuh.
8. Verify that they appear in the dashboard.

Useful Event IDs:

Event ID	Event
4624	Successful logon
4625	Failed logon
4688	Process creation, if enabled
4720	User account created
4740	User account locked out

Linux log collection

Tasks:

1. Open a terminal.
2. Examine system logs.

3. Examine authentication logs.
4. Generate a few controlled failed login attempts against your own lab account.
5. Review the resulting log entries.
6. Forward relevant logs to Wazuh.
7. Verify that the events are visible.

Useful commands:

```
journalctl
```

```
journalctl -b
```

```
sudo journalctl -u ssh
```

```
sudo tail -f /var/log/auth.log
```

The final command applies to systems that use `/var/log/auth.log`.

Expected learning: You should be able to identify the source, timestamp, user, event type, and outcome of a security event.

Project 3: Configure Wazuh

What you need to configure

1. Wazuh server.
2. Wazuh dashboard.
3. Windows agent.
4. Linux agent.
5. Security event collection.
6. File integrity monitoring.
7. Vulnerability detection.
8. Alert investigation.

What to investigate

Activity	What to look for
Failed login	Repeated authentication failures
New user	Unexpected account creation
File modification	Changes to monitored files
Service change	Unexpected service activity
Suspicious process	Unusual process execution
Vulnerability	Known weaknesses in installed software

Expected outcome

You should be able to open the Wazuh dashboard, identify monitored endpoints, locate generated alerts, inspect event details, and explain why an alert was generated.

Project 4: Generate Test Security Events

You need to generate activity that your monitoring system can observe.

Use only your own lab.

Test event	Expected observation
Failed login attempts	Authentication failures
Successful login	Successful authentication event
Create a test user	Account creation event
Modify a monitored file	File integrity event
Start or stop a service	Service-related system event
Run a test process	Process telemetry, if configured

For Windows, use Sysmon and Windows auditing to improve visibility.

For Linux, use systemd journal and authentication logs.

Do not assume every event automatically generates an alert. Log collection and alert generation are separate functions. You may need to configure a detection rule.

Project 5: Create Detection Rules

A detection rule defines conditions that indicate suspicious activity.

Start with simple rules.

Rule A: Repeated failed logins

```
IF
    one account has 10 failed login attempts
    within 5 minutes

THEN
    generate an alert
```

Rule B: Unexpected administrator creation

```
IF
    a new privileged account is created

THEN
```

```
    generate a high-priority alert
```

Rule C: Suspicious process activity

```
IF
    a process launches an unusual child process
    from an unexpected parent application

THEN
    generate an investigation alert
```

Rule D: Important file modification

```
IF
    a monitored sensitive file changes unexpectedly

THEN
    generate an alert
```

These are detection ideas, not ready-to-deploy rules. The actual implementation depends on your SIEM's rule syntax, event fields, logging configuration, and baseline activity.

Detection engineering checklist

- Identify the data source.
- Identify the relevant event.
- Define the suspicious condition.
- Write the detection rule.
- Generate test activity.
- Confirm whether the rule triggers.
- Review false positives.
- Improve the rule.
- Document the result.

Project 6: Investigate a Simulated Attack

Scenario: Suspicious Windows workstation activity

A Windows employee workstation generates the following events:

Time	Event
10:00	Multiple failed logins
10:05	Successful login

10:07 Unexpected PowerShell process

10:09 Suspicious outbound connection

10:12 Unexpected file modification

Your investigation

Answer these questions:

1. Which account was involved?
2. Did the login attempts eventually succeed?
3. Which process was executed?
4. What was the parent process?
5. Which destination did the endpoint contact?
6. Which files were modified?
7. Is there evidence of compromise?
8. What should be contained?
9. What evidence should be preserved?
10. What actions should be taken to recover safely?

Final report format

Section	Required information
Incident summary	What happened
Detection source	Which tool generated the alert
Timeline	Events in chronological order
Affected assets	Systems and accounts involved
Evidence	Relevant logs and telemetry
Analysis	What the evidence indicates
Severity	Impact and urgency
Containment	Actions to limit damage
Eradication	Removal of malicious activity
Recovery	Safe restoration
Recommendations	Improvements to prevent recurrence

9. SOC Investigation Workflow

The following is the basic workflow you should practice until it becomes familiar.

1. Security Event

System, endpoint, network, or user activity

2. SIEM Detection

Detection rule identifies suspicious activity

3. Alert Triage

Review context, severity, and reliability

4. Investigation

Correlate logs, endpoints, users, and network events

5. Classification

False positive, suspicious activity, or confirmed incident

6. Response and Escalation

Contain, escalate, eradicate, and recover as required

7. Documentation

Record evidence, findings, and recommendations

10. Important Differences — Quick Revision

Concept	Difference
SOC vs SIEM	SOC is the security operations function; SIEM is a technology used by it
Log vs Alert	Log records an event; alert identifies potentially suspicious activity
SIEM vs EDR	SIEM analyzes events from multiple sources; EDR focuses on endpoint detection and response
EDR vs XDR	EDR focuses on endpoints; XDR correlates security information across multiple domains
IOC vs IOA	IOC is observable evidence; IOA is suspicious behavior
Threat Intelligence vs Threat Hunting	Intelligence provides analyzed threat information; hunting proactively searches for threats
Threat Hunting vs Detection Engineering	Hunting investigates possible threats; detection engineering builds repeatable detection logic

False Positive vs False Negative	False positive flags legitimate activity; false negative misses malicious activity
Containment vs Eradication	Containment limits damage; eradication removes the threat
Recovery vs Lessons Learned	Recovery restores operations; lessons learned improve future security

11. Important Definitions — Revision Notes

Term	Definition
SOC	A centralized security function responsible for monitoring and responding to cyber threats.
SOC Analyst	A professional who investigates security alerts and monitors systems.
Alert Triage	Initial examination and prioritization of security alerts.
Security Monitoring	Continuous observation of systems and networks for threats.
Log	A record of an event in a system or application.
Endpoint Telemetry	Security-related activity data collected from endpoint devices.
SIEM	A platform that collects and analyzes security events from multiple sources.
Log Collection	Gathering logs from different systems.
Log Normalization	Converting different log formats into a consistent structure.
Correlation Rule	Logic that connects related events to identify suspicious activity.
Threat Hunting	Proactively searching for threats that may not have triggered alerts.
Antivirus	Software that detects and blocks malicious programs.
EDR	Endpoint Detection and Response.
XDR	Extended Detection and Response.
Sysmon	A Windows system monitoring service that records detailed activity.
Threat Intelligence	Analyzed information about cybersecurity threats.
IOC	Indicator of Compromise.
IOA	Indicator of Attack.
Threat Feed	An updated source of threat-related information.
MITRE ATT&CK	A knowledge base of adversary tactics and techniques.
Detection	The development and improvement of security detection rules.

Engineering	
Incident Response	The organized process of investigating and managing security incidents.
False Positive	A legitimate activity incorrectly identified as suspicious.
False Negative	Malicious activity that a detection system fails to identify.
Containment	Limiting the spread or impact of a security incident.
Eradication	Removing the attacker's presence and the underlying cause.
Recovery	Restoring affected systems and services.
Root Cause Analysis	Identifying the underlying reason an incident occurred.

12. Practical Skills Checklist

By the end of Phase 10, you should be capable of performing these tasks:

- Explain how a SOC operates.
- Explain L1, L2, and L3 analyst responsibilities.
- Investigate a failed login alert.
- Read Windows Event Logs.
- Read Linux authentication and system logs.
- Understand firewall and DNS logs.
- Explain endpoint telemetry.
- Explain SIEM architecture.
- Understand log collection and normalization.
- Create a basic correlation rule.
- Navigate a SIEM dashboard.
- Explain the differences between antivirus, EDR, and XDR.
- Investigate Sysmon events.
- Understand IOC and IOA.
- Map observed activity to MITRE ATT&CK.
- Perform a basic threat hunt.
- Create and test a detection rule.
- Follow all six incident response stages.
- Configure Wazuh.
- Collect logs from Windows and Linux.
- Investigate a simulated attack.
- Prepare a professional incident report.

14. Recommended Learning Order

Do not try to master all four SIEM platforms simultaneously. Learn one platform properly, then understand how the others differ.

Order	Skill	Priority
1	SOC fundamentals	Essential
2	Windows and Linux logs	Essential
3	SIEM fundamentals	Essential
4	Wazuh practical implementation	Essential
5	Sysmon and endpoint telemetry	High
6	Detection rules and correlation	High
7	MITRE ATT&CK	High
8	Threat hunting	High
9	Incident response	Essential
10	Splunk, Elastic, Microsoft Sentinel	Additional platform experience

Phase 11 — Digital Forensics and Malware Analysis

Beginner to advanced

1. Learning objectives

By the end of this phase, you should be able to:

1. Understand how digital evidence is identified, collected, preserved, and examined.
2. Maintain evidence integrity using a proper chain of custody.
3. Examine disks, memory, filesystems, browsers, and network activity.
4. Identify important Windows and Linux forensic artifacts.
5. Build timelines to reconstruct what happened on a computer.
6. Understand how deleted files can sometimes be recovered.
7. Classify malware and distinguish static from dynamic analysis.
8. Examine suspicious files without executing them on your normal computer.
9. Understand executable formats such as PE and ELF.
10. Use tools such as Autopsy, Volatility, Ghidra, YARA, and REMnux.

Important distinction: Digital forensics investigates evidence to reconstruct events. Malware analysis investigates suspicious software to understand what it does. They overlap, but they are not the same discipline.

Module 1: Digital Forensics Fundamentals

1.1 What is digital forensics?

Definition: Digital forensics is the process of identifying, collecting, preserving, examining, and presenting digital evidence in a systematic manner.

It helps investigators understand what happened on a digital device and whether there is evidence of unauthorized activity.

Practical example

Suppose an employee's laptop is suspected of being compromised.

An investigator may need to determine:

- When the suspicious activity started.
- Which files were accessed or modified.
- Whether malware was installed.
- Which external devices were connected.
- Whether information was transferred over a network.
- Which user account was involved.

Digital forensics attempts to answer these questions using evidence rather than assumptions.

Where is digital forensics used?

- Cybercrime investigations
- Corporate security investigations
- Data theft investigations
- Incident response
- Insider threat investigations
- Ransomware investigations
- Legal and regulatory investigations

1.2 What is digital evidence?

Definition: Digital evidence is information stored or transmitted in digital form that may help establish facts about an incident or investigation.

Examples include:

Evidence	What it may reveal
Documents	Created or modified information
System logs	Login and system activity
Browser history	Websites visited
Emails	Communication and attachments
Network captures	Network communication
Memory dump	Processes and information in RAM
Disk image	Files and filesystem information

USB artifacts	External device connections

Types of digital evidence

1. Volatile evidence

Information that may disappear when a device loses power or its current state changes.

Examples:

- Running processes
- Active network connections
- RAM contents
- Logged-in user sessions

2. Non-volatile evidence

Information that generally remains stored after the device is powered off.

Examples:

- Hard disk files
- SSD files
- System logs
- Browser databases
- Saved documents

Important: Volatile evidence may need to be collected before shutting down a system. However, evidence collection must follow an approved procedure because live collection itself can change the system.

1.3 Chain of custody

Definition: Chain of custody is a documented record of who collected, handled, transferred, examined, and stored digital evidence.

Its purpose is to demonstrate that evidence has been properly controlled and has not been improperly altered.

Example

Imagine investigators collect a laptop from an office.

1. Evidence collection

   The investigator records the laptop's make, model, serial number, condition, date, time, and location.

2. Evidence identification

   The device receives a unique evidence ID.

3. Evidence preservation

The investigator follows the appropriate procedure to preserve the original evidence.

4. Evidence transfer

   Every transfer is documented with the person responsible, date, time, and purpose.

5. Evidence examination

   The investigator examines a forensic copy whenever possible, rather than directly analyzing the original.

6. Evidence storage

   The original is stored securely with access controls and documented handling.

What should a chain-of-custody record contain?

- Evidence ID
- Description of the device or file
- Collection date and time
- Collector's name
- Collection location
- Hash values, where applicable
- Every transfer and handler
- Storage location
- Examination details

What is hashing?

Definition: Hashing converts data into a fixed-length value using a mathematical algorithm.

Common forensic hash algorithms include:

- MD5
- SHA-1
- SHA-256

SHA-256 is generally preferred for verifying evidence integrity.

Example:

If a forensic image has a SHA-256 hash before analysis and the same hash after transfer, that supports the conclusion that its contents have not changed.

A matching hash does not prove that evidence was collected correctly or that it was authentic before collection. It only supports integrity verification from the point at which the hash was calculated.

1.4 Disk forensics

Definition: Disk forensics is the examination of storage devices to identify, recover, and analyze digital evidence.

Storage devices include:

- HDD (Hard Disk Drive)
- SSD (Solid State Drive)
- USB flash drives
- Memory cards
- External storage devices

What can disk forensics reveal?

- Existing files
- Deleted file remnants
- File creation and modification times
- Installed applications
- User activity
- Malware files
- Partition information
- Hidden or suspicious files

What is a forensic disk image?

A forensic disk image is a copy of a storage device created for examination.

It may contain:

- Files and folders
- Filesystem metadata
- Unallocated space
- Partition information
- Other accessible storage structures

A forensic image should preserve as much relevant source information as the acquisition method allows.

Important distinction:

Normal file copy	Forensic disk image
Copies selected files	Can capture an entire storage device or partition

Usually excludes deleted-file remnants	May preserve unallocated space
Does not normally preserve all filesystem metadata	Can preserve filesystem structures and metadata
Suitable for ordinary file transfer	Designed for forensic examination

Write blockers

A write blocker is a hardware or software mechanism that prevents writes to a storage device during acquisition.

This helps prevent accidental modification of the original evidence.

For physical forensic acquisition, a validated hardware write blocker is generally preferable when compatible with the device and acquisition procedure.

1.5 Memory forensics

Definition: Memory forensics is the examination of a computer's RAM to identify evidence of running processes, malware, network activity, and other system behavior.

RAM can contain information that is not available in a disk image.

What can memory forensics reveal?

- Running processes
- Hidden or suspicious processes
- Active network connections
- Loaded modules
- Command-line information
- Injected code
- Some encryption keys and credentials, depending on the system and circumstances

Practical example

Suppose a computer contains ransomware.

The ransomware executable may have been deleted, but its process or related artifacts could still be present in a captured memory image.

Memory analysis may help identify:

- The suspicious process
- Its parent process
- Loaded libraries
- Network connections
- Evidence of code injection

Disk forensics vs memory forensics

Disk forensics	Memory forensics
Examines storage devices	Examines RAM
Focuses on stored information	Focuses on information in memory
Can examine files and filesystem artifacts	Can examine running processes and connections
Data usually persists after shutdown	Volatile data is generally lost after shutdown

Tool: Volatility is commonly used to analyze memory captures.

1.6 File system forensics

Definition: File system forensics is the examination of filesystem structures and metadata to understand how files were stored, accessed, changed, and deleted.

A filesystem manages how an operating system organizes data on storage devices.

Common filesystems:

Filesystem	Common use
NTFS	Windows
FAT32	USB drives and older storage
exFAT	Flash drives and removable storage
ext4	Linux
XFS	Linux servers
APFS	macOS

Important filesystem concepts

1. File metadata

Information describing a file, such as its name, size, permissions, and timestamps.

2. File allocation

The way a filesystem tracks which storage blocks belong to a file.

3. Unallocated space

Storage space that the filesystem currently considers available for reuse.

4. File slack

Unused space within the final allocated storage unit of a file. Depending on the filesystem and circumstances, it may contain remnants of earlier data.

5. Filesystem journal

A record used by certain filesystems to help maintain consistency after interruptions.

Example

An investigator finds that a document was modified shortly before a suspicious login.

Filesystem metadata may help establish when the document was changed. However, timestamps can be modified or affected by system settings, so they must be interpreted with other evidence.

1.7 Browser forensics

Definition: Browser forensics is the examination of web browser data to understand a user's browsing activity and interactions.

Browsers store different types of information locally.

Important browser artifacts

Artifact	What it can reveal
Browsing history	Websites visited
Download history	Files downloaded
Cookies	Website-related session and tracking data
Cache	Copies of previously accessed web resources
Bookmarks	Saved websites
Autofill data	Previously entered form information
Saved passwords	Stored login credentials, if accessible
Session data	Some recently opened tabs and browsing state

Practical example

Suppose an employee is suspected of downloading confidential company files.

An investigator might examine:

- Browser download history.
- Downloaded file metadata.
- Browser cache.
- Relevant timestamps.
- Network logs showing the download.

Browser history alone cannot prove that a person actually viewed or downloaded a particular file. Corroborating evidence is necessary.

1.8 Network forensics

Definition: Network forensics is the collection and analysis of network traffic and related records to investigate suspicious communication or security incidents.

It helps investigators understand how devices communicated over a network.

What can network forensics reveal?

- Communication between IP addresses.
- Suspicious connections.
- Data transfers.
- DNS queries.
- Unusual network protocols.
- Possible command-and-control communication.
- Evidence of data exfiltration.

Example

A computer unexpectedly communicates with an unfamiliar external server.

An investigator can examine:

1. Source and destination IP addresses.
2. Destination ports.
3. DNS queries.
4. Connection timestamps.
5. Transferred data volume.
6. Available packet contents.

Important network evidence

- PCAP files: captured network packets.
- Firewall logs: permitted or blocked connections.
- DNS logs: domain lookups.
- Proxy logs: web requests routed through a proxy.
- NetFlow records: summarized network communication.

Tool: Wireshark is used to inspect captured network packets.

1.9 Windows artifacts

Definition: Windows artifacts are files, registry entries, logs, and other traces generated by Windows that may provide evidence of user and system activity.

These artifacts are especially useful when investigating suspicious activity on Windows computers.

Important Windows forensic artifacts

Artifact	Investigative value
Windows Event Logs	Login, system, and application events
Registry	System configuration and user activity traces
Prefetch	Evidence related to application execution
Windows shortcuts (`.lnk`)	References to files and locations
Jump Lists	Some recently accessed files and application activity
Recycle Bin	Information about deleted files
USB device records	Evidence of connected removable devices
Amcache	Application inventory and execution-related information
SRUM	Some application and network resource usage information
Browser databases	Browser history, downloads, and related activity

Important Windows Event Log locations

- Security: `C:\Windows\System32\winevt\Logs\Security.evtx`
- System: `C:\Windows\System32\winevt\Logs\System.evtx`
- Application: `C:\Windows\System32\winevt\Logs\Application.evtx`

Useful Windows Event IDs

Event ID	Meaning
4624	Successful logon
4625	Failed logon
4634	Account logoff
4648	Logon using explicit credentials
4672	Special privileges assigned to a new logon
4688	Process creation, when auditing is enabled
4720	User account created
4726	User account deleted
4740	User account locked
1102	Security audit log cleared

Important: An event ID does not automatically prove malicious activity. Investigators must examine the account, time, source, surrounding events, and system context.

1.10 Linux artifacts

Definition: Linux forensic artifacts are logs, configuration files, shell records, filesystem metadata, and other traces that can help reconstruct activity on a Linux system.

Important Linux artifacts

Artifact	Investigative value
`/var/log/auth.log`	Authentication and authorization events on some distributions
`/var/log/secure`	Authentication-related logs on some distributions
`/var/log/syslog`	General system messages on some distributions
`/var/log/messages`	General system messages on some distributions
`/var/log/kern.log`	Kernel messages where configured
`~/.bash_history`	Bash command history, if saved
`/etc/passwd`	Local user account information
`/etc/shadow`	Password hash and account aging information; restricted access
`/etc/sudoers`	Sudo privilege configuration
`/var/log/wtmp`	Login and logout records
`/var/log/btmp`	Failed login records, where maintained
`/var/log/lastlog`	Last login information, where maintained
systemd journal	System and service logs

Useful Linux commands

View system logs:

```
journalctl
```

View logs from the current boot:

```
journalctl -b
```

View authentication-related logs on a system using this file:

```
sudo tail -f /var/log/auth.log
```

View recent login records:

```
last
```

View failed login records, if available:

```
sudo lastb
```

Check the history file of the current user:

```
cat ~/.bash_history
```

Important limitations:

- Shell history can be disabled, incomplete, or modified.
- Logs may rotate or be deleted.
- Different distributions use different log configurations.
- A command in shell history does not prove that it executed successfully.

1.11 Timeline analysis

Definition: Timeline analysis is the process of arranging digital events in chronological order to reconstruct what happened during an incident.

A timeline combines timestamps from different evidence sources.

Example

An investigator suspects that malware was installed on a workstation.

Time	Event
09:00	User logs into Windows
09:05	Browser downloads a suspicious file
09:07	File appears in Downloads
09:08	Suspicious executable starts
09:09	New outbound network connection appears
09:12	Security software raises an alert

This timeline helps connect events that might otherwise look unrelated.

Important timestamps

- Created time
- Modified time
- Accessed time
- Deleted time, where recoverable
- Log event time
- Network connection time
- Process start time

What is a timestamp problem?

Different evidence sources may use different time zones or timestamp formats.

For example:

- One log may use UTC.
- Another may use local time.
- A device may have an incorrect system clock.
- Some filesystems record timestamps with different precision.

Investigators must normalize timestamps carefully and document any assumptions.

1.12 Deleted file recovery

Definition: Deleted file recovery is the process of attempting to retrieve files that have been deleted from a storage device.

Deleting a file does not always immediately erase its contents.

How deletion works

In many filesystems, deleting a file initially removes or changes the filesystem's references to its data. The storage space may then be marked available for reuse.

If the original data has not been overwritten, recovery may be possible.

Example

Suppose a user deletes a document from a USB drive.

An investigator may find:

- The original file entry.
- Filesystem metadata.
- Data fragments in unallocated space.
- A recoverable copy of the file.

Factors affecting recovery

Factor	Effect
Data overwritten	Recovery becomes harder or impossible
TRIM on SSD	May make recovery difficult
Encryption	Data may be inaccessible without the required key
Filesystem damage	May complicate recovery
Time since deletion	More use of the device may reduce recovery chances

Important: SSD recovery is not the same as HDD recovery. TRIM and internal garbage collection can cause deleted data to become unrecoverable. Do not promise recovery simply because a file was recently deleted.

Module 2: Digital Forensics Tools

2.1 Autopsy

TryHackMe — Autopsy. Learn how to use Autopsy to investigate… | by Y3T1_ | Medium
medium.com

Definition: Autopsy is a digital forensics platform used to examine disk images and investigate files, deleted data, browser activity, and other artifacts.

Main uses

- Disk image analysis
- File recovery
- Browser history examination
- Keyword searches
- File metadata examination
- Timeline creation
- Hash-based identification

Practical example

You receive a forensic image of a USB drive.

Using Autopsy, you can:

1. Create a case.
2. Add the forensic image as a data source.
3. Run the available analysis modules.
4. Examine existing and deleted files.
5. Search for suspicious filenames.
6. Examine file metadata.
7. Export relevant findings for a report.

Official website: Autopsy

2.2 FTK Imager

AccessData FTK Imager 3.1 Download (Free) - FTK Imager.exe
accessdata-ftk-imager.software.informer.com

Definition: FTK Imager is a forensic acquisition and examination tool used to create and inspect forensic images of storage devices.

Main uses

- Acquire forensic images.
- Preview files and folders.
- Examine storage structures.
- Calculate hash values.
- Export selected evidence.

Autopsy vs FTK Imager

Autopsy	FTK Imager
Focuses on forensic examination and analysis	Focuses on acquisition and evidence preview
Supports timelines and artifact analysis	Supports forensic image creation and verification
Useful for investigating disk images	Useful for acquiring and checking evidence
Open-source platform	Proprietary tool with free availability options

Official resource: FTK Imager

2.3 Volatility

Memory Forensics using Volatility3 | by bl7ck0ut | Medium
bl7ck0ut.medium.com

Definition: Volatility is a memory forensics framework used to analyze captured RAM images.

Main uses

- Identify processes.
- Examine process relationships.
- Investigate loaded modules.
- Examine network-related artifacts.
- Search for suspicious memory activity.
- Investigate possible process injection.

Volatility 3 is the current major version to learn.

Example commands:

```
vol -h
```

This displays available options.

```
vol -f memory.raw windows.info
```

This attempts to identify information about a Windows memory image.

```
vol -f memory.raw windows.pslist
```

This lists processes identified by the relevant Windows plugin.

```
vol -f memory.raw windows.pstree
```

This displays process relationships in a tree-like structure.

The exact commands depend on the installed Volatility version and the memory image.

Official documentation: Volatility 3 documentation

2.4 Plaso

Deadhost Investigation and Super Timeline :: fiskeren.github.io
fiskeren.github.io

Definition: Plaso is a digital forensics framework used to extract timestamps and create timelines from different evidence sources.

Main uses

- Extract timestamped events.
- Analyze filesystem metadata.
- Examine supported log files.
- Combine events from multiple sources.
- Create a chronological investigation timeline.

Important terms

- `log2timeline`: Tool used to extract timestamped events.
- `psort`: Tool used to process, filter, and export timeline data.

Official project: Plaso

2.5 Tool selection guide

Investigation task	Suitable tool
Examine a disk image	Autopsy
Acquire a forensic image	FTK Imager
Analyze a RAM capture	Volatility
Inspect network packets	Wireshark
Build a forensic timeline	Plaso

Key point: No single tool can perform every forensic task. Investigators often combine several tools and compare their results.

Module 3: Malware Analysis Fundamentals

3.1 What is malware?

Definition: Malware is software intentionally designed to damage systems, disrupt operations, steal information, spy on users, or gain unauthorized access.

Malware is short for malicious software.

Common malware types

Malware type	Definition
Virus	Malicious code that attaches to another file or program and spreads when executed
Worm	Malware that can spread between systems without needing to attach to another program
Trojan	Malicious software disguised as legitimate software
Ransomware	Malware that encrypts or otherwise blocks access to data and demands payment
Spyware	Software that secretly collects information
Keylogger	Software or hardware that records keystrokes
Rootkit	Software designed to hide malicious activity or maintain privileged access
Botnet malware	Malware that connects infected devices to a remotely controlled network
Downloader	Malware that retrieves additional malicious components
Backdoor	A mechanism that provides unauthorized or hidden access

Important differences

Virus vs worm

- A virus generally depends on a host file or program.
- A worm can spread independently across systems.

Trojan vs ransomware

- A Trojan describes how malicious software is disguised or delivered.
- Ransomware describes malicious behavior involving extortion and denial of access to data.

A single malware family can fit more than one category.

3.2 Malware analysis

Definition: Malware analysis is the process of examining suspicious software to understand its functionality, behavior, capabilities, and potential impact.

Why is malware analysis performed?

- Understand how malware operates.
- Identify files and systems affected.
- Discover persistence mechanisms.
- Identify network communication.
- Extract indicators of compromise.
- Develop security detections.
- Support incident response.

Example

Suppose an organization discovers an unknown executable on an employee's laptop.

An analyst wants to determine:

- Is it malicious?
- Does it steal files?
- Does it connect to an external server?
- Does it create new processes?
- Does it modify system settings?
- How can defenders detect it?

Malware analysis helps answer these questions.

3.3 Malware classification

Malware classification is the process of grouping malicious software according to its behavior, purpose, structure, or family.

Classification methods

1. Based on behavior

- File encryption
- Credential theft
- Data exfiltration
- Process injection
- Remote access

2. Based on purpose

- Financial theft
- Espionage

- Destruction
- Unauthorized access
- Extortion

3. Based on technical characteristics

- File-based malware
- Fileless malware
- Script-based malware
- Bootkits
- Memory-resident malware

What is fileless malware?

Definition: Fileless malware is malware that relies heavily on legitimate system tools, scripts, memory, or other components rather than operating primarily through a traditional malicious executable stored on disk.

It can still create files or leave disk artifacts. "Fileless" does not mean that absolutely no files exist.

3.4 Static analysis

Definition: Static analysis is the examination of a suspicious file without executing it.

It focuses on the file's structure, contents, metadata, and code.

What can static analysis reveal?

- File type
- File size
- File hashes
- Embedded strings
- Imported functions
- Executable sections
- Suspicious URLs or IP addresses
- Possible packing or obfuscation

Example

An unknown executable contains strings such as:

```
http://example.invalid
cmd.exe
powershell.exe
CreateRemoteThread
```

These strings may suggest that the program uses command execution, PowerShell, or process-related functionality.

However, strings alone cannot prove malicious behavior. They may be unused, misleading, or part of legitimate software.

Advantages

- Does not require running the malware.
- Can reveal useful information quickly.
- Helps identify suspicious file structures.
- Can be performed before controlled execution.

Limitations

- Malware may hide strings.
- Code may be packed or encrypted.
- Static analysis cannot directly show runtime behavior.
- A file may contain misleading information.

3.5 Dynamic analysis

Definition: Dynamic analysis is the examination of software while it is running in a controlled environment.

It helps an analyst observe actual program behavior.

What can dynamic analysis reveal?

- Files created or modified.
- Processes started.
- Registry changes.
- Network connections.
- Services installed.
- Scheduled tasks created.
- Attempts to modify security settings.
- Attempts to persist after restart.

Example

An analyst executes a suspicious sample inside an isolated malware-analysis VM.

During execution, monitoring shows:

1. The program creates a file in a temporary directory.
2. It starts another process.

3. It attempts an outbound network connection.
4. It modifies a startup-related registry location.

These observations can help explain the sample's behavior.

Static vs dynamic analysis

Static analysis	Dynamic analysis
Does not execute the sample	Executes the sample in a controlled environment
Examines code and file structure	Observes runtime behavior
Can identify imports and strings	Can identify actual file and network activity
Safer when properly handled	Requires stronger isolation
May miss behavior hidden through obfuscation	May miss behavior triggered only under specific conditions

Professional practice: Use static analysis first, then perform dynamic analysis in a properly isolated environment when needed. Neither method is sufficient in every case.

3.6 Malware behavior

Definition: Malware behavior refers to the actions performed by malicious software when it executes or operates on a system.

Understanding behavior is more useful than relying only on a filename or antivirus label.

Common malware behaviors

Behavior	What it means
Persistence	Attempts to remain active after restart
Privilege escalation	Attempts to gain higher permissions
Process injection	Attempts to execute code inside another process
Credential theft	Attempts to obtain passwords or authentication data
Data exfiltration	Transfers information out of a system
Command and control (C2)	Communicates with infrastructure controlled by an attacker
Defense evasion	Attempts to avoid detection
Discovery	Collects information about the system or network
Lateral movement	Attempts to move to other systems
File encryption	Makes files inaccessible, often for extortion

What is persistence?

Definition: Persistence is a technique that allows software to continue running or regain execution after events such as a system restart or user logon.

Examples of persistence locations:

- Windows startup entries.
- Windows scheduled tasks.
- Windows services.
- Linux systemd services.
- Cron jobs.
- Shell startup configuration files.

A legitimate application may use these same mechanisms. Their presence alone is not proof of malware.

What is command and control?

Definition: Command and control is the communication mechanism through which an attacker remotely manages compromised systems.

For example, malware may periodically contact a remote server to receive instructions or send stolen information.

An analyst may investigate:

- Destination IP addresses.
- Domain names.
- DNS requests.
- Connection frequency.
- Network protocols.
- Unusual data transfers.

3.7 Reverse engineering fundamentals

Definition: Reverse engineering is the process of examining a program to understand how it works, even when its original source code is unavailable.

Most compiled software does not contain its original, human-readable source code.

Reverse engineering helps analysts examine the program's internal logic.

Important concepts

1. Source code

Human-readable instructions written by a programmer.

2. Compilation

The process of translating source code into executable or other machine-readable output.

3. Machine code

Instructions represented in a form that a processor can execute.

4. Assembly language

A human-readable representation of low-level processor instructions.

5. Disassembler

A tool that translates machine-code instructions into assembly language.

6. Decompiler

A tool that attempts to reconstruct higher-level code from a compiled program. Its output is an approximation, not necessarily the original source code.

7. Debugger

A tool used to inspect a running program or execution state, including registers, memory, and instructions.

Example

Suppose a suspicious program contains code that:

- Reads files from a directory.
- Creates a compressed archive.
- Sends the archive over a network connection.

A reverse engineer can inspect the program's functions and control flow to understand these actions.

Why reverse engineering matters in cybersecurity

- Understand unknown malware.
- Investigate suspicious program behavior.
- Identify vulnerabilities.
- Examine exploit functionality.
- Discover hidden configuration.
- Develop detection methods.

3.8 PE file format

Definition: PE (Portable Executable) is the executable file format used by Windows for programs, DLLs, and certain other executable components.

Common PE extensions include:

- `.exe`
- `.dll`

- `.sys`

Basic PE structure

Memory Forensics-Based Malware Detection Using Computer Vision and Machine Learning mdpi.com

Important components:

Component	Purpose
DOS Header	Contains the initial DOS-compatible header
PE Signature	Identifies the file as a PE image
COFF File Header	Contains information about the target machine and sections
Optional Header	Contains information such as entry point and image layout
Section Table	Describes the sections in the executable
`.text`	Commonly contains executable code
`.data`	Commonly contains initialized data
`.rdata`	Commonly contains read-only data
`.rsrc`	Commonly contains resources such as icons and dialogs
`.reloc`	Contains relocation information when present

What is an entry point?

Definition: The entry point is the location at which execution of a program begins after the operating system loader prepares it.

What are imports?

Imports describe functions and libraries that a program expects to use from other modules.

Examples include:

- File handling functions.
- Network functions.
- Process management functions.
- Cryptographic functions.

An unusual combination of imports can be worth investigating, but imported functions do not independently prove that a program is malicious.

Why PE analysis matters

Windows malware commonly uses PE files. Understanding their structure helps analysts:

- Identify suspicious executables.

- Examine imported functions.
- Inspect sections.
- Find embedded resources.
- Detect unusual file characteristics.

3.9 ELF file format

Definition: ELF (Executable and Linkable Format) is a common binary format used by Linux and many Unix-like systems.

It is used for:

- Executable programs.
- Shared libraries.
- Object files.
- Core dump files.

Basic ELF structure

ELF 文件格式详解 - fengchen
fengchen321.github.io

Component	Purpose
ELF Header	Identifies the file format and architecture
Program Header Table	Describes segments used when loading the program
Section Header Table	Describes sections used for linking and analysis
`.text`	Usually contains executable code
`.data`	Contains initialized writable data
`.rodata`	Contains read-only data
`.bss`	Represents uninitialized data
`.symtab`	Symbol table, when present
`.dynsym`	Dynamic symbol table, when present
`.dynamic`	Dynamic linking information

PE vs ELF

PE	ELF
Primarily associated with Windows	Commonly associated with Linux and Unix-like systems
Extensions often include `.exe`, `.dll`,	Often has no required filename extension

`.sys`

Uses PE headers and sections	Uses ELF headers, segments, and sections
Examined using PE analysis tools	Examined using ELF analysis tools

Important: File extensions are not reliable proof of file format. A file called `document.pdf` could contain executable data. Identify files by examining their actual contents.

3.10 Strings analysis

Definition: Strings analysis is the examination of readable text embedded inside a binary file.

A compiled program may contain text that reveals information about its functionality.

Examples of strings

```
http://example.invalid
C:\Users\Public\
cmd.exe
powershell.exe
Software\Microsoft\Windows\CurrentVersion\Run
Login failed
```

These strings may reveal:

- URLs and domain names.
- File paths.
- Registry paths.
- Commands.
- Error messages.
- Embedded configuration.

Useful commands

On Linux, `strings` can extract printable character sequences:

```
strings suspicious_file
```

Search for URLs:

```
strings suspicious_file | grep -Ei 'https?://|www\.'
```

Search for command-related strings:

```
strings suspicious_file | grep -Ei 'cmd|powershell|bash'
```

Limitations

Strings analysis cannot reliably reveal:

- Encrypted strings.

- Strings generated at runtime.
- Obfuscated configuration.
- Actual execution behavior.

It is an initial investigation technique, not a complete malware analysis.

3.11 Process monitoring

Definition: Process monitoring is the observation of running programs and their activities on a computer.

It helps analysts identify suspicious execution and relationships between processes.

What should be monitored?

- Process name.
- Process ID (PID).
- Parent process ID (PPID).
- Executable path.
- Command-line arguments.
- User account.
- Loaded libraries.
- Network connections.
- File operations.

Example

A document reader unexpectedly launches a command interpreter, which then starts another scripting process.

This process chain may deserve investigation because it is unusual for a document reader.

However, process relationships must be evaluated in context. Legitimate applications can launch other processes.

Useful tools

- Windows Process Monitor (Procmon).
- Windows Process Explorer.
- Sysmon.
- Linux `ps`.
- Linux `top` and `htop`.
- Linux `lsof`.

Example Linux commands:

`ps aux`

Shows running processes.

`pstree -p`

Displays process relationships.

`ss -tulpn`

Displays listening sockets and associated processes where permissions allow.

3.12 Sandbox analysis

Definition: Sandbox analysis is the examination of suspicious software inside a controlled environment designed to limit its access to the real system and observe its behavior.

Why use a sandbox?

Running malware on a normal computer can cause:

- File destruction.
- Data theft.
- Credential compromise.
- Network infection.
- Unauthorized remote access.

A sandbox reduces these risks, although it cannot guarantee complete safety.

How a sandbox works

Building a Malware Analysis Lab: A Threat Hunter’s Perspective (Part 1) | by Ajith Chandran | Medium
medium.com

1. Prepare an isolated analysis environment.
2. Record the sample's hash and metadata.
3. Take a clean VM snapshot.
4. Start monitoring tools.
5. Execute the sample inside the isolated VM.
6. Observe file, process, registry, and network activity.
7. Save the results.
8. Revert the VM to its clean snapshot.

What is VM snapshotting?

Definition: A VM snapshot records a virtual machine's state at a particular point, allowing an analyst to return to that state later.

It is useful for restoring a clean analysis environment.

Critical warning: A snapshot is not a security boundary. A compromised VM may still communicate with or attack other devices if networking is configured incorrectly.

For malware analysis, do not use:

- Your normal personal operating system.
- Shared folders containing personal documents.
- Shared clipboard with the host.
- Shared USB devices.
- Bridged networking connected to your normal network.

Start with no network connectivity. If network behavior must be examined, use a deliberately controlled and isolated network with monitoring. Do not connect an unknown sample directly to the public internet.

3.13 Indicators of compromise (IOC)

Definition: An Indicator of Compromise (IOC) is an observable piece of information that may indicate a system has been compromised or involved in malicious activity.

Common IOCs

IOC	Example
File hash	SHA-256 of a known malicious file
IP address	Known malicious destination IP
Domain	Suspicious command-and-control domain
File path	Unexpected executable in a temporary directory
Registry entry	Suspicious startup entry
Process name	Known malicious process name
Email address	Address associated with a phishing campaign
URL	Known malicious download URL

IOC vs IOA

IOC	IOA
Indicator of Compromise	Indicator of Attack
Identifies evidence associated with compromise	Identifies suspicious actions or behavior
Often focuses on artifacts	Focuses on activity patterns
Example: known malicious file hash	Example: unusual process injection behavior

What is a false positive?

A false positive occurs when a detection system flags legitimate activity as suspicious.

What is a false negative?

A false negative occurs when malicious activity is not detected.

Why are IOCs useful?

- Block known malicious domains.
- Detect known malware files.
- Search endpoint logs for suspicious artifacts.
- Identify affected systems.
- Support incident response.

Important: IOCs can become outdated. Malware authors can change file hashes, domains, and IP addresses. Combine IOC matching with behavior-based detection.

Module 4: Malware Analysis Tools

4.1 Ghidra

Install Ghidra on Linux | Flathub
flathub.org

Definition: Ghidra is a software reverse engineering framework used to examine compiled programs and understand their internal logic.

Main features

- Disassembly.
- Decompilation.
- Function identification.
- Code navigation.
- Cross-references.
- Data type analysis.
- Support for multiple processor architectures.

Example use

An analyst opens a suspicious executable in Ghidra and examines:

- Its entry point.
- Imported functions.
- Important functions.

- String references.
- Control flow.
- Suspicious network or file operations.

Official project: Ghidra

4.2 Detect It Easy (DIE)

Emansrepo Infostealer - PyInstaller, Deobfuscation and LLM | Nikhil “Kaido” Hegde
nikhilh-20.github.io

Definition: Detect It Easy is a tool that identifies executable file characteristics, including file format, compiler, packer, and related signatures.

Main uses

- Identify PE and ELF files.
- Detect possible packers.
- Identify compiler signatures.
- Examine executable characteristics.
- Help decide the next analysis step.

Official project: Detect It Easy

4.3 PE-bear

PE Bear - looking inside
institute.sektor7.net

Definition: PE-bear is a tool for examining Windows Portable Executable files.

Main uses

- Inspect PE headers.
- Examine sections.
- View imports and exports.
- Inspect entry point information.
- Examine resources.
- Identify unusual PE structures.

Official project: PE-bear

4.4 YARA

Malware Analysis: A Beginners Guide | ASTRA Labs
astra-labshq.github.io

Definition: YARA is a pattern-matching tool used to identify and classify files based on rules describing their contents or characteristics.

YARA rules can search for:

- Specific strings.
- Byte patterns.
- File properties.
- Combinations of conditions.

Simple YARA rule example

```
rule Example_Suspicious_File
{
    strings:
        $a = "suspicious_marker"
        $b = "example.invalid"

    condition:
        $a and $b
}
```

This rule matches a file when both strings are found.

The example is for learning only. It is not a reliable malware detector.

Why YARA is useful

- Detect known malware patterns.
- Search large collections of files.
- Classify malware families.
- Support threat hunting.
- Create custom detection rules.

Official documentation: YARA documentation

4.5 REMnux

Why REMnux Is My Go-To VM for Malware and Forensics Analysis | by Imane Ismail | HawkEyeSec | Medium
medium.com

Definition: REMnux is a Linux-based toolkit designed for malware analysis and reverse engineering.

Main uses

- Static malware analysis.
- Network behavior analysis.
- Malware document analysis.
- Memory and file investigation.
- Reverse engineering support.
- Examination of suspicious scripts and files.

Official website: REMnux

4.6 FLARE-VM

FLARE VM: The Windows Malware Analysis Distribution You've Always Needed! | Mandiant | Google Cloud Blog
cloud.google.com

Definition: FLARE-VM is a Windows-based environment that installs a collection of tools used for malware analysis and reverse engineering.

Main uses

- Static analysis.
- Dynamic analysis.
- Debugging.
- Reverse engineering.
- Executable inspection.
- Malware investigation.

Official project: FLARE-VM

REMnux vs FLARE-VM

REMnux	FLARE-VM
Linux-based analysis environment	Windows-based analysis environment
Useful for scripts, network analysis, and file examination	Useful for Windows executable analysis and debugging
Provides Linux malware-analysis tools	Provides Windows malware-analysis tools
Can complement a Windows analysis VM	Can complement a Linux analysis VM

4.7 Tool selection guide

Task	Tool

Reverse engineer a compiled program	Ghidra
Identify executable format and packer	Detect It Easy
Inspect Windows PE structure	PE-bear
Match files against detection patterns	YARA
Use a Linux malware-analysis toolkit	REMnux
Use a Windows malware-analysis toolkit	FLARE-VM
Monitor processes and file changes	Procmon, Process Explorer, Sysmon
Analyze memory	Volatility
Examine network traffic	Wireshark

Module 5: Safe Malware Analysis Lab

This module is important. Do not start executing unknown malware just because you have installed analysis tools.

Your computer has 16 GB RAM. You can build a useful lab without running several heavy virtual machines simultaneously.

5.1 Recommended lab architecture

How I Built a Safe & Isolated Malware Analysis Lab from Scratch
linkedin.com

Component	Purpose
Main Linux operating system	Your normal host
REMnux VM	Linux-based analysis
FLARE-VM	Windows-based analysis
Wireshark	Network traffic analysis
Ghidra	Reverse engineering
YARA	File pattern detection

Suggested resource allocation

Machine	Suggested RAM
Main operating system	4–6 GB
REMnux	2–3 GB
FLARE-VM	4–6 GB

Run only one analysis VM at a time initially. These are approximate allocations, not official minimum requirements.

Isolation requirements

Before analyzing any suspicious executable:

1. Use a dedicated virtual machine.
2. Take a clean snapshot.
3. Disable shared folders.
4. Disable shared clipboard and drag-and-drop.
5. Avoid USB passthrough.
6. Start with the virtual network adapter disconnected.
7. Keep personal accounts and credentials out of the VM.
8. Restore the clean snapshot after the investigation.

For network analysis, use an isolated virtual network and controlled services. Do not assume that host-only networking automatically makes every configuration safe.

5.2 Your first safe malware-analysis exercise

Do not download random malware from the internet for your first exercise.

Start with a harmless executable or a test file that you create yourself.

Exercise A — Static analysis

1. Create a harmless text file.
2. Calculate its SHA-256 hash.
3. Examine its file type.
4. Extract readable strings.
5. Record its size and metadata.
6. Create a simple YARA rule that matches a unique string in the file.

Example commands on Linux:

```
sha256sum sample.txt
```

```
file sample.txt
```

```
strings sample.txt
```

Exercise B — Process monitoring

1. Open a harmless application in your analysis VM.
2. Observe its process ID.
3. Identify its parent process.

4. Observe the files it accesses.
5. Observe any network connections it creates.
6. Record your findings.

Exercise C — Network analysis

1. Capture traffic generated by your own test application.
2. Open the capture in Wireshark.
3. Identify source and destination addresses.
4. Identify protocols.
5. Examine DNS queries.
6. Document the results.

These exercises teach the core analysis workflow without exposing your main system to malware.

Module 6: Practical Digital Forensics Projects

Project 1: USB forensic investigation

Beginner

Objective: Examine a USB drive and identify evidence.

Tasks:

1. Create a test USB drive containing sample documents.
2. Create a forensic image using an appropriate acquisition tool.
3. Calculate and record the image's SHA-256 hash.
4. Open the image in Autopsy.
5. Examine files and metadata.
6. Identify deleted-file entries where available.
7. Generate an investigation report.

Expected deliverables:

- Forensic image.
- Hash record.
- Evidence inventory.
- Screenshots.
- Investigation report.

Project 2: Windows forensic investigation

Intermediate

Objective: Reconstruct user and system activity from a Windows VM.

Tasks:

1. Create a Windows VM.
2. Perform ordinary activities such as logging in, creating files, and installing a harmless application.
3. Generate controlled test events.
4. Collect relevant Windows Event Logs.
5. Examine browser artifacts.
6. Inspect Prefetch and other available artifacts.
7. Build a chronological timeline.
8. Explain what the evidence establishes and what it does not establish.

Project 3: Linux forensic investigation

Objective: Investigate activity on a Linux VM.

Tasks:

1. Create a test Linux VM.
2. Create a new user.
3. Perform successful and failed login attempts.
4. Run ordinary commands.
5. Create and delete test files.
6. Examine authentication and system logs.
7. Examine shell history.
8. Create a timeline of the activity.

Project 4: Memory forensics

Objective: Learn to investigate running processes using a RAM capture.

Tasks:

1. Use a dedicated test VM.
2. Capture its memory using a suitable acquisition method.
3. Preserve the original capture.
4. Calculate its SHA-256 hash.

5. Examine the image with Volatility.
6. Identify running processes.
7. Examine process relationships.
8. Document suspicious or unexpected findings.

Do not assume that a process is malicious simply because its name looks unfamiliar.

Project 5: Network forensics

Objective: Investigate a controlled network capture.

Tasks:

1. Generate normal traffic in your lab.
2. Capture packets using Wireshark.
3. Identify DNS requests.
4. Identify TCP connections.
5. Examine source and destination addresses.
6. Identify unusual communication patterns.
7. Write a short investigation report.

Module 7: Practical Malware Analysis Projects

Project 1: Static malware analysis report

Objective: Examine a harmless test executable or an authorized training sample without executing it.

Tasks:

1. Calculate its SHA-256 hash.
2. Identify its file format.
3. Examine its headers.
4. Extract strings.
5. Inspect imported functions.
6. Check for signs of packing.
7. Record possible indicators.
8. Write a static analysis report.

Project 2: PE file analysis

Objective: Understand the structure of Windows executables.

Tasks:

1. Open a harmless Windows executable in Detect It Easy.
2. Identify its PE format.
3. Examine its sections using PE-bear.
4. Inspect its imports.
5. Identify its entry point.
6. Open it in Ghidra.
7. Examine selected functions.
8. Document the findings.

Project 3: YARA detection rules

Objective: Create rules that identify specific file patterns.

Tasks:

1. Create harmless sample files.
2. Add unique strings to different samples.
3. Write YARA rules for those strings.
4. Scan the sample directory.
5. Observe which files match.
6. Modify the rules to reduce unwanted matches.
7. Document detection limitations.

Project 4: Controlled dynamic analysis

Objective: Observe software behavior inside an isolated lab.

Tasks:

1. Prepare a clean analysis VM.
2. Record the test sample's hash.
3. Take a VM snapshot.
4. Start process and file monitoring.
5. Execute only a harmless test program.
6. Observe process creation and file operations.
7. Capture network activity if applicable.
8. Restore the clean VM snapshot.
9. Write a behavior analysis report.

Do not substitute a normal desktop or an ordinary internet-connected VM for a properly isolated malware-analysis environment.

Module 8: Digital Forensics vs Malware Analysis

These disciplines are closely related, but their goals differ.

Digital forensics	Malware analysis
Investigates digital evidence	Investigates suspicious software
Reconstructs events	Understands malicious code and behavior
Examines disks, memory, logs, and networks	Examines executable files, scripts, and runtime activity
Focuses on evidence preservation	Focuses on software behavior and capabilities
Supports investigations and incident response	Supports detection, response, and malware research
Example: Determine how data was stolen	Example: Determine how malware stole the data

How they work together

Suppose a company experiences a ransomware incident.

Digital forensics investigates:

- Which machine was affected first?
- When did the incident start?
- Which files were accessed?
- Which accounts were used?
- Which other machines were affected?

Malware analysis investigates:

- How does the ransomware work?
- What encryption-related behavior does it exhibit?
- How does it maintain persistence?
- Does it communicate with external servers?
- What detection opportunities exist?

Combining both disciplines provides a more complete investigation.

Module 9: Investigation and Reporting

Technical investigation is not complete until findings are documented.

A forensic report should separate facts, interpretations, and unanswered questions.

9.1 Standard investigation report structure

Section	Required information
Case information	Case ID, investigator, date
Investigation objective	What needs to be investigated
Evidence inventory	Devices and files examined
Acquisition method	How evidence was collected
Integrity verification	Hash algorithms and hash values
Tools used	Software and versions
Examination procedure	Steps performed
Findings	Evidence discovered
Timeline	Chronological events
Analysis	Interpretation of findings
Limitations	Missing evidence and uncertainty
Conclusion	What can reasonably be established
Recommendations	Further investigation or defensive actions

9.2 Example of a finding

Finding: A suspicious executable was present in a user's Downloads directory.

Supporting evidence:

- File path.
- File metadata.
- SHA-256 hash.
- Relevant event logs.
- Related process information.

Interpretation: The file's presence establishes that it was stored in that location at the time represented by the evidence. It does not, by itself, establish that the user executed it.

Further investigation: Examine process execution artifacts, memory evidence, and related system logs.

This is the difference between a defensible forensic conclusion and an unsupported accusation.

Module 10: Important Definitions for Revision

Term	Simple definition
Digital forensics	Investigation of digital evidence
Digital evidence	Digital information relevant to an investigation
Chain of custody	Record of evidence handling and transfers
Hashing	Converting data into a fixed-length hash value
Disk forensics	Examination of storage devices
Memory forensics	Examination of RAM
File system forensics	Examination of filesystem structures and metadata
Browser forensics	Examination of browser activity and artifacts
Network forensics	Investigation of network communication
Timeline analysis	Arranging events in chronological order
Deleted file recovery	Attempting to retrieve deleted data
Malware	Software designed to perform malicious activities
Static analysis	Examining a program without executing it
Dynamic analysis	Examining a program while it runs
Reverse engineering	Understanding a program by examining its internal structure
PE	Common Windows executable file format
ELF	Common Linux and Unix-like executable file format
Disassembler	Converts machine instructions into assembly representation
Decompiler	Attempts to reconstruct higher-level code
Sandbox	Controlled environment for examining software
Persistence	Maintaining execution or access across restarts or logins
C2	Command-and-control communication
IOC	Indicator of Compromise

IOA	Indicator of Attack
YARA	Pattern-matching tool used to identify files
Process monitoring	Observing running processes and their activity

Module 11: Key Differences You Must Understand

Comparison	Main difference
Volatile vs non-volatile evidence	Disappears with system state changes vs persists in storage
Disk vs memory forensics	Storage examination vs RAM examination
Normal copy vs forensic image	Selected files vs broader storage acquisition
Static vs dynamic analysis	No execution vs controlled execution
PE vs ELF	Windows-oriented format vs Linux/Unix-oriented format
Disassembler vs decompiler	Assembly output vs approximate higher-level output
IOC vs IOA	Compromise-related artifact vs suspicious behavior
Virus vs worm	Host-dependent spreading vs independent spreading
REMnux vs FLARE-VM	Linux analysis environment vs Windows analysis environment
Forensics vs malware analysis	Evidence reconstruction vs malicious software investigation

Phase 11 Completion Checklist

Before moving on, you should be able to perform these tasks independently:

- Explain digital evidence and chain of custody.
- Explain hashing and evidence integrity.
- Distinguish disk, memory, filesystem, browser, and network forensics.
- Identify important Windows and Linux artifacts.
- Build a basic forensic timeline.
- Explain deleted-file recovery limitations.
- Acquire and examine a test disk image.
- Analyze a memory capture using Volatility.
- Explain malware classification.
- Perform basic static analysis.
- Explain dynamic analysis and sandbox isolation.

- Identify PE and ELF structures.
- Extract strings from a test binary.
- Understand basic reverse engineering in Ghidra.
- Write a basic YARA rule.
- Produce a structured forensic or mal

Phase 12 — Active Directory and Enterprise Security

Beginner to advanced

1. Learning objectives

This phase teaches how organizations manage users, computers, authentication, permissions, and security across large Windows networks.

By the end, you should be able to:

1. Understand Windows Server and enterprise network environments.
2. Explain Active Directory architecture and domain controllers.
3. Manage domain users, groups, and Organizational Units.
4. Understand Group Policy and centralized administration.
5. Explain LDAP, Kerberos, NTLM, and Windows authentication.
6. Understand DNS and domain trusts in Active Directory.
7. Identify common Active Directory security weaknesses.
8. Understand Identity and Access Management (IAM) and Privileged Access Management (PAM).
9. Recognize common Active Directory attack techniques.
10. Use BloodHound to analyze relationships and privilege paths.
11. Identify signs of lateral movement and suspicious authentication.
12. Understand how enterprise environments defend their identity infrastructure.

Why this phase matters: Active Directory is widely used to centrally manage Windows users, computers, and access in enterprise environments. A weakness in its identity infrastructure can affect many systems at once.

Module 1: Windows Server Fundamentals

1.1 What is Windows Server?

Definition: Windows Server is a Microsoft operating system designed to provide services and manage resources for computers and users in a network.

Unlike a typical desktop operating system, Windows Server is designed for centralized administration, enterprise services, and server workloads.

Common uses

- User and computer management.
- File and printer sharing.
- Centralized authentication.
- DNS and DHCP services.
- Application hosting.
- Database hosting.
- Network policy management.
- Enterprise security administration.

Windows desktop vs Windows Server

Windows desktop	Windows Server
Designed primarily for individual users	Designed for server workloads and network administration
Runs desktop applications	Provides centralized network services
Commonly manages one user's environment	Can manage many users and computers
Usually used as a workstation	Commonly used as a server
Does not normally host an Active Directory domain controller role	Can host Active Directory Domain Services

1.2 What is a server?

Definition: A server is a computer or software system that provides services or resources to other computers, known as clients.

Example

In a company:

- An employee's laptop requests access to a shared folder.
- A file server provides the requested files.
- A domain controller helps authenticate the employee.
- A DNS server helps locate the relevant network services.

One physical server can host multiple services, although larger enterprises often separate important roles.

1.3 What is a client-server architecture?

Definition: Client-server architecture is a network model in which client devices request services and servers provide them.

Powering IT Infrastructure with Windows Server Windows Server is the engine behind modern IT networks—whether you're managing 10 users or 10,000. It's trusted in businesses of all sizes to keep… | Mohammed Ayaz | 31 comments
linkedin.com

Example:

1. An employee logs into a company computer.
2. The computer communicates with the domain controller.
3. The domain controller helps verify the user's identity.
4. The employee receives access according to assigned permissions.
5. The employee accesses approved company resources.

1.4 Important Windows Server roles

Server role	Purpose
Active Directory Domain Services (AD DS)	Centralized identity and domain management
DNS Server	Resolves names and helps locate services
DHCP Server	Assigns IP configuration to network clients
File Server	Provides shared file storage
Web Server (IIS)	Hosts websites and web applications
Active Directory Certificate Services (AD CS)	Provides certificate infrastructure
Remote Desktop Services	Provides remote desktop and application access

Important: These are different server roles. Installing Windows Server does not automatically make a computer a domain controller.

Module 2: Active Directory Architecture

2.1 What is Active Directory?

Definition: Active Directory (AD) is Microsoft's directory service used to centrally manage users, computers, groups, and other resources in a Windows domain environment.

It allows administrators to manage access and security policies across an organization.

Practical example

Imagine a company with:

- 500 employees.
- 300 laptops.
- 40 servers.
- 10 departments.

Without centralized identity management, administrators might need to manage separate accounts and permissions on many individual computers.

Active Directory allows administrators to centrally manage identities and access.

What does Active Directory manage?

- User accounts.
- Computer accounts.
- Security groups.
- Organizational Units.
- Authentication.
- Access-related policies.
- Domain-related information.

2.2 Active Directory architecture

Active Directory has a hierarchical structure.

A practical approach to Active Directory Domain Services, Part 1: A beginner's guide to Active Directory - ManageEngine Blog
blogs.manageengine.com

Important components:

1. Forest
2. Tree
3. Domain

4. Organizational Unit
5. Objects

What is an object?

Definition: An object is an individual entry stored in Active Directory that represents an entity such as a user, computer, group, or organizational unit.

Examples:

- User object.
- Computer object.
- Group object.
- Printer object.

2.3 What is a domain?

Definition: An Active Directory domain is a logical administrative boundary that contains users, computers, groups, and other directory objects managed under a common directory database and security policies.

Example:

`company.local`

A company might have domain accounts such as:

`alice@company.local`

`bob@company.local`

The `.local` suffix is an example used in private labs. For a real enterprise, use a properly planned DNS namespace, commonly a subdomain of a domain the organization owns.

2.4 What is a tree?

Definition: An Active Directory tree is a collection of one or more domains arranged in a hierarchical namespace with contiguous DNS naming.

Example:

```
company.com
    |
    |--- sales.company.com
    |
    |--- research.company.com
```

These domains share a hierarchical DNS namespace.

2.5 What is a forest?

Definition: An Active Directory forest is the highest-level logical container in a traditional Active Directory environment. It can contain one or more domain trees that share a common schema, configuration, and global catalog.

A forest establishes an important security and administrative boundary.

Domain vs tree vs forest

Component	Meaning
Domain	Manages a set of directory objects and domain identities
Tree	Related domains with a contiguous DNS namespace
Forest	One or more trees sharing common directory infrastructure

Important: Do not confuse an AD domain with a Windows workgroup. A workgroup does not provide the same centralized directory and authentication architecture.

2.6 What is Active Directory Domain Services?

Definition: Active Directory Domain Services (AD DS) is the Windows Server role that provides directory storage, authentication, and centralized management for an Active Directory domain.

AD DS stores directory information and helps authenticate domain identities.

2.7 What is a domain controller?

Definition: A domain controller (DC) is a Windows Server running AD DS that participates in authenticating domain users and managing Active Directory directory data.

Main responsibilities

- Authenticate domain users.
- Authenticate domain computers.
- Store directory information.
- Apply or help distribute Group Policy.
- Support directory queries.
- Participate in replication with other domain controllers.

Active Directory Fundamentals for Cybersecurity: The Foundation You Need Before Learning AD Attacks | by Umang Mishra | Jun, 2026 | Medium
medium.com

Example

An employee logs into a domain-joined laptop.

The computer contacts an appropriate domain controller to authenticate the user. The user can then access authorized enterprise resources.

Why have multiple domain controllers?

Organizations often deploy more than one domain controller for:

- Availability.
- Fault tolerance.
- Geographic distribution.
- Load distribution.
- Replication.

Important: Multiple domain controllers do not mean that each one has an unrelated copy of Active Directory. They participate in directory replication.

Module 3: Domain Users, Groups, and Organizational Units

3.1 Domain users

Definition: A domain user is a user account stored in Active Directory that can be used to authenticate to domain-connected resources.

Example:

```
Username: alice
Domain: company.local
Logon name: alice@company.local
```

A domain user can access resources according to the permissions assigned to that account.

Common account types

Account type	Purpose
Standard user	Performs ordinary employee activities
Administrator	Performs administrative tasks
Service account	Runs an application or service
Guest account	Provides limited guest access, if enabled
Managed service account	Provides managed credentials for supported services

Security rule: Employees should not use administrator accounts for routine work. Separate standard and privileged accounts reduce risk.

3.2 Domain groups

Definition: A domain group is a collection of Active Directory accounts or other groups used to manage permissions collectively.

Instead of granting permissions to 100 employees individually, an administrator can create a group and assign permissions to that group.

Example

A company has three departments:

```
Company
  |
  |--- HR Group
  |--- Finance Group
  |--- IT Group
```

The Finance group can receive access to finance-related resources without granting that access to every employee.

Types of groups

1. Security groups

Used to assign permissions and rights.

Examples:

- Finance Users
- IT Administrators
- Helpdesk Operators

2. Distribution groups

Used primarily for email distribution. They are not used to assign Windows resource permissions.

Group scope

Active Directory security groups have three main scopes:

Scope	Main purpose
Domain Local	Assign permissions to resources in its own domain
Global	Group users and groups from its own domain
Universal	Group users and groups across the forest

These scopes help administrators organize permissions across domains.

3.3 What is an Organizational Unit (OU)?

Definition: An Organizational Unit is a container inside an Active Directory domain used to organize directory objects and apply administrative delegation or Group Policy.

Example:

```
company.local
    |
    |--- Employees
    |       |--- HR
    |       |--- Finance
    |       |--- IT
    |
    |--- Computers
    |       |--- Laptops
    |       |--- Desktops
    |
    |--- Servers
```

An administrator can organize accounts based on department, location, or device type.

Why use OUs?

- Organize users and computers.
- Apply Group Policy.
- Delegate administrative tasks.
- Separate departments.
- Simplify enterprise administration.

OU vs security group

Organizational Unit	Security group
Organizes directory objects	Groups accounts or other groups
Can be used as a scope for Group Policy	Can be assigned permissions
Supports administrative delegation	Supports permission management
Is a directory container	Is a directory group object

Important: Putting a user inside an OU does not automatically grant that user access to a shared folder. Permissions must be assigned separately.

Module 4: Group Policy

4.1 What is Group Policy?

Definition: Group Policy is a Windows management feature that allows administrators to centrally configure settings for users and computers in an Active Directory environment.

It helps enforce consistent security and configuration settings across an organization.

Example

An organization wants all employee computers to:

- Require strong passwords.
- Lock screens after inactivity.
- Restrict certain control panel settings.
- Configure Windows Defender.
- Apply approved firewall settings.

Instead of configuring each computer individually, administrators can use Group Policy.

4.2 What is a Group Policy Object?

Definition: A Group Policy Object (GPO) is a collection of policy settings that can be applied to users and computers.

GPOs can contain settings related to:

- Password policies.
- Account lockout policies.
- Windows Firewall.
- Security options.
- Software deployment.
- Scripts.
- User interface restrictions.
- Administrative templates.

4.3 How does Group Policy work?

Microsoft Official Course Module 11 Implementing Group Policy
slidetodoc.com

A simplified process:

1. An administrator creates a GPO.
2. The GPO is linked to a site, domain, or OU.

3. A domain-joined computer or user processes applicable policies.
4. The configured settings are applied according to policy scope and precedence.

Common Group Policy processing order

The usual processing order is:

1. Local policy.
2. Site policy.
3. Domain policy.
4. Organizational Unit policy.

This is commonly called LSDOU.

Policies applied later can override earlier conflicting settings, subject to inheritance, enforcement, and other configuration rules.

4.4 Important Group Policy concepts

Concept	Meaning
GPO	Collection of policy settings
GPO link	Connects a GPO to a site, domain, or OU
Inheritance	Allows applicable policies from higher levels to affect lower levels
Enforced	Prevents certain lower-level policies from overriding an enforced policy
Block Inheritance	Prevents normal inheritance from higher levels, subject to enforcement
Security filtering	Controls which users or computers can apply a GPO
WMI filtering	Uses Windows Management Instrumentation conditions to filter GPO application

Useful Windows commands

Display applied Group Policy information:

```
gpresult /r
```

Generate a detailed HTML report:

```
gpresult /h gp-report.html
```

Refresh Group Policy:

```
gpupdate /force
```

Why Group Policy matters in security

Incorrect policies can expose an entire organization.

Examples:

- Weak password requirements.
- Excessive local administrator access.
- Disabled security protections.
- Unnecessary remote access.
- Insecure firewall configuration.

Group Policy is therefore an important enterprise security control.

Module 5: LDAP and Directory Communication

5.1 What is LDAP?

Definition: LDAP (Lightweight Directory Access Protocol) is a protocol used to access and manage information stored in directory services.

Active Directory supports LDAP for directory queries and management.

Example

An application wants to find a user named Alice.

It can query the directory for information such as:

- User name.
- Email address.
- Department.
- Group membership.
- Organizational Unit.

Common LDAP ports

Port	Purpose
TCP 389	LDAP
TCP 636	LDAP over TLS, commonly called LDAPS
TCP/UDP 3268	Global Catalog LDAP
TCP 3269	Global Catalog LDAP over TLS

Important: LDAP on port 389 is not automatically encrypted. LDAP can be configured with StartTLS, while LDAPS commonly uses TLS from the beginning of the connection. Proper certificate validation and secure configuration matter.

5.2 What is an LDAP directory structure?

LDAP organizes directory information using a hierarchical naming structure.

Example:

```
DC=company,DC=com
    |
    |--- OU=Employees
    |       |
    |       |--- CN=Alice
    |       |--- CN=Bob
    |
    |--- OU=Computers
```

Important terms:

Term	Meaning
DC	Domain Component
OU	Organizational Unit
CN	Common Name
DN	Distinguished Name
RDN	Relative Distinguished Name

Example Distinguished Name:

```
CN=Alice,OU=Employees,DC=company,DC=com
```

It identifies the Alice object within the directory hierarchy.

5.3 LDAP vs Active Directory

LDAP	Active Directory
A communication protocol	A directory service
Used to query and manage directory information	Stores and manages identities and directory objects
Can communicate with different directory services	Supports LDAP and other Windows authentication technologies
Not limited to Microsoft	Microsoft directory technology

LDAP is not another name for Active Directory. It is one of the protocols Active Directory supports.

Module 6: Kerberos, NTLM, and Windows Authentication

This is one of the most important modules in Active Directory security.

You must understand authentication before studying attacks such as Kerberoasting, Pass-the-Hash, and Pass-the-Ticket.

6.1 What is authentication?

Definition: Authentication is the process of verifying that a user, computer, or service is who or what it claims to be.

Examples:

- Entering a username and password.
- Using a smart card.
- Using a certificate.
- Using a security key.
- Using a Kerberos ticket.

6.2 What is authorization?

Definition: Authorization is the process of determining which resources or actions an authenticated identity is allowed to access.

Authentication vs authorization

Authentication	Authorization
Verifies identity	Determines permissions
Answers: Who are you?	Answers: What can you access?
Happens during identity verification	Uses permissions and access policies
Example: Logging into Windows	Example: Accessing a restricted folder

An account can authenticate successfully but still lack permission to access a particular resource.

6.3 What is Kerberos?

Definition: Kerberos is a network authentication protocol that uses tickets to authenticate users and services without repeatedly sending the user's password across the network.

Active Directory commonly uses Kerberos as its preferred domain authentication protocol when the necessary conditions are met.

Main Kerberos components

Component	Meaning
Client	User or computer requesting authentication
Key Distribution Center (KDC)	Trusted service that issues Kerberos tickets
Authentication Service (AS)	KDC component involved in initial authentication
Ticket Granting Service (TGS)	KDC component that issues service tickets
Ticket Granting Ticket (TGT)	Ticket used to request additional service tickets
Service Ticket	Ticket used to authenticate to a particular service

In Active Directory, the KDC runs on domain controllers.

How Kerberos authentication works

[What is Active Directory? A Beginner's Guide - Active Directory Pro
activedirectorypro.com]

Simplified flow:

1. A user logs into a domain-connected computer.
2. The client requests initial authentication from the KDC.
3. The KDC issues a Ticket Granting Ticket (TGT) after successful authentication.
4. The client requests access to a particular service.
5. The KDC issues a service ticket.
6. The client presents the service ticket to the service.
7. The service validates the ticket and grants access if authorized.

Example

Alice logs into her company computer and accesses a file server.

Kerberos can provide the authentication tickets needed for her to access the service without sending her password to the file server.

Important Kerberos ports

- TCP/UDP 88: Kerberos authentication.
- TCP/UDP 464: Kerberos password change operations.

Why time synchronization matters

Kerberos uses timestamps to help prevent replay attacks.

If the clocks of the client, domain controller, and service are significantly out of synchronization, authentication can fail.

6.4 What is NTLM?

Definition: NTLM (NT LAN Manager) is a Microsoft authentication protocol that uses a challenge-response mechanism to authenticate users.

It is an older authentication system that may still be encountered in Windows environments.

Simplified NTLM authentication

1. The client requests authentication.
2. The server sends a challenge.
3. The client calculates a response using credential-derived information.
4. The server or domain controller verifies the response.
5. Authentication succeeds or fails.

The password itself is not simply transmitted in plaintext during normal NTLM challenge-response authentication.

Why is NTLM a security concern?

NTLM has limitations compared with modern authentication approaches.

Potential concerns include:

- Credential relay attacks.
- Pass-the-Hash exposure.
- Legacy compatibility requirements.
- Weak or outdated configurations.
- Limited mutual authentication protections in some scenarios.

Organizations should reduce unnecessary NTLM usage and monitor remaining dependencies before restricting it.

6.5 Kerberos vs NTLM

Kerberos	NTLM
Ticket-based authentication	Challenge-response authentication
Uses a KDC	Uses challenge-response verification
Supports mutual authentication in appropriate configurations	Mutual authentication protections are more limited
Commonly preferred in Active Directory domains	Often used as a fallback or for legacy compatibility
Uses service tickets	Does not use Kerberos service tickets

Important: Kerberos is not automatically secure in every configuration. Weak service-account passwords, excessive privileges, and poor delegation settings can still create serious vulnerabilities.

6.6 Windows authentication

Windows can use multiple authentication mechanisms depending on the environment and application.

Examples:

- Kerberos.
- NTLM.
- Local account authentication.
- Smart card authentication.
- Certificate-based authentication.
- Windows Hello for Business.

Local vs domain authentication

Local authentication	Domain authentication
Account exists on an individual computer	Account exists in Active Directory
Managed locally	Managed centrally
Does not require a domain controller for ordinary local login	Normally depends on domain infrastructure for fresh domain authentication
Permissions are managed on the individual system	Identity and policies can be managed centrally

Windows may also use cached domain logon information when a domain controller is unavailable. This does not mean that the computer has successfully contacted the domain controller at that moment.

Module 7: DNS in Active Directory

7.1 What is DNS?

Definition: DNS (Domain Name System) translates domain names into IP addresses and helps computers locate network services.

For example:

```
server.company.com
        |
        v
192.168.1.10
```

DNS allows users and computers to use names instead of remembering IP addresses.

7.2 Why is DNS important in Active Directory?

Active Directory depends heavily on DNS.

A domain-joined computer uses DNS to locate domain controllers and other services.

For example, when a computer joins:

`company.com`

It needs to discover the appropriate domain controller.

Active Directory uses special DNS records, particularly SRV records, to advertise services.

Important DNS record types

Record	Purpose
A	Maps a hostname to an IPv4 address
AAAA	Maps a hostname to an IPv6 address
CNAME	Creates an alias for another hostname
MX	Identifies mail servers
PTR	Maps an IP address to a hostname
SRV	Identifies the location of a particular service
TXT	Stores text information

Example: Active Directory SRV records

An Active Directory DNS zone may contain records that help clients discover LDAP and Kerberos services.

Examples of service names:

`_ldap._tcp.dc._msdcs.company.com`

`_kerberos._tcp.company.com`

These records help domain-connected computers find appropriate services.

7.3 What is DNS misconfiguration?

DNS misconfiguration occurs when DNS records, permissions, servers, or settings are incorrectly configured.

Possible consequences:

- Clients cannot locate domain controllers.
- Authentication may fail.
- Services may become unavailable.

- Incorrect records may redirect clients.
- Dynamic DNS updates may be abused if poorly secured.

Security recommendations

- Restrict who can modify DNS records.
- Use secure dynamic updates where appropriate.
- Monitor unexpected DNS changes.
- Protect DNS servers.
- Maintain redundant DNS infrastructure.
- Monitor suspicious DNS queries.

Module 8: Domain Trusts

8.1 What is a domain trust?

Definition: A domain trust is a relationship that allows identities in one Active Directory domain to be recognized for authentication and access purposes in another domain.

Trusts help organizations share resources across domain boundaries.

Example

A company has two domains:

`company.com`

`research.company.com`

A trust relationship can allow authorized users from one domain to access resources in the other domain.

Important: A trust relationship does not automatically give every user access to every resource. Resource permissions and authorization still apply.

8.2 Types of Active Directory trusts

Trust type	Meaning
Parent-child trust	Relationship between parent and child domains in the same tree
Tree-root trust	Relationship between domain trees in the same forest
Shortcut trust	Direct trust created to improve authentication paths
External trust	Trust between domains in separate forests or domains that do not share the same forest
Forest trust	Trust between two Active Directory forests

Trust direction

One-way trust: One domain trusts identities from another domain in a particular direction.

Two-way trust: Both domains trust identities from the other domain.

A two-way trust still does not mean unrestricted resource access.

8.3 Why domain trusts matter for security

Incorrectly configured trust relationships can expand the impact of a compromise.

For example:

- Excessive permissions may cross domain boundaries.
- Poorly controlled administrative accounts may affect multiple domains.
- Trust misconfigurations may create unexpected access paths.

Security teams should regularly review:

- Trust relationships.
- Trust direction.
- Authentication settings.
- Administrative permissions.
- Cross-domain group memberships.

Module 9: Active Directory Security

9.1 What is Active Directory security?

Definition: Active Directory security is the set of controls and practices used to protect directory identities, domain controllers, authentication, and access to enterprise resources.

Active Directory is a high-value target because it controls access to many enterprise systems.

What can happen if AD is compromised?

- Unauthorized access to employee accounts.
- Compromise of privileged accounts.
- Access to sensitive files.
- Unauthorized changes to security policies.
- Compromise of additional computers.
- Potential control over large parts of an organization.

9.2 Common Active Directory security weaknesses

Weakness	Risk

Weak passwords	Easier credential compromise
Excessive privileges	Larger impact if an account is compromised
Unsecured service accounts	Potential credential exposure
Unnecessary NTLM	Increased legacy authentication risks
Misconfigured delegation	Possible impersonation or privilege abuse
Unpatched domain controllers	Exposure to known vulnerabilities
Poor GPO configuration	Weak or inconsistent security settings
Excessive domain administrator membership	High-impact account compromise
Insecure LDAP configuration	Potential exposure of directory communication
Unmonitored authentication	Attacks may go unnoticed

9.3 Domain Administrator

Definition: Domain Administrator is a highly privileged security group in an Active Directory domain.

Members have extensive administrative control over that domain.

A compromised Domain Administrator account can cause serious damage.

Security recommendations

- Keep membership extremely limited.
- Avoid using privileged accounts for daily browsing.
- Use separate administrative accounts.
- Require strong multifactor authentication where supported.
- Monitor privileged logons.
- Review group membership regularly.
- Use dedicated administrative workstations where appropriate.

9.4 Domain controller hardening

Definition: Domain controller hardening is the process of securing a domain controller by reducing unnecessary services, restricting access, applying updates, and enforcing appropriate security configurations.

Important practices:

1. Keep Windows Server patched.
2. Restrict interactive logons.
3. Limit local and domain administrative access.

4. Use secure authentication configurations.
5. Protect backup and recovery procedures.
6. Monitor privileged operations.
7. Secure DNS and directory communication.
8. Maintain reliable backups.
9. Restrict network access to required services.
10. Review security configuration regularly.

A domain controller should not be treated like an ordinary employee workstation.

9.5 Principle of least privilege

Definition: The principle of least privilege means that each user or system should receive only the permissions necessary to perform its assigned tasks.

Example:

An HR employee needs access to HR documents, not unrestricted access to finance servers or domain administration.

Least privilege reduces the damage caused by compromised accounts and accidental misuse.

Module 10: Identity and Access Management (IAM)

10.1 What is IAM?

Definition: Identity and Access Management (IAM) is the framework of processes, technologies, and policies used to manage digital identities and control access to organizational resources.

IAM answers four important questions:

1. Who is the user?
2. How is the user's identity verified?
3. What resources can the user access?
4. When should that access be removed?

Main IAM components

Component	Purpose
Identity management	Creates and manages digital identities
Authentication	Verifies identities
Authorization	Determines permissions

Access control	Enforces access decisions
MFA	Requires multiple authentication factors
Provisioning	Creates accounts and grants access
Deprovisioning	Removes access when no longer required
Access review	Checks whether permissions are still appropriate
Single Sign-On (SSO)	Allows access to multiple applications through a shared authentication process

10.2 What is Multi-Factor Authentication?

Definition: Multi-Factor Authentication (MFA) requires two or more different categories of authentication factors to verify an identity.

Three common categories:

- Something you know: password or PIN.
- Something you have: hardware security key or authentication device.
- Something you are: biometric characteristic.

Two passwords are not two independent authentication factors because both belong to the knowledge category.

Why is MFA important?

If an attacker obtains a password, MFA can provide an additional barrier to account compromise.

Phishing-resistant methods, such as properly implemented FIDO2 security keys, are stronger than relying only on SMS codes or basic push approvals.

10.3 What is Single Sign-On?

Definition: Single Sign-On (SSO) allows a user to authenticate once and access multiple connected applications without separately entering credentials for every application.

Example:

An employee signs in to a company identity platform and accesses:

- Email.
- HR portal.
- Internal applications.
- Approved cloud services.

SSO improves convenience, but it also makes the central identity provider a high-value security target.

10.4 What is Role-Based Access Control?

Definition: Role-Based Access Control (RBAC) assigns permissions according to a user's organizational role.

Example:

Role	Typical access
HR employee	HR systems
Finance employee	Finance systems
IT support	Approved support tools
Security analyst	Security monitoring tools
System administrator	Approved system administration

RBAC helps reduce the need to assign permissions individually.

10.5 IAM lifecycle

1. Identity creation: Create a new employee identity.
2. Provisioning: Grant required accounts and permissions.
3. Authentication: Verify the employee's identity.
4. Authorization: Allow access according to role and policy.
5. Access review: Check whether the employee still needs those permissions.
6. Deprovisioning: Disable or remove access when the employee leaves or changes roles.

A common enterprise failure is not removing old permissions after an employee changes departments or leaves the organization.

Module 11: Privileged Access Management (PAM)

11.1 What is PAM?

Definition: Privileged Access Management (PAM) is a set of security practices and technologies used to control, monitor, and protect accounts with elevated permissions.

Privileged accounts can perform sensitive operations such as:

- Creating users.
- Changing security policies.
- Managing servers.
- Accessing confidential data.

- Installing system software.
- Managing domain infrastructure.

IAM vs PAM

IAM	PAM
Manages identities and access broadly	Focuses on high-privilege access
Covers ordinary and privileged users	Focuses on powerful accounts and sessions
Handles account lifecycle and permissions	Controls, monitors, and secures privileged credentials
Example: Employee account management	Example: Domain administrator access management

11.2 What are privileged accounts?

Examples:

- Domain Administrator.
- Enterprise Administrator.
- Server administrator.
- Database administrator.
- Cloud administrator.
- Service accounts with powerful permissions.

11.3 PAM security controls

1. Just-in-Time (JIT) access

Provides elevated permissions only when needed and for a limited period.

2. Privileged credential vaulting

Stores privileged credentials in a protected system.

3. Session monitoring

Records or monitors privileged administrative sessions.

4. Credential rotation

Changes privileged passwords or secrets periodically or after relevant events.

5. Approval workflows

Requires authorization before sensitive privileged access is granted.

6. Separate administrative accounts

Keeps everyday user activity separate from administrative operations.

11.4 Why PAM matters

Suppose an employee's normal account is compromised.

If that account has permanent administrator privileges, the attacker may gain substantial control.

If privileged access is separated, restricted, and monitored, the attacker's ability to escalate privileges may be reduced.

Module 12: Enterprise Security Monitoring

12.1 What is enterprise security monitoring?

Definition: Enterprise security monitoring is the continuous collection and analysis of security-related information from an organization's systems, networks, applications, and identities to identify suspicious activity.

It helps security teams detect and investigate attacks.

What should an enterprise monitor?

- User logins.
- Failed authentication.
- Privileged account activity.
- Group membership changes.
- Domain controller events.
- GPO modifications.
- DNS queries.
- Endpoint processes.
- Network connections.
- Service account activity.
- Remote access sessions.

12.2 Important Windows security events

Event ID	Meaning	Why monitor it
4624	Successful logon	Detect unusual successful logins
4625	Failed logon	Identify repeated failed authentication
4648	Explicit credentials used	Investigate unusual credential use
4672	Special privileges assigned	Monitor privileged logons
4688	Process created, if auditing enabled	Investigate suspicious execution

4720	User account created	Detect unexpected account creation
4722	User account enabled	Detect unexpected account activation
4724	Password reset attempted	Monitor password administration
4728	Member added to a global security group	Monitor privileged group changes
4732	Member added to a local security group	Monitor sensitive group changes
4756	Member added to a universal security group	Monitor group changes
4768	Kerberos TGT requested	Monitor initial Kerberos authentication
4769	Kerberos service ticket requested	Monitor service-ticket activity
4771	Kerberos pre-authentication failed	Investigate authentication failures
4776	NTLM credential validation	Monitor NTLM authentication
4740	User account locked	Investigate repeated failed authentication
5136	Directory object modified, when auditing is configured	Investigate AD changes

Important: Event IDs are only investigative starting points. Their interpretation depends on auditing configuration, account identity, source device, timing, and surrounding activity.

12.3 SIEM in enterprise environments

Definition: A Security Information and Event Management (SIEM) system collects and analyzes security logs from multiple sources to identify suspicious activity.

Examples:

- Microsoft Sentinel.
- Splunk Enterprise Security.
- Elastic Security.
- Wazuh.

Example detection

An employee account:

1. Has several failed logins.
2. Successfully authenticates from an unusual workstation.
3. Receives unexpected group membership changes.
4. Accesses a sensitive server.

A SIEM can correlate these events and generate an alert for investigation.

A single unusual event may be harmless. A sequence of related events can provide stronger evidence.

Module 13: Advanced Active Directory Attacks

This module explains how attackers target Active Directory and how defenders identify and prevent these techniques.

Study these techniques in an isolated domain you own or have explicit permission to test. The objective is to understand the attack paths and their defensive implications, not to attack real organizations.

13.1 What is Kerberoasting?

Definition: Kerberoasting is an Active Directory attack technique in which an attacker with a valid domain identity requests Kerberos service tickets for accounts associated with Service Principal Names (SPNs), then attempts to crack the ticket's encrypted portion offline.

First understand SPN

Definition: A Service Principal Name (SPN) is an identifier that associates a service with the account under which that service runs.

Example:

```
MSSQLSvc/sqlserver.company.local:1433
```

This identifies a SQL Server service.

How Kerberoasting works conceptually

[What is Kerberoasting and why it matters for cybersecurity
expressvpn.com]

1. An attacker obtains access to an ordinary domain account.
2. The attacker identifies service accounts associated with SPNs.
3. The attacker requests relevant Kerberos service tickets.
4. The ticket's encrypted portion can be obtained.
5. The attacker attempts offline password guessing against the ticket.
6. A weak service-account password may be recovered.

Why is Kerberoasting possible?

Kerberos allows authenticated domain users to request service tickets for services they are permitted to access.

The security weakness is often not Kerberos itself. It is the use of weak or guessable passwords for service accounts.

Defensive measures

- Use long, random service-account passwords.
- Prefer Group Managed Service Accounts (gMSAs) where supported.
- Remove unnecessary SPNs.
- Avoid excessive service-account privileges.
- Monitor unusual service-ticket requests.
- Review accounts using older or weaker encryption configurations.
- Protect service-account credentials.

Important monitoring event

Windows Security Event ID 4769 records Kerberos service-ticket requests when the appropriate auditing is enabled.

A large number of unusual requests, particularly involving multiple service accounts, may justify investigation. However, legitimate applications can also generate high volumes of service-ticket requests.

13.2 What is AS-REP Roasting?

Definition: AS-REP Roasting is an Active Directory attack technique that targets accounts configured without Kerberos pre-authentication and attempts to recover their passwords by cracking obtained authentication data offline.

First understand Kerberos pre-authentication

Pre-authentication is a Kerberos security mechanism that requires a client to prove knowledge of a credential before the KDC issues the initial authentication response.

It helps prevent unauthenticated parties from freely obtaining password-cracking material for accounts.

How AS-REP Roasting works conceptually

1. An attacker identifies a domain account without Kerberos pre-authentication.
2. The attacker requests authentication information for that account.
3. The domain controller returns an AS-REP response.
4. The response contains encrypted information that may be subjected to offline password guessing.
5. A weak password may be recovered.

Why is it dangerous?

An account configured without pre-authentication may expose information that allows offline password attacks without requiring the attacker to know that account's password first.

Defensive measures

- Require Kerberos pre-authentication for ordinary accounts.
- Review accounts where it has been disabled.
- Use strong, unique passwords.
- Monitor unusual authentication activity.
- Investigate unexpected changes to account authentication settings.

Key difference:

Kerberoasting	AS-REP Roasting
Targets service accounts associated with SPNs	Targets accounts without Kerberos pre-authentication
Uses Kerberos service-ticket data	Uses Kerberos AS-REP response data
Requires an authenticated domain identity to request service tickets	Does not require prior authentication as the targeted account to obtain the relevant response
Weak service-account passwords are a common risk	Weak passwords on affected accounts are a common risk

13.3 What is Pass-the-Hash?

Definition: Pass-the-Hash (PtH) is an authentication attack technique in which an attacker uses a stolen password hash or hash-derived credential material to authenticate without first recovering the original plaintext password.

Why is it possible?

Some Windows authentication mechanisms, particularly NTLM, use password-derived cryptographic material.

If an attacker obtains usable credential material and the environment permits its use, the attacker may be able to authenticate without knowing the original password.

Example scenario

An attacker compromises a workstation and obtains credential material associated with a privileged account.

If the same account has excessive privileges on other systems, the attacker may attempt to use the stolen material to access those systems.

Defensive measures

- Reduce unnecessary NTLM authentication.
- Use unique administrative credentials.
- Restrict privileged logons to approved systems.

- Use Credential Guard where supported.
- Apply least privilege.
- Monitor suspicious NTLM authentication.
- Avoid reusing local administrator passwords.
- Use Windows LAPS to manage local administrator passwords.

13.4 What is Pass-the-Ticket?

Definition: Pass-the-Ticket (PtT) is an attack technique in which an attacker steals and reuses Kerberos authentication tickets to access services without necessarily knowing the user's plaintext password.

Understand the basic idea

Kerberos uses tickets to authenticate clients to services.

If an attacker obtains a usable ticket from a compromised system, the attacker may attempt to reuse it while it remains valid and accepted.

Types of Kerberos tickets

- TGT: Ticket Granting Ticket.
- TGS: Service ticket.

Both are important in understanding ticket-related attacks.

Defensive measures

- Protect endpoint memory and credentials.
- Use Credential Guard where supported.
- Limit privileged accounts on ordinary workstations.
- Monitor unusual ticket usage.
- Investigate unexpected logon patterns.
- Reduce the lifetime and exposure of privileged credentials according to security requirements.

Pass-the-Hash vs Pass-the-Ticket

Pass-the-Hash	Pass-the-Ticket
Uses stolen hash or hash-derived credential material	Uses stolen Kerberos tickets
Commonly associated with NTLM authentication abuse	Associated with Kerberos authentication abuse
Does not require recovering plaintext password	Does not necessarily require recovering plaintext

	password
Investigate suspicious NTLM activity	Investigate suspicious Kerberos ticket activity

13.5 What is delegation in Active Directory?

Definition: Delegation is an Active Directory and Kerberos capability that allows a service to act on behalf of a user when accessing another service, subject to configured restrictions.

Example

An employee accesses an internal web application.

The web application needs to retrieve information from a database using the employee's identity.

Delegation can allow the application to make that second request on the user's behalf.

Types of delegation

1. Unconstrained delegation

Allows a service to receive and potentially use delegated Kerberos credentials in a broad manner.

It presents significant risk if the server or service account is compromised.

2. Constrained delegation

Restricts delegation to specified services.

It reduces the scope compared with unconstrained delegation, but misconfiguration can still create security risks.

3. Resource-Based Constrained Delegation (RBCD)

Allows the target resource to control which identities are trusted to delegate to it.

This can simplify some administrative scenarios, but incorrectly configured permissions can create privilege-abuse paths.

Why is misconfigured delegation dangerous?

An attacker who compromises an account or server with overly broad delegation rights may be able to impersonate users or access services in ways the organization did not intend.

Defensive measures

- Avoid unconstrained delegation unless there is a justified requirement.
- Review delegation settings regularly.
- Restrict which accounts can configure delegation.
- Protect highly privileged accounts from delegation.
- Audit changes to delegation-related directory attributes.
- Investigate unusual service impersonation.

13.6 What is Active Directory privilege escalation?

Definition: Active Directory privilege escalation is the process of obtaining permissions beyond those initially assigned to an account, potentially reaching highly privileged domain-level access.

Common causes

Weakness	Potential consequence
Excessive group membership	Unauthorized administrative capabilities
Weak service-account passwords	Compromise of a privileged service identity
Dangerous delegated permissions	Unauthorized modification of directory objects
Misconfigured GPO permissions	Unauthorized policy changes
Unprotected credentials	Privileged credential exposure
Misconfigured delegation	Unauthorized impersonation or service access
Poorly controlled ACLs	Unauthorized changes to AD objects
Unpatched systems	Exploitation of known vulnerabilities

What is an ACL?

Definition: An Access Control List (ACL) is a list of permissions that determines which identities can perform which operations on an object.

Active Directory uses access control information to determine who can read, modify, or manage directory objects.

Example

Suppose an ordinary employee account has permission to modify a security group that grants access to sensitive servers.

Even if the employee is not an administrator, this permission could create a path to more powerful access.

This is why administrators must review not only group membership but also delegated permissions and object ACLs.

13.7 What is BloodHound?

Important AD security tool

Definition: BloodHound is a security analysis tool that maps relationships between Active Directory identities, computers, groups, permissions, and other directory objects to identify potential privilege paths.

BloodHound Setup | 4jai
4jai.net

What does BloodHound show?

It can help identify relationships such as:

- User membership in groups.
- Computer and domain relationships.
- Administrative privileges.
- Object control permissions.
- Session information.
- Delegation relationships.
- Potential paths to high-value accounts.

How does it work conceptually?

Pave the way to Domain Admins with BloodHound | Incognito Lab
incognitolab.com

1. Directory and relationship information is collected from an authorized environment.
2. The information is imported into the analysis platform.
3. BloodHound builds a relationship graph.
4. Analysts examine relationships and potential attack paths.
5. Security teams prioritize and remediate risky permissions.

What is an attack path?

Definition: An attack path is a sequence of relationships or permissions that could allow one identity to reach another identity or resource with greater privileges.

Example:

```
Ordinary User
     |
     v
Controls a Group
     |
     v
Group Can Modify a Server
     |
     v
Server Has Privileged Access
     |
     v
Potential Domain Compromise
```

This is a conceptual example, not proof that such a path is exploitable in every environment.

Why security teams use BloodHound

- Find excessive permissions.
- Identify risky group relationships.
- Discover indirect privilege paths.
- Audit delegated permissions.
- Prioritize Active Directory remediation.
- Validate security architecture.

Important limitation

BloodHound identifies relationships and potential paths. It does not automatically prove that every path can be exploited.

Analysts must validate findings against actual permissions, environment configuration, and security controls.

Official project: BloodHound

13.8 What is lateral movement?

Definition: Lateral movement is an attack technique in which an attacker moves from one compromised system to another within an organization to expand access or reach valuable resources.

Example

An attacker compromises an ordinary employee workstation.

The attacker then attempts to access:

1. Another workstation.
2. An application server.
3. A file server.
4. A sensitive database server.

This movement can help the attacker reach more valuable resources.

Common enterprise technologies involved

- Remote Desktop Protocol (RDP).
- Server Message Block (SMB).
- Windows Remote Management (WinRM).
- PowerShell remoting.
- Remote service administration.

- Remote scheduled tasks.

These are legitimate administrative technologies. Their use is not automatically malicious.

Defensive measures

- Restrict remote administration.
- Use unique credentials for privileged accounts.
- Apply network segmentation.
- Monitor remote logons.
- Restrict administrative tools.
- Use endpoint detection and response.
- Monitor unusual process and authentication chains.
- Limit which accounts can administer servers.

13.9 Lateral movement detection

Definition: Lateral movement detection is the process of identifying suspicious activity that suggests an attacker is moving between systems in a network.

Important warning signs

Warning sign	Possible meaning
One account logs into many computers unexpectedly	Possible credential abuse
Unusual administrative logon	Possible privilege misuse
Unexpected remote service creation	Possible remote execution
Unusual SMB activity	Possible file sharing or remote administration abuse
Unexpected RDP connections	Possible remote access
Unusual PowerShell remoting	Possible administrative tool abuse
New privileged group membership	Possible privilege escalation
Authentication from an unusual workstation	Possible credential misuse

Important Windows event IDs

Event ID	Meaning
4624	Successful logon
4625	Failed logon

4648	Explicit credentials used
4672	Special privileges assigned
4688	Process creation, if enabled
4697	Service installed
4728	Member added to global security group
4732	Member added to local security group
4769	Kerberos service ticket requested
4776	NTLM credential validation
7045	New service installed, in the System log

Important logon types

Windows Event ID 4624 includes a Logon Type field.

Logon type	Meaning
2	Interactive logon
3	Network logon
4	Batch logon
5	Service logon
7	Unlock
8	NetworkCleartext
9	NewCredentials
10	RemoteInteractive, commonly RDP
11	CachedInteractive

A network logon (Type 3) is not automatically an attack. Investigators need to examine the source address, account, destination computer, and related events.

Module 14: Active Directory Defense and Hardening

Understanding attacks is not enough. You must know how to secure the environment.

14.1 Account security

- Use long, unique passwords.

- Enable MFA where supported.
- Disable unused accounts.
- Remove stale accounts.
- Separate ordinary and privileged accounts.
- Review group membership.
- Use managed service accounts where appropriate.
- Avoid shared administrator credentials.

14.2 Domain controller security

- Keep domain controllers updated.
- Restrict administrative access.
- Monitor privileged operations.
- Secure DNS.
- Protect directory backups.
- Restrict unnecessary services.
- Apply secure baseline configurations.
- Maintain a tested recovery process.

14.3 Kerberos security

- Require pre-authentication for ordinary accounts.
- Use strong service-account credentials.
- Reduce unnecessary SPNs.
- Review delegation settings.
- Monitor unusual ticket requests.
- Prefer modern encryption configurations.
- Synchronize system clocks.

14.4 NTLM security

- Inventory NTLM dependencies.
- Reduce unnecessary NTLM use.
- Monitor NTLM authentication.
- Protect credential material.
- Restrict privileged account logons.
- Test compatibility before enforcing restrictions.

14.5 Group Policy security

- Restrict who can modify GPOs.
- Audit GPO changes.
- Use security filtering carefully.
- Review scripts and software deployment policies.
- Prevent unnecessary local administrator access.
- Apply appropriate endpoint security settings.

14.6 Enterprise identity security checklist

Security control	Objective
Least privilege	Reduce unnecessary permissions
MFA	Strengthen authentication
PAM	Protect privileged identities
LAPS	Manage local administrator passwords
Credential Guard	Protect supported credential material
Windows Defender	Detect and block threats
EDR	Monitor endpoint behavior
SIEM	Correlate security events
Network segmentation	Limit movement between systems
Regular access reviews	Remove unnecessary access
Tested backups	Support recovery after compromise

Module 15: Practical Active Directory Home Lab

Theory alone will not make you an Active Directory security analyst. You need to build a domain, manage identities, configure policies, and investigate security events.

15.1 Recommended lab architecture

How To Setup a Active Directory Home Lab | Oracle VirtualBox | Windows Server & PowerShell | by Shahzeb M | Medium
medium.com

For your 16 GB RAM laptop, start with just two virtual machines.

Machine	Role	Suggested RAM
Windows Server	Domain controller, DNS, AD DS	4 GB
Windows client	Domain-joined workstation	3–4 GB
Linux host	Your main operating system	Remaining RAM

You do not need Kali Linux for the initial Active Directory lab.

Network configuration

Use an isolated virtual network:

```
            Isolated Lab Network
                    |
            192.168.56.0/24
                    |
          +----------+----------+
          |                     |
    Windows Server          Windows Client
    192.168.56.10           192.168.56.20
          |
    Active Directory
          |
    company.test
```

Use an internal or host-only virtual network, not bridged networking to your normal home or college network.

The example IP addresses are for the lab only.

15.2 Lab setup stages

1. Install Windows Server

   Create a Windows Server VM using a legitimate installation image.

2. Configure networking

   Assign a static IP address to the server and configure its DNS settings appropriately.

3. Install AD DS

   Add the Active Directory Domain Services server role.

4. Create a domain

   Promote the server to a domain controller and create a test domain such as `company.test`.

5. Configure DNS

Verify that the domain's DNS records are available and that the client can locate the domain controller.

6. Install a Windows client

   Create a separate Windows client VM and connect it to the isolated lab network.

7. Join the domain

   Join the client to `company.test` using an authorized domain account.

8. Create test identities

   Create ordinary users, security groups, and OUs.

9. Configure Group Policy

   Create and apply a test GPO to a selected OU.

10. Verify authentication

    Sign in to the client using a domain user and verify the applied policies.

15.3 Practical project 1 — Active Directory administration

Beginner

Objective: Learn how to manage an enterprise domain.

Tasks:

1. Create a domain named `company.test`.
2. Create three OUs: HR, Finance, and IT.
3. Create three test users in each OU.
4. Create corresponding security groups.
5. Add users to the correct groups.
6. Create a test shared folder.
7. Assign folder permissions to the appropriate groups.
8. Test access using different domain accounts.

Expected result: You should understand how identities, groups, OUs, and resource permissions work together.

15.4 Practical project 2 — Group Policy security

Beginner

Objective: Centrally configure workstation security.

Tasks:

1. Create a GPO for the HR OU.
2. Configure an appropriate screen-lock policy.

3. Configure password and account-lockout policies at the correct scope.
4. Apply a suitable Windows Firewall policy.
5. Verify which policies are applied using `gpresult`.
6. Test the policy using a domain-joined client.
7. Document the result.

Do not confuse domain account password policy with every possible user-specific GPO setting. Password policy scope and fine-grained password policies require particular attention in Active Directory.

15.5 Practical project 3 — Authentication investigation

Intermediate

Objective: Understand and investigate domain authentication.

Tasks:

1. Sign in using a test domain account.
2. Generate successful and failed logons.
3. Access a test shared folder.
4. Examine relevant Windows Security logs.
5. Identify Event IDs 4624 and 4625.
6. Investigate Kerberos-related events where applicable.
7. Compare normal and unusual authentication activity.

Expected result: You should be able to explain what happened using event evidence rather than assumptions.

15.6 Practical project 4 — Active Directory security audit

Intermediate

Objective: Identify configuration weaknesses in your own domain.

Tasks:

1. Review privileged group membership.
2. Find unused test accounts.
3. Review service accounts.
4. Examine delegation configurations.
5. Review GPO permissions.
6. Check DNS configuration.
7. Review NTLM dependencies.

8. Document risky permissions.
9. Recommend remediation.

15.7 Practical project 5 — BloodHound analysis

Advanced

Objective: Understand directory relationships and privilege paths.

Tasks:

1. Build your own isolated AD domain.
2. Create several test users and groups.
3. Create controlled permission relationships.
4. Collect directory information using an authorized collection method.
5. Import the information into BloodHound.
6. Examine group relationships.
7. Identify potential paths to privileged accounts.
8. Verify whether the findings represent real security weaknesses.
9. Recommend changes to remove risky relationships.

Use test identities and deliberately created lab configurations. Do not collect directory information from a company or college network without explicit authorization.

15.8 Practical project 6 — Lateral movement detection

Advanced

Objective: Detect suspicious movement between domain-connected computers.

Tasks:

1. Create two domain-joined test computers.
2. Generate normal user authentication.
3. Generate approved remote administration activity.
4. Collect Windows Security logs.
5. Compare normal and unusual logon patterns.
6. Examine process creation and service installation events.
7. Build a simple SIEM detection.
8. Write an investigation report.

Expected result: You should be able to identify unusual remote access and explain why it deserves investigation.

Module 16: Phase 12 Final Revision

16.1 Important definitions

Term	Simple definition
Windows Server	Operating system designed for server and enterprise workloads
Active Directory	Microsoft directory service for centralized identity and resource management
AD DS	Windows Server role providing Active Directory domain services
Domain	Logical administrative environment containing directory objects
Domain Controller	Server running AD DS that participates in domain authentication and directory management
Forest	Highest-level logical Active Directory structure
Tree	Collection of domains with a contiguous DNS namespace
OU	Container used to organize directory objects and delegate administration
GPO	Collection of centralized user and computer policy settings
LDAP	Protocol for accessing and managing directory information
Kerberos	Ticket-based network authentication protocol
NTLM	Microsoft challenge-response authentication protocol
TGT	Kerberos Ticket Granting Ticket
SPN	Identifier associated with a service and its account
Domain Trust	Relationship allowing identities across domains to be recognized
IAM	Management of digital identities and access
PAM	Protection and control of privileged access
MFA	Authentication using multiple factor categories
SSO	One authentication process for access to multiple connected applications
RBAC	Assigning permissions based on roles
Kerberoasting	Attack targeting Kerberos service tickets for offline password cracking
AS-REP Roasting	Attack targeting accounts without Kerberos pre-authentication
Pass-the-Hash	Using stolen hash-derived credential material for authentication
Pass-the-Ticket	Reusing stolen Kerberos authentication tickets

Delegation	Allowing a service to act on behalf of a user
BloodHound	Tool for analyzing Active Directory relationships and potential attack paths
Lateral Movement	Moving between compromised systems in a network
ACL	List of permissions controlling access to an object

16.2 Critical comparisons

Comparison	Main difference
Domain vs workgroup	Centralized domain management vs decentralized workgroup management
Domain vs forest	Individual directory domain vs forest-level structure
Domain controller vs member server	Provides AD DS vs participates in the domain without hosting AD DS
User vs computer account	Represents a person or service identity vs a domain-joined computer
OU vs group	Organizes objects vs groups identities for permissions
LDAP vs AD	Protocol vs directory service
Kerberos vs NTLM	Ticket-based vs challenge-response authentication
Authentication vs authorization	Identity verification vs permission decisions
IAM vs PAM	Broad identity management vs privileged-access protection
Kerberoasting vs AS-REP Roasting	Service-ticket targeting vs pre-authentication-disabled account targeting
Pass-the-Hash vs Pass-the-Ticket	Reusing hash-derived credentials vs reusing Kerberos tickets
Unconstrained vs constrained delegation	Broad delegation vs delegation restricted to configured services

Module 17: Phase 12 Final Assessment

Answer all questions before checking the explanations.

Phase 12 quiz

0/16 answered

1. What is Active Directory primarily used for?

Video editing
Centralized identity and resource management
Disk partitioning

Packet capture

2. Which server hosts the AD DS role to participate in domain authentication?

Domain controller
Proxy server
Print client
Workstation

3. What is an Organizational Unit?

An authentication protocol
A directory container
A password hash
A network cable

4. What is the main purpose of Group Policy?

Capture packets
Centrally configure user and computer settings
Recover deleted files
Compile applications

5. Which protocol is used to query directory information?

LDAP
FTP
SMTP
ICMP

6. Which authentication protocol uses tickets?

NTLM
Kerberos
HTTP
DHCP

7. Which component issues Kerberos tickets in an AD domain?

DNS resolver
KDC
File Explorer
DHCP client

8. What is an SPN?

A service identifier associated with an account
A disk partition
A type of GPO
An encryption algorithm

9. What does Kerberoasting target?

Kerberos service tickets
Browser cookies

Linux shell history
DNS cache only

10. What is AS-REP Roasting associated with?

Accounts without Kerberos pre-authentication
Disabled DNS
Expired certificates only
File compression

11. What does Pass-the-Hash reuse?

A stolen hash or hash-derived credential material
A stolen browser bookmark
A DNS record
A GPO report

12. What does Pass-the-Ticket reuse?

Kerberos tickets
NTFS permissions
A file hash
A DHCP lease

13. What is PAM designed to protect?

Ordinary desktop wallpaper
Privileged accounts and access
DNS record formatting
Browser themes

14. What does BloodHound help identify?

Potential Active Directory privilege paths
Disk temperatures
Wireless signal strength
Video codecs

15. What is lateral movement?

Moving files to another folder
Moving between compromised systems
Changing a desktop background
Updating a password

16. Which Windows event records a Kerberos service-ticket request?

4624
4769
4720
1102
Submit answers

Phase 12 Completion Checklist

- Understand Windows Server fundamentals.
- Explain Active Directory architecture.
- Understand forests, trees, domains, and OUs.
- Create and manage domain users and groups.
- Understand Group Policy and GPO processing.
- Explain LDAP and directory structure.
- Understand Kerberos authentication.
- Understand NTLM authentication.
- Explain DNS in Active Directory.
- Understand domain trusts.
- Explain IAM, MFA, SSO, and RBAC.
- Understand PAM and least privilege.
- Recognize Kerberoasting and AS-REP Roasting.
- Explain Pass-the-Hash and Pass-the-Ticket.
- Understand delegation risks.
- Analyze Active Directory relationships using BloodHound.
- Identify suspicious lateral movement.
- Build a small Active Directory lab.
- Write a basic enterprise security investigation report.